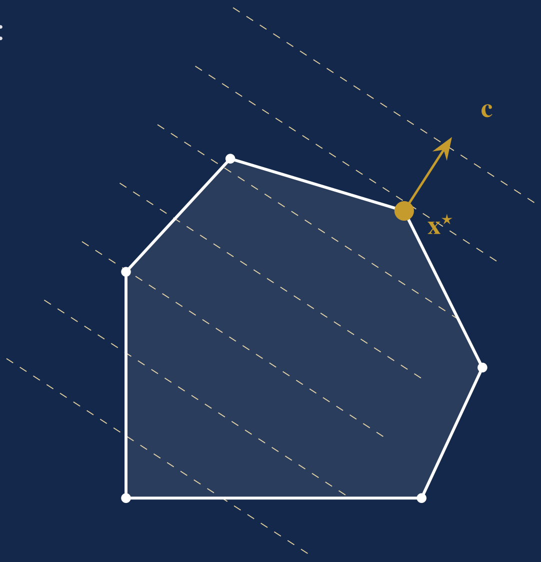
A COMPLETE STUDY MANUAL

Linear Programming

for Machine Learning

From First Principles to Modern Applications:
Theory, Geometry, Algorithms, Duality and
Learning Models, Derived Step by Step



25
Modules

150
Worked Examples

6
Solved Examples per Module

Anthony Chinedu Echem
Machine Learning Engineer and Data Scientist

FIRST EDITION • NOVEMBER 2026

Linear Programming for Machine Learning

A Complete Study Manual

First Edition, November 2026.

Typeset throughout in the Times New Roman typeface design (using TeX Gyre Termes and Nimbus Roman, its free digital versions) for text, headings, tables and mathematics.

Every numerical answer in the worked examples was checked independently against a linear programming solver before publication.

This manual may be used freely for personal study and teaching.

Preface

Linear programming is one of the quiet engines of modern computation. It schedules aircraft, routes data packets, balances investment portfolios and plans the production of almost everything we buy. It is also, far more often than most learners realise, the hidden mathematics behind machine learning. Robust regression, quantile regression, sparse signal recovery, support vector machines with the ℓ_1 norm, the Wasserstein distance between probability distributions, adversarial games, policy evaluation in reinforcement learning and inference in graphical models can all be written as linear programs. Once a learner understands linear programming deeply, a large part of machine learning stops looking like a collection of unrelated tricks and starts looking like one coherent subject.

This manual was written with a single goal: to make every concept lucid. It assumes only school algebra and a willingness to work carefully. Nothing is stated without a reason. Every equation is derived in numbered steps, every symbol is defined the first time it is used in each module, and every idea is tested immediately in worked examples whose solutions show each line of the calculation. There are twenty five modules, and each module contains six fully solved examples, giving one hundred and fifty worked problems in total.

The manual is organised in four parts.

- **Part I, Foundations** (Modules 1 to 7), builds the language of linear programming: formulation, the necessary linear algebra, convex geometry, modelling techniques, the graphical method, standard form and basic feasible solutions.
- **Part II, Algorithms** (Modules 8 to 11), develops the simplex method in full, including artificial variables, the Big M and two phase methods, the treatment of special cases and the revised simplex method in matrix form.
- **Part III, Duality and Analysis** (Modules 12 to 17), explains duality theory, complementary slackness, the dual simplex method, sensitivity analysis, interior point methods and integer programming.
- **Part IV, Linear Programming in Machine Learning** (Modules 18 to 25), applies everything to network flows, least absolute deviation regression, Chebyshev and quantile regression, linear classification, sparse recovery, optimal transport, games and adversarial learning, and Markov decision processes.

Readers who already know the simplex method may move quickly through Part II, but they are encouraged not to skip the derivations in Part III, because duality is the single idea that connects linear programming most strongly to machine learning.

How to Use This Manual

Each module follows the same rhythm so that you always know where you are. The coloured boxes below are used consistently throughout the book. Learn what each one means, and you will be able to scan any page and find exactly what you need.

Learning Objectives

Opens every module. It lists, in plain language, what you will be able to do once you have finished the module.

Definition (Name of the Concept)

States a precise definition. Every symbol in the definition is explained immediately afterwards.

Derivation (Name of the Result)

Builds an equation or a result in numbered steps. No step is skipped, and each step states the rule of algebra or logic that justifies it.

Key Result (Name of the Result)

Collects the final formula or principle that you should remember, after it has been derived.

Lucid Insight (Topic)

Gives an intuitive or geometric explanation, the kind a good tutor would draw on a whiteboard.

Common Pitfall

Warns about a mistake that learners frequently make, and shows how to avoid it.

Machine Learning Connection

Shows exactly where the idea of the module appears in machine learning practice.

Worked Example (Title of the Problem)

States a problem. Six of these appear in every module, numbered by module.

Solution

Solves the problem in labelled steps, with every calculation shown and the final answer clearly marked.

Module Summary

Closes every module with the essential ideas, so that the module can be revised in a few minutes.

A suggested way to study

1. Read the learning objectives and the motivation first, so that you know why the module matters.
2. Work through each derivation with pen and paper. Cover the next step, try to produce it yourself, then uncover it.
3. Attempt each worked example before reading its solution. Even an unsuccessful attempt makes the solution far easier to absorb.
4. Finish with the summary box and ask yourself whether you could explain each item to a colleague.

Notation and Conventions

The following conventions are used throughout. Where a module needs additional symbols, they are defined inside that module.

Symbol	Meaning
$\mathbb{R}$	The set of all real numbers.
$\mathbb{R}^n$	The set of all column vectors with n real components.
$\mathbb{R}^{m\times n}$	The set of all real matrices with m rows and n columns.
x, y, z (italic)	Scalars, that is, single real numbers.
$\mathbf{x}, \mathbf{y}, \mathbf{c}$ (bold lower case)	Column vectors. The jth component of $\mathbf{x}$ is written x_j.
$\mathbf{A}, \mathbf{B}, \mathbf{I}$ (bold upper case)	Matrices. The entry in row i and column j of $\mathbf{A}$ is written a_{ij}.
$\mathbf{A}^\top,\ \mathbf{x}^\top$	The transpose of a matrix or vector. A row vector is written as $\mathbf{x}^\top$.
$\mathbf{c}^\top\mathbf{x}$	The inner product $\sum_{j=1}^{n} c_j x_j$ of two vectors in $\mathbb{R}^n$.
$\mathbf{0},\ \mathbf{1}$	The vector of all zeros and the vector of all ones, of the appropriate size.
$\mathbf{I}$	The identity matrix, with ones on the diagonal and zeros elsewhere.
$\mathbf{x} \geq \mathbf{0}$	Every component of $\mathbf{x}$ is greater than or equal to zero. Inequalities between vectors always hold component by component.
$\mathbf{A}_j$ or $\mathbf{a}_j$	The jth column of the matrix $\mathbf{A}$.
$\mathbf{a}_i^\top$	The ith row of the matrix $\mathbf{A}$, written as a row vector.
$\|\mathbf{x}\|_1$	The ℓ_1 norm $\sum_j \left|x_j\right|$.
$\|\mathbf{x}\|_2$	The Euclidean norm $\sqrt{\sum_j x_j^2}$.
$\|\mathbf{x}\|_\infty$	The maximum norm $\max_j \left|x_j\right|$.
$\mathbf{x}^\star,\ z^\star$	An optimal solution and the optimal objective value.
$\mathbf{B},\ \mathbf{N}$	The basis matrix and the nonbasic matrix in the simplex method.
$\mathbf{x}_B,\ \mathbf{x}_N$	The basic and nonbasic variables.
$\mathbf{y}$ or $\boldsymbol{\lambda}$	Dual variables (also called shadow prices or Lagrange multipliers).
$\bar{c}_j$	The reduced cost of variable x_j.
$\Leftrightarrow$	"if and only if".
$:=$	"is defined to be equal to".

Typographical conventions

- Minus signs always appear inside mathematics, for example -3, and are typeset with the true minus symbol.
- Equations are numbered by module, so equation (8.4) is the fourth numbered equation of Module 8.
- Worked examples are numbered in the same way, so Worked Example 12.3 is the third example of Module 12.
- In simplex tableaux the pivot element is shown in a red frame, for example $\boxed{2}$.
- Final answers are placed in a gold frame, for example $\boxed{z^\star = 36}$.

Contents

Preface ii

How to Use This Manual iii

Notation and Conventions v

List of Worked Examples xiv

Part I Foundations 2

1 The Anatomy of a Linear Program 2

- 1.1 Why Linear Programming Matters . . . 2
- 1.2 Linear Functions and Linear Constraints . . . 3
- 1.3 The General Linear Program . . . 4
- 1.4 Matrix and Vector Form . . . 5
- 1.5 The Four Modelling Assumptions . . . 7
- 1.6 A Systematic Five Step Formulation Procedure . . . 7
- 1.7 Worked Examples . . . 8

2 Linear Algebra Essentials for Linear Programming 14

- 2.1 Why Linear Algebra Comes First . . . 14
- 2.2 Vectors and Their Operations . . . 14
- 2.3 Matrices and Matrix Multiplication . . . 16
- 2.4 Systems of Linear Equations and Gaussian Elimination . . . 17
- 2.5 Linear Independence, Span and Rank . . . 18
- 2.6 Hyperplanes and Half Spaces . . . 18
- 2.7 Worked Examples . . . 19

3 Convex Sets, Polyhedra and Extreme Points 24
3.1 Motivation: The Shape of the Feasible Region . . . 24
3.2 Convex Combinations and Convex Sets . . . 24
3.3 Polyhedra . . . 26
3.4 Extreme Points and Vertices . . . 26
3.5 Why Optima Occur at Vertices . . . 27
3.6 Unbounded Polyhedra and Recession Directions . . . 28
3.7 Worked Examples . . . 29

4 Modelling Techniques and Linearisation 33
4.1 Why Reformulation Is a Core Skill . . . 33
4.2 Absolute Values . . . 33
4.3 Minimax and Maximin Problems . . . 35
4.4 Convex Piecewise Linear Functions . . . 35
4.5 Free Variables . . . 36
4.6 Goal Programming . . . 36
4.7 Linear Fractional Programming . . . 37
4.8 Worked Examples . . . 37

5 The Graphical Method 42
5.1 Why Learn a Method That Only Works in Two Dimensions? . . . 42
5.2 Step 1: Drawing the Feasible Region . . . 42
5.3 Step 2: Contour Lines of the Objective . . . 43
5.4 Step 3: Corner Point Enumeration . . . 43
5.5 Step 4: Certifying Optimality . . . 43
5.6 Special Cases . . . 44
5.7 Worked Examples . . . 45

6 Standard Form, Canonical Form and Conversions 51
6.1 Why a Fixed Form Is Needed . . . 51
6.2 The Two Standard Shapes . . . 51
6.3 The Conversion Rules . . . 52
    6.3.1 Rule 1: maximisation to minimisation . . . 52
    6.3.2 Rule 2: inequality to equation with a slack variable . . . 53
    6.3.3 Rule 3: inequality to equation with a surplus variable . . . 53

6.3.4 Rule 4: negative right hand sides 53
6.3.5 Rule 5: variables that are not nonnegative 53
6.4 Counting Dimensions 54
6.5 Worked Examples 55

7 Basic Feasible Solutions and the Fundamental Theorem 59
7.1 From Geometry to Algebra 59
7.2 Bases and Basic Solutions 60
7.3 Basic Feasible Solutions Are Vertices 61
7.4 The Fundamental Theorem of Linear Programming 61
7.5 Worked Examples 63

Part II Algorithms 68

8 The Simplex Method 68
8.1 The Idea in One Paragraph 68
8.2 Deriving the Rules 69
8.3 The Simplex Tableau 70
8.4 Worked Examples 72

9 Artificial Variables: The Big M and Two Phase Methods 80
9.1 The Starting Point Problem 80
9.2 Artificial Variables 81
9.3 The Big M Method 81
9.4 The Two Phase Method 82
9.5 Worked Examples 83

10 Special Cases: Degeneracy, Cycling, Multiple Optima and Unboundedness 93
10.1 When the Tableau Tells a Story 93
10.2 Degeneracy 93
10.3 Cycling and Bland’s Rule 94
10.4 Multiple Optimal Solutions 95
10.5 Unboundedness 96
10.6 Worked Examples 97

11 The Revised Simplex Method in Matrix Form 105

11.1 Why a Revised Method? 105

11.2 The Tableau in Matrix Form 105

11.3 One Iteration of the Revised Simplex Method 107

11.4 Updating the Inverse: The Product Form 107

11.5 Worked Examples 108

Part III Duality and Analysis 114

12 Duality Theory 114

12.1 The Question That Creates the Dual 114

12.2 Deriving the Dual by Bounding 114

12.3 The Primal Dual Correspondence 116

12.4 The Duality Theorems 116

12.5 Worked Examples 118

13 Complementary Slackness, Farkas’ Lemma and Optimality Conditions 122

13.1 Where the Duality Gap Lives 122

13.2 Complementary Slackness 122

13.3 Farkas’ Lemma: Certificates of Infeasibility 124

13.4 The KKT Conditions for Linear Programs 124

13.5 Worked Examples 125

14 The Dual Simplex Method and Reoptimisation 130

14.1 Two Kinds of Feasibility in One Tableau 130

14.2 Deriving the Rules 131

14.3 Reoptimisation 132

14.4 Worked Examples 132

15 Sensitivity Analysis and Shadow Prices 139

15.1 Why Answers Must Come with Ranges 139

15.2 Changing a Right Hand Side 140

15.3 Changing an Objective Coefficient 141

15.4 Introducing a New Activity 141

15.5 Worked Examples 142

16 Interior Point Methods 146
16.1 Why Another Algorithm? . . . 146
16.2 The Logarithmic Barrier . . . 147
16.3 The Central Path . . . 148
16.4 The Primal Dual Newton Step . . . 148
16.5 Worked Examples . . . 150

17 Integer Programming, Branch and Bound, and Cutting Planes 154
17.1 When Fractions Are Not Allowed . . . 154
17.2 Modelling with Binary Variables . . . 155
17.3 Branch and Bound . . . 155
17.4 Cutting Planes . . . 156
17.5 Problems That Are Integral for Free . . . 157
17.6 Worked Examples . . . 157

Part IV Linear Programming in Machine Learning 164

18 Network Flow Models and Their Role in Machine Learning 164
18.1 Why Networks Deserve Their Own Module . . . 164
18.2 Networks and Flow Conservation . . . 164
18.3 Maximum Flow and Minimum Cut . . . 166
18.4 Worked Examples . . . 167

19 Least Absolute Deviation Regression 171
19.1 Motivation: When Squared Errors Mislead . . . 171
19.2 The Model . . . 171
19.3 The Dual of LAD Regression . . . 172
19.4 Interpolation and Robustness . . . 173
19.5 Worked Examples . . . 174

20 Chebyshev Regression and Quantile Regression 178
20.1 Beyond the Average Error . . . 178
20.2 Chebyshev Regression . . . 178
20.3 Quantile Regression . . . 179
20.4 Worked Examples . . . 181

21 Linear Classification and the ℓ_1 Support Vector Machine 185

21.1 Classification as Optimisation 185
21.2 Separability as a Feasibility Problem 186
21.3 The ℓ_1 Norm Support Vector Machine 186
21.4 Worked Examples 188

22 Sparse Recovery, Basis Pursuit and ℓ_1 Regularisation 192

22.1 The Value of Sparsity 192
22.2 Basis Pursuit 193
22.3 When Does ℓ_1 Recover Every Sparse Vector? 194
22.4 Noisy Measurements: The Dantzig Selector 195
22.5 Worked Examples 196

23 Optimal Transport and the Wasserstein Distance 200

23.1 Moving Mass at Minimum Cost 200
23.2 The Kantorovich Linear Program 200
23.3 Entropic Regularisation and the Sinkhorn Algorithm 202
23.4 Worked Examples 203

24 Zero Sum Games, Minimax and Adversarial Robustness 208

24.1 Learning against an Adversary 208
24.2 Matrix Games and Mixed Strategies 208
24.3 Each Player's Linear Program 209
24.4 Adversarial Robustness of Linear Classifiers 210
24.5 Distributionally Robust Model Selection 211
24.6 Worked Examples 211

25 Markov Decision Processes and Reinforcement Learning by Linear Programming 216

25.1 Sequential Decisions 216
25.2 Markov Decision Processes 217
25.3 The Primal Linear Program 218
25.4 The Dual: Occupancy Measures 218
25.5 Worked Examples 219

Appendix A Formula Reference 224

Appendix B Solving Linear Programs in Python 226

Appendix C Glossary 229

Appendix D Further Reading 231

Appendix E Practice Exercises with Answers 233

List of Worked Examples

1.1 Production Planning in a Furniture Workshop . . . 8
1.2 A Minimum Cost Diet . . . 9
1.3 Reading Off **c**, **A** and **b** from a Mixed Model . . . 10
1.4 Is It Linear? . . . 10
1.5 Allocating a GPU Budget across Experiments . . . 11
1.6 Designing a Training Data Mixture . . . 12
2.1 Computing a Matrix Vector Product Two Ways . . . 19
2.2 Solving a 3×3 System by Gaussian Elimination . . . 20
2.3 Testing Linear Independence and Finding the Rank . . . 21
2.4 Using the 2×2 Inverse to Find a Corner Point . . . 21
2.5 Describing a Half Space . . . 22
2.6 Distance to a Hyperplane and the Margin of a Classifier . . . 22
3.1 Writing a Point as a Convex Combination . . . 29
3.2 Proving or Disproving Convexity . . . 29
3.3 Finding All Vertices of a Polygon . . . 30
3.4 Vertex Test in Three Dimensions . . . 30
3.5 Optimising over the Vertices . . . 31
3.6 An Unbounded Polyhedron and Its Recession Direction . . . 31
4.1 Minimising a Sum of Absolute Deviations . . . 37
4.2 A Minimax Load Balancing Problem . . . 38
4.3 A Tiered Energy Tariff . . . 39
4.4 A Model with a Free Variable . . . 39
4.5 Goal Programming for a Labelling Team . . . 40
4.6 Maximising an Efficiency Ratio . . . 40
5.1 The Furniture Workshop Solved Graphically . . . 45
5.2 The Minimum Cost Diet Solved Graphically . . . 46
5.3 Multiple Optimal Solutions . . . 47

5.4 An Unbounded Problem . . . 48
5.5 An Infeasible Problem . . . 48
5.6 Routing Traffic between a Small and a Large Model . . . 49
6.1 A Complete Conversion . . . 55
6.2 Free and Nonpositive Variables with a Negative Right Hand Side . . . 55
6.3 Lower and Upper Bounds . . . 56
6.4 Interpreting Slack Values . . . 56
6.5 ℓ_1 Regression in Standard Form . . . 57
6.6 Sizing a Conversion and Going Back to Canonical Form . . . 58
7.1 Enumerating All Basic Solutions . . . 63
7.2 Computing a Basic Solution with $\mathbf{B}^{-1}$ . . . 63
7.3 A Degenerate Vertex with Several Bases . . . 64
7.4 From a Feasible Point to a Better BFS . . . 65
7.5 How Many Bases Could There Be? . . . 65
7.6 Moving Along an Edge to an Adjacent BFS . . . 66
8.1 The Furniture Workshop by the Simplex Method . . . 72
8.2 A Variable That Enters and Later Leaves . . . 73
8.3 Allocating Training Runs on a Shared Cluster . . . 74
8.4 Optimal after One Pivot, and the Meaning of Reduced Costs . . . 75
8.5 A Negative Objective Coefficient and a Minimisation in Disguise . . . 76
8.6 An Annotation Company's Product Mix . . . 77
9.1 The Minimum Cost Diet by the Big M Method . . . 83
9.2 An Equality Constraint with the Big M Method . . . 84
9.3 The Data Mixture Problem by the Two Phase Method . . . 86
9.4 Detecting Infeasibility in Phase I . . . 88
9.5 Detecting Infeasibility with the Big M Method . . . 89
9.6 A Labelling Quota by the Two Phase Method . . . 90
10.1 A Tie in the Ratio Test and a Degenerate Optimum . . . 97
10.2 A Problem That Cycles under the Largest Coefficient Rule . . . 98
10.3 Bland's Rule Breaks the Cycle . . . 100
10.4 Finding All Optimal Solutions from the Tableau . . . 101
10.5 Detecting Unboundedness and Writing the Ray . . . 102
10.6 Debugging an ML Resource Model with a Missing Budget . . . 102
11.1 Multipliers and Reduced Costs of the Furniture Optimum . . . 108

11.2 A Full Revised Simplex Iteration . . . 109
11.3 Checking the Eta Matrix by Direct Multiplication . . . 110
11.4 Solving a Problem Completely by the Revised Method . . . 110
11.5 How Much Work Does the Revised Method Save? . . . 111
11.6 Certifying a Proposed Production Plan . . . 111
12.1 The Dual of the Furniture Problem . . . 118
12.2 Dualising a Mixed Problem with the Rules Table . . . 118
12.3 Bounding an Optimum without Solving It . . . 119
12.4 The Diet Problem and the Pill Seller . . . 119
12.5 Reading the Dual from a Final Tableau . . . 120
12.6 The Dual of a Minimax Problem Is a Worst Case Mixture . . . 120
13.1 Recovering Dual Prices by Complementary Slackness . . . 125
13.2 Recovering the Primal from the Dual . . . 126
13.3 Rejecting a Proposed Solution . . . 126
13.4 A Farkas Certificate of Infeasibility . . . 127
13.5 Checking the KKT Conditions . . . 127
13.6 Complementary Slackness for the Median . . . 128
14.1 The Diet Problem without Artificial Variables . . . 132
14.2 A Three Variable Covering Problem . . . 133
14.3 Adding a New Constraint after Solving . . . 134
14.4 Reoptimising after a Capacity Change . . . 136
14.5 Detecting Infeasibility with the Dual Simplex Method . . . 137
14.6 A Licence Cap on a Model Family . . . 137
15.1 The Value and Range of Timber . . . 142
15.2 Explaining the Capacity Puzzle of Module 14 . . . 142
15.3 How Profitable Must an Unused Product Become? . . . 143
15.4 Range of Optimality for a Basic Variable . . . 143
15.5 Should a New Product Be Launched? . . . 144
15.6 The Value Curve of GPU Capacity . . . 144
16.1 A Barrier Trajectory in One Dimension . . . 150
16.2 Computing Points of the Central Path . . . 150
16.3 Gap, Duality Measure and Target . . . 151
16.4 One Primal Dual Newton Step by Hand . . . 151
16.5 Forming the Normal Equations . . . 152

16.6 How Many Iterations? 153
17.1 Why Rounding Fails 157
17.2 Branch and Bound by Hand 158
17.3 A Gomory Cut from the Optimal Tableau 159
17.4 Feature Selection under a Budget 159
17.5 A Fixed Charge Compute Rental 160
17.6 Assigning Annotators: An LP That Is Automatically Integral 161
18.1 A Transportation Problem for Data Centres 167
18.2 Shortest Path as a Linear Program 167
18.3 Maximum Flow and Minimum Cut 168
18.4 Minimum Cost Flow with Potentials 168
18.5 Image Segmentation by Minimum Cut 169
18.6 Total Unimodularity of an Incidence Matrix 170
19.1 A LAD Line through Data with an Outlier 174
19.2 A Dual Certificate for the LAD Line 174
19.3 Robustness When the Outlier Grows 175
19.4 The Median as LAD with an Intercept Only 175
19.5 LAD with Two Features 176
19.6 LAD with Prior Knowledge 177
20.1 A Chebyshev Line through Three Points 181
20.2 A Chebyshev Fit to Four Points 181
20.3 Sample Quantiles from the Pinball Loss 182
20.4 A Lower Quartile Regression Line 182
20.5 A Dual Certificate for Quantile Regression 183
20.6 A Prediction Band from Two Quantile Lines 183
21.1 Is the Training Set Separable? 188
21.2 Solving the Hard Margin ℓ_1 SVM 188
21.3 The Dual Solution and the Support Vectors 189
21.4 The XOR Problem Is Not Linearly Separable 189
21.5 A Soft Margin ℓ_1 SVM in One Dimension 190
21.6 The Effect of the Regularisation Constant 190
22.1 Basis Pursuit Finds the Sparse Solution, Least Norm Does Not 196
22.2 A Dual Certificate of Exact Recovery 196
22.3 When Basis Pursuit Fails 197

22.4 Checking the Null Space Property 197
22.5 The Dantzig Selector with an Orthonormal Design 197
22.6 A Regularisation Path for ℓ_1 Penalised LAD 198
23.1 An Exact Transport Plan 203
23.2 W_1 from Distribution Functions 204
23.3 Certifying the Plan with Kantorovich Potentials 204
23.4 Measuring Label Shift between Training and Deployment 205
23.5 One Sinkhorn Iteration 206
23.6 Matching Generated Samples to Real Samples 206
24.1 Rock, Paper, Scissors 211
24.2 A 2×2 Game by Formula and by LP 212
24.3 A 2×3 Game Solved Graphically 212
24.4 Group Robust Model Selection 213
24.5 Training a Robust Linear Classifier 214
24.6 Saddle Point Verification 215
25.1 Evaluating Fixed Policies 219
25.2 The Optimal Value by the Primal LP 220
25.3 The Dual Occupancy Measure and the Optimal Policy 221
25.4 Value Iteration Approaches the LP Solution 221
25.5 A Safety Constrained MDP 222
25.6 A Deterministic MDP Is a Shortest Path Problem 222

Part I Foundations

MODULE 1

The Anatomy of a Linear Program

Learning Objectives

After studying this module you will be able to:

- explain what a linear program is and name its three building blocks: decision variables, an objective function and constraints;
- recognise whether a function or a constraint is linear, and explain why linearity matters;
- state the four modelling assumptions of linear programming (proportionality, additivity, divisibility and certainty);
- translate a problem described in words into a precise mathematical model using a systematic five step procedure;
- rewrite any linear program in compact matrix and vector notation, identifying $\mathbf{c}$, $\mathbf{A}$ and $\mathbf{b}$;
- describe where linear programs appear inside machine learning workflows.

1.1 Why Linear Programming Matters

Almost every decision that involves limited resources can be phrased as a question of the form: *how should I choose a set of quantities so that some measure of quality is as large (or as small) as possible, while respecting a list of limits?* A factory chooses how many units of each product to make. A hospital chooses how many nurses to roster on each shift. A cloud provider chooses how many virtual machines to place on each server. A machine learning engineer chooses how many GPU hours to give each experiment, or which weights a model should use so that its prediction errors are small.

When the measure of quality and every limit can be written using only sums of quantities multiplied by constants, the question becomes a **linear program**, usually abbreviated as **LP**. Linear programs are special for three reasons.

1. **They are expressive.** A surprisingly large range of practical problems, including many machine learning problems, can be written as linear programs, sometimes after a clever reformulation.
2. **They are solvable.** There are algorithms (the simplex method of Modules 8 to 11 and the interior point methods of Module 16) that solve linear programs with millions of variables reliably.
3. **They come with certificates.** Duality theory (Modules 12 and 13) supplies a second linear program whose optimal value proves that a solution is optimal. Very few other classes of optimisation problems give such clean guarantees.

Lucid Insight What the word “programming” means

The word *programming* in “linear programming” has nothing to do with computer code. It comes from the military and economic planning of the 1940s, where a “programme” meant a plan or schedule of activities. George Dantzig, who invented the simplex method in 1947, used the phrase “programming in a linear structure”. A linear program is therefore a *linear plan*: a plan whose costs, benefits and resource usage are all linear.

1.2 Linear Functions and Linear Constraints

Before defining a linear program we must be precise about the word *linear*.

Definition Linear Function

A function $f : \mathbb{R}^n \to \mathbb{R}$ is called **linear** if it can be written in the form

$$f(x_1, x_2, \ldots, x_n) = c_1x_1 + c_2x_2 + \cdots + c_nx_n = \sum_{j=1}^{n} c_jx_j, \tag{1.1}$$

where

n is the number of variables, a positive whole number;

x_j is the jth variable, for $j = 1, 2, \ldots, n$;

c_j is a fixed real number called the *coefficient* of x_j;

$\sum_{j=1}^{n}$ means “add up the terms for $j = 1$, then $j = 2$, and so on up to $j = n$”.

Strictly speaking, a function $c_1x_1 + \cdots + c_nx_n + d$ with a constant $d \neq 0$ is called *affine*. In optimisation the constant d never changes which choice of $x_1, \ldots, x_n$ is best, because adding the same number to every possible value of the objective does not change the ordering of those values. We therefore follow common practice and call such functions linear as well.

Derivation The two defining properties of a linear function

We show that the function in (1.1) satisfies the two properties that characterise linearity. Let $\mathbf{x} = (x_1, \ldots, x_n)$ and $\mathbf{y} = (y_1, \ldots, y_n)$ be any two points and let α be any real number.

Step 1. **Additivity.** Evaluate f at the sum $\mathbf{x} + \mathbf{y}$, whose jth component is $x_j + y_j$:

$$f(\mathbf{x} + \mathbf{y}) = \sum_{j=1}^{n} c_j(x_j + y_j).$$

Step 2. Distribute each coefficient over the bracket, which is allowed because multiplication distributes over addition:

$$f(\mathbf{x} + \mathbf{y}) = \sum_{j=1}^{n} \left(c_jx_j + c_jy_j\right).$$

Step 3. Split the single sum into two sums, which is allowed because the order of addition does not matter:

$$f(\mathbf{x}+\mathbf{y}) = \sum_{j=1}^{n} c_j x_j + \sum_{j=1}^{n} c_j y_j = f(\mathbf{x}) + f(\mathbf{y}).$$

Step 4. **Homogeneity.** Evaluate f at the scaled point $\alpha\mathbf{x}$, whose jth component is αx_j:

$$f(\alpha\mathbf{x}) = \sum_{j=1}^{n} c_j(\alpha x_j) = \alpha \sum_{j=1}^{n} c_j x_j = \alpha f(\mathbf{x}),$$

where the common factor α has been taken outside the sum.

Hence a linear function satisfies $f(\mathbf{x}+\mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$ and $f(\alpha\mathbf{x}) = \alpha f(\mathbf{x})$. In words: *doubling every input doubles the output, and the effect of two changes together is the sum of their separate effects.*

Definition Linear Constraint

A **linear constraint** is a statement that a linear function is less than or equal to, greater than or equal to, or exactly equal to a constant:

$$\sum_{j=1}^{n} a_j x_j \le b, \qquad \sum_{j=1}^{n} a_j x_j \ge b, \qquad \text{or} \qquad \sum_{j=1}^{n} a_j x_j = b, \tag{1.2}$$

where

a_j	is the fixed coefficient of the variable x_j in this constraint, often the amount of a resource used by one unit of activity j;
b	is a fixed real number called the *right hand side*, often the total amount of a resource available.

Expressions such as x_1x_2, x_1^2, $\sqrt{x_1}$, e^{x_1}, $|x_1|$, $\max(x_1, x_2)$ and x_1/x_2 are *not* linear. Some of them, notably $|x_1|$, $\max(x_1, x_2)$ and certain ratios, can nevertheless be handled with the reformulation tricks of Module 4. Strict inequalities such as $x_1 < 5$ are never allowed in a linear program, because a strict inequality may have no best solution: the value $x_1 = 5$ is excluded, and for any $x_1 < 5$ there is a larger value that is still below 5.

1.3 The General Linear Program

Definition Linear Program

A **linear program** is an optimisation problem of the form

$$\begin{aligned} \text{maximise (or minimise)} \quad & z = c_1x_1 + c_2x_2 + \cdots + c_nx_n \\ \text{subject to} \quad & a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \ (\le, =, \ge)\ b_1 \\ & a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \ (\le, =, \ge)\ b_2 \\ & \qquad \vdots \\ & a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \ (\le, =, \ge)\ b_m \\ & x_1 \ge 0,\ x_2 \ge 0,\ \ldots,\ x_n \ge 0, \end{aligned} \tag{1.3}$$

where

$x_1, \dots, x_n$ are the **decision variables**, the quantities we are free to choose;

z is the **objective value**, the quantity to be maximised or minimised;

c_j is the **objective coefficient** of x_j, the contribution of one unit of x_j to z (a profit, a cost, an accuracy gain, and so on);

m is the number of **functional constraints**;

a_{ij} is the **technological coefficient** in row i and column j, the amount of resource i consumed (or requirement i satisfied) by one unit of x_j;

b_i is the **right hand side** of constraint i, the availability or requirement for item i;

$(\leq, =, \geq)$ indicates that each constraint carries exactly one of the three relations;

$x_j \geq 0$ are the **nonnegativity constraints**, which say that activities cannot be performed at negative levels.

Three further definitions describe the possible answers.

Definition Feasible Solution, Feasible Region and Optimal Solution

- A **feasible solution** is any choice of $(x_1, \dots, x_n)$ that satisfies every constraint, including nonnegativity.
- The **feasible region** is the set of all feasible solutions. We denote it by $\mathcal{F}$.
- An **optimal solution** $\mathbf{x}^\star$ is a feasible solution whose objective value is at least as good as that of every other feasible solution. For maximisation this means $z(\mathbf{x}^\star) \geq z(\mathbf{x})$ for every $\mathbf{x} \in \mathcal{F}$.
- The **optimal value** is $z^\star = z(\mathbf{x}^\star)$.

Key Result The three possible outcomes of a linear program

Every linear program falls into exactly one of three cases, a fact proved in Module 7:

1. **Infeasible**: the feasible region $\mathcal{F}$ is empty, so no choice satisfies all constraints.
2. **Unbounded**: the objective can be improved without limit while staying feasible.
3. **Optimal**: there is at least one optimal solution, and the optimal value $z^\star$ is a finite number.

1.4 Matrix and Vector Form

Writing out every term of (1.3) becomes tiresome once there are more than a handful of variables. Matrix notation compresses the whole problem into a single line, and it is the notation used by every solver and every research paper.

Derivation From scalar form to matrix form

Consider a maximisation problem in which every functional constraint has the relation $\leq$.

Step 1. **Collect the variables into a vector.** Define the column vector of decision variables

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n.$$

Step 2. **Collect the objective coefficients.** Define $\mathbf{c} = (c_1, c_2, \ldots, c_n)^\top \in \mathbb{R}^n$. The transpose symbol $\top$ turns the row into a column. The inner product of $\mathbf{c}$ and $\mathbf{x}$ is

$$\mathbf{c}^\top \mathbf{x} = \begin{pmatrix} c_1 & c_2 & \cdots & c_n \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n,$$

which is exactly the objective z. So $z = \mathbf{c}^\top \mathbf{x}$.

Step 3. **Collect the constraint coefficients into a matrix.** Place the coefficient a_{ij} in row i and column j of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

Step 4. **Multiply the matrix by the vector.** By the definition of matrix multiplication, the ith entry of $\mathbf{Ax}$ is the inner product of row i of $\mathbf{A}$ with $\mathbf{x}$:

$$(\mathbf{Ax})_i = a_{i1} x_1 + a_{i2} x_2 + \cdots + a_{in} x_n,$$

which is exactly the left hand side of constraint i.

Step 5. **Collect the right hand sides.** Define $\mathbf{b} = (b_1, \ldots, b_m)^\top \in \mathbb{R}^m$. The m constraints $(\mathbf{Ax})_i \leq b_i$ for $i = 1, \ldots, m$ are written together as the single vector inequality $\mathbf{Ax} \leq \mathbf{b}$, because inequalities between vectors are read component by component.

Step 6. **Write nonnegativity compactly.** The n conditions $x_j \geq 0$ become $\mathbf{x} \geq \mathbf{0}$.

Putting the pieces together gives the compact form below.

Key Result **Compact matrix form of a linear program**

$$\max_{\mathbf{x} \in \mathbb{R}^n} \ \mathbf{c}^\top \mathbf{x} \qquad \text{subject to} \qquad \mathbf{Ax} \leq \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}, \tag{1.4}$$

where

$\mathbf{x} \in \mathbb{R}^n$ is the vector of decision variables;

$\mathbf{c} \in \mathbb{R}^n$ is the vector of objective coefficients;

$\mathbf{A} \in \mathbb{R}^{m \times n}$ is the constraint matrix;

$\mathbf{b} \in \mathbb{R}^m$ is the right hand side vector.

The triple $(\mathbf{c}, \mathbf{A}, \mathbf{b})$ completely specifies the problem. Two other common forms, the *standard form* with equality constraints and the *canonical form*, are studied in Module 6.

1.5 The Four Modelling Assumptions

A linear model is only a faithful picture of reality when four assumptions hold, at least approximately. It is good professional practice to check each of them explicitly.

Definition Assumptions of Linear Programming

1. **Proportionality.** The contribution of each variable to the objective and to each constraint is directly proportional to its value. If one table yields a profit of 30, then ten tables yield 300. There are no set up costs and no discounts for volume.
2. **Additivity.** The total contribution is the sum of the separate contributions. There are no interaction effects such as "making chairs and tables together saves time".
3. **Divisibility.** Variables may take any real value, including fractions such as 7.5. If only whole numbers make sense, we need integer programming (Module 17).
4. **Certainty.** All coefficients c_j, a_{ij} and b_i are known constants. When they are uncertain we use sensitivity analysis (Module 15) or robust optimisation (Module 24).

Common Pitfall

A very common error is to write a constraint that mixes units, for example adding hours to kilograms. Before writing any constraint, state its units in words, for instance "hours of labour used $\leq$ hours of labour available". Every term on both sides must then carry the same unit.

1.6 A Systematic Five Step Formulation Procedure

Experienced modellers follow almost the same steps every time. Learning them early saves a great deal of confusion.

Key Result The five step procedure

1. **Understand the decision.** Ask: what exactly must be chosen? Write one sentence beginning "We must decide…".
2. **Define the decision variables**, with their units. For example, "x_1 is the number of tables produced per week".
3. **Write the objective function** as a linear expression in the variables, and state whether it is to be maximised or minimised.
4. **Write every constraint**, one for each limited resource or each requirement, checking units as you go. Do not forget nonnegativity, bounds and logical requirements such as "at least 20% of the total".

5. **Check the model** by testing a simple candidate solution, and confirm that the four assumptions are reasonable.

Machine Learning Connection

Training a machine learning model is itself an optimisation problem: we choose parameters $\mathbf{w}$ to minimise a loss $L(\mathbf{w})$ measured on data. When the loss is built from absolute values or maxima of linear functions of $\mathbf{w}$, such as the absolute error $|y_i - \mathbf{w}^\top \mathbf{x}_i|$ or the hinge loss $\max(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i)$, the training problem can be rewritten exactly as a linear program. Modules 19 to 22 show this in detail for robust regression, quantile regression, support vector machines and sparse recovery. Outside model training, linear programs also allocate compute budgets, design data mixtures, schedule labelling teams and enforce fairness requirements in decisions.

1.7 Worked Examples

Worked Example 1.1 Production Planning in a Furniture Workshop

A workshop makes tables and chairs. Each table earns a profit of £30 and each chair earns £20. Each table needs 4 units of timber and 2 hours of labour. Each chair needs 2 units of timber and 3 hours of labour. Each week 40 units of timber and 30 hours of labour are available. (a) Formulate a linear program that maximises weekly profit. (b) Write it in matrix form. (c) Evaluate the profit of the four plans $(0, 0)$, $(10, 0)$, $(0, 10)$ and $(7.5, 5)$, check which are feasible, and state the best of them.

Solution

Step 1. Decision. We must decide how many tables and chairs to make each week.

Step 2. Variables. Let x_1 be the number of tables per week and x_2 the number of chairs per week.

Step 3. Objective. Profit in pounds is 30 per table times x_1 tables plus 20 per chair times x_2 chairs:

$$\text{maximise } z = 30x_1 + 20x_2.$$

Step 4. Timber constraint (units of timber). Tables use $4x_1$ units and chairs use $2x_2$ units, and the total cannot exceed 40:

$$4x_1 + 2x_2 \le 40.$$

Step 5. Labour constraint (hours). Tables use $2x_1$ hours and chairs use $3x_2$ hours:

$$2x_1 + 3x_2 \le 30.$$

Step 6. Nonnegativity. $x_1 \ge 0$ and $x_2 \ge 0$.

Step 7. Matrix form. Reading off the coefficients,

$$\mathbf{c} = \begin{pmatrix} 30 \\ 20 \end{pmatrix}, \qquad \mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 40 \\ 30 \end{pmatrix},$$

so the model is max $\mathbf{c}^\top \mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$.

Step 8. **Test the plans.** For each plan we compute timber $4x_1 + 2x_2$, labour $2x_1 + 3x_2$ and profit $30x_1 + 20x_2$.

(x_1, x_2)	Timber (≤ 40)	Labour (≤ 30)	Feasible?	z
$(0, 0)$	0	0	yes	0
$(10, 0)$	40	20	yes	300
$(0, 10)$	20	30	yes	200
$(7.5, 5)$	$30 + 10 = 40$	$15 + 15 = 30$	yes	$225 + 100 = 325$

All four plans are feasible. The plan $(7.5, 5)$ uses all the timber and all the labour, and it gives the largest profit. In Module 5 we shall prove that it is in fact optimal for the whole problem, because it is the best corner of the feasible region.

Answer. $\boxed{x_1 = 7.5,\ x_2 = 5,\ z = 325}$ is the best of the four plans (and, as Module 5 shows, the overall optimum).

Worked Example 1.2 A Minimum Cost Diet

A nutritionist mixes two foods. Food A costs £2 per unit and contains 3 grams of protein and 1 gram of fibre per unit. Food B costs £3 per unit and contains 4 grams of protein and 3 grams of fibre per unit. The mixture must provide at least 24 grams of protein and at least 12 grams of fibre. Formulate the problem, write it in matrix form with $\le$ constraints only, and show that the plan $(4.8,\ 2.4)$ is feasible and costs less than the pure plans $(0, 6)$ and $(12, 0)$.

Solution

Step 1. **Variables.** Let x_1 be the units of food A and x_2 the units of food B in the mixture.

Step 2. **Objective.** The cost in pounds is $z = 2x_1 + 3x_2$, which we minimise.

Step 3. **Protein requirement** (grams): $3x_1 + 4x_2 \ge 24$.

Step 4. **Fibre requirement** (grams): $x_1 + 3x_2 \ge 12$.

Step 5. **Nonnegativity:** $x_1, x_2 \ge 0$.

Step 6. **Convert $\ge$ to $\le$.** Multiplying both sides of an inequality by -1 reverses its direction. So $3x_1 + 4x_2 \ge 24$ becomes $-3x_1 - 4x_2 \le -24$, and $x_1 + 3x_2 \ge 12$ becomes $-x_1 - 3x_2 \le -12$. Thus

$$\mathbf{c} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \qquad \mathbf{A} = \begin{pmatrix} -3 & -4 \\ -1 & -3 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} -24 \\ -12 \end{pmatrix},$$

and the model is min $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{Ax} \le \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$.

Step 7. **Check the plan $(4.8, 2.4)$.** Protein: $3(4.8) + 4(2.4) = 14.4 + 9.6 = 24 \ge 24$. Fibre: $4.8 + 3(2.4) = 4.8 + 7.2 = 12 \ge 12$. Both requirements are met exactly. Cost: $2(4.8) + 3(2.4) = 9.6 + 7.2 = 16.8$.

Step 8. **Check the pure plans.** For $(0, 6)$: protein 24, fibre 18, cost 18. For $(12, 0)$: protein 36, fibre 12, cost 24. Both are feasible but more expensive.

Answer. The mixture $\boxed{x_1 = 4.8,\ x_2 = 2.4}$ is feasible with cost £16.80, cheaper than £18 and £24. Module 5 confirms it is the minimum cost diet.

Worked Example 1.3 Reading Off c, A and b from a Mixed Model

Write the following linear program in the compact form max $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$:

$$\begin{aligned} \text{maximise} \quad & 5x_1 - 2x_2 + 4x_3 \\ \text{subject to} \quad & x_1 + x_2 + x_3 \le 10, \\ & 2x_1 - x_3 \ge -4, \\ & x_2 + 3x_3 = 9, \\ & x_1, x_2, x_3 \ge 0. \end{aligned}$$

Solution

Step 1. **The first constraint** is already of the ≤ type: row $(1,\ 1,\ 1)$ with right hand side 10.

Step 2. **The second constraint** is of the ≥ type. Multiply by -1 to obtain $-2x_1 + 0x_2 + x_3 \le 4$: row $(-2,\ 0,\ 1)$ with right hand side 4.

Step 3. **The equality constraint.** An equation $p = q$ holds exactly when both $p \le q$ and $p \ge q$ hold. Therefore $x_2 + 3x_3 = 9$ is replaced by the pair

$$x_2 + 3x_3 \le 9 \qquad \text{and} \qquad x_2 + 3x_3 \ge 9.$$

The second of these becomes $-x_2 - 3x_3 \le -9$ after multiplication by -1.

Step 4. **Assemble.** With the variables ordered as (x_1, x_2, x_3):

$$\mathbf{c} = \begin{pmatrix} 5 \\ -2 \\ 4 \end{pmatrix}, \qquad \mathbf{A} = \begin{pmatrix} 1 & 1 & 1 \\ -2 & 0 & 1 \\ 0 & 1 & 3 \\ 0 & -1 & -3 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 10 \\ 4 \\ 9 \\ -9 \end{pmatrix}.$$

Step 5. **Check dimensions.** There are $n = 3$ variables and, after the splitting, $m = 4$ rows, so $\mathbf{A} \in \mathbb{R}^{4\times3}$, $\mathbf{b} \in \mathbb{R}^4$ and $\mathbf{c} \in \mathbb{R}^3$. The product $\mathbf{A}\mathbf{x}$ is defined because $\mathbf{A}$ has 3 columns and $\mathbf{x}$ has 3 entries.

Answer. The model is max $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$ with the $\mathbf{c}$, $\mathbf{A}$ and $\mathbf{b}$ shown above.

Worked Example 1.4 Is It Linear?

For each expression decide whether it may appear directly in a linear program. Where it may not, say whether it can be rewritten linearly under a stated condition. (a) $3x_1 + 2x_2 + 7 \le 20$; (b) $x_1x_2 \le 6$; (c) $\frac{x_1}{x_2} \le 2$ with $x_2 > 0$; (d) $x_1^2 + x_2 \le 4$; (e) $\frac{x_1 + x_2}{2} \ge 3$; (f) $x_1 < 4$.

Solution

Step 1. **(a)** Subtract 7 from both sides: $3x_1 + 2x_2 \leq 13$. This is a linear constraint. *Linear.*

Step 2. **(b)** The product x_1x_2 multiplies two variables together. Doubling both variables multiplies the product by four, which violates homogeneity. *Not linear* and, in general, not rewritable as a single linear constraint.

Step 3. **(c)** A ratio of variables is not linear. However, because $x_2 > 0$, we may multiply both sides by x_2 without reversing the inequality: $x_1 \leq 2x_2$, that is $x_1 - 2x_2 \leq 0$. *Linear after rewriting*, provided x_2 is known to be positive.

Step 4. **(d)** The square x_1^2 is not linear because $(2x_1)^2 = 4x_1^2 \neq 2x_1^2$. *Not linear.*

Step 5. **(e)** Multiply both sides by 2 (a positive constant, so the direction is kept): $x_1 + x_2 \geq 6$. *Linear.*

Step 6. **(f)** The relation is a strict inequality, which is not allowed. In practice it is replaced by $x_1 \leq 4$, or by $x_1 \leq 4 - \varepsilon$ for a small chosen tolerance $\varepsilon > 0$. *Not allowed as written.*

Answer. (a) and (e) are linear; (c) is linear after multiplying by $x_2 > 0$; (b) and (d) are nonlinear; (f) must be replaced by a non strict inequality.

Worked Example 1.5 Allocating a GPU Budget across Experiments

A machine learning team has a budget of £100 for cloud GPUs and three experiments to run. Each GPU hour spent on experiment A, B or C is expected to improve validation accuracy by 0.08, 0.05 and 0.03 percentage points respectively, and costs £2, £1 and £0.50 respectively (the experiments use different machine types). Each experiment can use at most 40 GPU hours. Formulate an LP that maximises the total expected accuracy gain, and solve it by reasoning about value per pound.

Solution

Step 1. **Variables.** Let x_A, x_B, x_C be the GPU hours given to experiments A, B and C.

Step 2. **Objective** (percentage points): maximise $z = 0.08x_A + 0.05x_B + 0.03x_C$.

Step 3. **Budget** (pounds): $2x_A + 1x_B + 0.5x_C \leq 100$.

Step 4. **Bounds** (hours): $0 \leq x_A \leq 40$, $0 \leq x_B \leq 40$, $0 \leq x_C \leq 40$.

Step 5. **Value per pound.** Dividing gain per hour by cost per hour gives the gain bought by one pound:

$$\text{A: } \frac{0.08}{2} = 0.04, \qquad \text{B: } \frac{0.05}{1} = 0.05, \qquad \text{C: } \frac{0.03}{0.5} = 0.06.$$

Step 6. **Why the greedy rule is optimal here.** There is a single budget constraint and simple upper bounds. Moving one pound from a lower ratio experiment to a higher ratio one always increases z, so at the optimum we fill the highest ratio first. (This is the *fractional knapsack* argument; it fails when there are several shared constraints, which is why we need the simplex method in general.)

Step 7. **Fill C first.** $x_C = 40$ hours costs $0.5 \times 40 = 20$ pounds. Remaining budget $100 - 20 = 80$.

Step 8. **Then B.** $x_B = 40$ hours costs 40 pounds. Remaining budget $80 - 40 = 40$.

Step 9. **Then A.** The remaining 40 pounds buy $40/2 = 20$ hours, so $x_A = 20$, which is within its bound of 40.

Step 10. **Objective value.** $z = 0.08(20) + 0.05(40) + 0.03(40) = 1.6 + 2.0 + 1.2 = 4.8$.

Answer. $\boxed{x_A = 20,\ x_B = 40,\ x_C = 40,\ z^\star = 4.8 \text{ percentage points}}$, using the whole budget.

Worked Example 1.6 Designing a Training Data Mixture

A team needs at least 50 thousand labelled examples for fine tuning. Three sources are available. Source A costs £4 per thousand examples with a label noise rate of 2%. Source B costs £1 per thousand with noise 8%. Source C costs £2 per thousand with noise 6%. The average noise of the mixture must not exceed 5%, and at least 20% of the mixture must come from source C to cover a specialist domain. Formulate a linear program that minimises cost, rewrite the ratio constraints linearly, and verify that the plan $(a, b, c) = (12.5,\ 0,\ 37.5)$ is feasible. Its optimality is confirmed in Module 9 (Worked Example 9.3).

Solution

Step 1. **Variables.** Let a, b, c be the thousands of examples bought from sources A, B and C.

Step 2. **Objective** (pounds): minimise $z = 4a + b + 2c$.

Step 3. **Volume requirement** (thousands of examples): $a + b + c \geq 50$.

Step 4. **Noise requirement.** The number of noisy examples (in thousands) is $0.02a + 0.08b + 0.06c$, and the total is $a + b + c$. The average noise is their ratio, which must be at most 0.05:

$$\frac{0.02a + 0.08b + 0.06c}{a + b + c} \leq 0.05.$$

Because $a + b + c \geq 50 > 0$, multiply both sides by $a + b + c$ without changing the direction:

$$0.02a + 0.08b + 0.06c \leq 0.05a + 0.05b + 0.05c.$$

Move all terms to the left: $-0.03a + 0.03b + 0.01c \leq 0$. Multiply by 100 to clear decimals:

$$-3a + 3b + c \leq 0.$$

Step 5. **Domain coverage.** The share of C must be at least 20%: $c \geq 0.2(a + b + c)$. Expanding and collecting terms gives $0.8c - 0.2a - 0.2b \geq 0$, and multiplying by 5 gives

$$-a - b + 4c \geq 0.$$

Step 6. **Complete model.**

$$\begin{aligned} \min \quad & 4a + b + 2c \\ \text{subject to} \quad & a + b + c \geq 50, \\ & -3a + 3b + c \leq 0, \\ & -a - b + 4c \geq 0, \\ & a, b, c \geq 0. \end{aligned}$$

Step 7. **Check the plan** $(12.5, 0, 37.5)$**.** Volume: $12.5 + 0 + 37.5 = 50 \geq 50$. Noise: $-3(12.5) + 0 + 37.5 = -37.5 + 37.5 = 0 \leq 0$. Coverage: $-12.5 - 0 + 4(37.5) = 137.5 \geq 0$. All constraints hold, so the plan is feasible. Its cost is $4(12.5) + 0 + 2(37.5) = 50 + 75 = 125$.

Step 8. **Interpretation.** Source B is cheap but noisy; each thousand from B forces extra purchases from the clean but expensive source A. Source C alone is too noisy ($6\% > 5\%$), so the cheapest acceptable blend mixes C with just enough A to bring the average noise down to exactly 5%.

Answer. The plan $\boxed{a = 12.5,\ b = 0,\ c = 37.5,\ z = 125}$ is feasible; the two phase simplex method of Module 9 shows that no cheaper plan exists.

Module Summary

- A linear program chooses decision variables to maximise or minimise a linear objective subject to linear equality and inequality constraints.
- Linear functions satisfy additivity $f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$ and homogeneity $f(\alpha\mathbf{x}) = \alpha f(\mathbf{x})$.
- Any LP can be written compactly as max $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \leq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, after converting $\geq$ constraints by multiplying by -1 and splitting equalities into two inequalities.
- The four assumptions are proportionality, additivity, divisibility and certainty.
- Formulate with five steps: decision, variables with units, objective, constraints, check.
- Ratio constraints with a positive denominator become linear after multiplying through by the denominator.
- Many ML tasks (compute allocation, data mixture design and, later, robust regression and classification) are linear programs.

Module 2

Linear Algebra Essentials for Linear Programming

Learning Objectives

After studying this module you will be able to:

- perform and interpret vector addition, scalar multiplication, inner products and matrix vector products;
- solve a system of linear equations by Gaussian elimination and explain each row operation;
- decide whether a set of vectors is linearly independent and compute the rank of a matrix;
- derive and use the formula for the inverse of a 2×2 matrix;
- describe hyperplanes and half spaces, the geometric objects from which every feasible region is built;
- derive the distance from a point to a hyperplane and connect it to the margin of a linear classifier.

2.1 Why Linear Algebra Comes First

The simplex method, duality theory and interior point methods are all written in the language of vectors and matrices. A basic feasible solution (Module 7) is found by solving a square system of linear equations. A simplex pivot (Module 8) is a Gaussian elimination step. A constraint $\mathbf{a}^\top \mathbf{x} \leq b$ is a half space, and a feasible region is an intersection of half spaces. If this module is mastered, every later module becomes much easier to read.

2.2 Vectors and Their Operations

Definition Vector

A **vector** in $\mathbb{R}^n$ is an ordered list of n real numbers written as a column,

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix},$$

where

x_j is the jth *component* of $\mathbf{x}$, a real number;

n is the *dimension* of the vector.

Geometrically, a vector in $\mathbb{R}^2$ or $\mathbb{R}^3$ is a point, or equivalently an arrow from the origin to that point.

Definition Vector Addition and Scalar Multiplication

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and a scalar $\alpha \in \mathbb{R}$,

$$\mathbf{x} + \mathbf{y} = \begin{pmatrix} x_1 + y_1 \\ \vdots \\ x_n + y_n \end{pmatrix}, \qquad \alpha\mathbf{x} = \begin{pmatrix} \alpha x_1 \\ \vdots \\ \alpha x_n \end{pmatrix}. \tag{2.1}$$

Both operations act component by component.

Definition Inner Product and Euclidean Norm

The **inner product** (also called the dot product) of $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ is the scalar

$$\mathbf{x}^\top \mathbf{y} = x_1y_1 + x_2y_2 + \cdots + x_ny_n = \sum_{j=1}^{n} x_jy_j. \tag{2.2}$$

The **Euclidean norm** (length) of $\mathbf{x}$ is

$$\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}} = \sqrt{x_1^2 + \cdots + x_n^2}. \tag{2.3}$$

where

$\mathbf{x}^\top$ is the row vector obtained by transposing $\mathbf{x}$;

x_jy_j is the product of the jth components.

Lucid Insight What an inner product measures

The inner product $\mathbf{c}^\top\mathbf{x}$ measures how far $\mathbf{x}$ travels in the direction of $\mathbf{c}$, scaled by the length of $\mathbf{c}$. In a linear program the objective $\mathbf{c}^\top\mathbf{x}$ therefore rewards points that lie far along the direction $\mathbf{c}$. This single picture explains why optimal solutions sit on the boundary of the feasible region: to go as far as possible in a fixed direction you must keep moving until a wall stops you.

Derivation The angle formula for the inner product

We show that $\mathbf{x}^\top\mathbf{y} = \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2 \cos\theta$, where θ is the angle between $\mathbf{x}$ and $\mathbf{y}$.

Step 1. Consider the triangle whose sides are the arrows $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{x} - \mathbf{y}$. The law of cosines from elementary geometry says

$$\|\mathbf{x} - \mathbf{y}\|_2^2 = \|\mathbf{x}\|_2^2 + \|\mathbf{y}\|_2^2 - 2\,\|\mathbf{x}\|_2\,\|\mathbf{y}\|_2 \cos\theta.$$

Step 2. Expand the left side using the definition (2.3) and the distributive property of the inner product:

$$\|\mathbf{x}-\mathbf{y}\|_2^2 = (\mathbf{x}-\mathbf{y})^\top(\mathbf{x}-\mathbf{y}) = \mathbf{x}^\top\mathbf{x} - \mathbf{x}^\top\mathbf{y} - \mathbf{y}^\top\mathbf{x} + \mathbf{y}^\top\mathbf{y}.$$

Step 3. Since $\mathbf{x}^\top\mathbf{y} = \mathbf{y}^\top\mathbf{x}$ (multiplication of numbers is commutative), this equals $\|\mathbf{x}\|_2^2 - 2\mathbf{x}^\top\mathbf{y} + \|\mathbf{y}\|_2^2$.

Step 4. Set the two expressions equal and cancel $\|\mathbf{x}\|_2^2 + \|\mathbf{y}\|_2^2$ from both sides:

$$-2\mathbf{x}^\top\mathbf{y} = -2\,\|\mathbf{x}\|_2\,\|\mathbf{y}\|_2 \cos\theta.$$

Step 5. Divide by -2:

$$\mathbf{x}^\top\mathbf{y} = \|\mathbf{x}\|_2\,\|\mathbf{y}\|_2 \cos\theta. \tag{2.4}$$

Consequences: $\mathbf{x}^\top\mathbf{y} > 0$ when the angle is acute, $\mathbf{x}^\top\mathbf{y} = 0$ when the vectors are perpendicular (*orthogonal*), and $\mathbf{x}^\top\mathbf{y} < 0$ when the angle is obtuse.

2.3 Matrices and Matrix Multiplication

Definition Matrix and Matrix Vector Product

A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is a rectangular array with m rows and n columns; a_{ij} denotes the entry in row i and column j. For $\mathbf{x} \in \mathbb{R}^n$, the product $\mathbf{A}\mathbf{x} \in \mathbb{R}^m$ has components

$$(\mathbf{A}\mathbf{x})_i = \sum_{j=1}^{n} a_{ij}x_j, \qquad i = 1,\ldots,m. \tag{2.5}$$

Derivation Two equivalent views of Ax

Let $\mathbf{a}_i^\top$ denote row i of $\mathbf{A}$ and let $\mathbf{A}_j$ denote column j of $\mathbf{A}$.

Step 1. **Row view.** Formula (2.5) says that entry i of $\mathbf{A}\mathbf{x}$ is $a_{i1}x_1 + \cdots + a_{in}x_n$, which by (2.2) is the inner product $\mathbf{a}_i^\top\mathbf{x}$. Hence

$$\mathbf{A}\mathbf{x} = \begin{pmatrix} \mathbf{a}_1^\top\mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top\mathbf{x} \end{pmatrix}.$$

Step 2. **Column view.** Write the same entries column by column. Entry i of $x_1\mathbf{A}_1 + \cdots + x_n\mathbf{A}_n$ is $x_1a_{i1} + \cdots + x_na_{in}$, which is again (2.5). Hence

$$\mathbf{A}\mathbf{x} = x_1\mathbf{A}_1 + x_2\mathbf{A}_2 + \cdots + x_n\mathbf{A}_n. \tag{2.6}$$

The row view reads each constraint separately. The column view says that $\mathbf{A}\mathbf{x}$ is a *weighted combination of the columns*, with weights x_j. The column view is the heart of the simplex method: choosing a basis means choosing which columns may carry nonzero weights.

Definition   Matrix Product, Identity and Inverse

For $\mathbf{A} \in \mathbb{R}^{m\times p}$ and $\mathbf{B} \in \mathbb{R}^{p\times n}$, the product $\mathbf{AB} \in \mathbb{R}^{m\times n}$ has entries $(\mathbf{AB})_{ij} = \sum_{k=1}^{p} a_{ik}b_{kj}$. The **identity** $\mathbf{I}$ has ones on the diagonal and zeros elsewhere, so $\mathbf{Ix} = \mathbf{x}$. A square matrix $\mathbf{B}$ is **invertible** if there is a matrix $\mathbf{B}^{-1}$ with $\mathbf{BB}^{-1} = \mathbf{B}^{-1}\mathbf{B} = \mathbf{I}$. Then the system $\mathbf{Bx} = \mathbf{b}$ has the unique solution $\mathbf{x} = \mathbf{B}^{-1}\mathbf{b}$.

Derivation   The inverse of a 2×2 matrix

Let $\mathbf{B} = \begin{pmatrix} p & q \\ r & s \end{pmatrix}$ with real entries p, q, r, s. We look for $\mathbf{B}^{-1} = \begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix}$ with $\mathbf{BB}^{-1} = \mathbf{I}$.

Step 1. Multiply out $\mathbf{BB}^{-1}$ and match it with $\mathbf{I}$ entry by entry:

$$p\alpha + q\gamma = 1, \quad r\alpha + s\gamma = 0, \quad p\beta + q\delta = 0, \quad r\beta + s\delta = 1.$$

Step 2. Solve the first pair for α, γ. Multiply the first equation by s and the second by q, then subtract: $(ps - qr)\alpha = s$, so $\alpha = s/(ps - qr)$. Substituting back gives $\gamma = -r/(ps - qr)$.

Step 3. Solve the second pair in the same way: $\beta = -q/(ps - qr)$ and $\delta = p/(ps - qr)$.

Step 4. Define the *determinant* $\det \mathbf{B} = ps - qr$. Then

$$\mathbf{B}^{-1} = \frac{1}{ps - qr}\begin{pmatrix} s & -q \\ -r & p \end{pmatrix}, \qquad \text{valid when } ps - qr \neq 0. \tag{2.7}$$

Memory aid: *swap the diagonal, negate the off diagonal, divide by the determinant.*

2.4 Systems of Linear Equations and Gaussian Elimination

A system of m linear equations in n unknowns is written $\mathbf{Ax} = \mathbf{b}$. Gaussian elimination solves it using three **elementary row operations**, none of which changes the set of solutions:

1. swap two equations;
2. multiply an equation by a nonzero constant;
3. add a multiple of one equation to another equation.

Derivation   Why row operations preserve solutions

Step 1. Swapping two equations only changes the order in which the conditions are listed, not the conditions themselves.

Step 2. If $\mathbf{a}_i^\top \mathbf{x} = b_i$, then $k\mathbf{a}_i^\top \mathbf{x} = kb_i$. Conversely, if $k \neq 0$ we may divide by k to return. So scaling by a nonzero constant is reversible.

Step 3. If $\mathbf{a}_i^\top \mathbf{x} = b_i$ and $\mathbf{a}_k^\top \mathbf{x} = b_k$ both hold, then $(\mathbf{a}_k + \lambda\mathbf{a}_i)^\top \mathbf{x} = b_k + \lambda b_i$ also holds. Conversely, subtracting λ times equation i from the new equation returns equation k. So this operation is reversible too.

Every operation is reversible, so the old and new systems have exactly the same solutions.

Lucid Insight Pivoting is Gaussian elimination

A *pivot* chooses one nonzero entry, divides its row so that the entry becomes 1, then subtracts multiples of that row from all other rows so that every other entry in its column becomes 0. The column is then a unit column. Every iteration of the simplex method (Module 8) performs exactly one such pivot.

2.5 Linear Independence, Span and Rank

Definition Linear Combination, Span and Linear Independence

Given vectors $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$:

- a **linear combination** is any vector $\lambda_1 \mathbf{v}_1 + \cdots + \lambda_k \mathbf{v}_k$ with real weights $\lambda_1, \dots, \lambda_k$;
- the **span** is the set of all linear combinations;
- the vectors are **linearly independent** if the only weights solving $\lambda_1 \mathbf{v}_1 + \cdots + \lambda_k \mathbf{v}_k = \mathbf{0}$ are $\lambda_1 = \cdots = \lambda_k = 0$. Otherwise they are **linearly dependent**, which means at least one vector can be written as a combination of the others.

Definition Rank

The **rank** of a matrix $\mathbf{A}$, written $\operatorname{rank} \mathbf{A}$, is the largest number of linearly independent columns of $\mathbf{A}$ (equivalently, of rows). It equals the number of nonzero rows remaining after Gaussian elimination brings $\mathbf{A}$ to row echelon form.

Key Result Solvability of square systems

For a square matrix $\mathbf{B} \in \mathbb{R}^{m \times m}$ the following statements are equivalent: (i) $\mathbf{B}$ is invertible; (ii) $\det \mathbf{B} \neq 0$; (iii) the columns of $\mathbf{B}$ are linearly independent; (iv) $\operatorname{rank} \mathbf{B} = m$; (v) $\mathbf{B}\mathbf{x} = \mathbf{b}$ has exactly one solution for every $\mathbf{b}$. In the simplex method a *basis* is precisely a set of m columns of $\mathbf{A}$ forming such an invertible matrix $\mathbf{B}$.

2.6 Hyperplanes and Half Spaces

Definition Hyperplane and Half Space

For a nonzero vector $\mathbf{a} \in \mathbb{R}^n$ and a scalar $b \in \mathbb{R}$:

$$H = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^\top \mathbf{x} = b\} \quad \text{is a } \textbf{hyperplane}, \qquad H^- = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^\top \mathbf{x} \le b\} \quad \text{is a } \textbf{closed half space}. \tag{2.8}$$

where

$\mathbf{a}$ is the *normal vector*; it is perpendicular to the hyperplane and points towards the side where $\mathbf{a}^\top \mathbf{x}$ increases;

b is the *offset*, which shifts the hyperplane parallel to itself.

In $\mathbb{R}^2$ a hyperplane is a straight line; in $\mathbb{R}^3$ it is a plane.

Derivation   The normal vector is perpendicular to the hyperplane

Step 1. Take any two points $\mathbf{x}, \mathbf{y}$ on H. Then $\mathbf{a}^\top\mathbf{x} = b$ and $\mathbf{a}^\top\mathbf{y} = b$.

Step 2. Subtract the equations: $\mathbf{a}^\top(\mathbf{x} - \mathbf{y}) = b - b = 0$.

Step 3. The vector $\mathbf{x} - \mathbf{y}$ points along the hyperplane, and by (2.4) a zero inner product means a right angle. So $\mathbf{a}$ is perpendicular to every direction lying in H.

Derivation   Distance from a point to a hyperplane

Let $\mathbf{x}_0$ be any point and $H = \{\mathbf{x} : \mathbf{a}^\top\mathbf{x} = b\}$.

Step 1. The shortest route from $\mathbf{x}_0$ to H runs along the normal direction $\mathbf{a}$. So the closest point has the form $\mathbf{x}_0 - t\mathbf{a}$ for some scalar t.

Step 2. Require this point to lie on H: $\mathbf{a}^\top(\mathbf{x}_0 - t\mathbf{a}) = b$, that is $\mathbf{a}^\top\mathbf{x}_0 - t\,\mathbf{a}^\top\mathbf{a} = b$.

Step 3. Solve for t: $t = (\mathbf{a}^\top\mathbf{x}_0 - b)/\,\|\mathbf{a}\|_2^2$.

Step 4. The distance is the length of the step, $\|t\mathbf{a}\|_2 = |t|\;\|\mathbf{a}\|_2$. Substituting t,

$$\operatorname{dist}(\mathbf{x}_0, H) = \frac{\left|\mathbf{a}^\top\mathbf{x}_0 - b\right|}{\|\mathbf{a}\|_2}. \tag{2.9}$$

where

$\mathbf{x}_0$ is the point whose distance we measure;

$\mathbf{a}^\top\mathbf{x}_0 - b$ is the *signed residual*, positive on the side to which $\mathbf{a}$ points.

Machine Learning Connection

A linear classifier predicts the label $\operatorname{sign}(\mathbf{w}^\top\mathbf{x} + w_0)$ for a feature vector $\mathbf{x}$, where $\mathbf{w}$ is a weight vector and w_0 a bias. Its decision boundary is the hyperplane $\mathbf{w}^\top\mathbf{x} + w_0 = 0$, and formula (2.9) with $\mathbf{a} = \mathbf{w}$, $b = -w_0$ gives the distance of each data point from the boundary. Maximising the smallest such distance is the idea behind the support vector machine (Module 21). The column view (2.6) is also how a linear model makes predictions for a whole data set at once: the prediction vector $\mathbf{X}\mathbf{w}$ is a weighted sum of the feature columns of the data matrix $\mathbf{X}$.

2.7 Worked Examples

Worked Example 2.1   Computing a Matrix Vector Product Two Ways

Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 0 \\ 3 & -1 & 4 \end{pmatrix}$ and $\mathbf{x} = (2,\ 1,\ 3)^\top$. Compute $\mathbf{A}\mathbf{x}$ using both the row view and the column view, and interpret the result as resource usage if the rows are two resources and the columns are three activities.

Solution

Step 1. Check sizes. $\mathbf{A}$ is 2×3 and $\mathbf{x}$ has 3 entries, so $\mathbf{Ax}$ exists and has 2 entries.

Step 2. Row view. Row 1 is $(1, 2, 0)$: $1(2) + 2(1) + 0(3) = 2 + 2 + 0 = 4$. Row 2 is $(3, -1, 4)$: $3(2) + (-1)(1) + 4(3) = 6 - 1 + 12 = 17$.

Step 3. Column view. Using (2.6),

$$\mathbf{Ax} = 2\begin{pmatrix}1\\3\end{pmatrix} + 1\begin{pmatrix}2\\-1\end{pmatrix} + 3\begin{pmatrix}0\\4\end{pmatrix} = \begin{pmatrix}2\\6\end{pmatrix} + \begin{pmatrix}2\\-1\end{pmatrix} + \begin{pmatrix}0\\12\end{pmatrix} = \begin{pmatrix}4\\17\end{pmatrix}.$$

Step 4. Interpretation. Running activity 1 at level 2, activity 2 at level 1 and activity 3 at level 3 consumes 4 units of resource 1 and 17 units of resource 2. Each column of $\mathbf{A}$ is the "recipe" of one activity.

Answer. $\boxed{\mathbf{Ax} = (4,\ 17)^\top}$ by both methods.

Worked Example 2.2 Solving a 3 × 3 System by Gaussian Elimination

Solve

$$\begin{aligned} x_1 + x_2 + x_3 &= 6, \\ 2x_1 - x_2 + x_3 &= 3, \\ x_1 + 2x_2 - x_3 &= 2. \end{aligned}$$

Solution

We work on the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$, writing R_i for row i.

Step 1. Starting matrix.

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & -1 & 1 & 3 \\ 1 & 2 & -1 & 2 \end{array}\right)$$

Step 2. Eliminate x_1 from rows 2 and 3. $R_2 \leftarrow R_2 - 2R_1$ and $R_3 \leftarrow R_3 - R_1$:

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & -3 & -1 & -9 \\ 0 & 1 & -2 & -4 \end{array}\right)$$

Check row 2: $2 - 2(1) = 0$, $-1 - 2(1) = -3$, $1 - 2(1) = -1$, $3 - 2(6) = -9$.

Step 3. Swap rows 2 and 3 so that the pivot is 1 rather than −3, which keeps the arithmetic simple:

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -2 & -4 \\ 0 & -3 & -1 & -9 \end{array}\right)$$

Step 4. Eliminate x_2 from row 3. $R_3 \leftarrow R_3 + 3R_2$: entries 0, $-3+3 = 0$, $-1-6 = -7$, $-9-12 = -21$:

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -2 & -4 \\ 0 & 0 & -7 & -21 \end{array}\right)$$

Step 5. **Back substitution.** Row 3: $-7x_3 = -21$, so $x_3 = 3$. Row 2: $x_2 - 2(3) = -4$, so $x_2 = 2$. Row 1: $x_1 + 2 + 3 = 6$, so $x_1 = 1$.

Step 6. **Verify** in the original equations: $1 + 2 + 3 = 6$; $2 - 2 + 3 = 3$; $1 + 4 - 3 = 2$. All hold.

Answer. $\boxed{x_1 = 1,\ x_2 = 2,\ x_3 = 3}$.

Worked Example 2.3 Testing Linear Independence and Finding the Rank

Decide whether $\mathbf{v}_1 = (1, 0, 2)^\top$, $\mathbf{v}_2 = (0, 1, 1)^\top$ and $\mathbf{v}_3 = (1, 1, 3)^\top$ are linearly independent, and find the rank of the matrix with these vectors as columns.

Solution

Step 1. **Set up the test.** We look for weights with $\lambda_1\mathbf{v}_1 + \lambda_2\mathbf{v}_2 + \lambda_3\mathbf{v}_3 = \mathbf{0}$. Component by component:

$$\lambda_1 + \lambda_3 = 0, \qquad \lambda_2 + \lambda_3 = 0, \qquad 2\lambda_1 + \lambda_2 + 3\lambda_3 = 0.$$

Step 2. **Solve.** From the first two equations, $\lambda_1 = -\lambda_3$ and $\lambda_2 = -\lambda_3$. Substitute into the third: $-2\lambda_3 - \lambda_3 + 3\lambda_3 = 0$, which is $0 = 0$, true for *every* λ_3.

Step 3. **Conclusion.** Taking $\lambda_3 = -1$ gives $\lambda_1 = 1$, $\lambda_2 = 1$, so $\mathbf{v}_1 + \mathbf{v}_2 - \mathbf{v}_3 = \mathbf{0}$, that is $\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$. Nonzero weights exist, so the vectors are *dependent*.

Step 4. **Rank by elimination.** Form $\mathbf{A} = [\mathbf{v}_1\ \mathbf{v}_2\ \mathbf{v}_3]$ and eliminate:

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 3 \end{pmatrix} \xrightarrow{R_3 - 2R_1} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix} \xrightarrow{R_3 - R_2} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

Two nonzero rows remain, so $\operatorname{rank}\mathbf{A} = 2$.

Step 5. **Consequence for LP.** The three columns cannot together form a basis, because $\mathbf{A}$ is not invertible. Any two of them, for example $\mathbf{v}_1, \mathbf{v}_2$, can.

Answer. The vectors are linearly dependent ($\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$) and $\boxed{\operatorname{rank}\mathbf{A} = 2}$.

Worked Example 2.4 Using the 2×2 Inverse to Find a Corner Point

In Worked Example 1.1 both constraints $4x_1 + 2x_2 \leq 40$ and $2x_1 + 3x_2 \leq 30$ were tight at the best plan. Find that plan by inverting the matrix of the two tight constraints.

Solution

Step 1. **Tight constraints as a system.** $\mathbf{Bx} = \mathbf{b}$ with $\mathbf{B} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$ and $\mathbf{b} = (40,\ 30)^\top$.

Step 2. **Determinant.** With $p = 4$, $q = 2$, $r = 2$, $s = 3$: $\det\mathbf{B} = ps - qr = 12 - 4 = 8 \neq 0$, so $\mathbf{B}$ is invertible.

Step 3. Inverse by (2.7)**.**

$$\mathbf{B}^{-1} = \frac{1}{8}\begin{pmatrix} 3 & -2 \\ -2 & 4 \end{pmatrix}.$$

Step 4. Check. $\mathbf{B}\mathbf{B}^{-1} = \frac{1}{8}\begin{pmatrix} 12-4 & -8+8 \\ 6-6 & -4+12 \end{pmatrix} = \frac{1}{8}\begin{pmatrix} 8 & 0 \\ 0 & 8 \end{pmatrix} = \mathbf{I}.$

Step 5. Solve.

$$\mathbf{x} = \mathbf{B}^{-1}\mathbf{b} = \frac{1}{8}\begin{pmatrix} 3(40)-2(30) \\ -2(40)+4(30) \end{pmatrix} = \frac{1}{8}\begin{pmatrix} 60 \\ 40 \end{pmatrix} = \begin{pmatrix} 7.5 \\ 5 \end{pmatrix}.$$

Answer. $\boxed{(x_1, x_2) = (7.5,\ 5)}$, the corner where timber and labour are both fully used.

Worked Example 2.5 Describing a Half Space

For the constraint $2x_1 + 3x_2 \leq 12$: (a) find the normal vector and the intercepts of the boundary line; (b) decide which of the points $(0,0)$, $(3,3)$ and $(6,0)$ lie in the half space, on its boundary, or outside; (c) state the direction in which the left hand side grows fastest.

Solution

Step 1. Normal vector. Writing the constraint as $\mathbf{a}^\top\mathbf{x} \leq b$ gives $\mathbf{a} = (2,3)^\top$ and $b = 12$.

Step 2. Intercepts. On the boundary $2x_1 + 3x_2 = 12$: setting $x_2 = 0$ gives $x_1 = 6$; setting $x_1 = 0$ gives $x_2 = 4$. The line passes through $(6,0)$ and $(0,4)$.

Step 3. Test the points. Evaluate $2x_1 + 3x_2$: at $(0,0)$ the value is $0 < 12$ (inside); at $(3,3)$ it is $6 + 9 = 15 > 12$ (outside); at $(6,0)$ it is $12 = 12$ (on the boundary).

Step 4. Fastest growth. The gradient of $2x_1 + 3x_2$ is the constant vector $\mathbf{a} = (2,3)^\top$. Moving one unit of length along $\mathbf{a}/\|\mathbf{a}\|_2 = (2,3)/\sqrt{13}$ raises the left hand side by $\|\mathbf{a}\|_2 = \sqrt{13} \approx 3.606$, which is the largest possible rise per unit length by (2.4) with $\cos\theta = 1$.

Answer. Normal $\boxed{\mathbf{a} = (2,3)^\top}$; intercepts $(6,0)$ and $(0,4)$; $(0,0)$ inside, $(6,0)$ on the boundary, $(3,3)$ outside.

Worked Example 2.6 Distance to a Hyperplane and the Margin of a Classifier

(a) Find the distance from the point $\mathbf{x}_0 = (4,5)^\top$ to the line $3x_1 + 4x_2 = 12$. (b) A linear classifier has weight vector $\mathbf{w} = (3,4)^\top$ and bias $w_0 = -12$, and the two "margin lines" are $\mathbf{w}^\top\mathbf{x} + w_0 = 1$ and $\mathbf{w}^\top\mathbf{x} + w_0 = -1$. Find the distance between these two lines.

Solution

Step 1. Part (a), identify a and b. $\mathbf{a} = (3,4)^\top$, $b = 12$, so $\|\mathbf{a}\|_2 = \sqrt{9+16} = 5$.

Step 2. Signed residual. $\mathbf{a}^\top\mathbf{x}_0 - b = 3(4) + 4(5) - 12 = 12 + 20 - 12 = 20$.

Step 3. Distance by (2.9)**.** $\text{dist} = |20|/5 = 4$.

Step 4. **Part (b), rewrite the margin lines.** $\mathbf{w}^\top\mathbf{x} + w_0 = 1$ means $3x_1 + 4x_2 = 13$; $\mathbf{w}^\top\mathbf{x} + w_0 = -1$ means $3x_1 + 4x_2 = 11$.

Step 5. **Distance between parallel lines.** Pick any point on the second line, for example $\mathbf{x}_1 = (11/3,\ 0)$, and use (2.9) with $b = 13$: $|3(11/3) + 0 - 13|\,/5 = |-2|\,/5 = 0.4$.

Step 6. **General formula.** The same argument with b values $1 - w_0$ and $-1 - w_0$ gives a separation of $|(1 - w_0) - (-1 - w_0)|\,/\,\|\mathbf{w}\|_2 = 2/\,\|\mathbf{w}\|_2$. Here $2/5 = 0.4$, in agreement.

Answer. (a) $\boxed{4}$; (b) the margin width is $\boxed{2/\,\|\mathbf{w}\|_2 = 0.4}$. Smaller weight vectors give wider margins, the key idea of Module 21.

Module Summary

- The inner product $\mathbf{x}^\top\mathbf{y} = \sum_j x_j y_j = \|\mathbf{x}\|_2\,\|\mathbf{y}\|_2 \cos\theta$ measures alignment.
- $\mathbf{Ax}$ can be read row by row (one constraint per row) or column by column (a weighted sum of activity recipes).
- Gaussian elimination uses three reversible row operations; a simplex pivot is one elimination step.
- A square matrix is invertible exactly when its columns are independent and its determinant is nonzero; for 2×2 matrices, $\mathbf{B}^{-1} = \frac{1}{ps-qr}\begin{pmatrix} s & -q \\ -r & p \end{pmatrix}$.
- Each linear inequality defines a half space with normal vector $\mathbf{a}$; feasible regions are intersections of half spaces.
- The distance from $\mathbf{x}_0$ to $\mathbf{a}^\top\mathbf{x} = b$ is $\left|\mathbf{a}^\top\mathbf{x}_0 - b\right|\,/\,\|\mathbf{a}\|_2$, which yields the margin $2/\,\|\mathbf{w}\|_2$ of a linear classifier.

Module 3

Convex Sets, Polyhedra and Extreme Points

Learning Objectives

After studying this module you will be able to:

- define convex combinations and convex sets, and prove that a given set is or is not convex;
- prove that every feasible region of a linear program is a convex set called a polyhedron;
- define extreme points (vertices) geometrically and characterise them algebraically through active constraints;
- explain why a linear objective always attains its optimum at a vertex when an optimum exists;
- recognise unbounded polyhedra and use recession directions to decide whether an objective is bounded;
- explain why convexity guarantees that a local optimum of a linear program is a global optimum.

3.1 Motivation: The Shape of the Feasible Region

In Module 1 the best plan for the furniture workshop sat at a “corner” where two constraints met. This was not a coincidence. The feasible region of every linear program has a very particular shape: it is a convex polyhedron, a flat sided region with no dents or holes. Linear objectives over such regions are always optimised at corners. This geometric fact is the foundation of the simplex method, which jumps from corner to corner, and it is why linear programs can be solved reliably at enormous scale.

3.2 Convex Combinations and Convex Sets

Definition Line Segment and Convex Combination

For two points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the **line segment** joining them is the set

$$[\mathbf{x}, \mathbf{y}] = \{\lambda\mathbf{x} + (1 - \lambda)\mathbf{y} : 0 \leq \lambda \leq 1\}. \tag{3.1}$$

More generally, a **convex combination** of the points $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is

$$\lambda_1\mathbf{v}_1 + \lambda_2\mathbf{v}_2 + \cdots + \lambda_k\mathbf{v}_k, \qquad \lambda_i \geq 0, \qquad \sum_{i=1}^{k} \lambda_i = 1. \tag{3.2}$$

where

λ, λ_i are *weights*; they are nonnegative and add up to one, so they behave like probabilities;

$\mathbf{v}_i$ are the points being combined.

The **convex hull** of $\mathbf{v}_1, \dots, \mathbf{v}_k$ is the set of all their convex combinations.

Lucid Insight Reading the segment formula

When $\lambda = 1$ the point $\lambda\mathbf{x} + (1-\lambda)\mathbf{y}$ equals $\mathbf{x}$; when $\lambda = 0$ it equals $\mathbf{y}$; when $\lambda = \frac{1}{2}$ it is the midpoint. As λ slides from 1 down to 0, the point travels in a straight line from $\mathbf{x}$ to $\mathbf{y}$. A convex combination of several points is a "weighted average" that always lies inside the polygon spanned by them.

Definition Convex Set

A set $S \subseteq \mathbb{R}^n$ is **convex** if for every pair of points $\mathbf{x}, \mathbf{y} \in S$ and every $\lambda \in [0, 1]$, the point $\lambda\mathbf{x} + (1-\lambda)\mathbf{y}$ also belongs to S. In words: *the straight segment between any two points of the set stays entirely inside the set.*

A disc, a square, a triangle, a straight line and the whole space $\mathbb{R}^n$ are convex. A crescent moon, a ring and a set made of two separate pieces are not.

Derivation Every half space is convex

Let $H^- = \{\mathbf{x} : \mathbf{a}^\top\mathbf{x} \le b\}$ and take any $\mathbf{x}, \mathbf{y} \in H^-$ and any $\lambda \in [0, 1]$.

Step 1. By assumption, $\mathbf{a}^\top\mathbf{x} \le b$ and $\mathbf{a}^\top\mathbf{y} \le b$.

Step 2. Evaluate the left hand side at the combined point and use linearity (Module 1, additivity and homogeneity):

$$\mathbf{a}^\top(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) = \lambda\,\mathbf{a}^\top\mathbf{x} + (1-\lambda)\,\mathbf{a}^\top\mathbf{y}.$$

Step 3. Because $\lambda \ge 0$ and $1-\lambda \ge 0$, multiplying the two inequalities of Step 1 by these numbers keeps their direction:

$$\lambda\,\mathbf{a}^\top\mathbf{x} \le \lambda b, \qquad (1-\lambda)\,\mathbf{a}^\top\mathbf{y} \le (1-\lambda)b.$$

Step 4. Add the two inequalities:

$$\lambda\,\mathbf{a}^\top\mathbf{x} + (1-\lambda)\,\mathbf{a}^\top\mathbf{y} \le \lambda b + (1-\lambda)b = b.$$

Step 5. Combining Steps 2 and 4, $\mathbf{a}^\top(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \le b$, so the combined point is in H^-.

The same argument with $=$ in place of $\le$ shows that every hyperplane is convex.

Derivation The intersection of convex sets is convex

Let $S_1, \dots, S_m$ be convex and let $S = S_1 \cap \dots \cap S_m$.

Step 1. Take $\mathbf{x}, \mathbf{y} \in S$ and $\lambda \in [0, 1]$. Being in the intersection means $\mathbf{x}$ and $\mathbf{y}$ belong to every S_i.

Step 2. Each S_i is convex, so $\lambda\mathbf{x} + (1-\lambda)\mathbf{y} \in S_i$ for every i.

Step 3. A point that lies in every S_i lies in their intersection, so $\lambda\mathbf{x} + (1-\lambda)\mathbf{y} \in S$.

3.3 Polyhedra

Definition Polyhedron and Polytope

A **polyhedron** is a set of the form

$$P = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} \le \mathbf{b}\}, \tag{3.3}$$

that is, the intersection of finitely many half spaces $\mathbf{a}_i^\top \mathbf{x} \le b_i$, $i = 1, \ldots, m$. A bounded polyhedron is called a **polytope**.

where

$\mathbf{A} \in \mathbb{R}^{m\times n}$ has rows $\mathbf{a}_1^\top, \ldots, \mathbf{a}_m^\top$;

$\mathbf{b} \in \mathbb{R}^m$ collects the right hand sides $b_1, \ldots, b_m$.

Nonnegativity $x_j \ge 0$ is included by writing it as $-x_j \le 0$.

Key Result The feasible region of every LP is convex

Every feasible region of a linear program is a polyhedron, being an intersection of finitely many half spaces and hyperplanes. By the two derivations above, every polyhedron is convex.

3.4 Extreme Points and Vertices

Definition Extreme Point

A point $\mathbf{x} \in P$ is an **extreme point** of a convex set P if it cannot be written as $\lambda\mathbf{y} + (1-\lambda)\mathbf{z}$ with $\mathbf{y}, \mathbf{z} \in P$, $\mathbf{y} \ne \mathbf{z}$ and $0 < \lambda < 1$. In words: *an extreme point is not in the middle of any segment lying in P*. Extreme points of polyhedra are also called **vertices** or **corners**.

The geometric definition is intuitive but hard to check with a computer. The following algebraic test is what algorithms actually use.

Definition Active Constraint

A constraint $\mathbf{a}_i^\top \mathbf{x} \le b_i$ is **active** (or **tight**, or **binding**) at a point $\mathbf{x}^0$ if it holds with equality, $\mathbf{a}_i^\top \mathbf{x}^0 = b_i$. The **active set** is $I(\mathbf{x}^0) = \{i : \mathbf{a}_i^\top \mathbf{x}^0 = b_i\}$.

Key Result Algebraic characterisation of vertices

Let $P = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} \le \mathbf{b}\}$ and $\mathbf{x}^0 \in P$. Then $\mathbf{x}^0$ is a vertex of P if and only if among the constraints active at $\mathbf{x}^0$ there are n whose normal vectors $\mathbf{a}_i$ are linearly independent.

Derivation Why n independent active constraints pin down a vertex

Step 1. **If there are n independent active constraints, then $\mathbf{x}^0$ is a vertex.** Suppose $\mathbf{x}^0 = \lambda\mathbf{y}+(1-\lambda)\mathbf{z}$ with $\mathbf{y}, \mathbf{z} \in P$ and $0 < \lambda < 1$. For each active i: $b_i = \mathbf{a}_i^\top\mathbf{x}^0 = \lambda\mathbf{a}_i^\top\mathbf{y} + (1-\lambda)\mathbf{a}_i^\top\mathbf{z}$.

Step 2. Since $\mathbf{a}_i^\top\mathbf{y} \le b_i$ and $\mathbf{a}_i^\top\mathbf{z} \le b_i$, the weighted average equals b_i only if both terms equal b_i (if either were smaller, the average would be smaller than b_i, because both weights are strictly positive).

Step 3. Therefore $\mathbf{y}$ and $\mathbf{z}$ both satisfy the n independent equations $\mathbf{a}_i^\top\mathbf{x} = b_i$. A square system with independent rows has exactly one solution (Module 2), so $\mathbf{y} = \mathbf{z} = \mathbf{x}^0$. Hence $\mathbf{x}^0$ is not strictly inside any segment: it is a vertex.

Step 4. **If there are fewer than n independent active constraints, then $\mathbf{x}^0$ is not a vertex.** Then the active normals span a space of dimension less than n, so there is a nonzero direction $\mathbf{d}$ with $\mathbf{a}_i^\top\mathbf{d} = 0$ for all active i. For a small number $\varepsilon > 0$, the points $\mathbf{x}^0 \pm \varepsilon\mathbf{d}$ keep every active constraint at equality and, because inactive constraints have slack, keep them satisfied as well. So $\mathbf{x}^0$ is the midpoint of the segment between $\mathbf{x}^0 + \varepsilon\mathbf{d}$ and $\mathbf{x}^0 - \varepsilon\mathbf{d}$, both in P: not a vertex.

3.5 Why Optima Occur at Vertices

Derivation A linear objective over a polytope is maximised at a vertex

Let P be a polytope with vertices $\mathbf{v}_1, \ldots, \mathbf{v}_k$. A classical theorem (the Minkowski representation) states that every point of a polytope is a convex combination of its vertices. We use this fact to derive the result.

Step 1. Take any $\mathbf{x} \in P$ and write $\mathbf{x} = \sum_{i=1}^k \lambda_i\mathbf{v}_i$ with $\lambda_i \ge 0$ and $\sum_i \lambda_i = 1$.

Step 2. Apply the objective and use linearity:

$$\mathbf{c}^\top\mathbf{x} = \mathbf{c}^\top\Big(\sum_{i=1}^k \lambda_i\mathbf{v}_i\Big) = \sum_{i=1}^k \lambda_i\,\mathbf{c}^\top\mathbf{v}_i.$$

Step 3. Let $M = \max_i \mathbf{c}^\top\mathbf{v}_i$ be the best objective value among the vertices. Each term satisfies $\lambda_i\mathbf{c}^\top\mathbf{v}_i \le \lambda_i M$ because $\lambda_i \ge 0$.

Step 4. Sum over i:

$$\mathbf{c}^\top\mathbf{x} \le \sum_{i=1}^k \lambda_i M = M\sum_{i=1}^k \lambda_i = M.$$

Step 5. So no feasible point beats the best vertex, and the best vertex itself achieves M. Therefore

$$\max_{\mathbf{x}\in P} \mathbf{c}^\top\mathbf{x} = \max_{i=1,\ldots,k} \mathbf{c}^\top\mathbf{v}_i. \tag{3.4}$$

Lucid Insight A weighted average never beats its best ingredient

The heart of the derivation is Step 4: the objective at any feasible point is a weighted average of the objective values at the vertices, and an average can never exceed its largest ingredient. This is why searching the finitely many vertices is enough.

Derivation For linear programs, local optima are global optima

Suppose $\mathbf{x}^0 \in P$ is a *local* maximiser: no feasible point within some small distance $r > 0$ of $\mathbf{x}^0$ has a larger objective.

Step 1. Suppose, for contradiction, some $\mathbf{y} \in P$ has $\mathbf{c}^\top \mathbf{y} > \mathbf{c}^\top \mathbf{x}^0$.

Step 2. For small $\lambda > 0$ the point $\mathbf{x}_\lambda = (1-\lambda)\mathbf{x}^0 + \lambda \mathbf{y}$ lies in P (convexity) and lies within distance r of $\mathbf{x}^0$, because its distance is $\lambda \left\|\mathbf{y} - \mathbf{x}^0\right\|_2$.

Step 3. By linearity, $\mathbf{c}^\top \mathbf{x}_\lambda = \mathbf{c}^\top \mathbf{x}^0 + \lambda(\mathbf{c}^\top \mathbf{y} - \mathbf{c}^\top \mathbf{x}^0) > \mathbf{c}^\top \mathbf{x}^0$.

Step 4. This contradicts local optimality, so no such $\mathbf{y}$ exists: $\mathbf{x}^0$ is a global maximiser.

3.6 Unbounded Polyhedra and Recession Directions

Definition Recession Direction

A nonzero vector $\mathbf{d}$ is a **recession direction** of a nonempty polyhedron $P = \{\mathbf{x} : \mathbf{A}\mathbf{x} \le \mathbf{b}\}$ if $\mathbf{x} + t\mathbf{d} \in P$ for every $\mathbf{x} \in P$ and every $t \ge 0$. This holds exactly when $\mathbf{A}\mathbf{d} \le \mathbf{0}$.

Derivation Recession test and unboundedness

Step 1. For $\mathbf{x} \in P$ and $t \ge 0$: $\mathbf{A}(\mathbf{x} + t\mathbf{d}) = \mathbf{A}\mathbf{x} + t\mathbf{A}\mathbf{d}$. If $\mathbf{A}\mathbf{d} \le \mathbf{0}$ then $\mathbf{A}\mathbf{x} + t\mathbf{A}\mathbf{d} \le \mathbf{A}\mathbf{x} \le \mathbf{b}$, so the ray stays in P.

Step 2. Along the ray the objective is $\mathbf{c}^\top(\mathbf{x} + t\mathbf{d}) = \mathbf{c}^\top \mathbf{x} + t\,\mathbf{c}^\top \mathbf{d}$.

Step 3. If $\mathbf{c}^\top \mathbf{d} > 0$, this grows without limit as $t \to \infty$, so a maximisation problem is *unbounded*. If $\mathbf{c}^\top \mathbf{d} \le 0$ for every recession direction, moving along rays never helps.

Machine Learning Connection

Convexity is the property that makes optimisation trustworthy in machine learning. Logistic regression, support vector machines and ℓ_1 regression are convex problems, so any local optimum found by an algorithm is the global optimum, exactly as derived above. Polytopes appear directly as well: the set of probability vectors $\{\mathbf{p} \ge \mathbf{0} : \mathbf{1}^\top \mathbf{p} = 1\}$ (the *probability simplex*) is a polytope whose vertices are the one hot vectors, and the transport polytope of Module 23 contains all couplings between two distributions. Linear programming relaxations of discrete problems (Modules 17 and 24) are convex hulls of discrete solution sets.

3.7 Worked Examples

Worked Example 3.1 Writing a Point as a Convex Combination

Let $\mathbf{v}_1 = (0,0)$, $\mathbf{v}_2 = (4,0)$, $\mathbf{v}_3 = (0,4)$. Determine whether $\mathbf{x} = (1,2)$ and $\mathbf{y} = (3,2)$ belong to the convex hull of these points (the triangle they span) by finding weights.

Solution

Step 1. Set up the equations for x. We need $\lambda_1, \lambda_2, \lambda_3 \geq 0$ with $\lambda_1 + \lambda_2 + \lambda_3 = 1$ and

$$\lambda_1 \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \lambda_2 \begin{pmatrix} 4 \\ 0 \end{pmatrix} + \lambda_3 \begin{pmatrix} 0 \\ 4 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Step 2. Solve. First component: $4\lambda_2 = 1$, so $\lambda_2 = 0.25$. Second component: $4\lambda_3 = 2$, so $\lambda_3 = 0.5$. Then $\lambda_1 = 1 - 0.25 - 0.5 = 0.25$.

Step 3. Check. All weights are nonnegative and sum to one, so $\mathbf{x}$ is in the triangle; since all weights are strictly positive, it is in the interior.

Step 4. Repeat for $\mathbf{y} = (3,2)$. First component gives $\lambda_2 = 0.75$, second gives $\lambda_3 = 0.5$, so $\lambda_1 = 1 - 0.75 - 0.5 = -0.25 < 0$.

Step 5. Conclusion for y. The only weights solving the equations include a negative one, so $\mathbf{y}$ is not a convex combination. Indeed $3 + 2 = 5 > 4$ places $\mathbf{y}$ beyond the edge $x_1 + x_2 = 4$.

Answer. $\boxed{\mathbf{x} = 0.25\mathbf{v}_1 + 0.25\mathbf{v}_2 + 0.5\mathbf{v}_3}$ lies in the triangle; $\mathbf{y}$ does not.

Worked Example 3.2 Proving or Disproving Convexity

Decide which sets are convex: (a) the ℓ_1 ball $B = \{\mathbf{x} \in \mathbb{R}^2 : |x_1| + |x_2| \leq 1\}$; (b) $S = \{\mathbf{x} \in \mathbb{R}^2 : |x_1| \geq 1\}$; (c) the solution set $L = \{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b}\}$ of a linear system.

Solution

Step 1. (a) Tool: the triangle inequality $|u + v| \leq |u| + |v|$ for real numbers u, v. Take $\mathbf{x}, \mathbf{y} \in B$ and $\lambda \in [0,1]$, and let $\mathbf{z} = \lambda\mathbf{x} + (1-\lambda)\mathbf{y}$.

Step 2. For each component j: $\left|z_j\right| = \left|\lambda x_j + (1-\lambda)y_j\right| \leq \lambda\left|x_j\right| + (1-\lambda)\left|y_j\right|$, using the triangle inequality and $\lambda, 1-\lambda \geq 0$.

Step 3. Add over $j = 1, 2$: $|z_1| + |z_2| \leq \lambda(|x_1| + |x_2|) + (1-\lambda)(|y_1| + |y_2|) \leq \lambda + (1-\lambda) = 1$. So $\mathbf{z} \in B$, and B is convex.

Step 4. (b) Counterexample. $\mathbf{x} = (1,0)$ and $\mathbf{y} = (-1,0)$ both satisfy $|x_1| \geq 1$, but their midpoint $(0,0)$ has $|0| = 0 < 1$. So S is *not* convex; it consists of two separate pieces.

Step 5. (c) Take $\mathbf{x}, \mathbf{y} \in L$. Then $\mathbf{A}(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) = \lambda\mathbf{A}\mathbf{x} + (1-\lambda)\mathbf{A}\mathbf{y} = \lambda\mathbf{b} + (1-\lambda)\mathbf{b} = \mathbf{b}$. So L is convex.

Answer. (a) convex; (b) not convex; (c) convex.

Worked Example 3.3 Finding All Vertices of a Polygon

Find all vertices of $P = \{\mathbf{x} \in \mathbb{R}^2 : x_1 + x_2 \le 4,\ x_1 + 3x_2 \le 6,\ x_1 \ge 0,\ x_2 \ge 0\}$, verify each with the active constraint test, and show that $(2, 0)$ is not a vertex.

Solution

Step 1. **Strategy.** In $\mathbb{R}^2$ a vertex needs $n = 2$ independent active constraints. With four constraints there are $\binom{4}{2} = 6$ pairs to try. For each pair, solve the two equations and keep the result only if it satisfies all constraints.

Step 2. **Pair** $x_1 = 0,\ x_2 = 0$: point $(0, 0)$. Check: $0 \le 4$, $0 \le 6$. Feasible.

Step 3. **Pair** $x_2 = 0,\ x_1 + x_2 = 4$: point $(4, 0)$. Check: $4 + 0 = 4 \le 6$. Feasible.

Step 4. **Pair** $x_2 = 0,\ x_1 + 3x_2 = 6$: point $(6, 0)$. Check: $6 + 0 = 6 > 4$. Infeasible, discard.

Step 5. **Pair** $x_1 = 0,\ x_1 + x_2 = 4$: point $(0, 4)$. Check: $0 + 12 = 12 > 6$. Infeasible, discard.

Step 6. **Pair** $x_1 = 0,\ x_1 + 3x_2 = 6$: point $(0, 2)$. Check: $0 + 2 = 2 \le 4$. Feasible.

Step 7. **Pair** $x_1 + x_2 = 4,\ x_1 + 3x_2 = 6$: subtract the first equation from the second to get $2x_2 = 2$, so $x_2 = 1$ and $x_1 = 3$. Check nonnegativity: feasible. The normals $(1, 1)$ and $(1, 3)$ have determinant $1 \cdot 3 - 1 \cdot 1 = 2 \ne 0$, so they are independent.

Step 8. **The point** $(2, 0)$**.** Only $x_2 \ge 0$ is active (since $2 < 4$, $2 < 6$, $2 > 0$). One active constraint is fewer than $n = 2$, so it is not a vertex. Directly: $(2, 0) = \frac{1}{2}(0, 0) + \frac{1}{2}(4, 0)$, the midpoint of an edge.

Answer. The vertices are $\boxed{(0, 0),\ (4, 0),\ (3, 1),\ (0, 2)}$; the point $(2, 0)$ lies in the middle of an edge.

Worked Example 3.4 Vertex Test in Three Dimensions

Let $P = \{\mathbf{x} \in \mathbb{R}^3 : x_1 + x_2 + x_3 \le 3,\ x_1 \ge 0,\ x_2 \ge 0,\ x_3 \ge 0\}$. Classify each point as a vertex or not: (a) $(1, 1, 1)$; (b) $(3, 0, 0)$; (c) $(1.5,\ 1.5,\ 0)$; (d) $(0, 0, 0)$.

Solution

Here $n = 3$, so a vertex needs three linearly independent active constraints. The normals are $(1, 1, 1)$ for the sum constraint and $-\mathbf{e}_1, -\mathbf{e}_2, -\mathbf{e}_3$ for $x_j \ge 0$ (written $-x_j \le 0$).

Step 1. **(a)** $(1, 1, 1)$**.** Sum $= 3$, active. No coordinate is zero, so no nonnegativity constraint is active. Only 1 active constraint: *not a vertex*.

Step 2. **(b)** $(3, 0, 0)$**.** Sum $= 3$ active; $x_2 = 0$ and $x_3 = 0$ active. Normals $(1, 1, 1)$, $(0, -1, 0)$, $(0, 0, -1)$. The matrix with these rows has determinant

$$\det\begin{pmatrix} 1 & 1 & 1 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{pmatrix} = 1 \cdot (-1) \cdot (-1) = 1 \ne 0$$

(the matrix is upper triangular, so the determinant is the product of its diagonal). Three independent active constraints: *vertex*.

Step 3. **(c)** $(1.5, 1.5, 0)$**.** Sum $= 3$ active and $x_3 = 0$ active: only 2. *Not a vertex*; it is the midpoint of the edge from $(3, 0, 0)$ to $(0, 3, 0)$.

Step 4. **(d)** $(0, 0, 0)$**.** The three nonnegativity constraints are active, with normals $-\mathbf{e}_1, -\mathbf{e}_2, -\mathbf{e}_3$, which are independent. *Vertex*.

Answer. (b) and (d) are vertices; (a) and (c) are not. The polytope is a tetrahedron with vertices $(0, 0, 0)$, $(3, 0, 0)$, $(0, 3, 0)$, $(0, 0, 3)$.

Worked Example 3.5 Optimising over the Vertices

Maximise $z = 3x_1 + 5x_2$ over the polygon P of Worked Example 3.3. Then verify, using the convex combination argument, that the interior point $(2, 1)$ cannot beat the best vertex.

Solution

Step 1. **Evaluate at each vertex** found in Worked Example 3.3:

Vertex	$(0, 0)$	$(4, 0)$	$(3, 1)$	$(0, 2)$
$z = 3x_1 + 5x_2$	0	12	$9 + 5 = 14$	10

Step 2. **Apply** (3.4)**.** The maximum over the polygon equals the largest vertex value, 14, at $(3, 1)$.

Step 3. **Express** $(2, 1)$ **as a convex combination of vertices.** Try weights on $(0, 0)$, $(4, 0)$, $(3, 1)$: we need $4\lambda_2 + 3\lambda_3 = 2$ and $\lambda_3 = 1$ from the second coordinate, which forces $\lambda_2 = -0.25 < 0$. Try instead $(0, 0)$, $(3, 1)$, $(0, 2)$: second coordinate $\lambda_3 + 2\lambda_4 = 1$ and first coordinate $3\lambda_3 = 2$, so $\lambda_3 = 2/3$, $\lambda_4 = 1/6$, $\lambda_1 = 1 - 2/3 - 1/6 = 1/6$. All nonnegative.

Step 4. **Objective as an average.** $z(2, 1) = \frac{1}{6}(0) + \frac{2}{3}(14) + \frac{1}{6}(10) = 0 + \frac{28}{3} + \frac{5}{3} = \frac{33}{3} = 11$. Directly, $3(2) + 5(1) = 11$, in agreement, and $11 \le 14$ as the theory predicts.

Answer. $\boxed{z^\star = 14 \text{ at } (3, 1)}$; the interior point $(2, 1)$ gives 11, a weighted average of vertex values.

Worked Example 3.6 An Unbounded Polyhedron and Its Recession Direction

Let $P = \{\mathbf{x} \in \mathbb{R}^2 : x_1 - x_2 \le 2,\ -x_1 + x_2 \le 1,\ x_1 \ge 0,\ x_2 \ge 0\}$. (a) Find all recession directions. (b) Decide whether each problem is bounded: maximise $x_1 + x_2$; maximise $x_1 - x_2$; minimise $x_1 + x_2$.

Solution

Step 1. **Recession conditions.** $\mathbf{d}$ must satisfy $\mathbf{Ad} \le \mathbf{0}$ for the four rows: $d_1 - d_2 \le 0$, $-d_1 + d_2 \le 0$, $-d_1 \le 0$, $-d_2 \le 0$.

Step 2. **Solve.** The first two give $d_1 \le d_2$ and $d_2 \le d_1$, so $d_1 = d_2$. The last two give $d_1, d_2 \ge 0$. Hence the recession directions are $\mathbf{d} = t(1, 1)$ with $t > 0$.

Step 3. **Maximise** $x_1 + x_2$**.** Here $\mathbf{c} = (1, 1)$ and $\mathbf{c}^\top \mathbf{d} = 2t > 0$. Starting at the feasible point $(0, 0)$ and moving along $(1, 1)$ the objective increases without limit: *unbounded*.

Step 4. **Maximise $x_1 - x_2$. $\mathbf{c}^\top \mathbf{d} = t - t = 0$**, so rays do not help, and the first constraint gives $x_1 - x_2 \leq 2$ directly. The value 2 is attained at $(2, 0)$, which satisfies all constraints ($-2 \leq 1$). *Bounded, optimum 2.*

Step 5. **Minimise $x_1 + x_2$.** Since $x_1, x_2 \geq 0$, the objective is at least 0, attained at $(0, 0)$. *Bounded, optimum 0.*

Answer. Recession directions $\boxed{\mathbf{d} = t(1, 1),\ t > 0}$; max $x_1 + x_2$ is unbounded; max $x_1 - x_2 = 2$; min $x_1 + x_2 = 0$.

Module Summary

- A set is convex if it contains the segment between any two of its points; half spaces, hyperplanes and their intersections are convex.
- The feasible region of every LP is a polyhedron $\{\mathbf{x} : \mathbf{A}\mathbf{x} \leq \mathbf{b}\}$, hence convex.
- A point is a vertex exactly when n linearly independent constraints are active there.
- The objective at any point of a polytope is a weighted average of vertex values, so the optimum is attained at a vertex.
- For linear (indeed convex) problems, every local optimum is global.
- A direction $\mathbf{d}$ with $\mathbf{A}\mathbf{d} \leq \mathbf{0}$ is a recession direction; if $\mathbf{c}^\top \mathbf{d} > 0$ the maximisation problem is unbounded.

Module 4

Modelling Techniques and Linearisation

Learning Objectives

After studying this module you will be able to:

- replace absolute values in objectives and constraints by linear constraints, and prove that the replacement is exact;
- turn minimax and maximin problems into linear programs using an auxiliary "epigraph" variable;
- model convex piecewise linear costs as the maximum of affine functions;
- handle free (sign unrestricted) variables by splitting them into two nonnegative parts;
- build goal programming models with deviation variables;
- convert a linear fractional objective into a linear program with the Charnes and Cooper transformation.

4.1 Why Reformulation Is a Core Skill

Real problems rarely arrive in the neat form max $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$. Errors are measured in absolute value, worst cases are measured by a maximum, prices change in tiers, and quantities may be negative. A small set of reformulation techniques converts all of these into genuine linear programs *without approximation*. These same techniques produce, in Part IV, the linear programming forms of ℓ_1 regression, Chebyshev regression, quantile regression, the hinge loss of support vector machines and the ℓ_1 penalty of sparse models. Mastering this module therefore unlocks most of the machine learning applications in the book.

4.2 Absolute Values

Derivation The absolute value as two linear inequalities

Let u be any real number (later it will be a linear expression in the variables) and t another real number.

Step 1. By definition, $|u| = u$ if $u \ge 0$ and $|u| = -u$ if $u < 0$. In both cases $|u| = \max(u, -u)$.

Step 2. A number t is at least the larger of two numbers exactly when it is at least each of them. Therefore

$$t \ge |u| \quad \Longleftrightarrow \quad t \ge u \;\text{ and }\; t \ge -u. \tag{4.1}$$

Step 3. Both inequalities on the right are linear in (t, u).

Key Result Minimising a sum of absolute values

The problem

$$\min_{\mathbf{x}} \sum_{i=1}^{m} \left|\mathbf{a}_i^\top \mathbf{x} - b_i\right| \quad \text{subject to} \quad \mathbf{x} \in P \tag{4.2}$$

is equivalent to the linear program

$$\min_{\mathbf{x},\mathbf{t}} \sum_{i=1}^{m} t_i \quad \text{subject to} \quad t_i \geq \mathbf{a}_i^\top \mathbf{x} - b_i, \quad t_i \geq -(\mathbf{a}_i^\top \mathbf{x} - b_i), \quad i = 1, \dots, m, \quad \mathbf{x} \in P, \tag{4.3}$$

where

$\mathbf{a}_i^\top \mathbf{x} - b_i$ is the ith *residual*, a linear function of $\mathbf{x}$;

t_i is a new variable that acts as an upper bound on the ith absolute residual;

P is any polyhedral set of additional constraints.

Derivation Why the reformulation is exact

Step 1. By (4.1), every feasible point of (4.3) has $t_i \geq \left|\mathbf{a}_i^\top \mathbf{x} - b_i\right|$, so its objective is at least $\sum_i \left|\mathbf{a}_i^\top \mathbf{x} - b_i\right|$.

Step 2. For a fixed $\mathbf{x}$, the smallest allowed value of each t_i is exactly $\left|\mathbf{a}_i^\top \mathbf{x} - b_i\right|$. Because we are minimising $\sum_i t_i$ and each t_i has a positive coefficient, the optimiser always pushes each t_i down to that smallest value.

Step 3. Hence at the optimum $t_i^\star = \left|\mathbf{a}_i^\top \mathbf{x}^\star - b_i\right|$, and the two problems have the same optimal $\mathbf{x}$ and the same optimal value.

Common Pitfall

The trick works only in the "convex direction". Minimising $|u|$, or requiring $|u| \leq c$, is fine. Maximising $|u|$, or requiring $|u| \geq c$, is *not*: the constraints $t \geq u$, $t \geq -u$ allow t to grow without bound, so maximising t does not measure $|u|$. A constraint $|u| \geq c$ describes a non convex set (Worked Example 3.2) and needs integer variables.

An alternative formulation splits the residual into positive and negative parts. Write $\mathbf{a}_i^\top \mathbf{x} - b_i = u_i^+ - u_i^-$ with $u_i^+, u_i^- \geq 0$, and minimise $\sum_i (u_i^+ + u_i^-)$. At the optimum at most one of u_i^+, u_i^- is positive, so $u_i^+ + u_i^- = \left|\mathbf{a}_i^\top \mathbf{x} - b_i\right|$. Both formulations are used in practice; the second is the one used for quantile regression in Module 20.

4.3 Minimax and Maximin Problems

Derivation The epigraph reformulation of a maximum

Consider $\min_{\mathbf{x}\in P}\ \max_{k=1,\ldots,K}(\mathbf{d}_k^\top\mathbf{x}+e_k)$, where each $\mathbf{d}_k^\top\mathbf{x}+e_k$ is an affine function.

Step 1. Introduce a new variable t intended to equal the maximum. As in (4.1), $t \geq \max_k(\mathbf{d}_k^\top\mathbf{x}+e_k)$ holds exactly when $t \geq \mathbf{d}_k^\top\mathbf{x}+e_k$ for every k.

Step 2. Minimise t subject to these K linear constraints and $\mathbf{x} \in P$.

Step 3. At the optimum, t is pushed down until it meets the largest of the affine functions, so $t^\star$ equals the minimax value.

The result is the linear program

$$\min_{\mathbf{x},t}\ t \quad \text{subject to} \quad \mathbf{d}_k^\top\mathbf{x}+e_k \leq t, \quad k=1,\ldots,K, \qquad \mathbf{x}\in P. \tag{4.4}$$

where

K is the number of affine pieces;

$\mathbf{d}_k, e_k$ are the slope vector and intercept of piece k;

t is the auxiliary *epigraph variable*, a free real number.

By symmetry, $\max_{\mathbf{x}} \min_k(\mathbf{d}_k^\top\mathbf{x}+e_k)$ becomes $\max\ t$ subject to $t \leq \mathbf{d}_k^\top\mathbf{x}+e_k$ for all k.

Lucid Insight The name “epigraph”

The epigraph of a function is the region on or above its graph. Requiring t to lie above every piece places the point $(\mathbf{x}, t)$ in the epigraph of the maximum. Minimising t then slides the point down until it rests on the graph.

4.4 Convex Piecewise Linear Functions

A function such as a tiered tariff, in which each extra unit costs at least as much as the previous one, is *convex piecewise linear*. Every such function can be written as the maximum of its affine pieces,

$$f(e) = \max_{k=1,\ldots,K}(\alpha_k e + \beta_k), \tag{4.5}$$

where

e is the input quantity (for example energy used);

α_k is the slope (marginal cost) of piece k, with $\alpha_1 \leq \alpha_2 \leq \cdots \leq \alpha_K$;

β_k is the intercept of piece k, chosen so that consecutive pieces meet at the breakpoints.

Once in this form, the epigraph trick (4.4) applies directly.

Derivation Finding the intercepts of a tiered tariff

Suppose the marginal cost is α_1 up to the breakpoint e_1, then α_2 up to e_2, and so on.

Step 1. The first piece passes through the origin: $\beta_1 = 0$.

Step 2. Piece $k+1$ must agree with piece k at the breakpoint e_k: $\alpha_{k+1}e_k + \beta_{k+1} = \alpha_k e_k + \beta_k$.

Step 3. Rearranging gives the recursion

$$\beta_{k+1} = \beta_k - (\alpha_{k+1} - \alpha_k)\, e_k. \tag{4.6}$$

Step 4. Because $\alpha_{k+1} \geq \alpha_k$, each new piece lies below the previous one to the left of e_k and above it to the right, so the maximum selects the correct piece on each interval.

4.5 Free Variables

Derivation Splitting a free variable

Some quantities may be negative, for example a regression coefficient or a change in inventory. Linear programming algorithms usually assume nonnegative variables, so we write a free variable x as a difference.

Step 1. Every real number can be written as $x = x^+ - x^-$ with $x^+ = \max(x, 0) \geq 0$ and $x^- = \max(-x, 0) \geq 0$.

Step 2. Replace x by $x^+ - x^-$ everywhere in the model, and add $x^+, x^- \geq 0$.

Step 3. Any feasible x gives a feasible pair, and any feasible pair gives a feasible x, so the two models have the same optimal value.

Step 4. The representation is not unique: (x^+, x^-) and $(x^+ + s,\ x^- + s)$ give the same x. This causes no harm. In the simplex method the columns of x^+ and x^- are negatives of each other, so they can never both be basic, and a basic solution automatically has $x^+x^- = 0$.

4.6 Goal Programming

When there are several targets that cannot all be met, we introduce *deviation variables* that measure how far each target is missed, and minimise a weighted sum of the undesirable deviations.

Definition Goal Constraint

A goal "$\mathbf{a}^\top\mathbf{x}$ should equal the target g" is written

$$\mathbf{a}^\top\mathbf{x} + d^- - d^+ = g, \qquad d^- \geq 0, \quad d^+ \geq 0, \tag{4.7}$$

where

d^- is the *underachievement*, the amount by which $\mathbf{a}^\top\mathbf{x}$ falls short of g;

d^+ is the *overachievement*, the amount by which $\mathbf{a}^\top\mathbf{x}$ exceeds g.

Only the undesirable deviation receives a positive weight in the objective. For a profit goal it is d^-; for a cost limit it is d^+.

4.7 Linear Fractional Programming

Derivation The Charnes and Cooper transformation

Consider maximising a ratio of affine functions over a polyhedron,

$$\max_{\mathbf{x}} \frac{\mathbf{p}^\top \mathbf{x} + \alpha}{\mathbf{q}^\top \mathbf{x} + \beta} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} \le \mathbf{b},\ \mathbf{x} \ge \mathbf{0},$$

where $\mathbf{p}, \mathbf{q} \in \mathbb{R}^n$ and $\alpha, \beta \in \mathbb{R}$ are given, and the denominator is positive on the feasible region.

Step 1. Define the scalar $\tau = 1/(\mathbf{q}^\top \mathbf{x} + \beta) > 0$ and the vector $\mathbf{y} = \tau \mathbf{x}$.

Step 2. The objective becomes $\tau(\mathbf{p}^\top \mathbf{x} + \alpha) = \mathbf{p}^\top \mathbf{y} + \alpha\tau$, which is linear in $(\mathbf{y}, \tau)$.

Step 3. The definition of τ gives $\tau(\mathbf{q}^\top \mathbf{x} + \beta) = 1$, that is $\mathbf{q}^\top \mathbf{y} + \beta\tau = 1$, a linear equation.

Step 4. Multiply $\mathbf{A}\mathbf{x} \le \mathbf{b}$ by $\tau > 0$: $\mathbf{A}\mathbf{y} \le \tau\mathbf{b}$, that is $\mathbf{A}\mathbf{y} - \mathbf{b}\tau \le \mathbf{0}$. Also $\mathbf{y} \ge \mathbf{0}$ and $\tau \ge 0$.

Step 5. The resulting linear program is

$$\max_{\mathbf{y},\tau} \ \mathbf{p}^\top \mathbf{y} + \alpha\tau \quad \text{subject to} \quad \mathbf{A}\mathbf{y} - \mathbf{b}\tau \le \mathbf{0}, \quad \mathbf{q}^\top \mathbf{y} + \beta\tau = 1, \quad \mathbf{y} \ge \mathbf{0},\ \tau \ge 0. \tag{4.8}$$

Step 6. Recover the original solution by $\mathbf{x}^\star = \mathbf{y}^\star/\tau^\star$ (valid when $\tau^\star > 0$).

Machine Learning Connection

Every technique in this module reappears in learning problems. The absolute value trick turns ℓ_1 regression and the ℓ_1 penalty $\lambda \|\mathbf{w}\|_1$ into linear programs (Modules 19 and 22). The epigraph trick gives Chebyshev regression, which minimises the worst case error (Module 20), and the minimax strategies of games and adversarial training (Module 24). The hinge loss $\max(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i)$ of a support vector machine is a convex piecewise linear function, handled with (4.4) (Module 21). Free variable splitting is how regression weights, which can be negative, enter an LP solver. Ratio objectives appear in efficiency measurement, such as data envelopment analysis, and in fairness metrics defined as rates.

4.8 Worked Examples

Worked Example 4.1 Minimising a Sum of Absolute Deviations

Find $(x_1, x_2) \ge \mathbf{0}$ with $x_1 + x_2 \le 4$ that minimises $|x_1 - 2| + |x_2 - 3|$. Write the linear program and solve it.

Solution

Step 1. **New variables.** Let $t_1 \geq |x_1 - 2|$ and $t_2 \geq |x_2 - 3|$.

Step 2. **Apply** (4.1)**.** $t_1 \geq x_1 - 2$, $t_1 \geq 2 - x_1$, $t_2 \geq x_2 - 3$, $t_2 \geq 3 - x_2$.

Step 3. **Linear program.**

$$\begin{aligned} \min \;& t_1 + t_2 \\ \text{subject to} \;& x_1 - t_1 \leq 2, \quad -x_1 - t_1 \leq -2, \quad x_2 - t_2 \leq 3, \quad -x_2 - t_2 \leq -3, \\ & x_1 + x_2 \leq 4, \quad x_1, x_2 \geq 0. \end{aligned}$$

Step 4. **Reason about the solution.** The unconstrained ideal $(2, 3)$ has $x_1 + x_2 = 5 > 4$, so it is infeasible. A lower bound: by the triangle inequality,

$$|x_1 - 2| + |x_2 - 3| \;\geq\; |(2 - x_1) + (3 - x_2)| = |5 - (x_1 + x_2)| \;\geq\; 5 - 4 = 1,$$

where the last step uses $x_1 + x_2 \leq 4$.

Step 5. **Attain the bound.** At $(x_1, x_2) = (1, 3)$: $|1 - 2| + |3 - 3| = 1 + 0 = 1$, and $1 + 3 = 4 \leq 4$. So the value 1 is achieved, with $t_1 = 1$, $t_2 = 0$.

Step 6. **Alternative optima.** Every point with $x_1 + x_2 = 4$, $x_1 \leq 2$ and $x_2 \leq 3$, such as $(2, 2)$ or $(1.5,\ 2.5)$, also gives value 1.

Answer. Optimal value $\boxed{1}$, attained for example at $(x_1, x_2) = (1, 3)$ with $t_1 = 1$, $t_2 = 0$.

Worked Example 4.2 A Minimax Load Balancing Problem

Two servers receive work. Allocating x_1 units of job type A and x_2 units of job type B produces loads $2x_1 + x_2$ on server 1 and $x_1 + 3x_2$ on server 2. At least 4 units of work in total must be scheduled ($x_1 + x_2 \geq 4$). Minimise the maximum load.

Solution

Step 1. **Epigraph variable.** Let t be the maximum load.

Step 2. **Linear program** by (4.4):

$$\min \; t \quad \text{subject to} \quad 2x_1 + x_2 \leq t, \quad x_1 + 3x_2 \leq t, \quad x_1 + x_2 \geq 4, \quad x_1, x_2 \geq 0.$$

Step 3. **Intuition.** At the optimum the two loads should be balanced; if one were strictly larger, shifting work towards the other server would reduce the maximum.

Step 4. **Balance the loads.** $2x_1 + x_2 = x_1 + 3x_2$ gives $x_1 = 2x_2$.

Step 5. **Use the minimum total work** (scheduling more work only raises loads): $x_1 + x_2 = 4$, so $2x_2 + x_2 = 4$, giving $x_2 = 4/3$ and $x_1 = 8/3$.

Step 6. **Maximum load.** $t = 2(8/3) + 4/3 = 16/3 + 4/3 = 20/3 \approx 6.667$; check server 2: $8/3 + 3(4/3) = 8/3 + 12/3 = 20/3$.

Step 7. **Compare with the extremes.** Sending all work as type A, $(4,0)$, gives loads 8 and 4 (maximum 8); all type B, $(0,4)$, gives 4 and 12 (maximum 12). Both are worse.

Step 8. **Proof of optimality.** Add 2 times the first load constraint and 1 times the second: $2(2x_1 + x_2) + (x_1 + 3x_2) \le 3t$, that is $5(x_1 + x_2) \le 3t$. With $x_1 + x_2 \ge 4$ this gives $20 \le 3t$, so $t \ge 20/3$ for every feasible plan.

Answer. $\boxed{x_1 = 8/3,\ x_2 = 4/3,\ t^\star = 20/3}$.

Worked Example 4.3 A Tiered Energy Tariff

A data centre earns £0.22 per kWh of computation it sells, and can use at most 500 kWh per day. Electricity costs £0.10 per kWh for the first 100 kWh, £0.15 per kWh for the next 200 kWh, and £0.25 per kWh beyond 300 kWh. Model the daily profit as an LP and find the best energy use.

Solution

Step 1. **Slopes and breakpoints.** $\alpha_1 = 0.10$, $\alpha_2 = 0.15$, $\alpha_3 = 0.25$; breakpoints $e_1 = 100$, $e_2 = 300$.

Step 2. **Intercepts by** (4.6). $\beta_1 = 0$; $\beta_2 = 0-(0.15-0.10)(100) = -5$; $\beta_3 = -5-(0.25-0.15)(300) = -5 - 30 = -35$.

Step 3. **Cost function.** $f(e) = \max(0.10e,\ 0.15e - 5,\ 0.25e - 35)$. Check at $e = 300$: 30, 40, 40, so $f(300) = 40 = 0.10(100) + 0.15(200)$, as expected.

Step 4. **Linear program.** Let t be the cost:

$$\max\ 0.22e - t \quad \text{subject to} \quad 0.10e \le t, \quad 0.15e - 5 \le t, \quad 0.25e - 35 \le t, \quad 0 \le e \le 500.$$

Step 5. **Marginal reasoning.** Each extra kWh earns 0.22. In the first two tiers it costs 0.10 or 0.15, which is profitable. In the third tier it costs $0.25 > 0.22$, which loses money. So use energy up to the end of the second tier: $e^\star = 300$.

Step 6. **Profit.** $0.22(300) - 40 = 66 - 40 = 26$.

Answer. $\boxed{e^\star = 300 \text{ kWh},\ \text{profit} = £26}$ per day.

Worked Example 4.4 A Model with a Free Variable

Solve max $2x_1 + x_2$ subject to $x_1 + x_2 \le 6$, $x_1 - x_2 \le 2$, $x_1 \ge 0$ and x_2 free. Show the split formulation and prove optimality algebraically.

Solution

Step 1. **Split.** Write $x_2 = x_2^+ - x_2^-$ with $x_2^+, x_2^- \ge 0$. The model becomes

$$\max\ 2x_1 + x_2^+ - x_2^- \quad \text{subject to} \quad x_1 + x_2^+ - x_2^- \le 6, \quad x_1 - x_2^+ + x_2^- \le 2, \quad x_1, x_2^+, x_2^- \ge 0.$$

Step 2. **Upper bound on** x_1**.** Adding the two original constraints: $(x_1 + x_2) + (x_1 - x_2) \le 6 + 2$, so $2x_1 \le 8$ and $x_1 \le 4$.

Step 3. Upper bound on the objective. Write $2x_1 + x_2 = (x_1 + x_2) + x_1 \le 6 + 4 = 10$.

Step 4. Attain the bound. Equality needs $x_1 + x_2 = 6$ and $x_1 = 4$, so $x_2 = 2$. Check $x_1 - x_2 = 2 \le 2$.

Step 5. Split values. $x_2 = 2$ gives $x_2^+ = 2$, $x_2^- = 0$.

Answer. $\boxed{x_1 = 4,\ x_2 = 2,\ z^\star = 10}$ (with $x_2^+ = 2$, $x_2^- = 0$).

Worked Example 4.5 Goal Programming for a Labelling Team

A team spends x_1 hours on expert labelling and x_2 hours on crowd review, with at most 25 hours in total. Output (in batches) is $3x_1 + 2x_2$ with a target of 60, and cost (in units of £10) is $4x_1 + 2x_2$ with a target of at most 60. Each batch short of the output target is penalised by 2 and each unit of cost overrun by 1. Formulate the goal program and solve it.

Solution

Step 1. Goal constraints by (4.7).

$$3x_1 + 2x_2 + d_1^- - d_1^+ = 60, \qquad 4x_1 + 2x_2 + d_2^- - d_2^+ = 60.$$

Step 2. Objective. Penalise output shortfall d_1^- and cost overrun d_2^+: minimise $2d_1^- + d_2^+$.

Step 3. Hard constraint. $x_1 + x_2 \le 25$, and all variables are nonnegative.

Step 4. Analyse the case of no shortfall. If $3x_1+2x_2 = 60$, then cost $= 4x_1+2x_2 = (3x_1+2x_2)+x_1 = 60+x_1$, so the overrun is x_1. From $x_2 = (60-3x_1)/2$ the hour limit gives $x_1+30-1.5x_1 \le 25$, so $x_1 \ge 10$. The smallest overrun is 10, at $x_1 = 10$, $x_2 = 15$, with penalty 10.

Step 5. Analyse a shortfall $s > 0$. Output $= 60 - s$ costs $2s$ in penalty. Repeating Step 4 with $60 - s$ in place of 60, cost $= 60 - s + x_1$ and the hour limit gives $x_1 \ge 10 - s$, so the overrun is at least $(10 - s) - s = 10 - 2s$. The total penalty is at least $2s + (10 - 2s) = 10$ (for $s \le 5$) or $2s > 10$ (for $s > 5$).

Step 6. Conclusion. No plan achieves a penalty below 10. The plan $x_1 = 10$, $x_2 = 15$ attains it with $d_1^- = 0$ and $d_2^+ = 10$.

Answer. Minimum penalty $\boxed{10}$, attained at $x_1 = 10$, $x_2 = 15$ (output on target, cost overrun of 10); other plans with $0 < s \le 5$ tie.

Worked Example 4.6 Maximising an Efficiency Ratio

Maximise $\dfrac{2x_1 + x_2 + 1}{x_1 + x_2 + 2}$ subject to $x_1 + 2x_2 \le 4$, $x_1 \le 3$, $x_1, x_2 \ge 0$ using the Charnes and Cooper transformation.

Solution

Step 1. Identify the data. $\mathbf{p} = (2, 1)$, $\alpha = 1$, $\mathbf{q} = (1, 1)$, $\beta = 2$. The denominator is at least $2 > 0$ on the feasible region.

Step 2. Transform by (4.8)**.** With $\mathbf{y} = \tau\mathbf{x}$:

$$\begin{aligned} \max\ & 2y_1 + y_2 + \tau \\ \text{subject to}\ & y_1 + 2y_2 - 4\tau \le 0, \quad y_1 - 3\tau \le 0, \\ & y_1 + y_2 + 2\tau = 1, \quad y_1, y_2, \tau \ge 0. \end{aligned}$$

Step 3. Guess the active constraints. Large x_1 helps the ratio, because its coefficient ratio $2/1$ exceeds that of x_2 $(1/1)$. Try $y_2 = 0$ and $y_1 = 3\tau$.

Step 4. Use the normalisation. $3\tau + 0 + 2\tau = 1$, so $\tau = 0.2$ and $y_1 = 0.6$. Check: $0.6 + 0 - 0.8 = -0.2 \le 0$.

Step 5. Objective. $2(0.6) + 0 + 0.2 = 1.4$.

Step 6. Recover x. $\mathbf{x} = \mathbf{y}/\tau = (0.6/0.2,\ 0) = (3, 0)$. Check directly: $(6+0+1)/(3+0+2) = 7/5 = 1.4$.

Step 7. Compare with other vertices. $(0, 0)$ gives $1/2$; $(0, 2)$ gives $3/4$; $(3, 0.5)$ gives $7.5/5.5 \approx 1.364$. All are smaller, confirming the optimum.

Answer. $\boxed{\mathbf{x}^\star = (3, 0),\ \text{ratio}^\star = 1.4}$.

Module Summary

- $t \ge |u|$ is equivalent to $t \ge u$ and $t \ge -u$; minimising $\sum t_i$ makes each t_i equal to the absolute value.
- $\min \max_k(\mathbf{d}_k^\top \mathbf{x} + e_k)$ becomes $\min t$ subject to $t \ge \mathbf{d}_k^\top \mathbf{x} + e_k$ for every k.
- Convex piecewise linear costs are maxima of affine pieces with intercepts $\beta_{k+1} = \beta_k - (\alpha_{k+1} - \alpha_k)e_k$.
- A free variable is split as $x = x^+ - x^-$ with $x^+, x^- \ge 0$.
- Goal programming uses deviation variables d^-, d^+ and penalises only the undesirable ones.
- The ratio $(\mathbf{p}^\top \mathbf{x} + \alpha)/(\mathbf{q}^\top \mathbf{x} + \beta)$ is maximised through the LP in $(\mathbf{y}, \tau)$ with $\mathbf{y} = \tau\mathbf{x}$, $\mathbf{q}^\top \mathbf{y} + \beta\tau = 1$.
- These tricks only work in the convex direction: minimising maxima or absolute values, not maximising them.

Module 5

The Graphical Method

Learning Objectives

After studying this module you will be able to:

- draw the feasible region of a two variable linear program accurately;
- draw objective contour lines and explain why they are parallel;
- locate the optimal vertex by sliding the objective line, and by enumerating corner points;
- certify that a vertex is optimal by writing **c** as a nonnegative combination of the active constraint normals, an idea that previews duality;
- recognise graphically the special cases of multiple optima, unboundedness, infeasibility and redundant constraints;
- solve a small machine learning deployment problem graphically.

5.1 Why Learn a Method That Only Works in Two Dimensions?

Real linear programs have thousands or millions of variables, so the graphical method is never used to solve them. It is nevertheless the best possible teacher. Every idea in the simplex method, in duality and in sensitivity analysis has a picture in two dimensions: corners, edges, improving directions, active constraints, certificates of optimality and the ways a problem can fail. When an algebraic statement later feels abstract, return to the picture.

5.2 Step 1: Drawing the Feasible Region

Derivation Drawing one constraint

Consider a constraint $a_1x_1 + a_2x_2 \le b$ with $(a_1, a_2) \ne (0, 0)$.

Step 1. Draw the boundary line $a_1x_1 + a_2x_2 = b$. When $a_1 \ne 0$ and $a_2 \ne 0$ the intercepts are found by setting one variable to zero:

$$x_1 \text{ intercept} = \frac{b}{a_1}, \qquad x_2 \text{ intercept} = \frac{b}{a_2}. \tag{5.1}$$

If $a_2 = 0$ the line is vertical ($x_1 = b/a_1$); if $a_1 = 0$ it is horizontal ($x_2 = b/a_2$).

Step 2. Choose the side. Substitute a test point not on the line, usually the origin $(0, 0)$. If $0 \le b$ is true, the origin's side is feasible; otherwise the other side is.

Step 3. **Shade** the feasible side. The feasible region is the overlap of all shaded half planes and the first quadrant $x_1, x_2 \geq 0$.

5.3 Step 2: Contour Lines of the Objective

Definition Objective Contour (Iso Profit or Iso Cost Line)

For a constant k, the **contour** of the objective $z = c_1x_1 + c_2x_2$ at level k is the line

$$c_1x_1 + c_2x_2 = k. \tag{5.2}$$

where

c_1, c_2 are the objective coefficients;

k is the objective value shared by every point on the line.

Derivation All contours are parallel, and c points uphill

Step 1. Solve (5.2) for x_2 (assume $c_2 \neq 0$): $x_2 = \dfrac{k}{c_2} - \dfrac{c_1}{c_2}x_1$.

Step 2. The slope is $-c_1/c_2$, which does not depend on k. So every contour has the same slope: the contours are parallel lines.

Step 3. Changing k changes only the intercept k/c_2, so increasing k slides the line parallel to itself.

Step 4. By Module 2, the vector $\mathbf{c} = (c_1, c_2)$ is perpendicular to every contour and points in the direction in which z increases fastest.

Rule. For maximisation, slide the contour in the direction of $\mathbf{c}$ as far as possible while it still touches the feasible region; for minimisation, slide it in the direction of $-\mathbf{c}$. The last point (or points) touched is optimal.

5.4 Step 3: Corner Point Enumeration

By Module 3, if an optimum exists then it is attained at a vertex. For a bounded region we can therefore list every vertex, evaluate z at each, and pick the best. Each vertex is the intersection of two boundary lines; solving the pair of equations by the 2×2 inverse of Module 2, or by elimination, gives its coordinates.

5.5 Step 4: Certifying Optimality

Sliding a line by eye is not a proof. The following derivation gives a short algebraic certificate. It is our first glimpse of duality theory.

Derivation A certificate of optimality at a vertex

Consider max $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{a}_i^\top\mathbf{x} \le b_i$ for $i = 1, \dots, m$. Let $\mathbf{x}^\star$ be a feasible vertex at which constraints 1 and 2 are active, that is $\mathbf{a}_1^\top\mathbf{x}^\star = b_1$ and $\mathbf{a}_2^\top\mathbf{x}^\star = b_2$.

Step 1. Suppose we can find numbers $y_1 \ge 0$ and $y_2 \ge 0$ with

$$\mathbf{c} = y_1\mathbf{a}_1 + y_2\mathbf{a}_2. \tag{5.3}$$

Step 2. Take any feasible $\mathbf{x}$. Using (5.3) and linearity,

$$\mathbf{c}^\top\mathbf{x} = y_1\,\mathbf{a}_1^\top\mathbf{x} + y_2\,\mathbf{a}_2^\top\mathbf{x}.$$

Step 3. Feasibility gives $\mathbf{a}_1^\top\mathbf{x} \le b_1$ and $\mathbf{a}_2^\top\mathbf{x} \le b_2$; multiplying by $y_1, y_2 \ge 0$ preserves the inequalities:

$$\mathbf{c}^\top\mathbf{x} \le y_1b_1 + y_2b_2.$$

Step 4. At the vertex the two constraints are equalities, so the same calculation gives $\mathbf{c}^\top\mathbf{x}^\star = y_1b_1 + y_2b_2$.

Step 5. Therefore $\mathbf{c}^\top\mathbf{x} \le \mathbf{c}^\top\mathbf{x}^\star$ for every feasible $\mathbf{x}$, and $\mathbf{x}^\star$ is optimal.

where

$\mathbf{a}_1, \mathbf{a}_2$ are the normal vectors of the two active constraints;

y_1, y_2 are the *multipliers*; in Module 12 they become the dual variables, or shadow prices.

Geometric meaning. Condition (5.3) with $y_1, y_2 \ge 0$ says that $\mathbf{c}$ lies inside the angle (the *cone*) formed by the two active normals. For minimisation with constraints $\mathbf{a}_i^\top\mathbf{x} \ge b_i$ the same argument works with every inequality reversed.

Lucid Insight The slope test

In two dimensions, condition (5.3) is equivalent to a slope comparison: the vertex is optimal when the slope of the objective contour lies between the slopes of the two active boundary lines. Steeper objectives favour the vertex further along the x_1 axis; flatter objectives favour vertices further up the x_2 axis.

5.6 Special Cases

Key Result What can happen in two dimensions

1. **Unique optimum**: the last contour touches a single vertex.
2. **Multiple optima**: the objective contour is parallel to an edge of the region, and the whole edge is optimal. In (5.3) this shows up as $\mathbf{c}$ being a positive multiple of a single active normal.
3. **Unbounded**: the region extends without limit in a direction $\mathbf{d}$ with $\mathbf{c}^\top\mathbf{d} > 0$, so the contour can be slid forever.

4. **Infeasible**: the half planes have no common point.
5. **Redundant constraint**: a constraint whose removal does not change the region. It is harmless but adds work.

Machine Learning Connection

Deployment decisions in machine learning often have exactly two levers: how much traffic to send to a small fast model and how much to a large accurate model, or how to split a budget between data collection and compute. The graphical method gives immediate insight into such trade offs, and the certificate (5.3) tells us the value of an extra unit of each limited resource, which is the price an engineer should be willing to pay for more GPU time or memory.

5.7 Worked Examples

Worked Example 5.1 The Furniture Workshop Solved Graphically

Solve max $z = 30x_1 + 20x_2$ subject to $4x_1 + 2x_2 \le 40$ (timber), $2x_1 + 3x_2 \le 30$ (labour), $x_1, x_2 \ge 0$. Certify the answer.

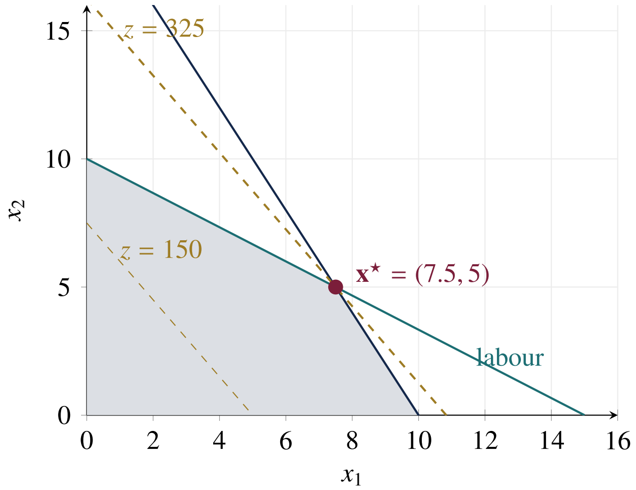
Solution

Step 1. **Timber line** $4x_1 + 2x_2 = 40$: intercepts $40/4 = 10$ and $40/2 = 20$, so through $(10, 0)$ and $(0, 20)$. The origin satisfies $0 \le 40$, so shade towards the origin.

Step 2. **Labour line** $2x_1 + 3x_2 = 30$: intercepts 15 and 10, through $(15, 0)$ and $(0, 10)$. Shade towards the origin.

Step 3. **Intersection** of the two lines (Worked Example 2.4): $(7.5, 5)$.

Step 4. **Feasible region** (figure below): the quadrilateral with vertices $(0, 0)$, $(10, 0)$, $(7.5, 5)$, $(0, 10)$.



Step 5. **Enumerate corners.** $z(0, 0) = 0$, $z(10, 0) = 300$, $z(7.5, 5) = 225 + 100 = 325$, $z(0, 10) = 200$. Best: $(7.5, 5)$.

Step 6. **Certificate.** Both constraints are active at $(7.5, 5)$. Solve $\mathbf{c} = y_1\mathbf{a}_1 + y_2\mathbf{a}_2$:

$$4y_1 + 2y_2 = 30, \qquad 2y_1 + 3y_2 = 20.$$

Multiply the second equation by 2 and subtract from the first: $(4y_1+2y_2)-(4y_1+6y_2) = 30-40$, so $-4y_2 = -10$ and $y_2 = 2.5$. Then $4y_1 = 30 - 5 = 25$, so $y_1 = 6.25$. Both are nonnegative.

Step 7. **Bound.** $y_1b_1 + y_2b_2 = 6.25(40) + 2.5(30) = 250 + 75 = 325 = z(7.5, 5)$. The vertex is optimal.

Answer. $\boxed{x_1^\star = 7.5,\ x_2^\star = 5,\ z^\star = 325}$. The multipliers say one extra unit of timber is worth £6.25 and one extra labour hour £2.50.

Worked Example 5.2 The Minimum Cost Diet Solved Graphically

Solve min $z = 2x_1 + 3x_2$ subject to $3x_1 + 4x_2 \geq 24$ (protein), $x_1 + 3x_2 \geq 12$ (fibre), $x_1, x_2 \geq 0$.

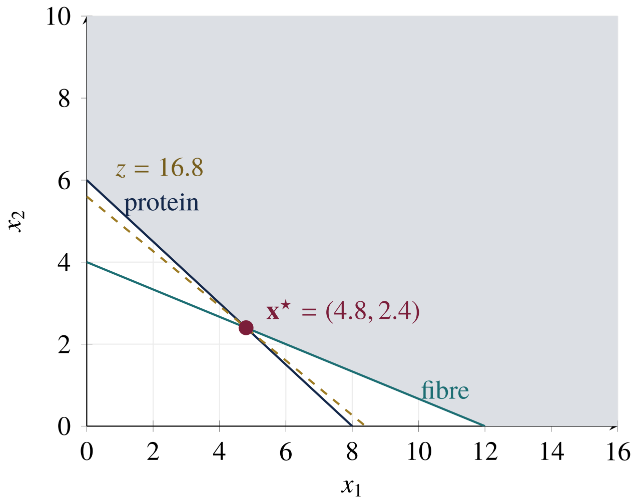
Solution

Step 1. **Protein line** through $(8, 0)$ and $(0, 6)$. The origin gives $0 \geq 24$, false, so the feasible side is *away* from the origin.

Step 2. **Fibre line** through $(12, 0)$ and $(0, 4)$. Again the feasible side is away from the origin.

Step 3. **Intersection.** From $x_1 = 12-3x_2$, substitute into the protein equation: $3(12-3x_2)+4x_2 = 24$, so $36 - 5x_2 = 24$, $x_2 = 2.4$, $x_1 = 12 - 7.2 = 4.8$.

Step 4. **Region.** Unbounded, with corner points $(0, 6)$, $(4.8, 2.4)$ and $(12, 0)$.



Step 5. **Is the minimum bounded?** The region is unbounded, but the cost has positive coefficients on nonnegative variables, so it cannot fall below 0; sliding the contour towards the origin (direction $-\mathbf{c}$) stops at a corner.

Step 6. **Corner values.** $z(0, 6) = 18$, $z(4.8, 2.4) = 16.8$, $z(12, 0) = 24$. Best: $(4.8, 2.4)$.

Step 7. **Certificate** (inequalities of type $\geq$, minimisation). Solve $\mathbf{c} = y_1(3,4) + y_2(1,3)$: $3y_1 + y_2 = 2$ and $4y_1 + 3y_2 = 3$. From the first, $y_2 = 2 - 3y_1$; substitute: $4y_1 + 6 - 9y_1 = 3$, so $y_1 = 0.6$ and $y_2 = 0.2$. Both nonnegative. Then for any feasible $\mathbf{x}$, $\mathbf{c}^\top\mathbf{x} = y_1(3x_1 + 4x_2) + y_2(x_1 + 3x_2) \geq 0.6(24) + 0.2(12) = 16.8$.

Answer. $\boxed{x_1^\star = 4.8,\ x_2^\star = 2.4,\ z^\star = 16.8}$. Each extra gram of protein required costs £0.60, each extra gram of fibre £0.20.

Worked Example 5.3 Multiple Optimal Solutions

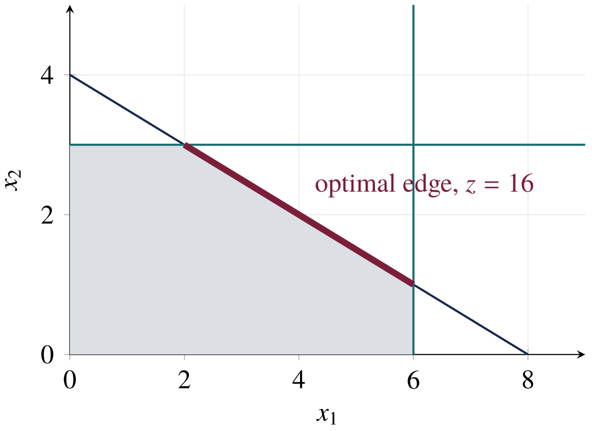
Solve max $z = 2x_1 + 4x_2$ subject to $x_1 + 2x_2 \leq 8$, $x_1 \leq 6$, $x_2 \leq 3$, $x_1, x_2 \geq 0$, and describe the full set of optimal solutions.

Solution

Step 1. **Vertices.** The lines are $x_1 + 2x_2 = 8$ (through $(8,0)$, $(0,4)$), $x_1 = 6$ and $x_2 = 3$. Intersections that are feasible: $(0,0)$, $(6,0)$, $(6,1)$ [from $x_1 = 6$ and $6 + 2x_2 = 8$], $(2,3)$ [from $x_2 = 3$ and $x_1 + 6 = 8$], $(0,3)$.

Step 2. **Corner values.** $z(0,0) = 0$, $z(6,0) = 12$, $z(6,1) = 12 + 4 = 16$, $z(2,3) = 4 + 12 = 16$, $z(0,3) = 12$.

Step 3. **Two corners tie** at $z = 16$. The objective $2x_1 + 4x_2 = 2(x_1 + 2x_2)$ is exactly twice the left hand side of the first constraint, so its contours are parallel to that constraint's boundary.



Step 4. **Every point of the edge is optimal.** Points on the segment are $\mathbf{x}(\lambda) = \lambda(2,3) + (1-\lambda)(6,1)$ for $\lambda \in [0,1]$. By linearity $z(\mathbf{x}(\lambda)) = 16\lambda + 16(1-\lambda) = 16$.

Step 5. **Certificate.** Take $y_1 = 2$ for the first constraint and 0 for the others: $\mathbf{c} = 2(1,2)$, and $\mathbf{c}^\top\mathbf{x} \leq 2 \cdot 8 = 16$ for every feasible $\mathbf{x}$.

Answer. $\boxed{z^\star = 16}$, attained on the whole edge from $(2,3)$ to $(6,1)$.

Worked Example 5.4 An Unbounded Problem

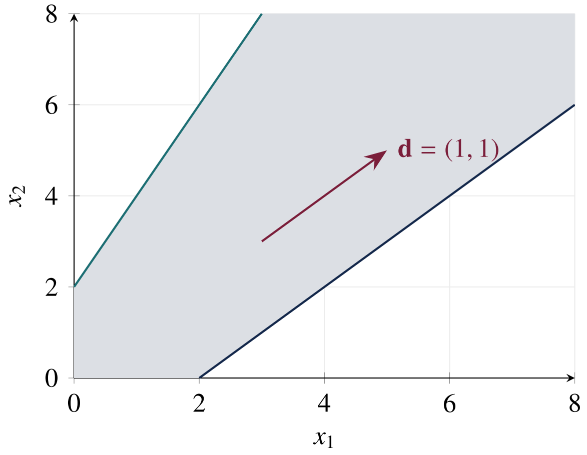
Investigate max $z = x_1 + x_2$ subject to $x_1 - x_2 \le 2$, $-2x_1 + x_2 \le 2$, $x_1, x_2 \ge 0$. Also solve the same problem with the objective minimised.

Solution

Step 1. **First line** $x_1 - x_2 = 2$ through $(2, 0)$ with slope 1. The origin gives $0 \le 2$: feasible side contains the origin (region above and to the left of the line).

Step 2. **Second line** $-2x_1 + x_2 = 2$ through $(0, 2)$ with slope 2. The origin gives $0 \le 2$: feasible side below and to the right.

Step 3. **Region.** It is bounded below by the axes and opens out between the two lines, which diverge because their slopes differ.



Step 4. **Recession direction.** $\mathbf{d} = (1, 1)$: $d_1 - d_2 = 0 \le 0$, $-2d_1 + d_2 = -1 \le 0$, $\mathbf{d} \ge \mathbf{0}$. So from the feasible point $(0, 0)$ the ray $t(1, 1)$ stays feasible for all $t \ge 0$.

Step 5. **Objective along the ray.** $z(t, t) = 2t \to \infty$. Since $\mathbf{c}^\top \mathbf{d} = 2 > 0$, the maximisation problem is *unbounded*.

Step 6. **Minimisation.** With $x_1, x_2 \ge 0$, $z \ge 0$, and $z(0, 0) = 0$. So the minimum is 0 at the origin.

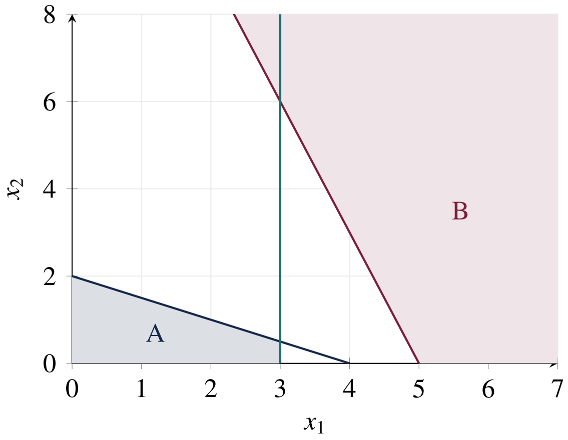
Answer. The maximum is unbounded; the minimum is 0 at $(0, 0)$.

Worked Example 5.5 An Infeasible Problem

Show that there is no $(x_1, x_2) \ge \mathbf{0}$ with $x_1 + 2x_2 \le 4$, $3x_1 + x_2 \ge 15$ and $x_1 \le 3$, both graphically and algebraically.

Solution

Step 1. **Graphical picture.** The first constraint keeps points below the line through $(4, 0)$ and $(0, 2)$. The third keeps $x_1 \le 3$. Together with the axes they form a small region near the origin with largest x_1 equal to 3. The second constraint requires points above the line through $(5, 0)$ and $(0, 15)$, which lies entirely beyond that small region.



Region A satisfies the first and third constraints; region B satisfies the second. They do not overlap.

Step 2. **Algebraic proof.** From $x_1 \le 3$ we get $3x_1 \le 9$. Combined with $3x_1 + x_2 \ge 15$ this forces $x_2 \ge 15 - 3x_1 \ge 15 - 9 = 6$.

Step 3. From $x_1 + 2x_2 \le 4$ and $x_1 \ge 0$ we get $2x_2 \le 4$, so $x_2 \le 2$.

Step 4. The conditions $x_2 \ge 6$ and $x_2 \le 2$ contradict each other, so no feasible point exists.

Step 5. **A certificate of infeasibility.** The proof combined the constraints with nonnegative multipliers to produce the impossible statement $0 \ge 4$ in disguise. Module 13 (Farkas' lemma) shows that such a certificate always exists for an infeasible LP.

Answer. The problem is $\boxed{\text{infeasible}}$.

Worked Example 5.6 Routing Traffic between a Small and a Large Model

An ML platform serves two models. Serving one thousand queries per hour on the small model uses 1 GPU unit and 2 GB of memory and earns £1; on the large model it uses 4 GPU units and 3 GB and earns £3. There are 40 GPU units and 45 GB available, and at least 10 thousand queries per hour must be served in total. Find the most profitable routing and the value of extra GPU capacity.

Solution

Step 1. **Variables.** x_1 and x_2 are thousands of queries per hour on the small and large models.

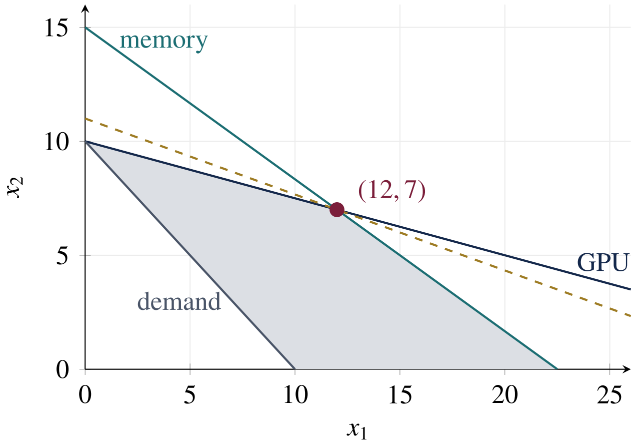
Step 2. **Model.**

$$\max\ z = x_1 + 3x_2 \quad \text{subject to} \quad x_1 + 4x_2 \le 40, \quad 2x_1 + 3x_2 \le 45, \quad x_1 + x_2 \ge 10, \quad x_1, x_2 \ge 0.$$

Step 3. **Lines.** GPU: $(40, 0)$ to $(0, 10)$. Memory: $(22.5, 0)$ to $(0, 15)$. Demand: $(10, 0)$ to $(0, 10)$, feasible side away from the origin.

Step 4. **GPU and memory intersection.** From GPU, $x_1 = 40 - 4x_2$. Substitute into memory: $80 - 8x_2 + 3x_2 = 45$, so $x_2 = 7$, $x_1 = 12$.

Step 5. **Vertices of the region.** $(10, 0)$, $(22.5, 0)$, $(12, 7)$, $(0, 10)$. (At $(0, 10)$ the GPU and demand lines meet; memory use is $30 \le 45$.)



Step 6. **Corner values.** $z(10, 0) = 10$, $z(22.5, 0) = 22.5$, $z(12, 7) = 12 + 21 = 33$, $z(0, 10) = 30$. Best: $(12, 7)$.

Step 7. **Certificate.** Active: GPU $(1, 4)$ and memory $(2, 3)$. Solve $y_1 + 2y_2 = 1$ and $4y_1 + 3y_2 = 3$. From the first, $y_1 = 1 - 2y_2$; substitute: $4 - 8y_2 + 3y_2 = 3$, so $y_2 = 0.2$, $y_1 = 0.6$. Both nonnegative, and $0.6(40) + 0.2(45) = 24 + 9 = 33$.

Step 8. **Value of extra GPU capacity.** The multiplier $y_1 = 0.6$ means one more GPU unit raises the optimal profit by about £0.60 per hour (as long as the same two constraints stay active; Module 15 makes this precise).

Answer. $\boxed{x_1^\star = 12,\ x_2^\star = 7,\ z^\star = \text{£}33 \text{ per hour}}$; an extra GPU unit is worth about £0.60 per hour.

Module Summary

- Draw each boundary line from its intercepts, test the origin to choose the side, and intersect the half planes.
- Objective contours $c_1x_1 + c_2x_2 = k$ are parallel; slide them along $\mathbf{c}$ to maximise, along $-\mathbf{c}$ to minimise.
- For bounded regions, evaluating z at every vertex finds the optimum.
- A vertex is optimal when $\mathbf{c} = y_1\mathbf{a}_1 + y_2\mathbf{a}_2$ with $y_1, y_2 \ge 0$ for its active constraints; then $y_1b_1 + y_2b_2$ is a matching bound. The multipliers are shadow prices.
- An objective parallel to an active edge gives multiple optima; a recession direction with $\mathbf{c}^\top\mathbf{d} > 0$ gives unboundedness; disjoint half planes give infeasibility.

Module 6

Standard Form, Canonical Form and Conversions

Learning Objectives

After studying this module you will be able to:

- state the standard form and the canonical form of a linear program, and explain why algorithms need a fixed form;
- convert maximisation to minimisation and prove the two problems have the same optimal solutions;
- introduce slack and surplus variables and prove that the conversion is exact;
- handle free variables, nonpositive variables, lower bounds, upper bounds and negative right hand sides;
- recover the original solution, and interpret the slack values, after solving the converted problem;
- convert a machine learning problem, ℓ_1 regression, into standard form and count its dimensions.

6.1 Why a Fixed Form Is Needed

A linear program can be written in many ways: maximise or minimise, constraints of three kinds, variables that are nonnegative, free or bounded. Algorithms such as the simplex method are much easier to describe and to implement if they accept only one shape of problem. We therefore agree on a *standard form* and learn to convert any linear program into it. The conversions are mechanical and lose no information: the converted problem has exactly the same optimal solutions, once the extra variables are discarded.

6.2 The Two Standard Shapes

Definition Standard Form

A linear program is in **standard form** if it reads

$$\min_{\mathbf{x}\in\mathbb{R}^n} \mathbf{c}^\top \mathbf{x} \qquad \text{subject to} \qquad \mathbf{A}\mathbf{x} = \mathbf{b}, \qquad \mathbf{x} \geq \mathbf{0}, \tag{6.1}$$

where

$\mathbf{x} \in \mathbb{R}^n$ is the vector of all variables, including any slack or surplus variables added during conversion;

$\mathbf{c} \in \mathbb{R}^n$ is the cost vector;

$\mathbf{A} \in \mathbb{R}^{m \times n}$ is the constraint matrix, usually assumed to have rank m (no redundant equations);

$\mathbf{b} \in \mathbb{R}^m$ is the right hand side, conventionally arranged so that $\mathbf{b} \geq \mathbf{0}$.

All constraints are equations and all variables are nonnegative.

Definition Canonical (Inequality) Form

A linear program is in **canonical form** if it reads

$$\max_{\mathbf{x} \in \mathbb{R}^n} \ \mathbf{c}^\top \mathbf{x} \qquad \text{subject to} \qquad \mathbf{A}\mathbf{x} \leq \mathbf{b}, \qquad \mathbf{x} \geq \mathbf{0}. \tag{6.2}$$

This is the form of Module 1 and is the natural form for writing the dual problem in Module 12.

Lucid Insight Why two forms?

The standard form is algebraically convenient: equations can be manipulated with Gaussian elimination, and basic solutions are defined by choosing columns. The canonical form is geometrically convenient: every constraint is a half space you can draw. Being fluent in both is like being able to switch between a map and a set of written directions.

6.3 The Conversion Rules

6.3.1 Rule 1: maximisation to minimisation

Derivation $\max f = -\min(-f)$

Let f be any function on a set $\mathcal{F}$ that attains its maximum at $\mathbf{x}^\star$.

Step 1. By definition $f(\mathbf{x}^\star) \geq f(\mathbf{x})$ for every $\mathbf{x} \in \mathcal{F}$.

Step 2. Multiply by -1, which reverses inequalities: $-f(\mathbf{x}^\star) \leq -f(\mathbf{x})$ for every $\mathbf{x} \in \mathcal{F}$.

Step 3. So $\mathbf{x}^\star$ minimises $-f$, and the minimum value is $-f(\mathbf{x}^\star)$.

Step 4. Hence

$$\max_{\mathbf{x} \in \mathcal{F}} f(\mathbf{x}) = -\min_{\mathbf{x} \in \mathcal{F}} \left(-f(\mathbf{x}) \right), \tag{6.3}$$

and both problems have the same set of optimal solutions.

For a linear program: replace max $\mathbf{c}^\top \mathbf{x}$ by min $(-\mathbf{c})^\top \mathbf{x}$, solve, and negate the optimal value at the end.

6.3.2 Rule 2: inequality to equation with a slack variable

Derivation Slack variables

Consider the constraint $\mathbf{a}_i^\top \mathbf{x} \le b_i$.

Step 1. Define the **slack** $s_i = b_i - \mathbf{a}_i^\top \mathbf{x}$, the unused amount of resource i.

Step 2. The original inequality holds exactly when $s_i \ge 0$.

Step 3. Rearranging the definition gives an equation: $\mathbf{a}_i^\top \mathbf{x} + s_i = b_i$.

Step 4. Therefore

$$\mathbf{a}_i^\top \mathbf{x} \le b_i \quad \Longleftrightarrow \quad \mathbf{a}_i^\top \mathbf{x} + s_i = b_i \text{ for some } s_i \ge 0. \tag{6.4}$$

Step 5. The slack has cost coefficient 0, because leaving a resource unused neither earns nor costs anything in the model.

6.3.3 Rule 3: inequality to equation with a surplus variable

Derivation Surplus variables

Consider $\mathbf{a}_i^\top \mathbf{x} \ge b_i$. Define the **surplus** $e_i = \mathbf{a}_i^\top \mathbf{x} - b_i$, the amount by which a requirement is exceeded. Exactly as before,

$$\mathbf{a}_i^\top \mathbf{x} \ge b_i \quad \Longleftrightarrow \quad \mathbf{a}_i^\top \mathbf{x} - e_i = b_i \text{ for some } e_i \ge 0, \tag{6.5}$$

and e_i receives cost coefficient 0.

6.3.4 Rule 4: negative right hand sides

If $b_i < 0$ in an equation $\mathbf{a}_i^\top \mathbf{x} = b_i$, multiply both sides by -1 to obtain $(-\mathbf{a}_i)^\top \mathbf{x} = -b_i > 0$. Multiplying an equation by a nonzero number does not change its solutions (Module 2). For an inequality, multiply by -1 *before* adding the slack or surplus, remembering that the direction reverses.

6.3.5 Rule 5: variables that are not nonnegative

Derivation Free, nonpositive and bounded variables

Step 1. **Free variable** $x_j \in \mathbb{R}$: substitute $x_j = x_j^+ - x_j^-$ with $x_j^+, x_j^- \ge 0$ (Module 4). The column of x_j in $\mathbf{A}$ is copied for x_j^+ and copied with the opposite sign for x_j^-; similarly for its cost.

Step 2. **Nonpositive variable** $x_j \le 0$: substitute $x_j = -x_j'$ with $x_j' \ge 0$. The column and cost of x_j change sign.

Step 3. **Lower bound** $x_j \ge \ell_j$ with $\ell_j \ne 0$: substitute $x_j = x_j' + \ell_j$ with $x_j' \ge 0$. Each constraint $\mathbf{a}_i^\top \mathbf{x}\ (\le, =, \ge)\ b_i$ becomes $\ldots + a_{ij}x_j' + \cdots\ (\le, =, \ge)\ b_i - a_{ij}\ell_j$, and the objective gains the constant $c_j\ell_j$.

Step 4. **Upper bound** $x_j \le u_j$: keep it as an ordinary constraint and add a slack, $x_j + s = u_j$. (Production solvers treat bounds specially for efficiency, but mathematically this is enough.)

Key Result   The conversion recipe

1. If maximising, negate **c** and minimise.
2. Make every variable nonnegative using Rule 5.
3. Make every right hand side nonnegative using Rule 4.
4. Add a slack to each ≤ constraint and subtract a surplus from each ≥ constraint.
5. Collect all variables into one vector **x** and read off **c**, **A** and **b**.

Common Pitfall

When a constraint has a negative right hand side, flip it *before* deciding between slack and surplus. For example $x_1 - x_2 \geq -3$ becomes $-x_1 + x_2 \leq 3$, which needs a *slack*, not a surplus. Doing the steps in the wrong order produces a surplus variable with a negative right hand side, which breaks the convention $\mathbf{b} \geq \mathbf{0}$.

6.4 Counting Dimensions

Derivation   Size of the standard form

Suppose an LP has $m_\leq$ constraints of type ≤, $m_\geq$ of type ≥ and $m_=$ equations, and has n_+ nonnegative variables and n_f free variables.

Step 1. The number of equations in standard form is $m = m_\leq + m_\geq + m_=$, because each constraint becomes exactly one equation.

Step 2. Each free variable becomes two variables, each ≤ constraint adds one slack and each ≥ constraint adds one surplus.

Step 3. Hence the number of variables is

$$n = n_+ + 2n_f + m_\leq + m_\geq. \tag{6.6}$$

Machine Learning Connection

General purpose solvers such as HiGHS, Gurobi and the linprog routine of SciPy accept problems in a convenient mixed form and convert them internally to a standard form. Knowing what happens inside helps you to read solver logs (which report the number of rows and columns after conversion), to predict memory use for large data sets, and to interpret the slack values that a solver returns. In ℓ_1 regression with N data points and p features, for example, the standard form has N equations and $2p + 2N$ variables, a size that grows linearly with the data. This tells you immediately whether an LP approach is practical for your data set.

6.5 Worked Examples

Worked Example 6.1 A Complete Conversion

Convert to standard form:

$$\max\ 3x_1 + 2x_2 \quad \text{subject to} \quad 2x_1 + x_2 \le 10, \quad x_1 + 3x_2 \ge 6, \quad x_1 - x_2 = 1, \quad x_1, x_2 \ge 0.$$

Solution

Step 1. **Objective.** By (6.3), minimise $-3x_1 - 2x_2$.

Step 2. **Variables** are already nonnegative, and **right hand sides** 10, 6, 1 are already nonnegative.

Step 3. **First constraint (≤).** Add slack $s_1 \ge 0$: $2x_1 + x_2 + s_1 = 10$.

Step 4. **Second constraint (≥).** Subtract surplus $e_2 \ge 0$: $x_1 + 3x_2 - e_2 = 6$.

Step 5. **Third constraint** is already an equation.

Step 6. **Assemble** with variable order (x_1, x_2, s_1, e_2):

$$\min \begin{pmatrix} -3 & -2 & 0 & 0 \end{pmatrix} \mathbf{x} \quad \text{subject to} \quad \begin{pmatrix} 2 & 1 & 1 & 0 \\ 1 & 3 & 0 & -1 \\ 1 & -1 & 0 & 0 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 10 \\ 6 \\ 1 \end{pmatrix}, \quad \mathbf{x} \ge \mathbf{0}.$$

Step 7. **Check the count** with (6.6): $n = 2 + 0 + 1 + 1 = 4$ variables, $m = 3$ equations.

Answer. $\mathbf{c} = (-3, -2, 0, 0)^\top$, $\mathbf{A}$ and $\mathbf{b} = (10, 6, 1)^\top$ as above, with $\mathbf{x} = (x_1, x_2, s_1, e_2)^\top \ge \mathbf{0}$.

Worked Example 6.2 Free and Nonpositive Variables with a Negative Right Hand Side

Convert to standard form:

$$\min\ x_1 - 2x_2 + x_3 \quad \text{subject to} \quad x_1 + x_2 - x_3 \ge -3, \quad 2x_1 - x_2 \le 5, \quad x_1 \ge 0,\ x_2 \text{ free},\ x_3 \le 0.$$

Solution

Step 1. **Free** x_2**.** Write $x_2 = x_2^+ - x_2^-$.

Step 2. **Nonpositive** x_3**.** Write $x_3 = -x_3'$ with $x_3' \ge 0$.

Step 3. **Objective.** $x_1 - 2(x_2^+ - x_2^-) + (-x_3') = x_1 - 2x_2^+ + 2x_2^- - x_3'$.

Step 4. **First constraint.** Substitute: $x_1 + x_2^+ - x_2^- + x_3' \ge -3$. The right hand side is negative, so multiply by -1 first: $-x_1 - x_2^+ + x_2^- - x_3' \le 3$. It is now a ≤ constraint, so add a slack: $-x_1 - x_2^+ + x_2^- - x_3' + s_1 = 3$.

Step 5. **Second constraint.** Substitute: $2x_1 - x_2^+ + x_2^- \le 5$; add slack: $2x_1 - x_2^+ + x_2^- + s_2 = 5$.

Step 6. **Assemble** with order $(x_1, x_2^+, x_2^-, x_3', s_1, s_2)$:

$$\mathbf{c} = \begin{pmatrix} 1 \\ -2 \\ 2 \\ -1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{A} = \begin{pmatrix} -1 & -1 & 1 & -1 & 1 & 0 \\ 2 & -1 & 1 & 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}.$$

Step 7. **Recovering the original variables** after solving: $x_2 = x_2^+ - x_2^-$ and $x_3 = -x_3'$.

Answer. The standard form has 6 nonnegative variables and 2 equations, as shown above.

Worked Example 6.3 Lower and Upper Bounds

Convert to standard form: max $4x_1 + x_2$ subject to $x_1 + x_2 \le 9$, $2 \le x_1$, $-1 \le x_2 \le 4$.

Solution

Step 1. **Shift x_1.** Let $x_1 = x_1' + 2$ with $x_1' \ge 0$.

Step 2. **Shift x_2.** Let $x_2 = x_2' - 1$ with $x_2' \ge 0$ (lower bound $\ell_2 = -1$).

Step 3. **Objective.** $4(x_1' + 2) + (x_2' - 1) = 4x_1' + x_2' + 7$. The constant 7 does not affect the optimiser; we remember to add it back.

Step 4. **Sum constraint.** $(x_1' + 2) + (x_2' - 1) \le 9$ gives $x_1' + x_2' \le 8$. Add slack: $x_1' + x_2' + s_1 = 8$.

Step 5. **Upper bound on x_2.** $x_2' - 1 \le 4$ gives $x_2' \le 5$. Add slack: $x_2' + s_2 = 5$.

Step 6. **Standard form.**

$$\min\ -4x_1' - x_2' \quad \text{subject to} \quad x_1' + x_2' + s_1 = 8, \quad x_2' + s_2 = 5, \quad x_1', x_2', s_1, s_2 \ge 0.$$

Step 7. **Solve by inspection** to check the conversion. In the shifted problem x_1' has the larger profit per unit of the shared resource, so put $x_1' = 8$, $x_2' = 0$, giving shifted objective 32. Original: $x_1 = 10$, $x_2 = -1$, objective $32 + 7 = 39$. Directly: $4(10) + (-1) = 39$, and $10 + (-1) = 9 \le 9$.

Answer. $\boxed{x_1 = 10,\ x_2 = -1,\ z^\star = 39}$; the conversion preserved the problem exactly.

Worked Example 6.4 Interpreting Slack Values

In the furniture workshop (Worked Example 1.1) the standard form is $4x_1 + 2x_2 + s_1 = 40$, $2x_1 + 3x_2 + s_2 = 30$. Compute and interpret the slacks for the plans $(7.5, 5)$, $(5, 5)$ and $(10, 0)$.

Solution

Step 1. **Formulas.** By the definition of slack, $s_1 = 40 - 4x_1 - 2x_2$ (unused timber) and $s_2 = 30 - 2x_1 - 3x_2$ (unused labour hours).

Step 2. **Plan** $(7.5, 5)$. $s_1 = 40 - 30 - 10 = 0$; $s_2 = 30 - 15 - 15 = 0$. Both resources are fully used.

Step 3. **Plan** $(5, 5)$. $s_1 = 40 - 20 - 10 = 10$; $s_2 = 30 - 10 - 15 = 5$. Ten units of timber and five hours of labour are idle, which suggests the plan can be improved.

Step 4. **Plan** $(10, 0)$. $s_1 = 40 - 40 - 0 = 0$; $s_2 = 30 - 20 - 0 = 10$. Timber is exhausted but ten labour hours are idle.

Step 5. **Link to geometry.** A slack equals zero exactly when its constraint is active. The optimal vertex $(7.5, 5)$ has both slacks zero, matching its two active constraints in Module 5.

Answer. $(s_1, s_2) = (0, 0), (10, 5)$ and $(0, 10)$ respectively; zero slack means the resource is a bottleneck.

Worked Example 6.5 ℓ_1 Regression in Standard Form

Fit the line $\hat{y} = w_0 + w_1 x$ to the data $(x, y) = (0, 1), (1, 2), (2, 2)$ by minimising the sum of absolute residuals. Write the problem in standard form, count its dimensions, and find the optimal line.

Solution

Step 1. **Residuals.** $r_i = y_i - w_0 - w_1 x_i$: $r_1 = 1 - w_0$, $r_2 = 2 - w_0 - w_1$, $r_3 = 2 - w_0 - 2w_1$.

Step 2. **Split residuals** into positive and negative parts: $r_i = u_i - v_i$ with $u_i, v_i \geq 0$; at the optimum $|r_i| = u_i + v_i$.

Step 3. **Split the free weights**: $w_0 = w_0^+ - w_0^-$, $w_1 = w_1^+ - w_1^-$.

Step 4. **Equations.** $w_0 + w_1 x_i + u_i - v_i = y_i$ for each i:

$$\begin{aligned} w_0^+ - w_0^- + u_1 - v_1 &= 1, \\ w_0^+ - w_0^- + w_1^+ - w_1^- + u_2 - v_2 &= 2, \\ w_0^+ - w_0^- + 2w_1^+ - 2w_1^- + u_3 - v_3 &= 2. \end{aligned}$$

Step 5. **Objective.** Minimise $u_1 + v_1 + u_2 + v_2 + u_3 + v_3$; the weight variables have cost 0.

Step 6. **Dimensions.** $m = N = 3$ equations and $n = 2p + 2N = 4 + 6 = 10$ variables, with $p = 2$ parameters.

Step 7. **Solve.** An optimal ℓ_1 line passes through at least two data points (this follows from the vertex property of Module 3 and is derived in Module 19). Try each pair:

- through $(0, 1), (1, 2)$: $\hat{y} = 1 + x$, residual at $x = 2$ is $2 - 3 = -1$; total 1;
- through $(1, 2), (2, 2)$: $\hat{y} = 2$, residual at $x = 0$ is $1 - 2 = -1$; total 1;
- through $(0, 1), (2, 2)$: $\hat{y} = 1 + 0.5x$, residual at $x = 1$ is $2 - 1.5 = 0.5$; total 0.5.

Answer. The standard form has 3 equations and 10 variables; the optimal line is $\boxed{\hat{y} = 1 + 0.5x}$ with total absolute error 0.5.

Worked Example 6.6 Sizing a Conversion and Going Back to Canonical Form

(a) An LP has 3 constraints of type $\leq$, 2 of type $\geq$, 1 equation, 4 nonnegative variables and 2 free variables. How many equations and variables does its standard form have? (b) Convert the standard form problem min $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$ into canonical form.

Solution

Step 1. **(a) Equations.** $m = 3 + 2 + 1 = 6$.

Step 2. **(a) Variables** by (6.6): $n = n_+ + 2n_f + m_\leq + m_\geq = 4 + 2(2) + 3 + 2 = 13$.

Step 3. **(b) Objective.** min $\mathbf{c}^\top\mathbf{x} = -\max\,(-\mathbf{c})^\top\mathbf{x}$ by (6.3).

Step 4. **(b) Equations.** $\mathbf{A}\mathbf{x} = \mathbf{b}$ holds exactly when both $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ and $\mathbf{A}\mathbf{x} \geq \mathbf{b}$ hold. The second becomes $-\mathbf{A}\mathbf{x} \leq -\mathbf{b}$.

Step 5. **(b) Result.**

$$\max\,(-\mathbf{c})^\top\mathbf{x} \quad \text{subject to} \quad \begin{pmatrix} \mathbf{A} \\ -\mathbf{A} \end{pmatrix}\mathbf{x} \leq \begin{pmatrix} \mathbf{b} \\ -\mathbf{b} \end{pmatrix}, \quad \mathbf{x} \geq \mathbf{0},$$

whose optimal value is the negative of the original optimal value.

Answer. (a) $\boxed{\text{6 equations and 13 variables}}$; (b) stack $\mathbf{A}$ above $-\mathbf{A}$ and $\mathbf{b}$ above $-\mathbf{b}$, and maximise $-\mathbf{c}^\top\mathbf{x}$.

Module Summary

- Standard form: min $\mathbf{c}^\top\mathbf{x}$, $\mathbf{A}\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, $\mathbf{b} \geq \mathbf{0}$. Canonical form: max $\mathbf{c}^\top\mathbf{x}$, $\mathbf{A}\mathbf{x} \leq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$.
- $\max f = -\min(-f)$; the optimal solutions are the same.
- $\leq$ constraints gain a slack $s_i = b_i - \mathbf{a}_i^\top\mathbf{x} \geq 0$; $\geq$ constraints lose a surplus $e_i = \mathbf{a}_i^\top\mathbf{x} - b_i \geq 0$.
- Free variables split into two nonnegative parts; nonpositive variables change sign; bounded variables are shifted or given a slack.
- Flip negative right hand sides before choosing slack or surplus.
- Standard form size: $m_\leq + m_\geq + m_=$ equations and $n_+ + 2n_f + m_\leq + m_\geq$ variables.
- Zero slack means an active constraint and a bottleneck resource.

MODULE 7

Basic Feasible Solutions and the Fundamental Theorem

Learning Objectives

After studying this module you will be able to:

- define a basis, a basic solution and a basic feasible solution (BFS) for a problem in standard form;
- compute a basic solution with the formula $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$, $\mathbf{x}_N = \mathbf{0}$;
- prove that basic feasible solutions are exactly the vertices of the feasible region;
- recognise degenerate basic feasible solutions;
- state and derive the Fundamental Theorem of Linear Programming, and use its constructive proof to move from any feasible point to a BFS that is no worse;
- explain why enumerating all bases is hopeless for realistic problems, motivating the simplex method.

7.1 From Geometry to Algebra

Modules 3 and 5 showed that optimal solutions sit at vertices. A computer, however, does not see pictures; it manipulates equations. This module gives the algebraic name for a vertex, the *basic feasible solution*, and proves the theorem that makes linear programming finite: if a linear program has an optimal solution, then it has one among the finitely many basic feasible solutions.

Throughout this module the problem is in standard form (Module 6):

$$\min\ \mathbf{c}^\top\mathbf{x} \qquad \text{subject to} \qquad \mathbf{A}\mathbf{x} = \mathbf{b}, \qquad \mathbf{x} \geq \mathbf{0}, \tag{7.1}$$

where

$\mathbf{A} \in \mathbb{R}^{m\times n}$ has full row rank m and $m \leq n$;

$\mathbf{A}_j$ denotes column j of $\mathbf{A}$, for $j = 1, \ldots, n$;

$\mathbf{b} \in \mathbb{R}^m$ is the right hand side and $\mathbf{c} \in \mathbb{R}^n$ the cost vector.

7.2 Bases and Basic Solutions

Definition Basis, Basic and Nonbasic Variables

A **basis** is a set $\mathcal{B} = \{B(1), \dots, B(m)\}$ of m column indices such that the columns $\mathbf{A}_{B(1)}, \dots, \mathbf{A}_{B(m)}$ are linearly independent. Collect them into the **basis matrix**

$$\mathbf{B} = [\mathbf{A}_{B(1)} \;\; \mathbf{A}_{B(2)} \;\; \cdots \;\; \mathbf{A}_{B(m)}] \in \mathbb{R}^{m\times m}, \tag{7.2}$$

which is invertible by Module 2. The variables $x_{B(1)}, \dots, x_{B(m)}$ are the **basic variables**, gathered in $\mathbf{x}_B$. The remaining $n - m$ variables are **nonbasic**, gathered in $\mathbf{x}_N$, and their columns form the matrix $\mathbf{N} \in \mathbb{R}^{m\times(n-m)}$.

Derivation The basic solution associated with a basis

Step 1. Reorder the columns of $\mathbf{A}$ so the basic columns come first. Then $\mathbf{A} = [\mathbf{B} \;\; \mathbf{N}]$ and $\mathbf{x} = \begin{pmatrix}\mathbf{x}_B \\ \mathbf{x}_N\end{pmatrix}$.

Step 2. By the column view of Module 2, $\mathbf{A}\mathbf{x} = \mathbf{B}\mathbf{x}_B + \mathbf{N}\mathbf{x}_N$. So the constraints read

$$\mathbf{B}\mathbf{x}_B + \mathbf{N}\mathbf{x}_N = \mathbf{b}. \tag{7.3}$$

Step 3. Set every nonbasic variable to zero, $\mathbf{x}_N = \mathbf{0}$. Then (7.3) becomes $\mathbf{B}\mathbf{x}_B = \mathbf{b}$.

Step 4. Because $\mathbf{B}$ is invertible, multiply on the left by $\mathbf{B}^{-1}$:

$$\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}, \qquad \mathbf{x}_N = \mathbf{0}. \tag{7.4}$$

The vector defined by (7.4) is the **basic solution** for the basis $\mathcal{B}$.

Definition Basic Feasible Solution and Degeneracy

A basic solution is a **basic feasible solution** (BFS) if it also satisfies $\mathbf{x} \geq \mathbf{0}$, that is, if $\mathbf{B}^{-1}\mathbf{b} \geq \mathbf{0}$. A BFS is **degenerate** if at least one basic variable equals zero; otherwise it is **nondegenerate**.

Lucid Insight Counting zeros

A basic solution has at least $n - m$ zeros (the nonbasic variables). In the canonical picture of Module 5, a variable equal to zero means a constraint is active: $x_j = 0$ is the active bound $x_j \geq 0$, and $s_i = 0$ is the active resource constraint i. Setting $n - m$ well chosen variables to zero is therefore the same as choosing enough active constraints to pin down a corner.

7.3 Basic Feasible Solutions Are Vertices

Key Result Equivalence of vertices and BFS

Let $P = \{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b},\ \mathbf{x} \geq \mathbf{0}\}$. A point $\mathbf{x} \in P$ is a vertex of P if and only if it is a basic feasible solution. Moreover, $\mathbf{x}$ is a BFS exactly when the columns $\mathbf{A}_j$ with $x_j > 0$ are linearly independent.

Derivation Proof of the equivalence

Let $J(\mathbf{x}) = \{j : x_j > 0\}$ be the *support* of $\mathbf{x} \in P$.

Step 1. **Independent support implies BFS.** If the columns $\{\mathbf{A}_j : j \in J(\mathbf{x})\}$ are independent, there are at most m of them. Since rank $\mathbf{A} = m$, we can add further columns of $\mathbf{A}$ until we have m independent columns: a basis $\mathcal{B} \supseteq J(\mathbf{x})$. Every nonbasic variable lies outside the support and so equals zero, and the basic variables solve $\mathbf{B}\mathbf{x}_B = \mathbf{b}$. Hence $\mathbf{x}$ is the basic solution of $\mathcal{B}$, and it is feasible.

Step 2. **BFS implies vertex.** Let $\mathbf{x}$ be a BFS with basis $\mathcal{B}$, and suppose $\mathbf{x} = \lambda\mathbf{y} + (1-\lambda)\mathbf{z}$ with $\mathbf{y}, \mathbf{z} \in P$ and $0 < \lambda < 1$. For a nonbasic index j, $0 = x_j = \lambda y_j + (1-\lambda)z_j$ with $y_j, z_j \geq 0$ and positive weights, so $y_j = z_j = 0$. Then $\mathbf{y}$ and $\mathbf{z}$ both satisfy $\mathbf{B}\mathbf{y}_B = \mathbf{b}$ and $\mathbf{B}\mathbf{z}_B = \mathbf{b}$, whose unique solution is $\mathbf{B}^{-1}\mathbf{b} = \mathbf{x}_B$. Hence $\mathbf{y} = \mathbf{z} = \mathbf{x}$, so $\mathbf{x}$ is a vertex.

Step 3. **Vertex implies independent support.** Suppose the support columns are dependent: there is $\mathbf{d} \neq \mathbf{0}$ with $d_j = 0$ for $j \notin J(\mathbf{x})$ and $\mathbf{A}\mathbf{d} = \sum_{j \in J} d_j \mathbf{A}_j = \mathbf{0}$. For small $\varepsilon > 0$, the points $\mathbf{x} \pm \varepsilon\mathbf{d}$ satisfy $\mathbf{A}(\mathbf{x} \pm \varepsilon\mathbf{d}) = \mathbf{b}$ and remain nonnegative (the positive components stay positive; the zero components are unchanged). So $\mathbf{x}$ is the midpoint of two distinct points of P and is not a vertex.

7.4 The Fundamental Theorem of Linear Programming

Key Result Fundamental Theorem of Linear Programming

For a linear program in standard form (7.1) with rank $\mathbf{A} = m$:

1. if there is a feasible solution, there is a basic feasible solution;
2. if there is an optimal solution, there is an optimal basic feasible solution.

Derivation Constructive proof by support reduction

Start from any feasible $\mathbf{x}$ (for part 2, an optimal one). If its support columns are independent, $\mathbf{x}$ is already a BFS. Otherwise:

Step 1. **Find a direction inside the support.** As in the previous derivation, pick $\mathbf{d} \neq \mathbf{0}$ with $\mathbf{A}\mathbf{d} = \mathbf{0}$ and $d_j = 0$ whenever $x_j = 0$. Replacing $\mathbf{d}$ by $-\mathbf{d}$ if necessary, arrange that $\mathbf{c}^\top\mathbf{d} \leq 0$ (moving along $\mathbf{d}$ does not increase cost), and if $\mathbf{c}^\top\mathbf{d} = 0$ arrange that $\mathbf{d}$ has at least one negative component.

Step 2. **Move along the direction.** Consider $\mathbf{x}(\theta) = \mathbf{x} + \theta\mathbf{d}$ for $\theta \geq 0$. It satisfies the equations, since $\mathbf{A}\mathbf{x}(\theta) = \mathbf{b} + \theta\mathbf{A}\mathbf{d} = \mathbf{b}$, and its cost is $\mathbf{c}^\top\mathbf{x} + \theta\mathbf{c}^\top\mathbf{d} \leq \mathbf{c}^\top\mathbf{x}$.

Step 3. **Ratio test.** Feasibility requires $x_j + \theta d_j \geq 0$ for every j. Only components with $d_j < 0$ can become negative, and they hit zero at $\theta = x_j/(-d_j)$. Define

$$\theta^\star = \min_{j\,:\,d_j<0} \frac{x_j}{-d_j}. \tag{7.5}$$

Step 4. **Shrink the support.** At $\theta^\star$ the minimising component becomes zero, so $\mathbf{x}(\theta^\star)$ is feasible, no more costly, and has *strictly smaller* support.

Step 5. **Repeat.** The support size is a nonnegative integer that decreases each time, so after finitely many repetitions the support columns become independent, giving a BFS no more costly than the starting point.

Edge case. If $\mathbf{c}^\top \mathbf{d} < 0$ and $\mathbf{d} \geq \mathbf{0}$, no ratio exists and the cost falls without limit, so the problem is unbounded and has no optimum; this cannot happen when $\mathbf{x}$ is optimal.

Lucid Insight The theorem makes linear programming finite

There are at most $\binom{n}{m}$ ways to choose m columns from n, so there are finitely many BFS. By the Fundamental Theorem we could, in principle, solve any LP by listing them all. The ratio test (7.5) you have just met is the same ratio test that drives the simplex method in Module 8.

Common Pitfall

Finite does not mean small. The number $\binom{n}{m}$ grows explosively: a modest ℓ_1 regression with 100 data points and 5 features has $n = 210$ variables and $m = 100$ equations, and $\binom{210}{100}$ is about 7×10^{61}. Enumeration is a theoretical tool, never a practical algorithm.

Definition Adjacent Basic Feasible Solutions

Two bases are **adjacent** if they share $m-1$ columns, that is, they differ by exchanging one basic variable for one nonbasic variable. Their basic feasible solutions are joined by an edge of the polyhedron. The simplex method moves only between adjacent BFS.

Machine Learning Connection

The Fundamental Theorem explains a striking phenomenon in machine learning: *linear programming solutions are sparse*. A BFS has at most m nonzero variables. In ℓ_1 regression this forces the fitted line or hyperplane to pass exactly through at least p data points (Module 19). In sparse recovery with basis pursuit (Module 22), it guarantees at most m nonzero coefficients when there are m measurements. In optimal transport (Module 23), an optimal transport plan between distributions with n_1 and n_2 support points needs at most $n_1 + n_2 - 1$ nonzero flows. Each of these is the vertex property of this module in disguise.

7.5 Worked Examples

Worked Example 7.1 Enumerating All Basic Solutions

The furniture workshop in standard form has constraints $4x_1 + 2x_2 + s_1 = 40$ and $2x_1 + 3x_2 + s_2 = 30$ with all variables nonnegative. List all basic solutions, decide which are feasible, and match the feasible ones with the vertices found in Module 5.

Solution

Step 1. Setup. $m = 2$, $n = 4$, variables (x_1, x_2, s_1, s_2), so there are at most $\binom{4}{2} = 6$ bases. For each basis we set the other two variables to zero and solve the 2×2 system.

Step 2. Basis $\{x_1, x_2\}$**.** $s_1 = s_2 = 0$: $4x_1 + 2x_2 = 40$, $2x_1 + 3x_2 = 30$, giving $(7.5, 5)$ (Worked Example 2.4). Feasible.

Step 3. Basis $\{x_1, s_1\}$**.** $x_2 = s_2 = 0$: second equation gives $2x_1 = 30$, $x_1 = 15$; first gives $s_1 = 40 - 60 = -20$. Infeasible.

Step 4. Basis $\{x_1, s_2\}$**.** $x_2 = s_1 = 0$: $4x_1 = 40$, $x_1 = 10$; $s_2 = 30 - 20 = 10$. Feasible.

Step 5. Basis $\{x_2, s_1\}$**.** $x_1 = s_2 = 0$: $3x_2 = 30$, $x_2 = 10$; $s_1 = 40 - 20 = 20$. Feasible.

Step 6. Basis $\{x_2, s_2\}$**.** $x_1 = s_1 = 0$: $2x_2 = 40$, $x_2 = 20$; $s_2 = 30 - 60 = -30$. Infeasible.

Step 7. Basis $\{s_1, s_2\}$**.** $x_1 = x_2 = 0$: $s_1 = 40$, $s_2 = 30$. Feasible.

Step 8. Summary.

Basis	x_1	x_2	s_1	s_2	Feasible?	$z = 30x_1 + 20x_2$
$\{x_1, x_2\}$	7.5	5	0	0	yes	325
$\{x_1, s_1\}$	15	0	−20	0	no	
$\{x_1, s_2\}$	10	0	0	10	yes	300
$\{x_2, s_1\}$	0	10	20	0	yes	200
$\{x_2, s_2\}$	0	20	0	−30	no	
$\{s_1, s_2\}$	0	0	40	30	yes	0

Answer. Four BFS, matching the four vertices $(7.5, 5)$, $(10, 0)$, $(0, 10)$, $(0, 0)$; the infeasible basic solutions $(15, 0)$ and $(0, 20)$ are intersections of boundary lines lying outside the region. The best BFS is $\boxed{(7.5, 5) \text{ with } z = 325}$.

Worked Example 7.2 Computing a Basic Solution with $\mathbf{B}^{-1}$

Consider $x_1 + x_2 + s_1 = 6$, $x_1 - x_2 + s_2 = 2$, $x_2 + s_3 = 3$ with all variables nonnegative. Compute the basic solutions for the bases (a) $\{x_1, x_2, s_3\}$ and (b) $\{x_1, x_2, s_1\}$ using $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$.

Solution

Step 1. **Columns.** $\mathbf{A}_{x_1} = (1,1,0)^\top$, $\mathbf{A}_{x_2} = (1,-1,1)^\top$, $\mathbf{A}_{s_1} = (1,0,0)^\top$, $\mathbf{A}_{s_2} = (0,1,0)^\top$, $\mathbf{A}_{s_3} = (0,0,1)^\top$; $\mathbf{b} = (6,2,3)^\top$.

Step 2. **(a) Basis matrix.** $\mathbf{B} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & -1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$.

Step 3. **(a) Solve $\mathbf{Bx}_B = \mathbf{b}$** rather than forming the inverse explicitly (this is how software does it). Rows 1 and 2: $x_1 + x_2 = 6$, $x_1 - x_2 = 2$. Adding gives $2x_1 = 8$, $x_1 = 4$, and then $x_2 = 2$. Row 3: $x_2 + s_3 = 3$, so $s_3 = 1$.

Step 4. **(a) Verdict.** $\mathbf{x}_B = (4,2,1)^\top \geq \mathbf{0}$, and $s_1 = s_2 = 0$: a BFS, nondegenerate.

Step 5. **(b) Basis matrix.** $\mathbf{B} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$. Row 3 gives $x_2 = 3$; row 2 gives $x_1 - 3 = 2$, so $x_1 = 5$; row 1 gives $5 + 3 + s_1 = 6$, so $s_1 = -2$.

Step 6. **(b) Verdict.** $s_1 < 0$, so this basic solution is infeasible.

Step 7. **The inverse for (a)**, for completeness. One can check by multiplication that

$$\mathbf{B}^{-1} = \begin{pmatrix} 0.5 & 0.5 & 0 \\ 0.5 & -0.5 & 0 \\ -0.5 & 0.5 & 1 \end{pmatrix}, \qquad \mathbf{B}^{-1}\mathbf{b} = \begin{pmatrix} 3+1+0 \\ 3-1+0 \\ -3+1+3 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \\ 1 \end{pmatrix}.$$

Answer. (a) $\boxed{(x_1, x_2, s_3) = (4,2,1)}$, a BFS; (b) $(x_1, x_2, s_1) = (5, 3, -2)$, not feasible.

Worked Example 7.3 A Degenerate Vertex with Several Bases

For $x_1 + x_2 \leq 4$, $x_1 \leq 2$, $x_2 \leq 2$, $x_1, x_2 \geq 0$, show that the vertex $(2,2)$ corresponds to three different bases, and explain why it is degenerate.

Solution

Step 1. **Standard form.** $x_1 + x_2 + s_1 = 4$, $x_1 + s_2 = 2$, $x_2 + s_3 = 2$; $m = 3$, $n = 5$.

Step 2. **At** $(2,2)$: $s_1 = 4 - 4 = 0$, $s_2 = 0$, $s_3 = 0$. So the point is $(x_1, x_2, s_1, s_2, s_3) = (2,2,0,0,0)$, with only two positive entries although $m = 3$.

Step 3. **Basis** $\{x_1, x_2, s_1\}$ $(s_2 = s_3 = 0)$: $x_1 = 2$, $x_2 = 2$, $s_1 = 0$.

Step 4. **Basis** $\{x_1, x_2, s_2\}$ $(s_1 = s_3 = 0)$: $x_2 = 2$, $x_1 = 4 - 2 = 2$, $s_2 = 2 - 2 = 0$.

Step 5. **Basis** $\{x_1, x_2, s_3\}$ $(s_1 = s_2 = 0)$: $x_1 = 2$, $x_2 = 2$, $s_3 = 0$.

Step 6. **Degeneracy.** In every case one basic variable equals zero. Geometrically, three constraint lines pass through $(2,2)$ although only two are needed to fix a point in the plane. Degenerate vertices are where the simplex method can make a pivot without moving (Module 10).

Answer. The vertex $\boxed{(2,2)}$ is a degenerate BFS represented by three bases.

Worked Example 7.4 From a Feasible Point to a Better BFS

In the system of Worked Example 7.2, the point $\mathbf{x} = (x_1, x_2, s_1, s_2, s_3) = (3, 1, 2, 0, 2)$ is feasible. With the objective *maximise* $x_1 + 2x_2$ (equivalently, minimise $-x_1 - 2x_2$), use the support reduction procedure to reach a BFS that is at least as good.

Solution

Step 1. **Check feasibility.** $3 + 1 + 2 = 6$, $3 - 1 + 0 = 2$, $1 + 2 = 3$. All nonnegative. Objective $3 + 2 = 5$.

Step 2. **Support.** Positive components: x_1, x_2, s_1, s_3, four columns in $\mathbb{R}^3$, which must be dependent.

Step 3. **Direction.** Find $\mathbf{d} = (d_1, d_2, d_{s_1}, 0, d_{s_3})$ with $\mathbf{Ad} = \mathbf{0}$:

$$d_1 + d_2 + d_{s_1} = 0, \qquad d_1 - d_2 = 0, \qquad d_2 + d_{s_3} = 0.$$

Take $d_1 = d_2 = 1$; then $d_{s_1} = -2$ and $d_{s_3} = -1$. So $\mathbf{d} = (1, 1, -2, 0, -1)$.

Step 4. **Improving sign.** The change in the objective $x_1 + 2x_2$ along $\mathbf{d}$ is $1 + 2 = 3 > 0$, so moving along $+\mathbf{d}$ improves it.

Step 5. **Ratio test (7.5).** Negative components of $\mathbf{d}$: s_1 with ratio $2/2 = 1$ and s_3 with ratio $2/1 = 2$. Minimum $\theta^\star = 1$.

Step 6. **New point.** $\mathbf{x} + 1 \cdot \mathbf{d} = (4,\ 2,\ 0,\ 0,\ 1)$. Objective $4 + 4 = 8$.

Step 7. **Check it is basic.** Support $\{x_1, x_2, s_3\}$, three columns which are independent (they form the basis matrix of Worked Example 7.2(a)). So it is a BFS.

Answer. We reach the BFS $\boxed{(4, 2, 0, 0, 1)}$, improving the objective from 5 to 8.

Worked Example 7.5 How Many Bases Could There Be?

(a) Give the upper bound on the number of BFS for a standard form LP with $n = 10$ variables and $m = 4$ equations. (b) Estimate the bound for ℓ_1 regression with $N = 100$ data points and $p = 5$ features. (c) Explain what this means for algorithm design.

Solution

Step 1. **(a) Formula.** A basis is a choice of m columns from n, so the number of bases is at most $\binom{n}{m} = \dfrac{n!}{m!\,(n-m)!}$.

Step 2. **(a) Compute.** $\binom{10}{4} = \dfrac{10 \cdot 9 \cdot 8 \cdot 7}{4 \cdot 3 \cdot 2 \cdot 1} = \dfrac{5040}{24} = 210.$

Step 3. **(b) Size of the problem.** From Module 6, $m = N = 100$ and $n = 2p + 2N = 10 + 200 = 210$.

Step 4. **(b) Bound.** $\binom{210}{100} \approx 7 \times 10^{61}$. Even checking a billion bases per second would take about 2×10^{45} years.

Step 5. **(c) Consequence.** We need an algorithm that visits only a tiny, well chosen fraction of the BFS. The simplex method moves to an adjacent BFS that improves the objective at each step, and in practice usually needs a number of steps comparable to a small multiple of m.

Answer. (a) $\boxed{210}$; (b) about 7×10^{61}; (c) enumeration is hopeless, so we need the simplex method.

Worked Example 7.6 Moving Along an Edge to an Adjacent BFS

In the furniture workshop, start from the BFS with basis $\{s_1, s_2\}$ (the origin). Increase the nonbasic variable x_1 from zero while keeping $x_2 = 0$. Find how far x_1 can increase, the new BFS, and the change in profit.

Solution

Step 1. Current BFS. $x_1 = x_2 = 0$, $s_1 = 40$, $s_2 = 30$, $z = 0$.

Step 2. Parametrise. Let $x_1 = \theta \geq 0$ and $x_2 = 0$. The equations force the basic variables to adjust:

$$s_1 = 40 - 4\theta, \qquad s_2 = 30 - 2\theta.$$

Step 3. Ratio test. Nonnegativity needs $40 - 4\theta \geq 0$, i.e. $\theta \leq 10$, and $30 - 2\theta \geq 0$, i.e. $\theta \leq 15$. The binding limit is $\theta^\star = \min(10, 15) = 10$.

Step 4. New BFS. $x_1 = 10$, $s_1 = 0$, $s_2 = 10$. The variable s_1 has dropped to zero and leaves the basis; x_1 enters. The new basis is $\{x_1, s_2\}$, adjacent to the old one.

Step 5. Profit change. z rises by 30 per unit of x_1, so by $30 \times 10 = 300$.

Answer. x_1 can rise to $\boxed{10}$; the new BFS is $(x_1, x_2, s_1, s_2) = (10, 0, 0, 10)$ with $z = 300$. This single move is one iteration of the simplex method.

Module Summary

- A basis is a set of m independent columns; its basic solution is $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$, $\mathbf{x}_N = \mathbf{0}$.
- A basic solution is a BFS when $\mathbf{B}^{-1}\mathbf{b} \geq \mathbf{0}$; it is degenerate if some basic variable is zero.
- BFS are exactly the vertices of $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$; equivalently, a feasible point is basic when its positive components have independent columns.
- Fundamental Theorem: feasibility implies a BFS exists; optimality implies an optimal BFS exists. The proof moves along null space directions using the ratio test $\theta^\star = \min_{d_j<0} x_j/(-d_j)$.
- There are at most $\binom{n}{m}$ BFS: finite but astronomically many, so we move between adjacent BFS instead.
- Vertex solutions are sparse, which explains sparsity in ℓ_1 regression, basis pursuit and optimal transport.

Part II　Algorithms

MODULE 8

The Simplex Method

Learning Objectives

After studying this module you will be able to:

- derive, from first principles, how the objective changes when a nonbasic variable is increased, and define the reduced cost;
- state and justify the optimality test, the entering rule, the ratio test and the pivot operation;
- set up an initial simplex tableau for a problem with $\le$ constraints and nonnegative right hand sides;
- carry out the simplex method by hand, recording each tableau, ratio and pivot;
- read the optimal solution, the optimal value, the slack values and the shadow prices from the final tableau;
- explain why the method terminates on nondegenerate problems.

8.1 The Idea in One Paragraph

The simplex method, invented by George Dantzig in 1947, starts at a vertex of the feasible region (a basic feasible solution) and repeatedly moves along an edge to an adjacent vertex with a better objective value. When no edge leads uphill, the current vertex is optimal, because for a linear program a local optimum is global (Module 3). Algebraically, each move exchanges one basic variable for one nonbasic variable and is carried out by a single Gaussian elimination pivot. In this module we derive every rule of the method and then apply it, step by step, to six problems.

Throughout the module we treat problems in canonical form with a nonnegative right hand side:

$$\max\ \mathbf{c}^\top\mathbf{x} \qquad \text{subject to} \qquad \mathbf{A}\mathbf{x} \le \mathbf{b}, \qquad \mathbf{x} \ge \mathbf{0}, \qquad \mathbf{b} \ge \mathbf{0}. \tag{8.1}$$

Adding one slack variable per constraint gives $\mathbf{A}\mathbf{x} + \mathbf{s} = \mathbf{b}$, and setting $\mathbf{x} = \mathbf{0}$, $\mathbf{s} = \mathbf{b}$ gives an immediate starting BFS (the origin), feasible precisely because $\mathbf{b} \ge \mathbf{0}$. Problems without such an obvious start are handled in Module 9.

8.2 Deriving the Rules

Derivation The objective in terms of the nonbasic variables

Let all variables (original and slack) be collected in $\mathbf{x} \in \mathbb{R}^n$ with constraint matrix $\mathbf{A}$ (now including the slack columns) and costs $\mathbf{c}$ (zero for slacks). Fix a basis with matrix $\mathbf{B}$ and nonbasic matrix $\mathbf{N}$.

Step 1. From Module 7, the constraints read $\mathbf{B}\mathbf{x}_B + \mathbf{N}\mathbf{x}_N = \mathbf{b}$.

Step 2. Multiply on the left by $\mathbf{B}^{-1}$ and rearrange to express the basic variables through the nonbasic ones:

$$\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b} - \mathbf{B}^{-1}\mathbf{N}\mathbf{x}_N. \tag{8.2}$$

Step 3. Split the objective into basic and nonbasic parts: $z = \mathbf{c}_B^\top \mathbf{x}_B + \mathbf{c}_N^\top \mathbf{x}_N$.

Step 4. Substitute (8.2):

$$z = \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{b} - \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{N}\mathbf{x}_N + \mathbf{c}_N^\top \mathbf{x}_N.$$

Step 5. Collect the terms in $\mathbf{x}_N$:

$$z = \underbrace{\mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{b}}_{\text{current value}} + \sum_{j \in N} \underbrace{\left(c_j - \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{A}_j\right)}_{\bar{c}_j} x_j. \tag{8.3}$$

where

$\mathbf{c}_B, \mathbf{c}_N$ are the costs of the basic and nonbasic variables;

N is the set of nonbasic indices and $\mathbf{A}_j$ is column j of $\mathbf{A}$;

$\bar{c}_j$ is the **reduced cost** of x_j: the rate at which z changes when x_j is increased from 0 while the other nonbasic variables stay at 0 and the basic variables adjust to keep the equations satisfied.

Key Result Optimality test (maximisation)

At the current BFS every nonbasic variable equals zero and may only increase. By (8.3):

- if $\bar{c}_j \leq 0$ for every nonbasic j, no increase can raise z, so the BFS is **optimal**;
- if some $\bar{c}_j > 0$, increasing x_j raises z, so the BFS can be improved.

Derivation The ratio test: how far can the entering variable go?

Suppose x_q is chosen to enter, with $\bar{c}_q > 0$. Increase it to $\theta \geq 0$, keeping the other nonbasic variables at zero.

Step 1. By (8.2), the basic variables become $\mathbf{x}_B(\theta) = \bar{\mathbf{b}} - \theta\,\bar{\mathbf{a}}_q$, where $\bar{\mathbf{b}} = \mathbf{B}^{-1}\mathbf{b}$ (the current right hand side column) and $\bar{\mathbf{a}}_q = \mathbf{B}^{-1}\mathbf{A}_q$ (the current column of x_q in the tableau).

Step 2. Row by row: $x_{B(i)}(\theta) = \bar{b}_i - \theta \bar{a}_{iq}$. Feasibility needs this to stay nonnegative.

Step 3. If $\bar{a}_{iq} \leq 0$, the row places no limit on θ (the basic variable stays equal or grows).

Step 4. If $\bar{a}_{iq} > 0$, the row requires $\theta \leq \bar{b}_i / \bar{a}_{iq}$.

Step 5. The largest feasible step is therefore

$$\theta^\star = \min_{i\,:\,\bar{a}_{iq}>0} \frac{\bar{b}_i}{\bar{a}_{iq}}, \tag{8.4}$$

and the row r attaining the minimum identifies the **leaving variable** $x_{B(r)}$, which drops to zero.

Step 6. If no $\bar{a}_{iq}$ is positive, θ can grow forever while z rises at rate $\bar{c}_q > 0$: the problem is **unbounded**.

Step 7. The objective increases by $\bar{c}_q\theta^\star \geq 0$.

Derivation The pivot updates everything at once

After choosing the entering column q and the leaving row r, the new basis replaces $x_{B(r)}$ by x_q. We need the tableau expressed in terms of the new basis, which means making column q a unit column with a 1 in row r.

Step 1. **Normalise the pivot row:** divide row r by the pivot element $\bar{a}_{rq} > 0$, so the entry becomes 1.

Step 2. **Eliminate elsewhere:** for every other row i (including the objective row), subtract $\bar{a}_{iq}$ times the new pivot row, so that the entry in column q becomes 0.

Step 3. These are elementary row operations, so the new tableau describes the same equations (Module 2), now solved for the new basic variables.

8.3 The Simplex Tableau

We record the problem in a table called the **tableau**. The objective row (row 0) stores the equation $z - \mathbf{c}^\top\mathbf{x} = 0$, so its entries are the *negatives* of the reduced costs.

Key Result Tableau conventions used in this book

- Row 0 holds the coefficients of $z - \sum_j c_j x_j = 0$, initially $-c_j$ under each x_j and 0 under each slack. Its right hand side is the current objective value.
- Rows 1 to m hold the constraints, each labelled by its basic variable. The RHS column holds the current values of the basic variables.
- Each basic variable has a unit column: 1 in its own row and 0 elsewhere, including row 0.
- **Optimal** when every entry of row 0 is ≥ 0 (that is, every $\bar{c}_j \leq 0$).
- **Entering variable**: the most negative entry in row 0 (Dantzig's rule).
- **Leaving variable**: the minimum ratio RHS / entry over rows with a positive entry in the entering column.

Key Result The simplex algorithm (tableau form, maximisation)

1. Write the problem with slacks and form the initial tableau with the slacks as the basis.
2. **Optimality test.** If all row 0 entries are ≥ 0, stop: the tableau is optimal.
3. **Entering variable.** Choose the column q with the most negative row 0 entry.
4. **Ratio test.** For rows with a positive entry in column q, compute RHS divided by that entry. If no entry is positive, stop: the problem is unbounded. Otherwise the row r with the smallest ratio gives the leaving variable.
5. **Pivot** on the entry in row r, column q. Relabel row r with the entering variable. Return to step 2.

Key Result Reading the final tableau

- Basic variables equal their RHS values; nonbasic variables equal zero.
- The optimal value is the RHS of row 0.
- The row 0 entry under slack s_i is the **shadow price** y_i of constraint i: the increase in the optimal value per unit increase in b_i (derived in Modules 12 and 15).
- The row 0 entry under a nonbasic original variable x_j is $-\bar{c}_j$: the loss in objective per unit of x_j forced into the solution.

Derivation Why the method terminates

Step 1. At each pivot the objective rises by $\bar{c}_q\theta^\star$. If the BFS is nondegenerate, every $\bar{b}_i > 0$, so $\theta^\star > 0$ and the objective rises *strictly*.

Step 2. A strictly rising objective means no BFS can be visited twice.

Step 3. There are finitely many BFS (at most $\binom{n}{m}$, Module 7), so the method must stop after finitely many pivots, either at an optimal BFS or by detecting unboundedness.

When degeneracy occurs, $\theta^\star$ may be 0 and the method could in principle cycle; Module 10 presents Bland's rule, which prevents this.

Lucid Insight What a pivot means geometrically

Choosing the entering variable chooses an edge leaving the current corner along which the objective improves. The ratio test walks along that edge until the first constraint becomes active, which is exactly the next corner. The pivot then rewrites the equations from the viewpoint of the new corner.

Machine Learning Connection

The simplex method remains one of the two workhorse algorithms inside commercial and open source LP solvers (the other is the interior point method of Module 16). In machine learning pipelines it is especially valuable when problems are solved again many times with small changes, for example refitting ℓ_1 regression as new data arrive, or updating an optimal transport plan between mini batches, because the final basis of one solve is an excellent "warm start" for the next. Reduced costs and shadow prices, which the tableau delivers for free, also tell an engineer which resource (GPU hours, memory, annotation budget) is truly limiting performance.

8.4 Worked Examples

Worked Example 8.1 The Furniture Workshop by the Simplex Method

Solve max $z = 30x_1 + 20x_2$ subject to $4x_1 + 2x_2 \le 40$, $2x_1 + 3x_2 \le 30$, $x_1, x_2 \ge 0$ by the simplex method, and read off the shadow prices.

Solution

Step 1. **Add slacks.** $4x_1 + 2x_2 + s_1 = 40$, $2x_1 + 3x_2 + s_2 = 30$. Row 0: $z - 30x_1 - 20x_2 = 0$. Initial basis $\{s_1, s_2\}$.

Step 2. **Tableau 0.** The most negative entry in row 0 is -30, so x_1 enters. Ratios: $40/4 = 10$ and $30/2 = 15$; the minimum is in the s_1 row, so s_1 leaves and the pivot is 4.

Basic	x_1	x_2	s_1	s_2	RHS	Ratio
z	-30	-20	0	0	0	
s_1	$\boxed{4}$	2	1	0	40	$40/4 = 10$
s_2	2	3	0	1	30	$30/2 = 15$

Step 3. **Pivot.** New row 1 = old row 1 ÷4: $(1,\ 0.5,\ 0.25,\ 0 \mid 10)$. Row 2 − 2×new row 1: $(0,\ 2,\ -0.5,\ 1 \mid 10)$. Row 0 + 30×new row 1: $(0,\ -5,\ 7.5,\ 0 \mid 300)$.

Step 4. **Tableau 1.** Row 0 still has -5 under x_2, so x_2 enters. Ratios: $10/0.5 = 20$ and $10/2 = 5$; s_2 leaves; pivot 2.

Basic	x_1	x_2	s_1	s_2	RHS	Ratio
z	0	-5	7.5	0	300	
x_1	1	0.5	0.25	0	10	$10/0.5 = 20$
s_2	0	$\boxed{2}$	-0.5	1	10	$10/2 = 5$

Step 5. **Pivot.** New row 2 = row 2 ÷2: $(0, 1, -0.25, 0.5 \mid 5)$. Row 1 − 0.5×new row 2: $(1, 0, 0.375, -0.25 \mid 7.5)$. Row 0 + 5×new row 2: $(0, 0, 6.25, 2.5 \mid 325)$.

Step 6. **Tableau 2.** All row 0 entries are nonnegative, so this tableau is optimal.

Basic	x_1	x_2	s_1	s_2	RHS
z	0	0	6.25	2.5	325
x_1	1	0	0.375	−0.25	7.5
x_2	0	1	−0.25	0.5	5

Step 7. **Read the answer.** $x_1 = 7.5$, $x_2 = 5$, $s_1 = s_2 = 0$, $z = 325$. The row 0 entries under s_1 and s_2 are the shadow prices 6.25 and 2.5, exactly the multipliers found graphically in Worked Example 5.1.

Answer. $\boxed{x_1^\star = 7.5,\ x_2^\star = 5,\ z^\star = 325}$ after two pivots; shadow prices 6.25 (timber) and 2.5 (labour).

Worked Example 8.2 A Variable That Enters and Later Leaves

Solve max $z = 3x_1 + 2x_2 + 4x_3$ subject to $x_1 + x_2 + 2x_3 \le 4$, $2x_1 + 3x_3 \le 5$, $2x_1 + x_2 + 3x_3 \le 7$, $x_1, x_2, x_3 \ge 0$.

Solution

Step 1. **Tableau 0.** The most negative row 0 entry is −4, so x_3 enters. Ratios $4/2 = 2$, $5/3 \approx 1.667$, $7/3 \approx 2.333$. The minimum is $5/3$, in the s_2 row; pivot 3.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	−3	−2	−4	0	0	0	0	
s_1	1	1	2	1	0	0	4	$4/2 = 2$
s_2	2	0	$\boxed{3}$	0	1	0	5	$5/3 = \frac{5}{3}$
s_3	2	1	3	0	0	1	7	$7/3 = \frac{7}{3}$

Step 2. **Pivot.** Divide row 2 by 3: $(\frac{2}{3}, 0, 1, 0, \frac{1}{3}, 0 \mid \frac{5}{3})$. Row 1 minus 2× it, row 3 minus 3× it, row 0 plus 4× it.

Step 3. **Tableau 1.** The most negative entry is −2 under x_2. Ratios: row s_1: $\frac{2}{3}/1 = \frac{2}{3}$; row x_3: entry 0, no ratio; row s_3: $2/1 = 2$. So s_1 leaves; pivot 1.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	$-\frac{1}{3}$	−2	0	0	$\frac{4}{3}$	0	$\frac{20}{3}$	
s_1	$-\frac{1}{3}$	$\boxed{1}$	0	1	$-\frac{2}{3}$	0	$\frac{2}{3}$	$\frac{2}{3}/1 = \frac{2}{3}$
x_3	$\frac{2}{3}$	0	1	0	$\frac{1}{3}$	0	$\frac{5}{3}$	none
s_3	0	1	0	0	−1	1	2	$2/1 = 2$

Step 4. **Tableau 2.** Now x_1 has row 0 entry −1 and enters. Ratios: row x_2 has a negative entry $(-\frac{1}{3})$, no ratio; row x_3: $\frac{5}{3} \div \frac{2}{3} = 2.5$; row s_3: $\frac{4}{3} \div \frac{1}{3} = 4$. So x_3 *leaves*, although it entered earlier;

pivot $\frac{2}{3}$.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	-1	0	0	2	0	0	8	
x_2	$-\frac{1}{3}$	1	0	1	$-\frac{2}{3}$	0	$\frac{2}{3}$	none
x_3	$\boxed{\frac{2}{3}}$	0	1	0	$\frac{1}{3}$	0	$\frac{5}{3}$	$\frac{5}{3}/\frac{2}{3} = 2.5$
s_3	$\frac{1}{3}$	0	0	-1	$-\frac{1}{3}$	1	$\frac{4}{3}$	$\frac{4}{3}/\frac{1}{3} = 4$

Step 5. **Tableau 3.** All row 0 entries are nonnegative: optimal.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS
z	0	0	1.5	2	0.5	0	10.5
x_2	0	1	0.5	1	-0.5	0	1.5
x_1	1	0	1.5	0	0.5	0	2.5
s_3	0	0	-0.5	-1	-0.5	1	0.5

Step 6. **Check.** Constraint 1: $2.5+1.5+0 = 4$. Constraint 2: $5+0 = 5$. Constraint 3: $5+1.5 = 6.5 \le 7$ (slack $s_3 = 0.5$, as in the tableau). Objective $3(2.5) + 2(1.5) = 7.5 + 3 = 10.5$.

Step 7. **Lesson.** A variable that looks attractive at first (x_3 has the largest profit) need not be in the final solution. The simplex method corrects early choices automatically.

Answer. $\boxed{x_1^\star = 2.5,\ x_2^\star = 1.5,\ x_3^\star = 0,\ z^\star = 10.5}$.

Worked Example 8.3 Allocating Training Runs on a Shared Cluster

A research group trains three model families. One run of family j earns a "research value" of 5, 4 and 3 points for $j = 1, 2, 3$. Runs consume CPU hours $(2, 3, 1)$, GPU hours $(4, 1, 2)$ and storage terabytes $(3, 4, 2)$, with 5 CPU hours, 11 GPU hours and 8 terabytes available (in suitable units). Fractional runs are allowed (they represent shares of a long schedule). Maximise total research value.

Solution

Step 1. **Model.** max $5x_1+4x_2+3x_3$ subject to $2x_1+3x_2+x_3 \le 5$, $4x_1+x_2+2x_3 \le 11$, $3x_1+4x_2+2x_3 \le 8$, $\mathbf{x} \ge \mathbf{0}$.

Step 2. **Tableau 0.** x_1 enters (-5). Ratios 2.5, 2.75, $8/3 \approx 2.667$; s_1 leaves; pivot 2.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	−5	−4	−3	0	0	0	0	
s_1	$\boxed{2}$	3	1	1	0	0	5	5/2 = 2.5
s_2	4	1	2	0	1	0	11	11/4 = 2.75
s_3	3	4	2	0	0	1	8	$8/3 = \frac{8}{3}$

Step 3. **Tableau 1.** Only x_3 has a negative row 0 entry (-0.5), so it enters. Ratios: row x_1: $2.5/0.5 = 5$; row s_2: entry 0, none; row s_3: $0.5/0.5 = 1$. So s_3 leaves; pivot 0.5.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	0	3.5	−0.5	2.5	0	0	12.5	
x_1	1	1.5	0.5	0.5	0	0	2.5	2.5/0.5 = 5
s_2	0	−5	0	−2	1	0	1	none
s_3	0	−0.5	$\boxed{0.5}$	−1.5	0	1	0.5	0.5/0.5 = 1

Step 4. **Tableau 2.** All row 0 entries are nonnegative: optimal.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS
z	0	3	0	1	0	1	13
x_1	1	2	0	2	0	−1	2
s_2	0	−5	0	−2	1	0	1
x_3	0	−1	1	−3	0	2	1

Step 5. **Interpretation.** $x_1 = 2$, $x_3 = 1$, $x_2 = 0$, value 13. CPU (shadow price 1) and storage (shadow price 1) are bottlenecks; GPU has slack $s_2 = 1$, so its shadow price is 0. The entry 3 under x_2 says forcing one run of family 2 would *lose* 3 points of value.

Answer. $\boxed{x^\star = (2, 0, 1),\ z^\star = 13}$; buying more GPU time is pointless, but extra CPU or storage would each add about 1 point per unit.

Worked Example 8.4 Optimal after One Pivot, and the Meaning of Reduced Costs

Solve max $z = 2x_1 + 3x_2 + 4x_3$ subject to $3x_1 + 2x_2 + x_3 \le 10$, $2x_1 + 5x_2 + 3x_3 \le 15$, $\mathbf{x} \ge \mathbf{0}$. Interpret every row 0 entry of the final tableau.

Solution

Step 1. **Tableau 0.** x_3 enters (row 0 entry −4). Ratios $10/1 = 10$ and $15/3 = 5$; s_2 leaves; pivot 3.

Basic	x_1	x_2	x_3	s_1	s_2	RHS	Ratio
z	-2	-3	-4	0	0	0	
s_1	3	2	1	1	0	10	$10/1 = 10$
s_2	2	5	$\boxed{3}$	0	1	15	$15/3 = 5$

Step 2. **Pivot.** Row 2 $\div 3$: $(\frac{2}{3}, \frac{5}{3}, 1, 0, \frac{1}{3} \mid 5)$. Row 1 $-$ it: $(\frac{7}{3}, \frac{1}{3}, 0, 1, -\frac{1}{3} \mid 5)$. Row 0 $+ 4\times$ it: $(-2 + \frac{8}{3}, -3 + \frac{20}{3}, 0, 0, \frac{4}{3} \mid 20) = (\frac{2}{3}, \frac{11}{3}, 0, 0, \frac{4}{3} \mid 20)$.

Basic	x_1	x_2	x_3	s_1	s_2	RHS
z	$\frac{2}{3}$	$\frac{11}{3}$	0	0	$\frac{4}{3}$	20
s_1	$\frac{7}{3}$	$\frac{1}{3}$	0	1	$-\frac{1}{3}$	5
x_3	$\frac{2}{3}$	$\frac{5}{3}$	1	0	$\frac{1}{3}$	5

Step 3. **Optimal**, since every row 0 entry is nonnegative. Solution $x_3 = 5$, $s_1 = 5$, others zero; $z = 20$.

Step 4. **Reduced cost of x_1.** The entry $\frac{2}{3}$ means $\bar{c}_1 = -\frac{2}{3}$. Check with (8.3): $\bar{c}_1 = c_1 - \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{A}_1$. Here the only priced resource is constraint 2 with shadow price $\frac{4}{3}$, so $\mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{A}_1 = \frac{4}{3} \times 2 = \frac{8}{3}$ and $\bar{c}_1 = 2 - \frac{8}{3} = -\frac{2}{3}$. Making one unit of x_1 earns 2 but uses 2 units of the scarce resource worth $\frac{8}{3}$: a net loss of $\frac{2}{3}$.

Step 5. **Reduced cost of x_2.** $\bar{c}_2 = 3 - \frac{4}{3} \times 5 = 3 - \frac{20}{3} = -\frac{11}{3}$, matching the entry $\frac{11}{3}$.

Step 6. **Shadow prices.** 0 for constraint 1 (it has slack 5) and $\frac{4}{3}$ for constraint 2 (one extra unit lets x_3 rise by $\frac{1}{3}$, worth $4 \times \frac{1}{3} = \frac{4}{3}$).

Answer. $\boxed{x^\star = (0, 0, 5),\ z^\star = 20}$; reduced costs $-\frac{2}{3}$ and $-\frac{11}{3}$ explain why x_1 and x_2 stay at zero.

Worked Example 8.5 A Negative Objective Coefficient and a Minimisation in Disguise

Solve min $w = -x_1 - 2x_2 + x_3$ subject to $2x_1 + x_2 + x_3 \le 14$, $x_1 + x_2 \le 8$, $x_2 + x_3 \le 6$, $\mathbf{x} \ge \mathbf{0}$.

Solution

Step 1. **Convert to maximisation** (Module 6): max $z = -w = x_1 + 2x_2 - x_3$. Row 0 is $z - x_1 - 2x_2 + x_3 = 0$, so the entry under x_3 is $+1$.

Step 2. **Tableau 0.** x_2 enters (-2). Ratios 14, 8, 6; s_3 leaves; pivot 1.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	-1	-2	1	0	0	0	0	
s_1	2	1	1	1	0	0	14	$14/1 = 14$
s_2	1	1	0	0	1	0	8	$8/1 = 8$
s_3	0	$\boxed{1}$	1	0	0	1	6	$6/1 = 6$

Step 3. **Tableau 1.** x_1 enters (-1). Ratios: row s_1: $8/2 = 4$; row s_2: $2/1 = 2$; row x_2: entry 0, none. So s_2 leaves; pivot 1.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	-1	0	3	0	0	2	12	
s_1	2	0	0	1	0	-1	8	$8/2 = 4$
s_2	$\boxed{1}$	0	-1	0	1	-1	2	$2/1 = 2$
x_2	0	1	1	0	0	1	6	none

Step 4. **Tableau 2.** All entries nonnegative: optimal.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS
z	0	0	2	0	1	1	14
s_1	0	0	2	1	-2	1	4
x_1	1	0	-1	0	1	-1	2
x_2	0	1	1	0	0	1	6

Step 5. **Answer in the original sense.** $x_1 = 2$, $x_2 = 6$, $x_3 = 0$, $z = 14$, hence $w = -14$. Variable x_3, which only adds cost, correctly stays at zero (its row 0 entry is 2).

Answer. $\boxed{x^\star = (2, 6, 0),\ w^\star = -14}$.

Worked Example 8.6 An Annotation Company's Product Mix

An annotation company sells three labelled data products: image boxes, text spans and audio transcripts, earning £6, £5 and £4 (hundreds) per batch. Each batch needs hours from three teams as follows: team A (2, 1, 1), team B (1, 3, 2), team C (2, 1, 2), with capacities 180, 300 and 240 hours. Find the best weekly mix, and say which team the company should expand.

Solution

Step 1. **Model.** max $6x_1 + 5x_2 + 4x_3$ subject to $2x_1 + x_2 + x_3 \le 180$, $x_1 + 3x_2 + 2x_3 \le 300$, $2x_1 + x_2 + 2x_3 \le 240$, $\mathbf{x} \ge \mathbf{0}$.

Step 2. **Tableau 0.** x_1 enters. Ratios 90, 300, 120; s_1 leaves; pivot 2.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	-6	-5	-4	0	0	0	0	
s_1	$\boxed{2}$	1	1	1	0	0	180	$180/2 = 90$
s_2	1	3	2	0	1	0	300	$300/1 = 300$
s_3	2	1	2	0	0	1	240	$240/2 = 120$

Step 3. **Tableau 1.** x_2 enters (-2). Ratios: row x_1: $90/0.5 = 180$; row s_2: $210/2.5 = 84$; row s_3: entry 0, none. So s_2 leaves; pivot 2.5.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS	Ratio
z	0	-2	-1	3	0	0	540	
x_1	1	0.5	0.5	0.5	0	0	90	$90/0.5 = 180$
s_2	0	$\boxed{2.5}$	1.5	-0.5	1	0	210	$210/2.5 = 84$
s_3	0	0	1	-1	0	1	60	none

Step 4. **Tableau 2.** All row 0 entries are nonnegative: optimal.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS
z	0	0	0.2	2.6	0.8	0	708
x_1	1	0	0.2	0.6	-0.2	0	48
x_2	0	1	0.6	-0.2	0.4	0	84
s_3	0	0	1	-1	0	1	60

Step 5. **Check.** Team A: $2(48) + 84 = 180$. Team B: $48 + 252 = 300$. Team C: $96 + 84 = 180 \le 240$ (slack 60). Profit $6(48) + 5(84) = 288 + 420 = 708$.

Step 6. **Which team to expand?** Shadow prices: team A 2.6, team B 0.8, team C 0. An extra hour in team A adds about £260 (since profits are in hundreds), team B about £80, team C nothing.

Answer. $\boxed{x^\star = (48, 84, 0),\ z^\star = 708}$ (£70,800); expand team A first.

Module Summary

- In terms of the nonbasic variables, $z = \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{b} + \sum_{j\in N} \bar{c}_j x_j$ with reduced cost $\bar{c}_j = c_j - \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{A}_j$.
- A BFS is optimal for maximisation when all reduced costs are ≤ 0, which in the tableau means all row 0 entries are ≥ 0.
- Enter the variable with the most negative row 0 entry; leave by the minimum ratio $\bar{b}_i/\bar{a}_{iq}$ over positive $\bar{a}_{iq}$; no positive entry means unbounded.

- A pivot is a Gaussian elimination step that makes the entering column a unit column.
- The final row 0 gives the optimal value, the shadow prices (under the slacks) and the losses from forcing nonbasic variables (under the originals).
- Without degeneracy the objective rises strictly at each pivot, so the method terminates.

MODULE 9

Artificial Variables: The Big M and Two Phase Methods

Learning Objectives

After studying this module you will be able to:

- explain why problems with ≥ or = constraints have no obvious starting basic feasible solution;
- introduce artificial variables and explain why they must be driven to zero;
- derive and apply the Big M method, including the pricing out of artificial variables in row 0;
- derive and apply the two phase method, and interpret the Phase I optimal value;
- detect infeasibility with either method;
- choose between the two methods for hand calculation and for software.

9.1 The Starting Point Problem

In Module 8 every constraint had the form $\mathbf{a}_i^\top \mathbf{x} \le b_i$ with $b_i \ge 0$, so the slack variables gave a ready made basis and the origin was a basic feasible solution. As soon as a model contains a requirement such as "at least 24 grams of protein" (≥) or a balance condition such as "total production equals 18" (=), that convenience disappears.

Derivation Why surplus variables cannot start the method

Consider $3x_1 + 4x_2 \ge 24$. In standard form (Module 6) it becomes $3x_1 + 4x_2 - e_1 = 24$ with surplus $e_1 \ge 0$.

Step 1. To use e_1 as the basic variable of this row we would set $x_1 = x_2 = 0$, which gives $-e_1 = 24$, that is $e_1 = -24$.

Step 2. This violates $e_1 \ge 0$, so the basic solution is infeasible.

Step 3. The coefficient of e_1 is -1, not $+1$, so its column is not a unit column with a positive entry, and no rescaling helps without making the right hand side negative.

An equation constraint $3x_1 + 2x_2 = 18$ is even worse: it has no slack or surplus at all.

9.2 Artificial Variables

Definition Artificial Variable

For each constraint of type $\geq$ or $=$ (after making $b_i \geq 0$), add a new nonnegative variable a_i with coefficient $+1$:

$$\mathbf{a}_i^\top \mathbf{x} - e_i + a_i = b_i \quad (\text{for } \geq), \qquad \mathbf{a}_i^\top \mathbf{x} + a_i = b_i \quad (\text{for } =). \tag{9.1}$$

where

a_i is an **artificial variable**. It has no meaning in the real problem; it is scaffolding that supplies a unit column, so that {slacks of $\leq$ rows} ∪ {artificials} forms a starting basis with values $\mathbf{b} \geq \mathbf{0}$;

e_i is the surplus variable of a $\geq$ row.

Key Result The central fact about artificial variables

A point of the enlarged problem with every $a_i = 0$ is a feasible point of the original problem, and conversely. The original problem is feasible *if and only if* the enlarged problem has a feasible solution with all artificial variables equal to zero. Both methods in this module are ways of forcing the artificial variables out.

9.3 The Big M Method

Derivation The penalised objective

Let M denote a symbolic positive number, larger than any number it will ever be compared with.

Step 1. For a **maximisation** problem, subtract M times each artificial variable from the objective:

$$\max \; z = \mathbf{c}^\top \mathbf{x} - M \sum_{i \in \mathcal{A}} a_i, \tag{9.2}$$

where $\mathcal{A}$ is the set of rows that received an artificial variable. For a minimisation problem, convert it to maximisation first (Module 6), or equivalently add $+M \sum a_i$ to the cost.

Step 2. Every unit of an artificial variable now costs an enormous penalty, so any solution with a positive artificial variable is worse than any solution with all artificials at zero, provided such a solution exists.

Step 3. Row 0 initially reads $z - \mathbf{c}^\top \mathbf{x} + M \sum_i a_i = 0$, so the entry under each a_i is $+M$.

Derivation Pricing out the artificial variables

The artificial variables are basic, so their row 0 entries must be zero (a basic column must be a unit column, Module 8). They are $+M$, so we eliminate them.

Step 1. For each row i whose basic variable is a_i, subtract M times row i from row 0.

Step 2. Under any column j this changes the row 0 entry from $-c_j$ to $-c_j - M \sum_{i \in \mathcal{A}} a_{ij}$, and the right hand side from 0 to $-M \sum_{i \in \mathcal{A}} b_i$.

Step 3. Every row 0 entry is now of the form $\alpha M + \beta$. Compare two such entries by their M coefficients first, and use the constant part only to break ties, because M dominates every ordinary number.

Key Result Outcomes of the Big M method

1. The final tableau has all artificials at zero: its solution is optimal for the original problem.
2. The final tableau is optimal but some artificial variable is positive: the original problem is **infeasible**.
3. The method detects unboundedness with all artificials at zero: the original problem is unbounded.

Common Pitfall

In software one cannot use a symbolic M, and choosing a numerical value is dangerous. If M is too small the penalty may not dominate and a wrong answer results; if M is very large it swamps the genuine costs and causes rounding errors. This is why solvers use the two phase approach (or more elaborate variants) instead.

9.4 The Two Phase Method

Derivation Phase I: find a feasible basis

Step 1. Ignore the real objective. Solve the auxiliary problem

$$\min \sum_{i\in\mathcal{A}} a_i \qquad \text{subject to the constraints (9.1) and all variables } \geq 0. \tag{9.3}$$

In tableau form we maximise $w = -\sum_i a_i$, so row 0 reads $w + \sum_i a_i = 0$.

Step 2. This problem is always feasible (start from the artificial basis) and bounded below by zero, so it has an optimal solution.

Step 3. Price out the artificial variables exactly as in Big M, but now with coefficient 1 instead of M: subtract each artificial row from row 0.

Step 4. Run the simplex method of Module 8.

Step 5. **If the optimum has** $\sum a_i > 0$ (that is, $w^\star < 0$), no feasible point of the original problem exists, by the central fact above: the original problem is infeasible.

Step 6. **If the optimum has** $\sum a_i = 0$, the final basis gives a basic feasible solution of the original problem.

Derivation Phase II: optimise from the feasible basis

Step 1. Delete the artificial columns from the final Phase I tableau. (If an artificial variable is still basic at value zero, pivot it out on any nonzero entry in its row under a non artificial column;

if there is none, the row is redundant and can be deleted.)

Step 2. Write the original objective in row 0: entries $-c_j$ under each x_j, zero under slacks and surpluses.

Step 3. Restore the unit column property for the current basic variables: for each basic x_j with a nonzero row 0 entry, subtract that entry times its row from row 0.

Step 4. Run the simplex method of Module 8 to optimality.

Lucid Insight Two phases, one geometry

Phase I wanders through an enlarged space, using artificial coordinates, until it lands on a vertex of the true feasible region. Phase II then behaves exactly like Module 8, walking uphill from that vertex. The Big M method merges the two phases into a single walk by making the artificial coordinates so expensive that the walk leaves them as quickly as possible.

Machine Learning Connection

Phase I is a feasibility problem, and feasibility questions are common in machine learning practice: “Is there any labelling budget allocation meeting all quality and coverage requirements?”, “Is there a classifier that separates these training points with margin at least one?” (Module 21), “Is there a transport plan with these marginals and this sparsity pattern?” When Phase I ends with a positive value, it tells the engineer not only that the requirements conflict but, through the final artificial values, by how much. Modern solvers report this as an “infeasibility” status, and their presolve and crossover stages use the same artificial variable idea.

9.5 Worked Examples

Worked Example 9.1 The Minimum Cost Diet by the Big M Method

Solve min $2x_1 + 3x_2$ subject to $3x_1 + 4x_2 \geq 24$, $x_1 + 3x_2 \geq 12$, $x_1, x_2 \geq 0$ by the Big M method.

Solution

Step 1. Convert to maximisation. max $z = -2x_1 - 3x_2 - Ma_1 - Ma_2$.

Step 2. Constraints with surplus and artificial variables.

$$3x_1 + 4x_2 - e_1 + a_1 = 24, \qquad x_1 + 3x_2 - e_2 + a_2 = 12.$$

Step 3. Initial tableau (before pricing out). Row 0 reads $z + 2x_1 + 3x_2 + Ma_1 + Ma_2 = 0$.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	RHS
z	2	3	0	M	0	M	0
a_1	3	4	−1	1	0	0	24
a_2	1	3	0	0	−1	1	12

Step 4. Price out. Subtract $M\times$ row a_1 and $M\times$ row a_2 from row 0. Under x_1: $2-3M-M=-4M+2$. Under x_2: $3-4M-3M=-7M+3$. Under e_1 and e_2: $0-M(-1)=M$. RHS: $-M(24+12)=-36M$.

Step 5. Tableau 1. The most negative entry is $-7M+3$ (largest M coefficient), so x_2 enters. Ratios 6 and 4; a_2 leaves.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	RHS	Ratio
z	$-4M+2$	$-7M+3$	M	0	M	0	$-36M$	
a_1	3	4	-1	1	0	0	24	$24/4=6$
a_2	1	$\boxed{3}$	0	0	-1	1	12	$12/3=4$

Step 6. Tableau 2. After the pivot, x_1 has the most negative entry and enters; the ratio test selects the a_1 row.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	RHS	Ratio
z	$-\frac{5}{3}M+1$	0	M	0	$-\frac{4}{3}M+1$	$\frac{7}{3}M-1$	$-8M-12$	
a_1	$\boxed{\frac{5}{3}}$	0	-1	1	$\frac{4}{3}$	$-\frac{4}{3}$	8	$8/\frac{5}{3}=4.8$
x_2	$\frac{1}{3}$	1	0	0	$-\frac{1}{3}$	$\frac{1}{3}$	4	$4/\frac{1}{3}=12$

Step 7. Tableau 3. No row 0 entry is negative (the entries $M-0.6$ and $M-0.2$ are positive because M is huge). Optimal, and both artificials are nonbasic, hence zero.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	RHS
z	0	0	0.6	$M-0.6$	0.2	$M-0.2$	-16.8
x_1	1	0	-0.6	0.6	0.8	-0.8	4.8
x_2	0	1	0.2	-0.2	-0.6	0.6	2.4

Step 8. Read the answer. $x_1=4.8$, $x_2=2.4$, $z=-16.8$, so the minimum cost is 16.8. The entries 0.6 and 0.2 under e_1 and e_2 are the prices of protein and fibre, matching Worked Example 5.2.

Answer. $\boxed{x_1^\star=4.8,\ x_2^\star=2.4,\ \text{cost}^\star=16.8}$.

Worked Example 9.2 An Equality Constraint with the Big M Method

Solve max $z=3x_1+5x_2$ subject to $x_1\le 4$, $2x_2\le 12$, $3x_1+2x_2=18$, $x_1,x_2\ge 0$.

Solution

Step 1. Standard form. $x_1+s_1=4$, $2x_2+s_2=12$, $3x_1+2x_2+a_3=18$. Objective max $3x_1+5x_2-Ma_3$.

Step 2. Initial tableau.

Basic	x_1	x_2	s_1	s_2	a_3	RHS
z	-3	-5	0	0	M	0
s_1	1	0	1	0	0	4
s_2	0	2	0	1	0	12
a_3	3	2	0	0	1	18

Step 3. Price out by subtracting $M\times$ row a_3 from row 0: under x_1, $-3-3M$; under x_2, $-5-2M$; RHS $-18M$.

Step 4. Tableau 1. The M coefficient -3 beats -2, so x_1 enters even though x_2 has the larger profit. Ratios 4 and 6; s_1 leaves.

Basic	x_1	x_2	s_1	s_2	a_3	RHS	Ratio
z	$-3M-3$	$-2M-5$	0	0	0	$-18M$	
s_1	$\boxed{1}$	0	1	0	0	4	$4/1=4$
s_2	0	2	0	1	0	12	none
a_3	3	2	0	0	1	18	$18/3=6$

Step 5. Tableau 2. x_2 enters ($-2M-5$). Ratios 6 and 3; a_3 leaves. The artificial variable has now left the basis.

Basic	x_1	x_2	s_1	s_2	a_3	RHS	Ratio
z	0	$-2M-5$	$3M+3$	0	0	$-6M+12$	
x_1	1	0	1	0	0	4	none
s_2	0	2	0	1	0	12	$12/2=6$
a_3	0	$\boxed{2}$	-3	0	1	6	$6/2=3$

Step 6. Tableau 3. The artificial column carries $M+2.5$, but the slack s_1 has -4.5, so s_1 enters. Ratios: $4/1=4$ and $6/3=2$; s_2 leaves.

Basic	x_1	x_2	s_1	s_2	a_3	RHS	Ratio
z	0	0	-4.5	0	$M+2.5$	27	
x_1	1	0	1	0	0	4	$4/1=4$
s_2	0	0	$\boxed{3}$	1	-1	6	$6/3=2$
x_2	0	1	-1.5	0	0.5	3	none

Step 7. Tableau 4. All entries nonnegative: optimal.

Basic	x_1	x_2	s_1	s_2	a_3	RHS
z	0	0	0	1.5	$M+1$	36
x_1	1	0	0	$-\frac{1}{3}$	$\frac{1}{3}$	2
s_1	0	0	1	$\frac{1}{3}$	$-\frac{1}{3}$	2
x_2	0	1	0	0.5	0	6

Step 8. Check. $x_1 = 2 \le 4$; $2x_2 = 12 \le 12$; $3(2) + 2(6) = 18$. Objective $6 + 30 = 36$.

Answer. $\boxed{x_1^\star = 2,\ x_2^\star = 6,\ z^\star = 36}$.

Worked Example 9.3 The Data Mixture Problem by the Two Phase Method

Solve the data mixture problem of Worked Example 1.6, min $4x_A + x_B + 2x_C$ subject to $x_A + x_B + x_C \ge 50$, $-3x_A + 3x_B + x_C \le 0$, $-x_A - x_B + 4x_C \ge 0$, all variables nonnegative, by the two phase method.

Solution

Step 1. Standard form. Row 1: $x_A + x_B + x_C - e_1 + a_1 = 50$. Row 2: $-3x_A + 3x_B + x_C + s_2 = 0$. Row 3: $-x_A - x_B + 4x_C - e_3 + a_3 = 0$. Starting basis $\{a_1, s_2, a_3\}$.

Step 2. Phase I objective. Maximise $w = -a_1 - a_3$. Row 0 initially has $+1$ under a_1 and a_3:

Basic	x_A	x_B	x_C	e_1	a_1	s_2	e_3	a_3	RHS
w	0	0	0	0	1	0	0	1	0
a_1	1	1	1	−1	1	0	0	0	50
s_2	−3	3	1	0	0	1	0	0	0
a_3	−1	−1	4	0	0	0	−1	1	0

Step 3. Price out. Subtract rows 1 and 3 from row 0. Under x_A: $0 - 1 - (-1) = 0$. Under x_B: $0 - 1 - (-1) = 0$. Under x_C: $0 - 1 - 4 = -5$. Under e_1 and e_3: $+1$ each. RHS: -50.

Step 4. Phase I, tableau 1. x_C enters. The ratios in rows s_2 and a_3 are both 0: a tie at a degenerate vertex. We take the first row, so s_2 leaves. The step length is zero (Module 10 studies this).

Basic	x_A	x_B	x_C	e_1	a_1	s_2	e_3	a_3	RHS	Ratio
w	0	0	−5	1	0	0	1	0	−50	
a_1	1	1	1	−1	1	0	0	0	50	$50/1 = 50$
s_2	−3	3	$\boxed{1}$	0	0	1	0	0	0	$0/1 = 0$
a_3	−1	−1	4	0	0	0	−1	1	0	$0/4 = 0$

Step 5. Phase I, further pivots. Two more pivots bring x_A in for a_3 and then x_B in for a_1:

Basic	x_A	x_B	x_C	e_1	a_1	s_2	e_3	a_3	RHS	Ratio
w	-15	15	0	1	0	5	1	0	-50	
a_1	4	-2	0	-1	1	-1	0	0	50	$50/4 = 12.5$
x_C	-3	3	1	0	0	1	0	0	0	none
a_3	$\boxed{11}$	-13	0	0	0	-4	-1	1	0	$0/11 = 0$

Basic	x_A	x_B	x_C	e_1	a_1	s_2	e_3	a_3	RHS	Ratio
w	0	$-\frac{30}{11}$	0	1	0	$-\frac{5}{11}$	$-\frac{4}{11}$	$\frac{15}{11}$	-50	
a_1	0	$\boxed{\frac{30}{11}}$	0	-1	1	$\frac{5}{11}$	$\frac{4}{11}$	$-\frac{4}{11}$	50	$50/\frac{30}{11} = \frac{55}{3}$
x_C	0	$-\frac{6}{11}$	1	0	0	$-\frac{1}{11}$	$-\frac{3}{11}$	$\frac{3}{11}$	0	none
x_A	1	$-\frac{13}{11}$	0	0	0	$-\frac{4}{11}$	$-\frac{1}{11}$	$\frac{1}{11}$	0	none

Step 6. End of Phase I. Row 0 is nonnegative and $w^\star = 0$: both artificials are zero, so the original problem is feasible and the basis $\{x_B, x_C, x_A\}$ is a feasible basis.

Basic	x_A	x_B	x_C	e_1	a_1	s_2	e_3	a_3	RHS
w	0	0	0	0	1	0	0	1	0
x_B	0	1	0	$-\frac{11}{30}$	$\frac{11}{30}$	$\frac{1}{6}$	$\frac{2}{15}$	$-\frac{2}{15}$	$\frac{55}{3}$
x_C	0	0	1	-0.2	0.2	0	-0.2	0.2	10
x_A	1	0	0	$-\frac{13}{30}$	$\frac{13}{30}$	$-\frac{1}{6}$	$\frac{1}{15}$	$-\frac{1}{15}$	$\frac{65}{3}$

Step 7. Phase II setup. Delete the artificial columns, and write the original objective as the maximisation $z = -4x_A - x_B - 2x_C$, so row 0 has $+4$, $+1$, $+2$ under x_A, x_B, x_C:

Basic	x_A	x_B	x_C	e_1	s_2	e_3	RHS
z	4	1	2	0	0	0	0
x_B	0	1	0	$-\frac{11}{30}$	$\frac{1}{6}$	$\frac{2}{15}$	$\frac{55}{3}$
x_C	0	0	1	-0.2	0	-0.2	10
x_A	1	0	0	$-\frac{13}{30}$	$-\frac{1}{6}$	$\frac{1}{15}$	$\frac{65}{3}$

Step 8. Restore unit columns. Subtract 1× row x_B, 2× row x_C and 4× row x_A from row 0. Under e_1: $0 + \frac{11}{30} + 2(0.2) + 4 \cdot \frac{13}{30} = \frac{11}{30} + \frac{12}{30} + \frac{52}{30} = \frac{75}{30} = 2.5$. Under s_2: $-\frac{1}{6} - 0 + \frac{4}{6} = 0.5$. Under e_3: $-\frac{2}{15} + 0.4 - \frac{4}{15} = 0$. RHS: $-\frac{55}{3} - 20 - \frac{260}{3} = -125$.

Basic	x_A	x_B	x_C	e_1	s_2	e_3	RHS
z	0	0	0	2.5	0.5	0	−125
x_B	0	1	0	$-\frac{11}{30}$	$\frac{1}{6}$	$\frac{2}{15}$	$\frac{55}{3}$
x_C	0	0	1	−0.2	0	−0.2	10
x_A	1	0	0	$-\frac{13}{30}$	$-\frac{1}{6}$	$\frac{1}{15}$	$\frac{65}{3}$

Step 9. **Phase II is already optimal**: no row 0 entry is negative. The solution is $x_A = \frac{65}{3} \approx 21.67$, $x_B = \frac{55}{3} \approx 18.33$, $x_C = 10$, with $z = -125$, so the minimum cost is 125.

Step 10. **Alternative optima.** The nonbasic surplus e_3 has a zero row 0 entry. Bringing it into the basis would change the solution without changing the cost: there are multiple optimal mixtures. One of them is (12.5, 0, 37.5) from Worked Example 1.6, also with cost $4(12.5) + 2(37.5) = 125$. Both meet the noise limit exactly.

Answer. Minimum cost $\boxed{125}$, attained for example by $(x_A, x_B, x_C) = (\frac{65}{3}, \frac{55}{3}, 10)$ and by $(12.5, 0, 37.5)$; the zero reduced cost of e_3 reveals the tie.

Worked Example 9.4 Detecting Infeasibility in Phase I

Apply Phase I to the system of Worked Example 5.5: $x_1 + 2x_2 \le 4$, $3x_1 + x_2 \ge 15$, $x_1 \le 3$, $x_1, x_2 \ge 0$.

Solution

Step 1. **Standard form.** $x_1 + 2x_2 + s_1 = 4$; $3x_1 + x_2 - e_2 + a_2 = 15$; $x_1 + s_3 = 3$. Starting basis $\{s_1, a_2, s_3\}$.

Step 2. **Phase I row 0.** Maximise $w = -a_2$. Pricing out (subtract row 2 from row 0) gives entries −3, −1, 0, 1, 0, 0 and RHS −15.

Basic	x_1	x_2	s_1	e_2	a_2	s_3	RHS	Ratio
w	−3	−1	0	1	0	0	−15	
s_1	1	2	1	0	0	0	4	4/1 = 4
a_2	3	1	0	−1	1	0	15	15/3 = 5
s_3	$\boxed{1}$	0	0	0	0	1	3	3/1 = 3

Step 3. **First pivot.** x_1 enters. Ratios 4, 5, 3; s_3 leaves.

Step 4. **Second pivot.** x_2 enters (−1). Ratios 0.5 and 6; s_1 leaves.

Basic	x_1	x_2	s_1	e_2	a_2	s_3	RHS	Ratio
w	0	−1	0	1	0	3	−6	
s_1	0	$\boxed{2}$	1	0	0	−1	1	1/2 = 0.5
a_2	0	1	0	−1	1	−3	6	6/1 = 6
x_1	1	0	0	0	0	1	3	none

Step 5. **Phase I optimum.** No negative entries remain, but $a_2 = 5.5$ is still basic and positive, and $w^\star = -5.5$.

Basic	x_1	x_2	s_1	e_2	a_2	s_3	RHS
w	0	0	0.5	1	0	2.5	−5.5
x_2	0	1	0.5	0	0	−0.5	0.5
a_2	0	0	−0.5	−1	1	−2.5	5.5
x_1	1	0	0	0	0	1	3

Step 6. **Conclusion.** The smallest possible total of artificial variables is $5.5 > 0$, so no point satisfies all original constraints. Indeed, the best Phase I point $x_1 = 3$, $x_2 = 0.5$ gives $3x_1 + x_2 = 9.5$, short of 15 by exactly 5.5.

Answer. Phase I ends with $\boxed{\sum a_i = 5.5 > 0}$: the problem is infeasible, and the protein style requirement is missed by at least 5.5 units.

Worked Example 9.5 Detecting Infeasibility with the Big M Method

Apply the Big M method to max $2x_1 + x_2$ subject to $x_1 + x_2 \le 3$, $x_1 + x_2 \ge 5$, $x_1, x_2 \ge 0$.

Solution

Step 1. **Standard form.** $x_1 + x_2 + s_1 = 3$, $x_1 + x_2 - e_2 + a_2 = 5$; objective max $2x_1 + x_2 - Ma_2$.

Step 2. **Initial tableau and pricing out.** Subtract $M\times$ row a_2 from row 0: under x_1, $-2 - M$; under x_2, $-1 - M$; under e_2, M; RHS $-5M$.

Basic	x_1	x_2	s_1	e_2	a_2	RHS
z	−2	−1	0	0	M	0
s_1	1	1	1	0	0	3
a_2	1	1	0	−1	1	5

Step 3. **Pivot.** x_1 enters ($-M - 2$ is the most negative). Ratios 3 and 5; s_1 leaves.

Basic	x_1	x_2	s_1	e_2	a_2	RHS	Ratio
z	$-M-2$	$-M-1$	0	M	0	$-5M$	
s_1	$\boxed{1}$	1	1	0	0	3	$3/1=3$
a_2	1	1	0	-1	1	5	$5/1=5$

Step 4. **Final tableau.** All row 0 entries are nonnegative (1, $M+2$, M, 0), so the Big M problem is optimal. But $a_2 = 2$ remains basic.

Basic	x_1	x_2	s_1	e_2	a_2	RHS
z	0	1	$M+2$	M	0	$-2M+6$
x_1	1	1	1	0	0	3
a_2	0	0	-1	-1	1	2

Step 5. **Conclusion.** An optimal Big M solution with a positive artificial variable means the original problem is infeasible. Directly: $x_1 + x_2$ cannot be both at most 3 and at least 5.

Answer. The problem is infeasible, signalled by $a_2 = 2 > 0$ in the optimal Big M tableau.

Worked Example 9.6 A Labelling Quota by the Two Phase Method

A team must label at least 10 thousand items, of which at least 3 thousand must be done by experts. Expert labelling (x_1, thousands) costs £5 hundred per thousand; crowd labelling (x_2) costs £3 hundred per thousand. Review capacity limits $2x_1 + x_2 \leq 24$. Minimise cost by the two phase method.

Solution

Step 1. **Model.** min $5x_1 + 3x_2$ subject to $x_1 + x_2 \geq 10$, $x_1 \geq 3$, $2x_1 + x_2 \leq 24$, $x_1, x_2 \geq 0$.

Step 2. **Standard form.** $x_1 + x_2 - e_1 + a_1 = 10$; $x_1 - e_2 + a_2 = 3$; $2x_1 + x_2 + s_3 = 24$.

Step 3. **Phase I.** Maximise $w = -a_1 - a_2$; after pricing out, row 0 is $(-2, -1, 1, 0, 1, 0, 0 \mid -13)$. x_1 enters; ratios 10, 3, 12; a_2 leaves.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	s_3	RHS	Ratio
w	-2	-1	1	0	1	0	0	-13	
a_1	1	1	-1	1	0	0	0	10	$10/1=10$
a_2	$\boxed{1}$	0	0	0	-1	1	0	3	$3/1=3$
s_3	2	1	0	0	0	0	1	24	$24/2=12$

Step 4. **Second Phase I pivot.** x_2 enters (-1); ratios 7 (row a_1) and 18 (row s_3); a_1 leaves.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	s_3	RHS	Ratio
w	0	−1	1	0	−1	2	0	−7	
a_1	0	$\boxed{1}$	−1	1	1	−1	0	7	$7/1 = 7$
x_1	1	0	0	0	−1	1	0	3	none
s_3	0	1	0	0	2	−2	1	18	$18/1 = 18$

Step 5. **End of Phase I.** $w^\star = 0$; both artificials are out. Feasible basis $\{x_2, x_1, s_3\}$ with $x_1 = 3$, $x_2 = 7$, $s_3 = 11$.

Basic	x_1	x_2	e_1	a_1	e_2	a_2	s_3	RHS
w	0	0	0	1	0	1	0	0
x_2	0	1	−1	1	1	−1	0	7
x_1	1	0	0	0	−1	1	0	3
s_3	0	0	1	−1	1	−1	1	11

Step 6. **Phase II setup.** Delete artificial columns; maximise $z = -5x_1 - 3x_2$, so row 0 has 5 and 3 under x_1 and x_2:

Basic	x_1	x_2	e_1	e_2	s_3	RHS
z	5	3	0	0	0	0
x_2	0	1	−1	1	0	7
x_1	1	0	0	−1	0	3
s_3	0	0	1	1	1	11

Step 7. **Restore unit columns.** Subtract 3× row x_2 and 5× row x_1. Under e_1: $0 - 3(-1) - 0 = 3$. Under e_2: $0 - 3(1) - 5(-1) = 2$. RHS: $-21 - 15 = -36$.

Basic	x_1	x_2	e_1	e_2	s_3	RHS
z	0	0	3	2	0	−36
x_2	0	1	−1	1	0	7
x_1	1	0	0	−1	0	3
s_3	0	0	1	1	1	11

Step 8. **Optimal immediately.** Both reduced cost entries (3 and 2) are positive. Minimum cost 36 hundred pounds.

Step 9. **Interpretation.** The entry 3 under e_1 says each extra thousand items required costs £300 (one more crowd batch). The entry 2 under e_2 says each extra thousand demanded from experts costs £200 more, the price difference between expert and crowd work.

Answer. $\boxed{x_1^\star = 3,\ x_2^\star = 7,\ \text{cost}^\star = £3{,}600}$, with 11 units of review capacity to spare.

Module Summary

- Constraints of type ≥ and = give no obvious starting basis; artificial variables $a_i \geq 0$ with coefficient +1 supply one.
- The original problem is feasible exactly when the enlarged problem has a solution with every $a_i = 0$.
- Big M: penalise each artificial by M in the objective, price out row 0, compare entries by their M coefficient first. A positive artificial in the optimal tableau means infeasibility.
- Two phase: Phase I minimises $\sum a_i$; a positive minimum proves infeasibility; a zero minimum gives a feasible basis. Phase II restores the real objective, prices out the basic variables and continues.
- Two phase is preferred in software because it avoids choosing a numerical M.

MODULE 10

Special Cases: Degeneracy, Cycling, Multiple Optima and Unboundedness

Learning Objectives

After studying this module you will be able to:

- recognise a degenerate basic feasible solution in a tableau and explain why a degenerate pivot may not improve the objective;
- explain cycling, reproduce a classical cycling example, and prove that Bland's rule prevents it;
- detect multiple optimal solutions from a final tableau and generate the alternative optima;
- detect unboundedness from a tableau and write down the improving ray explicitly;
- use these signals to debug models, for example to find a missing constraint.

10.1 When the Tableau Tells a Story

The simplex method of Module 8 does more than compute numbers. The pattern of zeros and signs in each tableau describes the geometry of the problem: whether the current vertex is crowded by extra constraints, whether a whole edge is optimal, or whether the objective can grow without limit. This module teaches you to read those patterns. Each one has a practical meaning for a modeller, and in machine learning applications each one appears regularly: degeneracy is the rule rather than the exception in ℓ_1 regression and in optimal transport, multiple optima appear whenever the data have symmetries, and unboundedness almost always signals a modelling mistake.

10.2 Degeneracy

Definition Degenerate Tableau

A tableau is **degenerate** if at least one basic variable has value zero, that is, some entry of the RHS column (other than row 0) is 0. Geometrically, more than n constraints are active at the current vertex (Module 7).

Derivation A degenerate pivot can leave the objective unchanged

Step 1. From Module 8, the step length is $\theta^\star = \min\{\bar{b}_i/\bar{a}_{iq} : \bar{a}_{iq} > 0\}$.

Step 2. If the minimising row has $\bar{b}_r = 0$, then $\theta^\star = 0$.

Step 3. The objective changes by $\bar{c}_q\theta^\star = 0$, and every variable keeps its value: the vertex does not move.

Step 4. Only the *description* of the vertex changes: a different set of active constraints is now labelled basic.

Degeneracy also shows up *before* it happens: a tie in the ratio test means that after the pivot, the variable of the losing tied row will equal zero, so the next tableau is degenerate.

10.3 Cycling and Bland's Rule

Definition Cycling

Cycling occurs when a sequence of degenerate pivots returns to a basis that was visited before. Since the pivoting rules are deterministic, the method then repeats the same sequence for ever and never terminates.

Cycling is rare in practice, but it is possible, and Worked Example 10.2 shows a famous instance. The cure is a careful tie breaking rule.

Key Result Bland's rule (smallest subscript rule)

Number all variables (original, slack and surplus) $1, 2, \ldots, n$ in a fixed order.

1. **Entering:** among all variables with an improving reduced cost (negative row 0 entry for maximisation), choose the one with the *smallest index*.
2. **Leaving:** among all rows that tie for the minimum ratio, choose the row whose basic variable has the *smallest index*.

With Bland's rule the simplex method never cycles and therefore always terminates.

Derivation Outline of why Bland's rule cannot cycle

Step 1. Suppose, for contradiction, that Bland's rule cycles. During a cycle the objective never changes, so every pivot in the cycle is degenerate.

Step 2. Call a variable *fickle* if it enters and leaves during the cycle. Let x_t be the fickle variable with the *largest* index.

Step 3. Consider the tableau where x_t leaves, with entering variable x_s ($s < t$), and a later tableau where x_t enters again. When x_t enters again, it has the smallest index among improving variables, so every fickle variable with a smaller index has a non improving reduced cost there.

Step 4. Write the objective row of the later tableau as a combination of the equations of the earlier tableau. Comparing the coefficient of x_s in two different expressions for the same objective change leads to a sum of terms that must be positive (because x_s improves in the earlier tableau) yet, term by term, is shown to be nonpositive using the sign information from Step 3 and the leaving rule.

Step 5. This contradiction proves that no cycle exists.

The full bookkeeping is lengthy; the essential idea is that always preferring small indices imposes an order which a cycle would have to violate.

Lucid Insight Practice versus theory

Commercial solvers rarely use Bland's rule throughout, because it tends to choose poor entering variables and slows the method down. Instead they perturb the right hand side by tiny random amounts (which removes ties), or switch to Bland's rule only after detecting many degenerate pivots in a row. The theoretical guarantee nevertheless matters: it proves that the simplex method is a genuine algorithm that always finishes.

10.4 Multiple Optimal Solutions

Derivation The zero reduced cost signal

Step 1. Suppose the tableau is optimal and some *nonbasic* variable x_q has row 0 entry exactly zero, so $\bar{c}_q = 0$.

Step 2. By the objective formula of Module 8, $z = z_0 + \sum_{j \in N} \bar{c}_j x_j$. Increasing x_q changes z at rate $\bar{c}_q = 0$: the objective stays at its optimal value.

Step 3. If the ratio test for column q gives $\theta^\star > 0$, pivoting x_q in moves to a *different* optimal vertex.

Step 4. By convexity (Module 3), every point on the segment between the two optimal vertices,

$$\mathbf{x}(\lambda) = \lambda \mathbf{x}^{(1)} + (1 - \lambda)\mathbf{x}^{(2)}, \qquad 0 \le \lambda \le 1, \tag{10.1}$$

is feasible, and by linearity it has the same optimal objective value.

where

$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}$ are the two optimal basic feasible solutions;

λ is the weight that slides between them.

Common Pitfall

A zero row 0 entry under a *basic* variable is normal: every basic column has zero there. Only a zero under a *nonbasic* variable signals alternative optima. Also, if that nonbasic column has $\theta^\star = 0$ (a degenerate case), pivoting gives a new basis for the same vertex, not a new optimal point.

10.5 Unboundedness

Derivation Reading an improving ray from the tableau

Step 1. Suppose x_q has a negative row 0 entry (it improves the objective) and every entry of its column in rows 1 to m is ≤ 0.

Step 2. Increase x_q to $t \geq 0$. Each basic variable becomes $x_{B(i)} = \bar{b}_i - t\bar{a}_{iq}$. Since $\bar{a}_{iq} \leq 0$, this is at least $\bar{b}_i \geq 0$ for every t, so feasibility is never lost.

Step 3. The objective becomes $z = z_0 + \bar{c}_q t$ with $\bar{c}_q > 0$, which tends to infinity.

Step 4. Collecting the changes gives the **ray**

$$\mathbf{x}(t) = \mathbf{x}^0 + t\,\mathbf{d}, \qquad d_q = 1, \quad d_{B(i)} = -\bar{a}_{iq}, \quad d_j = 0 \text{ for other nonbasic } j, \tag{10.2}$$

where

$\mathbf{x}^0$ is the current basic feasible solution;

$\mathbf{d}$ is a recession direction (Module 3) with $\mathbf{c}^\top \mathbf{d} = \bar{c}_q > 0$.

Key Result Tableau signals at a glance (maximisation)

What you see	What it means
A zero in the RHS column of a constraint row	Degenerate vertex; the next pivot may not move.
A tie in the ratio test	The next tableau will be degenerate.
All row 0 entries ≥ 0	Optimal.
Optimal, with a zero row 0 entry under a nonbasic variable	Possibly multiple optima (a whole edge or face is optimal).
Negative row 0 entry with no positive entry below it	Unbounded; read the ray from that column.
Optimal Phase I with positive artificial value	Infeasible (Module 9).

Machine Learning Connection

In ℓ_1 regression (Module 19) the optimal fit passes exactly through at least p data points, and when the data contain ties or collinear points more than p residuals are zero: the optimal vertex is degenerate, and several bases describe it. This is why ℓ_1 regression coefficients can have non unique "standard errors" and why the median, the simplest ℓ_1 estimator, is not unique for an even number of observations (multiple optima). Unboundedness, in contrast, is almost always a bug: a missing regularisation term, a forgotten budget, or a sign error in a loss function. Reading the improving ray tells you exactly which quantity is escaping to infinity.

10.6 Worked Examples

Worked Example 10.1 A Tie in the Ratio Test and a Degenerate Optimum

Solve max $z = 3x_1 + 2x_2$ subject to $x_1 + x_2 \le 4$, $x_1 \le 2$, $x_2 \le 2$, $x_1, x_2 \ge 0$, and identify the degeneracy.

Solution

Step 1. Tableau 0. x_1 enters (-3). Ratios 4 and 2 (row s_3 has entry 0); s_2 leaves.

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
z	-3	-2	0	0	0	0	
s_1	1	1	1	0	0	4	$4/1 = 4$
s_2	$\boxed{1}$	0	0	1	0	2	$2/1 = 2$
s_3	0	1	0	0	1	2	none

Step 2. Tableau 1. x_2 enters (-2). Ratios: row s_1: $2/1 = 2$; row s_3: $2/1 = 2$. *Tie.* Whichever row leaves, the other basic variable will become zero. We choose the first row, so s_1 leaves.

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
z	0	-2	0	3	0	6	
s_1	0	$\boxed{1}$	1	-1	0	2	$2/1 = 2$
x_1	1	0	0	1	0	2	none
s_3	0	1	0	0	1	2	$2/1 = 2$

Step 3. Tableau 2. Optimal. The basic variable s_3 equals zero: the tableau is degenerate.

Basic	x_1	x_2	s_1	s_2	s_3	RHS
z	0	0	2	1	0	10
x_2	0	1	1	-1	0	2
x_1	1	0	0	1	0	2
s_3	0	0	-1	1	1	0

Step 4. Geometry. At $(2, 2)$ three constraints are active ($x_1 + x_2 = 4$, $x_1 = 2$, $x_2 = 2$), one more than needed in the plane, exactly as in Worked Example 7.3.

Step 5. Shadow prices. Row 0 gives $y_1 = 2$, $y_2 = 1$, $y_3 = 0$. Had we broken the tie the other way, we would obtain different but equally valid prices ($y_1 = 0$, $y_2 = 3$, $y_3 = 2$; check: $3\cdot2+2\cdot2 = 10$ and $0 \cdot 4 + 3 \cdot 2 + 2 \cdot 2 = 10$). At degenerate optima shadow prices are not unique (Module 15).

Answer. $\boxed{x^\star = (2, 2),\ z^\star = 10}$ at a degenerate vertex with basic $s_3 = 0$.

Worked Example 10.2 A Problem That Cycles under the Largest Coefficient Rule

Apply the simplex method with the rule "enter the most negative row 0 entry; break ratio ties by the smallest index" to

$$\begin{aligned} \max\quad & 10x_1 - 57x_2 - 9x_3 - 24x_4 \\ \text{subject to}\quad & 0.5x_1 - 5.5x_2 - 2.5x_3 + 9x_4 \le 0, \\ & 0.5x_1 - 1.5x_2 - 0.5x_3 + x_4 \le 0, \\ & x_1 \le 1, \qquad x_1, x_2, x_3, x_4 \ge 0. \end{aligned}$$

Solution

Step 1. Starting tableau. The first two right hand sides are 0, so the starting BFS is already degenerate. x_1 enters (-10); rows s_1 and s_2 tie with ratio 0; the smaller index s_1 leaves.

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	-10	57	9	24	0	0	0	0	
s_1	$\boxed{0.5}$	-5.5	-2.5	9	1	0	0	0	$0/0.5 = 0$
s_2	0.5	-1.5	-0.5	1	0	1	0	0	$0/0.5 = 0$
s_3	1	0	0	0	0	0	1	1	$1/1 = 1$

Step 2. Tableau 1. x_2 enters; only row s_2 has a positive entry in its column; ratio 0; s_2 leaves.

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	0	-53	-41	204	20	0	0	0	
x_1	1	-11	-5	18	2	0	0	0	none
s_2	0	$\boxed{4}$	2	-8	-1	1	0	0	$0/4 = 0$
s_3	0	11	5	-18	-2	0	1	1	$1/11 = \frac{1}{11}$

Step 3. Tableau 2. x_3 enters; x_1 leaves (ratio 0).

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	0	0	-14.5	98	6.75	13.25	0	0	
x_1	1	0	$\boxed{0.5}$	-4	-0.75	2.75	0	0	$0/0.5 = 0$
x_2	0	1	0.5	-2	-0.25	0.25	0	0	$0/0.5 = 0$
s_3	0	0	-0.5	4	0.75	-2.75	1	1	none

Step 4. Tableau 3. x_4 enters; x_2 leaves (ratio 0).

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	29	0	0	−18	−15	93	0	0	
x_3	2	0	1	−8	−1.5	5.5	0	0	none
x_2	−1	1	0	$\boxed{2}$	0.5	−2.5	0	0	$0/2 = 0$
s_3	1	0	0	0	0	0	1	1	none

Step 5. **Tableau 4.** s_1 enters; x_3 leaves (ratio 0).

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	20	9	0	0	−10.5	70.5	0	0	
x_3	−2	4	1	0	$\boxed{0.5}$	−4.5	0	0	$0/0.5 = 0$
x_4	−0.5	0.5	0	1	0.25	−1.25	0	0	$0/0.25 = 0$
s_3	1	0	0	0	0	0	1	1	none

Step 6. **Tableau 5.** s_2 enters; x_4 leaves (ratio 0).

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	−22	93	21	0	0	−24	0	0	
s_1	−4	8	2	0	1	−9	0	0	none
x_4	0.5	−1.5	−0.5	1	0	$\boxed{1}$	0	0	$0/1 = 0$
s_3	1	0	0	0	0	0	1	1	none

Step 7. **Tableau 6** is identical to Tableau 0.

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS
z	−10	57	9	24	0	0	0	0
s_1	0.5	−5.5	−2.5	9	1	0	0	0
s_2	0.5	−1.5	−0.5	1	0	1	0	0
s_3	1	0	0	0	0	0	1	1

Step 8. **Summary of the cycle.** The bases visited were

$$\{s_1, s_2, s_3\} \to \{x_1, s_2, s_3\} \to \{x_1, x_2, s_3\} \to \{x_3, x_2, s_3\}$$
$$\to \{x_3, x_4, s_3\} \to \{s_1, x_4, s_3\} \to \{s_1, s_2, s_3\},$$

and the objective stayed at 0 throughout. Every pivot had step length zero. The method would repeat this loop for ever.

Answer. The method cycles through six bases without moving from $\mathbf{x} = \mathbf{0}$.

Worked Example 10.3 Bland's Rule Breaks the Cycle

Solve the problem of Worked Example 10.2 with Bland's rule, using the variable order $x_1, x_2, x_3, x_4, s_1, s_2, s_3$.

Solution

Step 1. Pivots 1 to 5 are the same as before. In Tableaux 0 to 4 the most negative entry also has the smallest index among the improving variables, and every ratio tie is broken by the smallest index under both rules. We therefore reach Tableau 5 with basis $\{s_1, x_4, s_3\}$, still at $\mathbf{x} = \mathbf{0}$.

Step 2. The decisive tableau. In Tableau 5 two variables improve: x_1 (entry -22) and s_2 (entry -24). The largest coefficient rule chose s_2 and closed the cycle. Bland's rule chooses the smaller index, x_1. Only rows x_4 and s_3 have positive entries; the ratios are 0 and 1, so x_4 leaves.

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	-22	93	21	0	0	-24	0	0	
s_1	-4	8	2	0	1	-9	0	0	none
x_4	$\boxed{0.5}$	-1.5	-0.5	1	0	1	0	0	$0/0.5 = 0$
s_3	1	0	0	0	0	0	1	1	$1/1 = 1$

Step 3. Tableau 6. The only improving variable is x_3 (entry -1). Only row s_3 has a positive entry, with ratio $1/1 = 1$: at last a pivot with a positive step, so the objective rises from 0 to 1.

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS	Ratio
z	0	27	-1	44	0	20	0	0	
s_1	0	-4	-2	8	1	-1	0	0	none
x_1	1	-3	-1	2	0	2	0	0	none
s_3	0	3	$\boxed{1}$	-2	0	-2	1	1	$1/1 = 1$

Step 4. Final tableau. All row 0 entries are nonnegative.

Basic	x_1	x_2	x_3	x_4	s_1	s_2	s_3	RHS
z	0	30	0	42	0	18	1	1
s_1	0	2	0	4	1	-5	2	2
x_1	1	0	0	0	0	0	1	1
x_3	0	3	1	-2	0	-2	1	1

Step 5. Read the answer. $x_1 = 1$, $x_3 = 1$, $x_2 = x_4 = 0$, $z = 10 - 9 = 1$. Check: constraint 1: $0.5 - 2.5 = -2 \le 0$; constraint 2: $0.5 - 0.5 = 0 \le 0$; constraint 3: $1 \le 1$.

Answer. Bland's rule terminates at $\boxed{x^\star = (1, 0, 1, 0),\ z^\star = 1}$ after seven pivots, the last of which moves the solution.

Worked Example 10.4 Finding All Optimal Solutions from the Tableau

Solve max $z = 2x_1 + 4x_2$ subject to $x_1 + 2x_2 \le 8$, $x_1 \le 6$, $x_2 \le 3$, $\mathbf{x} \ge \mathbf{0}$, and use the final tableau to find every optimal solution.

Solution

Step 1. Simplex iterations. x_2 enters first (ratios 4 and 3, s_3 leaves); then x_1 enters (ratios 2 and 6, s_1 leaves).

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
z	-2	-4	0	0	0	0	
s_1	1	2	1	0	0	8	$8/2 = 4$
s_2	1	0	0	1	0	6	none
s_3	0	$\boxed{1}$	0	0	1	3	$3/1 = 3$

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
z	-2	0	0	0	4	12	
s_1	$\boxed{1}$	0	1	0	-2	2	$2/1 = 2$
s_2	1	0	0	1	0	6	$6/1 = 6$
x_2	0	1	0	0	1	3	none

Step 2. Final tableau.

Basic	x_1	x_2	s_1	s_2	s_3	RHS
z	0	0	2	0	0	16
x_1	1	0	1	0	-2	2
s_2	0	0	-1	1	2	4
x_2	0	1	0	0	1	3

Step 3. First optimum. $\mathbf{x}^{(1)} = (x_1, x_2) = (2, 3)$, $z = 16$.

Step 4. Signal. The nonbasic variable s_3 has row 0 entry 0: alternative optima may exist.

Step 5. Pivot s_3 in. Ratios in the s_3 column: row x_1 has -2 (none); row s_2: $4/2 = 2$; row x_2: $3/1 = 3$. So s_2 leaves with $\theta^\star = 2 > 0$.

Step 6. Second optimum. Increasing s_3 by 2 changes $x_1 = 2 - (-2)(2) = 6$ and $x_2 = 3 - 1(2) = 1$. So $\mathbf{x}^{(2)} = (6, 1)$, with $z = 12 + 4 = 16$.

Step 7. All optima by (10.1): $\mathbf{x}(\lambda) = \lambda(2, 3) + (1 - \lambda)(6, 1) = (6 - 4\lambda,\ 1 + 2\lambda)$ for $\lambda \in [0, 1]$.

Answer. Every point $\boxed{(6 - 4\lambda,\ 1 + 2\lambda),\ 0 \le \lambda \le 1}$ is optimal with $z^\star = 16$, matching Worked Example 5.3.

Worked Example 10.5 Detecting Unboundedness and Writing the Ray

Apply the simplex method to max $z = x_1 + x_2$ subject to $x_1 - x_2 \le 2$, $-2x_1 + x_2 \le 2$, $\mathbf{x} \ge \mathbf{0}$.

Solution

Step 1. **Tableau 0.** Both entries are -1; with the smallest index rule x_1 enters. Only row s_1 has a positive entry; s_1 leaves.

Basic	x_1	x_2	s_1	s_2	RHS	Ratio
z	-1	-1	0	0	0	
s_1	$\boxed{1}$	-1	1	0	2	$2/1 = 2$
s_2	-2	1	0	1	2	none

Step 2. **Tableau 1.** x_2 has row 0 entry -2, so it improves the objective, but its column entries are -1 and -1: no positive entry, no ratio.

Basic	x_1	x_2	s_1	s_2	RHS
z	0	-2	1	0	2
x_1	1	-1	1	0	2
s_2	0	-1	2	1	6

Step 3. **Conclusion: unbounded.** Increasing $x_2 = t$ makes $x_1 = 2 + t$ and $s_2 = 6 + t$, both of which only grow.

Step 4. **Ray by** (10.2)**.** In the order (x_1, x_2, s_1, s_2):

$$\mathbf{x}(t) = (2, 0, 0, 6) + t(1, 1, 0, 1), \qquad z(t) = 2 + 2t \to \infty.$$

Step 5. **Check feasibility** of the ray in the original constraints: $(2+t) - t = 2 \le 2$ and $-2(2+t) + t = -4 - t \le 2$ for all $t \ge 0$.

Answer. The problem is unbounded along $(x_1, x_2) = (2 + t,\ t)$, with $z = 2 + 2t$.

Worked Example 10.6 Debugging an ML Resource Model with a Missing Budget

A platform team models revenue $4x_1 + 3x_2$ (hundreds of pounds) from x_1 thousand inference calls on a premium model and x_2 thousand on a standard model. The constraints are a latency rule $x_1 - x_2 \le 4$ and a fairness rule $-2x_1 + x_2 \le 3$. (a) Run the simplex method and diagnose the result. (b) Add the forgotten GPU budget $x_1 + x_2 \le 10$ and solve again.

Solution

Step 1. **(a) First pivot.** x_1 enters (-4); only row s_1 has a positive entry; s_1 leaves.

Basic	x_1	x_2	s_1	s_2	RHS	Ratio
z	-4	-3	0	0	0	
s_1	$\boxed{1}$	-1	1	0	4	$4/1 = 4$
s_2	-2	1	0	1	3	none

Step 2. **(a) Unbounded.** The x_2 column has entry -7 in row 0 but -1 and -1 below.

Basic	x_1	x_2	s_1	s_2	RHS
z	0	-7	4	0	16
x_1	1	-1	1	0	4
s_2	0	-1	2	1	11

Step 3. **(a) Diagnosis.** The ray is $(x_1, x_2) = (4 + t,\ t)$ with revenue $16 + 7t$. Both kinds of call grow together without limit, which is impossible in reality because calls consume GPU time. The ray points exactly at the missing resource constraint: something must limit $x_1 + x_2$.

Step 4. **(b) Add the budget** $x_1 + x_2 + s_3 = 10$. The first pivot is the same; then x_2 enters, and now row s_3 has a positive entry (2), so s_3 leaves with ratio 3.

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
z	0	-7	4	0	0	16	
x_1	1	-1	1	0	0	4	none
s_2	0	-1	2	1	0	11	none
s_3	0	$\boxed{2}$	-1	0	1	6	$6/2 = 3$

Step 5. **(b) Optimal.**

Basic	x_1	x_2	s_1	s_2	s_3	RHS
z	0	0	0.5	0	3.5	37
x_1	1	0	0.5	0	0.5	7
s_2	0	0	1.5	1	0.5	14
x_2	0	1	-0.5	0	0.5	3

Step 6. **(b) Read and check.** $x_1 = 7$, $x_2 = 3$, revenue $28 + 9 = 37$. Latency: $7 - 3 = 4 \le 4$ (active). Fairness: $-14 + 3 = -11 \le 3$ (slack 14). Budget: $10 \le 10$ (active). Shadow price of the budget: 3.5, so each extra thousand GPU calls of capacity is worth £350.

Answer. (a) Unbounded along $(4 + t,\ t)$, revealing the missing budget. (b) $\boxed{x^\star = (7, 3),\ z^\star = 37}$ (£3,700).

Module Summary

- A zero basic variable means degeneracy; a degenerate pivot may have step length zero and leave the vertex and objective unchanged.
- Cycling is a loop of degenerate pivots; Bland's smallest subscript rule (for both entering and leaving) prevents it.
- An optimal tableau with a zero row 0 entry under a nonbasic variable signals alternative optima; pivoting it in gives another optimal vertex, and all convex combinations are optimal.
- An improving column with no positive entries proves unboundedness, and the column gives the ray $\mathbf{x}^0 + t\mathbf{d}$ explicitly.
- In applications, degeneracy and multiple optima are common and harmless; unboundedness usually reveals a missing constraint.

Module 11

The Revised Simplex Method in Matrix Form

Learning Objectives

After studying this module you will be able to:

- express every entry of a simplex tableau through the basis inverse $\mathbf{B}^{-1}$;
- define the simplex multipliers $\mathbf{y}^\top = \mathbf{c}_B^\top \mathbf{B}^{-1}$ and compute reduced costs by "pricing";
- carry out a complete iteration of the revised simplex method using only $\mathbf{B}^{-1}$ and the original data;
- update $\mathbf{B}^{-1}$ with an elementary (eta) matrix, the product form of the inverse;
- verify the optimality of a proposed basis without running the method;
- estimate the computational savings of the revised method for large sparse problems such as those in machine learning.

11.1 Why a Revised Method?

The tableau method of Module 8 updates every entry of an $(m+1) \times (n+1)$ table at every pivot. For a problem with $m = 1{,}000$ constraints and $n = 100{,}000$ variables, that is about 10^8 arithmetic operations per pivot, almost all of them wasted on columns that will never enter the basis. Worse, the original matrix $\mathbf{A}$ in large applications is *sparse* (most entries are zero), but the tableau quickly fills with nonzeros. The revised simplex method keeps the original data untouched and carries only the information needed to make the next decision: the basis inverse $\mathbf{B}^{-1}$ (or, in practice, a factorisation of $\mathbf{B}$). It is the form used inside every serious simplex code.

11.2 The Tableau in Matrix Form

We work with the standard form problem written for maximisation,

$$\max \ \mathbf{c}^\top \mathbf{x} \qquad \text{subject to} \qquad \mathbf{A}\mathbf{x} = \mathbf{b}, \qquad \mathbf{x} \geq \mathbf{0}, \tag{11.1}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ already includes the slack columns. A basis has matrix $\mathbf{B}$, basic costs $\mathbf{c}_B$, nonbasic matrix $\mathbf{N}$ and nonbasic costs $\mathbf{c}_N$.

Derivation Every tableau entry from $\mathbf{B}^{-1}$

Step 1. **Constraint rows.** Multiplying $\mathbf{B}\mathbf{x}_B + \mathbf{N}\mathbf{x}_N = \mathbf{b}$ by $\mathbf{B}^{-1}$ gives

$$\mathbf{I}\mathbf{x}_B + \mathbf{B}^{-1}\mathbf{N}\mathbf{x}_N = \mathbf{B}^{-1}\mathbf{b}.$$

So in the current tableau the column of any variable x_j is $\bar{\mathbf{a}}_j = \mathbf{B}^{-1}\mathbf{A}_j$ and the right hand side is $\bar{\mathbf{b}} = \mathbf{B}^{-1}\mathbf{b}$.

Step 2. **Objective row.** From Module 8, $z = \mathbf{c}_B^\top\mathbf{B}^{-1}\mathbf{b} + \sum_{j\in N}(c_j - \mathbf{c}_B^\top\mathbf{B}^{-1}\mathbf{A}_j)x_j$. Row 0 stores $z - \sum_j \bar{c}_j x_j = \mathbf{c}_B^\top\mathbf{B}^{-1}\mathbf{b}$, so its entries are $-\bar{c}_j$ and its right hand side is the current objective value.

Step 3. **Define the simplex multipliers.**

$$\mathbf{y}^\top = \mathbf{c}_B^\top\mathbf{B}^{-1} \qquad \text{(equivalently } \mathbf{B}^\top\mathbf{y} = \mathbf{c}_B\text{)}. \tag{11.2}$$

Step 4. **Reduced costs by pricing.** Substituting (11.2),

$$\bar{c}_j = c_j - \mathbf{y}^\top\mathbf{A}_j. \tag{11.3}$$

where

$\mathbf{y} \in \mathbb{R}^m$ is the vector of **simplex multipliers**, one per constraint; at optimality these are the shadow prices and, in Module 12, the dual variables;

$\mathbf{y}^\top\mathbf{A}_j$ is the "imputed value" of the resources consumed by one unit of x_j;

$\bar{c}_j$ compares the direct profit c_j with that imputed value.

Key Result The tableau in one picture

$$\begin{array}{c|c|c} & \mathbf{x}^\top & \text{RHS} \\ \hline z & \mathbf{y}^\top\mathbf{A} - \mathbf{c}^\top & \mathbf{y}^\top\mathbf{b} \\ \hline \mathbf{x}_B & \mathbf{B}^{-1}\mathbf{A} & \mathbf{B}^{-1}\mathbf{b} \end{array} \qquad \text{with} \quad \mathbf{y}^\top = \mathbf{c}_B^\top\mathbf{B}^{-1}. \tag{11.4}$$

Any single entry can be computed on demand from $\mathbf{B}^{-1}$ and the original data, without ever forming the whole tableau.

Lucid Insight Pricing is a market test

Think of y_i as the market price of one unit of resource i. An activity j uses the bundle of resources $\mathbf{A}_j$, whose market value is $\mathbf{y}^\top\mathbf{A}_j$. If the activity earns more than the value of the resources it uses ($c_j > \mathbf{y}^\top\mathbf{A}_j$), it is worth starting; that is exactly $\bar{c}_j > 0$. At an optimum no activity beats the market.

11.3 One Iteration of the Revised Simplex Method

Key Result Revised simplex iteration (maximisation)

Given a feasible basis with known $\mathbf{B}^{-1}$ and current values $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$:

1. **Price:** compute $\mathbf{y}^\top = \mathbf{c}_B^\top \mathbf{B}^{-1}$ and $\bar{c}_j = c_j - \mathbf{y}^\top \mathbf{A}_j$ for nonbasic j.
2. **Optimality test:** if every $\bar{c}_j \le 0$, stop. The current basis is optimal.
3. **Choose entering** q with $\bar{c}_q > 0$ (for example the largest).
4. **Compute the direction** $\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_q$ (the entering column of the tableau).
5. **Ratio test:** if $\mathbf{d} \le \mathbf{0}$, the problem is unbounded. Otherwise $\theta^\star = \min\{(\mathbf{x}_B)_i/d_i : d_i > 0\}$, attained at row r.
6. **Update values:** $\mathbf{x}_B \leftarrow \mathbf{x}_B - \theta^\star \mathbf{d}$, then set the entering variable to $\theta^\star$ in position r.
7. **Update the inverse** with the eta matrix (below) and return to step 1.

11.4 Updating the Inverse: The Product Form

Derivation The eta matrix update

After the pivot, the new basis matrix $\mathbf{B}'$ equals $\mathbf{B}$ with column r replaced by $\mathbf{A}_q$.

Step 1. Multiply the new basis matrix by the old inverse: $\mathbf{B}^{-1}\mathbf{B}'$. Its columns are $\mathbf{B}^{-1}$ times the columns of $\mathbf{B}'$. For every column $i \ne r$, $\mathbf{B}^{-1}\mathbf{B}_i = \mathbf{e}_i$ (a unit vector), and column r is $\mathbf{B}^{-1}\mathbf{A}_q = \mathbf{d}$. So

$$\mathbf{B}^{-1}\mathbf{B}' = [\mathbf{e}_1 \ \cdots \ \mathbf{e}_{r-1} \ \ \mathbf{d} \ \ \mathbf{e}_{r+1} \ \cdots \ \mathbf{e}_m] =: \mathbf{F}.$$

Step 2. The matrix $\mathbf{F}$ is the identity except for column r. Its inverse is again the identity except for column r, which becomes the **eta vector**

$$\boldsymbol{\eta} = \Big(-\frac{d_1}{d_r}, \ \ldots, \ -\frac{d_{r-1}}{d_r}, \ \frac{1}{d_r}, \ -\frac{d_{r+1}}{d_r}, \ \ldots, \ -\frac{d_m}{d_r}\Big)^\top. \tag{11.5}$$

One checks directly that $\mathbf{F}\mathbf{E} = \mathbf{I}$ where $\mathbf{E}$ has column r equal to $\boldsymbol{\eta}$.

Step 3. From $\mathbf{B}^{-1}\mathbf{B}' = \mathbf{F}$ we get $\mathbf{B}' = \mathbf{B}\mathbf{F}$, and therefore

$$(\mathbf{B}')^{-1} = \mathbf{F}^{-1}\mathbf{B}^{-1} = \mathbf{E}\,\mathbf{B}^{-1}. \tag{11.6}$$

Multiplying by $\mathbf{E}$ is exactly the pivot operation applied to the rows of $\mathbf{B}^{-1}$: divide row r by d_r and subtract d_i times the new row r from every other row i. Starting from $\mathbf{B}^{-1} = \mathbf{I}$ (the slack basis), after k pivots $\mathbf{B}^{-1} = \mathbf{E}_k \cdots \mathbf{E}_2\mathbf{E}_1$, the **product form of the inverse**.

Common Pitfall

Never compute $\mathbf{B}^{-1}$ by a fresh matrix inversion at each iteration in real software, and never store a dense inverse for large problems. Production codes keep an LU factorisation of $\mathbf{B}$, solve $\mathbf{B}\mathbf{d}$ =

$\mathbf{A}_q$ and $\mathbf{B}^\top \mathbf{y} = \mathbf{c}_B$ by forward and back substitution, apply eta style updates between occasional "refactorisations", and thereby control both speed and rounding error.

Derivation Operation counts

Let v be the average number of nonzeros per column of $\mathbf{A}$.

Step 1. **Tableau method**: each pivot updates $(m+1)(n+1)$ entries, about mn operations.

Step 2. **Revised method**: computing $\mathbf{y}$ costs about m^2 (dense $\mathbf{B}^{-1}$); pricing all nonbasic columns costs about vn because only nonzeros of $\mathbf{A}_j$ are used; computing $\mathbf{d}$ costs about m^2; updating $\mathbf{B}^{-1}$ costs about m^2. In total roughly $3m^2 + vn$.

Step 3. When $n \gg m$ and v is small, $3m^2 + vn \ll mn$.

Machine Learning Connection

Machine learning linear programs are typically wide and sparse: an ℓ_1 regression with N samples has $2N$ residual columns that each contain a single nonzero, and an optimal transport problem between two histograms with n_1 and n_2 bins has $n_1 n_2$ columns each with exactly two nonzeros. The pricing step (11.3) is then cheap, and it can even be done lazily, generating only promising columns. That idea, *column generation*, lets solvers handle LPs whose columns are too numerous to list, such as the cutting stock problem and the large scale boosting formulation LPBoost, where each column is a weak learner chosen by solving a pricing problem.

11.5 Worked Examples

Worked Example 11.1 Multipliers and Reduced Costs of the Furniture Optimum

For the furniture workshop in standard form with $\mathbf{A} = \begin{pmatrix} 4 & 2 & 1 & 0 \\ 2 & 3 & 0 & 1 \end{pmatrix}$, $\mathbf{c} = (30, 20, 0, 0)^\top$, $\mathbf{b} = (40, 30)^\top$, take the basis $\{x_1, x_2\}$. Compute $\mathbf{B}^{-1}$, $\mathbf{x}_B$, $\mathbf{y}$, every reduced cost and z, and decide whether the basis is optimal.

Solution

Step 1. **Basis matrix and inverse** (Worked Example 2.4): $\mathbf{B} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$, $\mathbf{B}^{-1} = \begin{pmatrix} 3/8 & -1/4 \\ -1/4 & 1/2 \end{pmatrix}$.

Step 2. **Basic values.** $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b} = (\frac{3}{8}(40) - \frac{1}{4}(30),\ -\frac{1}{4}(40) + \frac{1}{2}(30)) = (15-7.5,\ -10+15) = (7.5,\ 5)$.

Step 3. **Multipliers** by (11.2): $\mathbf{y}^\top = \mathbf{c}_B^\top \mathbf{B}^{-1} = (30,\ 20)\begin{pmatrix} 3/8 & -1/4 \\ -1/4 & 1/2 \end{pmatrix} = (11.25 - 5,\ -7.5 + 10) = (6.25,\ 2.5)$.

Step 4. **Reduced costs** by (11.3): for basic variables they are zero automatically. For the slacks: $\bar{c}_{s_1} = 0 - \mathbf{y}^\top \mathbf{e}_1 = -6.25$ and $\bar{c}_{s_2} = 0 - \mathbf{y}^\top \mathbf{e}_2 = -2.5$.

Step 5. Objective. $z = \mathbf{y}^\top\mathbf{b} = 6.25(40) + 2.5(30) = 250 + 75 = 325$, equal to $\mathbf{c}_B^\top\mathbf{x}_B = 30(7.5) + 20(5)$.

Step 6. Optimality. All reduced costs are ≤ 0, so the basis is optimal, in agreement with Worked Example 8.1, whose final row 0 holds $-\bar{c}_j$: $(0,\ 0,\ 6.25,\ 2.5 \mid 325)$.

Answer. $\boxed{\mathbf{y} = (6.25,\ 2.5),\ \bar{c}_{s_1} = -6.25,\ \bar{c}_{s_2} = -2.5,\ z = 325}$; the basis is optimal.

Worked Example 11.2 A Full Revised Simplex Iteration

For the cluster problem of Worked Example 8.3, max $5x_1 + 4x_2 + 3x_3$ with $\mathbf{A} = \begin{pmatrix} 2 & 3 & 1 & 1 & 0 & 0 \\ 4 & 1 & 2 & 0 & 1 & 0 \\ 3 & 4 & 2 & 0 & 0 & 1 \end{pmatrix}$ (columns $x_1, x_2, x_3, s_1, s_2, s_3$) and $\mathbf{b} = (5, 11, 8)^\top$, start from the basis (x_1, s_2, s_3) reached after one pivot. Carry out one complete revised simplex iteration.

Solution

Step 1. Basis matrix (columns of x_1, s_2, s_3): $\mathbf{B} = \begin{pmatrix} 2 & 0 & 0 \\ 4 & 1 & 0 \\ 3 & 0 & 1 \end{pmatrix}$. Its inverse, found by elimination, is

$$\mathbf{B}^{-1} = \begin{pmatrix} 1/2 & 0 & 0 \\ -2 & 1 & 0 \\ -3/2 & 0 & 1 \end{pmatrix}.$$

Check one entry: row 2 of $\mathbf{B}^{-1}$ times column 1 of $\mathbf{B}$ is $-2(2) + 1(4) + 0 = 0$, as required.

Step 2. Basic values. $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b} = (\frac{5}{2},\ -10 + 11,\ -\frac{15}{2} + 8) = (2.5,\ 1,\ 0.5)$. Objective $5(2.5) = 12.5$.

Step 3. Price. $\mathbf{y}^\top = \mathbf{c}_B^\top\mathbf{B}^{-1} = (5, 0, 0)\mathbf{B}^{-1} = (2.5,\ 0,\ 0)$.

Step 4. Reduced costs of nonbasic variables:

$$\begin{aligned} \bar{c}_2 &= 4 - \mathbf{y}^\top(3, 1, 4)^\top = 4 - 7.5 = -3.5, \\ \bar{c}_3 &= 3 - \mathbf{y}^\top(1, 2, 2)^\top = 3 - 2.5 = 0.5, \\ \bar{c}_{s_1} &= 0 - 2.5 = -2.5. \end{aligned}$$

Step 5. Entering. Only $\bar{c}_3 = 0.5 > 0$, so x_3 enters.

Step 6. Direction. $\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_3 = (\frac{1}{2},\ -2 + 2,\ -\frac{3}{2} + 2) = (0.5,\ 0,\ 0.5)$.

Step 7. Ratio test. Row 1 (x_1): $2.5/0.5 = 5$. Row 2 (s_2): $d_2 = 0$, no ratio. Row 3 (s_3): $0.5/0.5 = 1$. Minimum $\theta^\star = 1$ at row 3, so s_3 leaves.

Step 8. Update values. $\mathbf{x}_B \leftarrow (2.5, 1, 0.5) - 1 \cdot (0.5, 0, 0.5) = (2,\ 1,\ 0)$, and $x_3 = 1$ replaces s_3. New basis (x_1, s_2, x_3) with values $(2, 1, 1)$. Objective $12.5 + 0.5 \times 1 = 13$.

Step 9. Eta update by (11.5) with $r = 3$, $d_r = 0.5$: $\boldsymbol{\eta} = (-0.5/0.5,\ -0/0.5,\ 1/0.5) = (-1,\ 0,\ 2)$. So

$$\mathbf{E} = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \qquad \mathbf{B}'^{-1} = \mathbf{E}\mathbf{B}^{-1} = \begin{pmatrix} 2 & 0 & -1 \\ -2 & 1 & 0 \\ -3 & 0 & 2 \end{pmatrix}.$$

Step 10. Next pricing. $\mathbf{y}^\top = (5,0,3)\mathbf{B}'^{-1} = (10{-}9,\ 0,\ {-5}{+}6) = (1,\ 0,\ 1)$. Then $\bar{c}_2 = 4-(3+4) = -3$, $\bar{c}_{s_1} = -1$, $\bar{c}_{s_3} = -1$. All nonpositive: optimal.

Answer. One iteration brings x_3 in and s_3 out, giving $\boxed{x^\star = (2,0,1),\ z^\star = 13,\ \mathbf{y} = (1,0,1)}$, exactly as in Worked Example 8.3.

Worked Example 11.3 Checking the Eta Matrix by Direct Multiplication

Verify that the matrix $\mathbf{B}'^{-1}$ of Worked Example 11.2 really is the inverse of the new basis matrix $\mathbf{B}' = [\mathbf{A}_1\ \mathbf{A}_{s_2}\ \mathbf{A}_3]$.

Solution

Step 1. New basis matrix. Columns of x_1, s_2, x_3: $\mathbf{B}' = \begin{pmatrix} 2 & 0 & 1 \\ 4 & 1 & 2 \\ 3 & 0 & 2 \end{pmatrix}$.

Step 2. Row 1 of $\mathbf{B}'^{-1}$ is $(2,0,-1)$. Times the columns of $\mathbf{B}'$: $2(2)+0-1(3) = 1$; $0+0+0 = 0$; $2(1)+0-1(2) = 0$. Gives $(1,0,0)$.

Step 3. Row 2 is $(-2,1,0)$: $-4+4+0 = 0$; $0+1+0 = 1$; $-2+2+0 = 0$. Gives $(0,1,0)$.

Step 4. Row 3 is $(-3,0,2)$: $-6+0+6 = 0$; 0; $-3+0+4 = 1$. Gives $(0,0,1)$.

Step 5. Conclusion. $\mathbf{B}'^{-1}\mathbf{B}' = \mathbf{I}$, so the eta update produced the correct inverse without inverting anything.

Step 6. Values check. $\mathbf{B}'^{-1}\mathbf{b} = (10-8,\ -10+11,\ -15+16) = (2,1,1)$, matching the updated values.

Answer. $\boxed{\mathbf{B}'^{-1}\mathbf{B}' = \mathbf{I}}$; the product form update is exact.

Worked Example 11.4 Solving a Problem Completely by the Revised Method

Solve max $2x_1 + 3x_2 + 4x_3$ subject to $3x_1 + 2x_2 + x_3 \le 10$, $2x_1 + 5x_2 + 3x_3 \le 15$, $\mathbf{x} \ge \mathbf{0}$ (Worked Example 8.4) using only matrix operations.

Solution

Step 1. Start. Basis (s_1, s_2), $\mathbf{B} = \mathbf{I}$, $\mathbf{B}^{-1} = \mathbf{I}$, $\mathbf{x}_B = (10,15)$, $\mathbf{c}_B = (0,0)$, so $\mathbf{y} = \mathbf{0}$.

Step 2. Price. $\bar{c}_j = c_j - 0 = (2,3,4)$ for x_1, x_2, x_3. Largest is $\bar{c}_3 = 4$: x_3 enters.

Step 3. Direction. $\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_3 = (1,3)$. Ratios $10/1 = 10$ and $15/3 = 5$; s_2 leaves ($r = 2$).

Step 4. Update. $\mathbf{x}_B = (10,15) - 5(1,3) = (5,0)$, then $x_3 = 5$. New basis (s_1, x_3) with values $(5,5)$.

Step 5. Eta. $\boldsymbol{\eta} = (-1/3,\ 1/3)$, so $\mathbf{B}^{-1} = \begin{pmatrix} 1 & -1/3 \\ 0 & 1/3 \end{pmatrix}$.

Step 6. Price again. $\mathbf{y}^\top = (0,\ 4)\mathbf{B}^{-1} = (0,\ 4/3)$. Reduced costs:

$$\bar{c}_1 = 2 - \tfrac{4}{3}(2) = -\tfrac{2}{3}, \qquad \bar{c}_2 = 3 - \tfrac{4}{3}(5) = -\tfrac{11}{3}, \qquad \bar{c}_{s_2} = 0 - \tfrac{4}{3} = -\tfrac{4}{3}.$$

Step 7. Optimal. All are negative. $z = \mathbf{y}^\top \mathbf{b} = \tfrac{4}{3}(15) = 20$.

Answer. $\boxed{x^\star = (0,0,5),\ z^\star = 20,\ \mathbf{y} = (0,\ \tfrac{4}{3})}$, matching the tableau solution.

Worked Example 11.5 How Much Work Does the Revised Method Save?

An ℓ_1 regression LP has $m = 1{,}000$ constraints and $n = 100{,}000$ columns with an average of $\nu = 5$ nonzeros per column. Compare the operations per pivot for the tableau and revised methods, and estimate the total for 3,000 pivots.

Solution

Step 1. Tableau method. About $mn = 10^3 \times 10^5 = 10^8$ operations per pivot.

Step 2. Revised method. About $3m^2 + \nu n = 3 \times 10^6 + 5 \times 10^5 = 3.5 \times 10^6$ per pivot.

Step 3. Ratio. $10^8/(3.5 \times 10^6) \approx 28.6$: the revised method is roughly 29 times cheaper per pivot.

Step 4. Totals for 3,000 pivots. Tableau: 3×10^{11}. Revised: 1.05×10^{10}. On a machine doing 10^9 useful operations per second, about 300 seconds against about 10 seconds.

Step 5. Memory. The tableau stores 10^8 numbers (800 MB in double precision) and becomes dense. The revised method stores the sparse $\mathbf{A}$ (5×10^5 nonzeros) and an $m\times m$ inverse or factorisation (10^6 numbers at most), a few megabytes.

Answer. About $\boxed{\text{29 times fewer operations}}$ per pivot, and hundreds of times less memory.

Worked Example 11.6 Certifying a Proposed Production Plan

For the annotation company of Worked Example 8.6 (columns $x_1, x_2, x_3, s_1, s_2, s_3$), a manager proposes the basis (x_1, x_2, s_3). Use matrix formulas to check feasibility and optimality without running the simplex method.

Solution

Step 1. Basis matrix from columns $x_1 = (2,1,2)$, $x_2 = (1,3,1)$, $s_3 = (0,0,1)$:

$$\mathbf{B} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 0 \\ 2 & 1 & 1 \end{pmatrix}, \qquad \mathbf{B}^{-1} = \begin{pmatrix} 3/5 & -1/5 & 0 \\ -1/5 & 2/5 & 0 \\ -1 & 0 & 1 \end{pmatrix}.$$

(The upper left 2×2 block has determinant $6-1 = 5$, and its inverse follows from the formula of Module 2.)

Step 2. Feasibility. $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b} = (\tfrac{3}{5}(180) - \tfrac{1}{5}(300),\ -\tfrac{1}{5}(180) + \tfrac{2}{5}(300),\ -180+240) = (48,\ 84,\ 60) \geq \mathbf{0}$. Feasible.

Step 3. **Multipliers.** $\mathbf{y}^\top = (6,5,0)\mathbf{B}^{-1} = (\frac{18}{5} - 1,\ -\frac{6}{5} + 2,\ 0) = (2.6,\ 0.8,\ 0)$.

Step 4. **Reduced costs.** $\bar{c}_3 = 4 - (2.6 \cdot 1 + 0.8 \cdot 2 + 0 \cdot 2) = 4 - 4.2 = -0.2$; $\bar{c}_{s_1} = -2.6$; $\bar{c}_{s_2} = -0.8$.

Step 5. **Verdict.** Feasible and all reduced costs ≤ 0: the basis is optimal, with $z = \mathbf{y}^\top\mathbf{b} = 2.6(180) + 0.8(300) = 468 + 240 = 708$.

Answer. The proposed basis is $\boxed{\text{feasible and optimal}}$, with plan $(48, 84, 0)$ and profit 708.

Module Summary

- In basis form: tableau columns are $\mathbf{B}^{-1}\mathbf{A}_j$, the RHS is $\mathbf{B}^{-1}\mathbf{b}$, row 0 is $\mathbf{y}^\top\mathbf{A} - \mathbf{c}^\top$ with $\mathbf{y}^\top = \mathbf{c}_B^\top\mathbf{B}^{-1}$, and the objective is $\mathbf{y}^\top\mathbf{b}$.
- Reduced costs are found by pricing: $\bar{c}_j = c_j - \mathbf{y}^\top\mathbf{A}_j$.
- A revised iteration: price, choose entering, compute $\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_q$, ratio test, update values, update $\mathbf{B}^{-1}$.
- The new inverse is $\mathbf{E}\mathbf{B}^{-1}$, where $\mathbf{E}$ is the identity with column r replaced by the eta vector $\boldsymbol{\eta}$.
- Any proposed basis can be certified by checking $\mathbf{B}^{-1}\mathbf{b} \geq \mathbf{0}$ and $c_j - \mathbf{y}^\top\mathbf{A}_j \leq 0$.
- For wide, sparse problems the revised method is far cheaper in time and memory, and it enables column generation.

Part III Duality and Analysis

Module 12

Duality Theory

Learning Objectives

After studying this module you will be able to:

- derive the dual of a linear program from the idea of bounding the primal objective, and from the Lagrangian;
- write the dual of any linear program using the primal dual correspondence table;
- state and prove weak duality, and use it to bound optimal values;
- state strong duality and prove it from the optimal simplex tableau;
- read the optimal dual solution from a final simplex tableau;
- interpret dual variables economically, and recognise duals that appear in machine learning.

12.1 The Question That Creates the Dual

In Module 5 we certified that the furniture plan $(7.5, 5)$ was optimal by finding multipliers $y_1 = 6.25$ and $y_2 = 2.5$ such that adding y_1 times the timber constraint to y_2 times the labour constraint produced an upper bound of 325 on every feasible profit. That trick raises a natural question: *what is the best upper bound obtainable in this way?* Finding the best multipliers is itself a linear program, called the **dual**. Duality is the single most important idea in linear programming. It supplies certificates of optimality, prices for resources, alternative algorithms, sensitivity information and, in machine learning, the support vector formulation, the Kantorovich potentials of optimal transport and the equilibrium strategies of games.

12.2 Deriving the Dual by Bounding

Derivation The dual of a canonical maximisation problem

Consider the **primal** problem

$$(P) \qquad \max\ \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} \le \mathbf{b}, \quad \mathbf{x} \ge \mathbf{0}, \tag{12.1}$$

with $\mathbf{A} \in \mathbb{R}^{m\times n}$ having rows $\mathbf{a}_1^\top, \dots, \mathbf{a}_m^\top$.

Step 1. Combine the constraints. Choose multipliers $y_1, \dots, y_m \ge 0$, one per constraint. Multiplying constraint i by $y_i \ge 0$ keeps its direction, and adding gives, for every feasible

x,

$$\sum_{i=1}^{m} y_i \mathbf{a}_i^\top \mathbf{x} \le \sum_{i=1}^{m} y_i b_i, \qquad \text{that is} \qquad \mathbf{y}^\top \mathbf{A}\mathbf{x} \le \mathbf{y}^\top \mathbf{b}.$$

Step 2. Compare with the objective. Write $\mathbf{y}^\top \mathbf{A}\mathbf{x} = \sum_{j=1}^{n} (\mathbf{A}^\top \mathbf{y})_j x_j$. If every coefficient satisfies $(\mathbf{A}^\top \mathbf{y})_j \ge c_j$, then, because each $x_j \ge 0$,

$$\mathbf{c}^\top \mathbf{x} = \sum_j c_j x_j \le \sum_j (\mathbf{A}^\top \mathbf{y})_j x_j = \mathbf{y}^\top \mathbf{A}\mathbf{x}.$$

Step 3. Chain the inequalities. For every feasible $\mathbf{x}$ and every $\mathbf{y} \ge \mathbf{0}$ with $\mathbf{A}^\top \mathbf{y} \ge \mathbf{c}$,

$$\mathbf{c}^\top \mathbf{x} \le \mathbf{y}^\top \mathbf{A}\mathbf{x} \le \mathbf{y}^\top \mathbf{b}.$$

So $\mathbf{b}^\top \mathbf{y}$ is an upper bound on the primal objective.

Step 4. Ask for the best bound. The tightest such bound solves

$$(D) \qquad \min\ \mathbf{b}^\top \mathbf{y} \quad \text{subject to} \quad \mathbf{A}^\top \mathbf{y} \ge \mathbf{c}, \quad \mathbf{y} \ge \mathbf{0}. \tag{12.2}$$

where

$\mathbf{y} \in \mathbb{R}^m$ are the **dual variables**, one for each primal constraint;

$\mathbf{A}^\top$ is the transpose of $\mathbf{A}$: each dual constraint corresponds to one primal variable;

$\mathbf{b}$ becomes the dual objective, and $\mathbf{c}$ becomes the dual right hand side.

Derivation The same dual from the Lagrangian

Step 1. Attach a multiplier $y_i \ge 0$ to each constraint $\mathbf{a}_i^\top \mathbf{x} \le b_i$ and form the **Lagrangian**

$$L(\mathbf{x}, \mathbf{y}) = \mathbf{c}^\top \mathbf{x} + \mathbf{y}^\top (\mathbf{b} - \mathbf{A}\mathbf{x}) = \mathbf{b}^\top \mathbf{y} + (\mathbf{c} - \mathbf{A}^\top \mathbf{y})^\top \mathbf{x}.$$

For feasible $\mathbf{x}$ the term $\mathbf{y}^\top (\mathbf{b} - \mathbf{A}\mathbf{x})$ is nonnegative, so $L(\mathbf{x}, \mathbf{y}) \ge \mathbf{c}^\top \mathbf{x}$.

Step 2. Define the **dual function** $g(\mathbf{y}) = \max_{\mathbf{x} \ge \mathbf{0}} L(\mathbf{x}, \mathbf{y})$. If some component of $\mathbf{c} - \mathbf{A}^\top \mathbf{y}$ is positive, increasing the matching x_j sends L to $+\infty$. Otherwise the maximum is at $\mathbf{x} = \mathbf{0}$. Hence

$$g(\mathbf{y}) = \begin{cases} \mathbf{b}^\top \mathbf{y} & \text{if } \mathbf{A}^\top \mathbf{y} \ge \mathbf{c}, \\ +\infty & \text{otherwise.} \end{cases}$$

Step 3. Minimising g over $\mathbf{y} \ge \mathbf{0}$ gives exactly (12.2).

This route generalises to nonlinear problems (support vector machines, Module 21) and is the route used throughout machine learning.

12.3 The Primal Dual Correspondence

Key Result Rules for writing the dual

For a primal *maximisation* problem, the dual is a *minimisation* problem with one variable per primal constraint and one constraint per primal variable:

Primal (maximise $\mathbf{c}^\top\mathbf{x}$)	**Dual (minimise $\mathbf{b}^\top\mathbf{y}$)**
constraint i: $\mathbf{a}_i^\top\mathbf{x} \le b_i$	variable $y_i \ge 0$
constraint i: $\mathbf{a}_i^\top\mathbf{x} \ge b_i$	variable $y_i \le 0$
constraint i: $\mathbf{a}_i^\top\mathbf{x} = b_i$	variable y_i free
variable $x_j \ge 0$	constraint j: $\mathbf{A}_j^\top\mathbf{y} \ge c_j$
variable $x_j \le 0$	constraint j: $\mathbf{A}_j^\top\mathbf{y} \le c_j$
variable x_j free	constraint j: $\mathbf{A}_j^\top\mathbf{y} = c_j$

To dualise a minimisation problem, read the table from right to left.

Derivation Why an equality gives a free dual variable

Step 1. An equation $\mathbf{a}_i^\top\mathbf{x} = b_i$ is the pair $\mathbf{a}_i^\top\mathbf{x} \le b_i$ and $-\mathbf{a}_i^\top\mathbf{x} \le -b_i$, with multipliers $y_i^+ \ge 0$ and $y_i^- \ge 0$.

Step 2. In the dual these always appear together as $y_i^+\mathbf{a}_i - y_i^-\mathbf{a}_i$ and $y_i^+ b_i - y_i^- b_i$, so only their difference $y_i = y_i^+ - y_i^-$ matters.

Step 3. A difference of two nonnegative numbers can be any real number, so y_i is free.

Likewise a free primal variable $x_j = x_j^+ - x_j^-$ produces two dual constraints $\mathbf{A}_j^\top\mathbf{y} \ge c_j$ and $-\mathbf{A}_j^\top\mathbf{y} \ge -c_j$, that is, an equation.

Key Result The dual of the dual is the primal

Write (12.2) as max $(-\mathbf{b})^\top\mathbf{y}$ subject to $(-\mathbf{A}^\top)\mathbf{y} \le -\mathbf{c}$, $\mathbf{y} \ge \mathbf{0}$, which has the canonical form (12.1). Its dual is min $(-\mathbf{c})^\top\mathbf{x}$ subject to $(-\mathbf{A})\mathbf{x} \ge -\mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$, which is (12.1) again. Duality is therefore a perfectly symmetric relationship.

12.4 The Duality Theorems

Key Result Weak duality

If $\mathbf{x}$ is feasible for (P) and $\mathbf{y}$ is feasible for (D), then

$$\mathbf{c}^\top\mathbf{x} \le \mathbf{b}^\top\mathbf{y}. \tag{12.3}$$

Consequences: (i) if both are feasible, both have finite optimal values; (ii) if (P) is unbounded, (D) is infeasible, and if (D) is unbounded, (P) is infeasible; (iii) if $\mathbf{c}^\top\mathbf{x} = \mathbf{b}^\top\mathbf{y}$ for feasible $\mathbf{x}, \mathbf{y}$, both are optimal.

The proof of (12.3) is Step 3 of the first derivation. Consequence (iii) holds because no primal value can exceed $\mathbf{b}^\top\mathbf{y}$, which $\mathbf{x}$ already attains, and symmetrically for the dual.

Key Result Strong duality

If (P) has an optimal solution $\mathbf{x}^\star$, then (D) has an optimal solution $\mathbf{y}^\star$, and

$$\mathbf{c}^\top\mathbf{x}^\star = \mathbf{b}^\top\mathbf{y}^\star. \tag{12.4}$$

Derivation Proof of strong duality from the simplex method

Step 1. Add slacks so that (P) reads max $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} + \mathbf{s} = \mathbf{b}$, $\mathbf{x}, \mathbf{s} \geq \mathbf{0}$. By the Fundamental Theorem (Module 7) and the termination of the simplex method with Bland's rule (Module 10), there is an optimal basis $\mathbf{B}$.

Step 2. Define $\mathbf{y}^{\star\top} = \mathbf{c}_B^\top\mathbf{B}^{-1}$, the simplex multipliers of Module 11.

Step 3. Optimality means every reduced cost is ≤ 0. For an original variable x_j: $c_j - \mathbf{y}^{\star\top}\mathbf{A}_j \leq 0$, that is $\mathbf{A}^\top\mathbf{y}^\star \geq \mathbf{c}$.

Step 4. For slack s_i, whose column is $\mathbf{e}_i$ and cost 0: $0 - \mathbf{y}^{\star\top}\mathbf{e}_i \leq 0$, that is $y_i^\star \geq 0$.

Step 5. So $\mathbf{y}^\star$ is feasible for (D). Its objective is $\mathbf{b}^\top\mathbf{y}^\star = \mathbf{c}_B^\top\mathbf{B}^{-1}\mathbf{b} = \mathbf{c}_B^\top\mathbf{x}_B^\star = \mathbf{c}^\top\mathbf{x}^\star$.

Step 6. By consequence (iii) of weak duality, $\mathbf{y}^\star$ is optimal for (D) and the values coincide.

Key Result Reading the dual from the final tableau

In the optimal tableau of a canonical maximisation problem, the row 0 entry under slack s_i equals $y_i^\star$, and the row 0 entry under original variable x_j equals the dual surplus $(\mathbf{A}^\top\mathbf{y}^\star)_j - c_j \geq 0$.

Key Result All possible combinations

	(D) optimal	**(D) unbounded**	**(D) infeasible**
(P) optimal	possible, equal values	impossible	impossible
(P) unbounded	impossible	impossible	possible
(P) infeasible	impossible	possible	possible

Lucid Insight The economic story

Imagine a firm that runs the primal: it uses resources $\mathbf{b}$ to make products with profits $\mathbf{c}$. A buyer offers to purchase all the firm's resources at unit prices $\mathbf{y}$. The firm accepts only if, for every product, the resources it needs are worth at least the profit it would earn: $\mathbf{A}^\top\mathbf{y} \geq \mathbf{c}$. The buyer wants to pay as little as possible, min $\mathbf{b}^\top\mathbf{y}$. Strong duality says that at the best prices the buyer pays exactly the firm's maximum profit: the resources are worth precisely what can be made from them.

Machine Learning Connection

Duality is everywhere in machine learning. The support vector machine is almost always solved through its dual, in which each training point receives a multiplier and only the support vectors receive nonzero ones (Module 21). The Wasserstein distance between distributions has a dual in terms of Lipschitz "potential" functions, the basis of the Wasserstein GAN critic (Module 23). In game theory and adversarial training, the two players' optimal mixed strategies are a primal dual pair (Module 24). In reinforcement learning, the dual of the Bellman LP has occupancy measures as variables (Module 25). Recognising a dual pair often halves the work of understanding a model.

12.5 Worked Examples

Worked Example 12.1 The Dual of the Furniture Problem

Write and solve the dual of max $30x_1 + 20x_2$ subject to $4x_1 + 2x_2 \le 40$, $2x_1 + 3x_2 \le 30$, $\mathbf{x} \ge \mathbf{0}$. Interpret the dual and confirm strong duality.

Solution

Step 1. Apply (12.2) with $\mathbf{A}^\top = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$, $\mathbf{b} = (40, 30)$, $\mathbf{c} = (30, 20)$:

$$\min\ 40y_1 + 30y_2 \quad \text{subject to} \quad 4y_1 + 2y_2 \ge 30, \quad 2y_1 + 3y_2 \ge 20, \quad y_1, y_2 \ge 0.$$

Step 2. Interpretation. y_1 is a price per unit of timber and y_2 a price per labour hour. The constraints say the resources in a table must be worth at least £30 and those in a chair at least £20.

Step 3. Vertices of the dual region. Intersect $4y_1 + 2y_2 = 30$ and $2y_1 + 3y_2 = 20$: from the second, $y_1 = 10 - 1.5y_2$; substitute: $40 - 6y_2 + 2y_2 = 30$, so $y_2 = 2.5$ and $y_1 = 6.25$. Axis corners: $(0, 15)$ (check $3 \cdot 15 = 45 \ge 20$) and $(10, 0)$ (check $20 \ge 20$).

Step 4. Evaluate. $w(0, 15) = 450$; $w(6.25, 2.5) = 250 + 75 = 325$; $w(10, 0) = 400$. Minimum 325 at $(6.25, 2.5)$.

Step 5. Strong duality. The primal optimum is 325 (Worked Example 8.1), equal to the dual optimum, and the dual solution equals the row 0 entries under s_1, s_2 in the final tableau.

Answer. $\boxed{\mathbf{y}^\star = (6.25,\ 2.5),\ w^\star = 325 = z^\star}$.

Worked Example 12.2 Dualising a Mixed Problem with the Rules Table

Write the dual of

$$\max\ 5x_1 - 2x_2 + 4x_3 \quad \text{subject to} \quad x_1 + x_2 + x_3 \le 10, \quad 2x_1 - x_3 \ge -4, \quad x_2 + 3x_3 = 9, \quad x_1, x_2, x_3 \ge 0,$$

and verify strong duality using the primal solution $\mathbf{x}^\star = (7, 0, 3)$ and the dual solution $\mathbf{y}^\star = (5,\ 0,\ -\frac{1}{3})$.

Solution

Step 1. **Dual variables** from the table: $y_1 \geq 0$ for the $\leq$ row, $y_2 \leq 0$ for the $\geq$ row, y_3 free for the equation.

Step 2. **Dual objective**: minimise $10y_1 - 4y_2 + 9y_3$.

Step 3. **Dual constraints**, one per primal variable (all $x_j \geq 0$, so all are $\geq$). Read the columns of the primal:

$$\begin{aligned} x_1 &: \quad y_1 + 2y_2 \geq 5, \\ x_2 &: \quad y_1 + y_3 \geq -2, \\ x_3 &: \quad y_1 - y_2 + 3y_3 \geq 4. \end{aligned}$$

Step 4. **Primal feasibility of** $(7, 0, 3)$: $7 + 0 + 3 = 10 \leq 10$; $14 - 3 = 11 \geq -4$; $0 + 9 = 9$. Value $35 - 0 + 12 = 47$.

Step 5. **Dual feasibility of** $(5, 0, -\frac{1}{3})$: $y_1 = 5 \geq 0$, $y_2 = 0 \leq 0$, y_3 free. Constraints: $5 + 0 = 5 \geq 5$; $5 - \frac{1}{3} = \frac{14}{3} \geq -2$; $5 - 0 - 1 = 4 \geq 4$. Value $50 - 0 - 3 = 47$.

Step 6. **Conclusion.** Both feasible with equal values, so both are optimal by weak duality.

Answer. Dual: min $10y_1 - 4y_2 + 9y_3$ subject to the three constraints above, $y_1 \geq 0$, $y_2 \leq 0$, y_3 free; $\boxed{z^\star = w^\star = 47}$.

Worked Example 12.3 Bounding an Optimum without Solving It

For the cluster problem max $5x_1 + 4x_2 + 3x_3$ subject to $2x_1 + 3x_2 + x_3 \leq 5$, $4x_1 + x_2 + 2x_3 \leq 11$, $3x_1 + 4x_2 + 2x_3 \leq 8$, $\mathbf{x} \geq \mathbf{0}$, a colleague offers $\mathbf{x} = (1, 0, 1)$ and an analyst offers prices $\mathbf{y} = (1, 1, 1)$. Use weak duality to bracket the optimal value. Then improve the bracket with $\mathbf{y}' = (1, 0, 1)$.

Solution

Step 1. **Primal feasibility of x**: $2 + 1 = 3 \leq 5$; $4 + 2 = 6 \leq 11$; $3 + 2 = 5 \leq 8$. Value $z = 5 + 3 = 8$.

Step 2. **Dual feasibility of y** $= (1, 1, 1)$: the dual constraints are $\mathbf{A}^\top \mathbf{y} \geq \mathbf{c}$. Column sums: $2 + 4 + 3 = 9 \geq 5$; $3 + 1 + 4 = 8 \geq 4$; $1 + 2 + 2 = 5 \geq 3$; and $\mathbf{y} \geq \mathbf{0}$. Value $5 + 11 + 8 = 24$.

Step 3. **Bracket.** By (12.3), $8 \leq z^\star \leq 24$.

Step 4. **Better prices** $\mathbf{y}' = (1, 0, 1)$: $2 + 3 = 5 \geq 5$; $3 + 4 = 7 \geq 4$; $1 + 2 = 3 \geq 3$. Feasible, with value $5 + 8 = 13$.

Step 5. **New bracket.** $8 \leq z^\star \leq 13$. Since the plan $(2, 0, 1)$ achieves 13 (Worked Example 8.3), the bound is tight and both $(2, 0, 1)$ and $\mathbf{y}'$ are optimal.

Answer. First $\boxed{8 \leq z^\star \leq 24}$, then $8 \leq z^\star \leq 13$; in fact $z^\star = 13$.

Worked Example 12.4 The Diet Problem and the Pill Seller

Write the dual of the diet problem min $2x_1 + 3x_2$ subject to $3x_1 + 4x_2 \geq 24$, $x_1 + 3x_2 \geq 12$, $\mathbf{x} \geq \mathbf{0}$, interpret it, and solve it.

Solution

Step 1. **Primal is a minimisation with $\geq$ rows**, so reading the table from right to left: the dual is a maximisation with variables $y_1, y_2 \geq 0$ and constraints of type $\leq$:

$$\max\ 24y_1 + 12y_2 \quad \text{subject to} \quad 3y_1 + y_2 \leq 2, \quad 4y_1 + 3y_2 \leq 3, \quad y_1, y_2 \geq 0.$$

Step 2. **Interpretation.** A seller of pure protein and fibre pills sets prices y_1, y_2 per gram. To compete with food A (cost 2, containing 3 g protein and 1 g fibre) the pills replacing it must cost at most 2; similarly for food B. The seller maximises revenue from supplying 24 g and 12 g.

Step 3. **Solve graphically.** Intersect $3y_1 + y_2 = 2$ and $4y_1 + 3y_2 = 3$: from the first, $y_2 = 2 - 3y_1$; substitute: $4y_1 + 6 - 9y_1 = 3$, so $y_1 = 0.6$ and $y_2 = 0.2$. Other corners: $(0, 1)$ with value 12; $(\frac{2}{3}, 0)$ with value 16; $(0, 0)$. The intersection gives $24(0.6) + 12(0.2) = 14.4 + 2.4 = 16.8$, the largest.

Step 4. **Strong duality.** The minimum diet cost is 16.8 (Worked Example 5.2), equal to the maximum pill revenue.

Answer. $\boxed{y^\star = (0.6,\ 0.2),\ w^\star = 16.8}$, equal to the minimum diet cost.

Worked Example 12.5 Reading the Dual from a Final Tableau

From the final tableau of the annotation company problem (Worked Example 8.6), read the optimal dual solution, verify dual feasibility directly, and confirm strong duality.

Solution

Step 1. **Final row 0** was $(0,\ 0,\ 0.2,\ 2.6,\ 0.8,\ 0 \mid 708)$ under $(x_1, x_2, x_3, s_1, s_2, s_3)$.

Step 2. **Dual solution** = entries under the slacks: $\mathbf{y}^\star = (2.6,\ 0.8,\ 0)$.

Step 3. **Dual problem.** $\min\ 180y_1 + 300y_2 + 240y_3$ subject to $2y_1 + y_2 + 2y_3 \geq 6$, $y_1 + 3y_2 + y_3 \geq 5$, $y_1 + 2y_2 + 2y_3 \geq 4$, $\mathbf{y} \geq \mathbf{0}$.

Step 4. **Check feasibility.** $5.2 + 0.8 + 0 = 6 \geq 6$; $2.6 + 2.4 + 0 = 5 \geq 5$; $2.6 + 1.6 + 0 = 4.2 \geq 4$. The surpluses 0, 0, 0.2 are exactly the row 0 entries under x_1, x_2, x_3.

Step 5. **Dual objective.** $180(2.6) + 300(0.8) + 0 = 468 + 240 = 708$, equal to $z^\star$.

Answer. $\boxed{\mathbf{y}^\star = (2.6,\ 0.8,\ 0),\ w^\star = 708 = z^\star}$.

Worked Example 12.6 The Dual of a Minimax Problem Is a Worst Case Mixture

Write the dual of the load balancing problem of Worked Example 4.2,

$$\min\ t \quad \text{subject to} \quad t - 2x_1 - x_2 \geq 0, \quad t - x_1 - 3x_2 \geq 0, \quad x_1 + x_2 \geq 4, \quad x_1, x_2 \geq 0,\ t \text{ free},$$

solve it, and interpret the dual variables.

Solution

Step 1. **Dual variables.** The primal is a minimisation with three ≥ rows, so the dual has $u_1, u_2, v \geq 0$ and is a maximisation of $0u_1 + 0u_2 + 4v$.

Step 2. **Dual constraints.** The free variable t gives an equation; $x_1, x_2 \geq 0$ give ≤ constraints. Reading columns:

$$t: \ u_1 + u_2 = 1, \qquad x_1: \ -2u_1 - u_2 + v \leq 0, \qquad x_2: \ -u_1 - 3u_2 + v \leq 0.$$

Step 3. **Simplify.** With $u_2 = 1 - u_1$, the constraints become $v \leq 2u_1 + (1 - u_1) = 1 + u_1$ and $v \leq u_1 + 3(1 - u_1) = 3 - 2u_1$. We maximise $4v$, so $v = \min(1 + u_1,\ 3 - 2u_1)$.

Step 4. **Maximise the minimum.** The first expression increases and the second decreases in u_1, so the best u_1 makes them equal: $1 + u_1 = 3 - 2u_1$, giving $u_1 = \frac{2}{3}$, $u_2 = \frac{1}{3}$, $v = \frac{5}{3}$.

Step 5. **Value.** $4v = \frac{20}{3}$, equal to the primal optimum $t^\star = \frac{20}{3}$.

Step 6. **Interpretation.** (u_1, u_2) is a probability distribution over the two servers (they are nonnegative and sum to one). The dual asks: which weighted average of the server loads is hardest to keep small? The primal minimises the worst server; the dual maximises over mixtures of servers. These multipliers are exactly the weights 2 and 1 (scaled to $\frac{2}{3}$ and $\frac{1}{3}$) used in the optimality proof of Worked Example 4.2.

Answer. $\boxed{u^\star = (\frac{2}{3}, \frac{1}{3}),\ v^\star = \frac{5}{3},\ w^\star = \frac{20}{3} = t^\star}$. Minimax problems have dual “adversarial mixtures”, the idea behind Module 24.

Module Summary

- The dual of $\max\{\mathbf{c}^\top\mathbf{x} : \mathbf{A}\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ is $\min\{\mathbf{b}^\top\mathbf{y} : \mathbf{A}^\top\mathbf{y} \geq \mathbf{c}, \mathbf{y} \geq \mathbf{0}\}$; it seeks the best upper bound obtainable by combining constraints.
- Constraints map to variables and variables to constraints; = constraints give free variables and free variables give = constraints. The dual of the dual is the primal.
- Weak duality: $\mathbf{c}^\top\mathbf{x} \leq \mathbf{b}^\top\mathbf{y}$ for all feasible pairs. Equal values certify optimality.
- Strong duality: optimal values coincide; $\mathbf{y}^{\star\top} = \mathbf{c}_B^\top\mathbf{B}^{-1}$ is the dual optimum and appears under the slacks in row 0.
- Dual variables are resource prices; in ML they are support vector weights, transport potentials and adversarial mixtures.

MODULE 13

Complementary Slackness, Farkas' Lemma and Optimality Conditions

Learning Objectives

After studying this module you will be able to:

- derive the duality gap identity and from it the complementary slackness conditions;
- use complementary slackness to recover a dual solution from a primal one, and the reverse;
- test whether a proposed solution is optimal without running the simplex method;
- state Farkas' lemma, prove it from LP duality, and build certificates of infeasibility;
- write the Karush Kuhn Tucker (KKT) conditions for a linear program and explain why they are necessary and sufficient;
- interpret complementary slackness in machine learning, for example as the "above, below or on" rule of the median.

13.1 Where the Duality Gap Lives

Weak duality says $\mathbf{c}^\top\mathbf{x} \le \mathbf{b}^\top\mathbf{y}$ for any feasible primal and dual pair. The difference $\mathbf{b}^\top\mathbf{y} - \mathbf{c}^\top\mathbf{x}$ is called the **duality gap**. Strong duality says it can be closed. This module asks: *exactly which terms make up the gap, and what must happen for every one of them to vanish?* The answer, complementary slackness, is a precise and very practical description of optimality.

Throughout, the primal and dual are the canonical pair of Module 12:

$$(P)\ \max\ \mathbf{c}^\top\mathbf{x}\ \text{ subject to }\ \mathbf{A}\mathbf{x} \le \mathbf{b},\ \mathbf{x} \ge \mathbf{0}, \qquad (D)\ \min\ \mathbf{b}^\top\mathbf{y}\ \text{ subject to }\ \mathbf{A}^\top\mathbf{y} \ge \mathbf{c},\ \mathbf{y} \ge \mathbf{0}. \tag{13.1}$$

13.2 Complementary Slackness

Derivation The duality gap identity

Introduce the primal slack $\mathbf{s} = \mathbf{b} - \mathbf{A}\mathbf{x} \ge \mathbf{0}$ and the dual surplus $\mathbf{t} = \mathbf{A}^\top\mathbf{y} - \mathbf{c} \ge \mathbf{0}$.

Step 1. Substitute $\mathbf{b} = \mathbf{A}\mathbf{x} + \mathbf{s}$ into the dual objective: $\mathbf{b}^\top\mathbf{y} = (\mathbf{A}\mathbf{x} + \mathbf{s})^\top\mathbf{y} = \mathbf{x}^\top\mathbf{A}^\top\mathbf{y} + \mathbf{s}^\top\mathbf{y}$.

Step 2. Substitute $\mathbf{c} = \mathbf{A}^\top\mathbf{y} - \mathbf{t}$ into the primal objective: $\mathbf{c}^\top\mathbf{x} = (\mathbf{A}^\top\mathbf{y} - \mathbf{t})^\top\mathbf{x} = \mathbf{x}^\top\mathbf{A}^\top\mathbf{y} - \mathbf{t}^\top\mathbf{x}$.

Step 3. Subtract:

$$\mathbf{b}^\top\mathbf{y} - \mathbf{c}^\top\mathbf{x} = \mathbf{s}^\top\mathbf{y} + \mathbf{t}^\top\mathbf{x} = \sum_{i=1}^{m} s_i y_i + \sum_{j=1}^{n} t_j x_j. \tag{13.2}$$

where

$s_i = b_i - \mathbf{a}_i^\top\mathbf{x}$ is the unused amount of primal resource i;

y_i is the dual price of resource i;

$t_j = \mathbf{A}_j^\top\mathbf{y} - c_j$ is the amount by which the imputed cost of activity j exceeds its profit;

x_j is the level of activity j.

Every term on the right is a product of two nonnegative numbers, so the gap is a sum of nonnegative terms.

Key Result Complementary slackness theorem

Let $\mathbf{x}$ be feasible for (P) and $\mathbf{y}$ feasible for (D). Then both are optimal if and only if

$$y_i\,(b_i - \mathbf{a}_i^\top\mathbf{x}) = 0 \quad \text{for every } i, \qquad \text{and} \qquad x_j\,(\mathbf{A}_j^\top\mathbf{y} - c_j) = 0 \quad \text{for every } j. \tag{13.3}$$

In words: *if a resource is not used up, its price is zero; if a resource has a positive price, it is used up; if an activity is used, it exactly breaks even at the dual prices; if an activity is unprofitable at the dual prices, it is not used.*

Derivation Proof of the theorem

Step 1. By (13.2), the gap is zero exactly when $\sum_i s_i y_i + \sum_j t_j x_j = 0$.

Step 2. A sum of nonnegative terms is zero if and only if every term is zero, which is (13.3).

Step 3. By weak and strong duality (Module 12), a feasible pair is optimal if and only if the gap is zero.

Common Pitfall

Complementary slackness says that *at least one* factor in each product is zero; it does not forbid both from being zero. A tight constraint can have a zero price (this happens at degenerate vertices), and a nonbasic variable can have zero reduced cost (this happens with multiple optima, Module 10).

Key Result Recipe: recovering the dual from a primal optimum

1. For every constraint with positive slack at $\mathbf{x}^\star$, set $y_i = 0$.
2. For every variable with $x_j^\star > 0$, write the dual constraint as an equation: $\mathbf{A}_j^\top\mathbf{y} = c_j$.
3. Solve the resulting linear system for the remaining y_i.
4. Check $\mathbf{y} \geq \mathbf{0}$ and all remaining dual constraints. If they hold, $\mathbf{x}^\star$ and $\mathbf{y}$ are both optimal; if any fails, $\mathbf{x}^\star$ was not optimal (provided the system had a unique solution).

13.3 Farkas' Lemma: Certificates of Infeasibility

Duality also answers the question "how can we *prove* that a system has no solution?"

Key Result Farkas' lemma

For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{b} \in \mathbb{R}^{m}$, exactly one of the following holds:

1. there exists $\mathbf{x} \geq \mathbf{0}$ with $\mathbf{A}\mathbf{x} \leq \mathbf{b}$;
2. there exists $\mathbf{y} \geq \mathbf{0}$ with $\mathbf{A}^\top\mathbf{y} \geq \mathbf{0}$ and $\mathbf{b}^\top\mathbf{y} < 0$.

A vector $\mathbf{y}$ as in (2) is a **certificate of infeasibility** for system (1).

Derivation Proof of Farkas' lemma from duality

Step 1. **Both cannot hold.** If $\mathbf{x}$ satisfies (1) and $\mathbf{y}$ satisfies (2), then $0 \leq (\mathbf{A}^\top\mathbf{y})^\top\mathbf{x} = \mathbf{y}^\top\mathbf{A}\mathbf{x} \leq \mathbf{y}^\top\mathbf{b} < 0$, a contradiction. (The first inequality uses $\mathbf{A}^\top\mathbf{y} \geq \mathbf{0}$ and $\mathbf{x} \geq \mathbf{0}$; the second uses $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ and $\mathbf{y} \geq \mathbf{0}$.)

Step 2. **At least one holds.** Consider the LP max $\mathbf{0}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \leq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$. Its dual is min $\mathbf{b}^\top\mathbf{y}$ subject to $\mathbf{A}^\top\mathbf{y} \geq \mathbf{0}$, $\mathbf{y} \geq \mathbf{0}$.

Step 3. The dual is always feasible ($\mathbf{y} = \mathbf{0}$ works). Also, if $\mathbf{y}$ is dual feasible then so is $\lambda\mathbf{y}$ for every $\lambda \geq 0$.

Step 4. Suppose (1) fails, so the primal is infeasible. By the table of Module 12, a feasible dual paired with an infeasible primal must be unbounded. Hence some dual feasible $\mathbf{y}$ has $\mathbf{b}^\top\mathbf{y} < 0$, which is (2).

Lucid Insight What a certificate looks like

A Farkas certificate is a recipe for combining the constraints with nonnegative weights so that the left hand side is "obviously nonnegative" while the right hand side is negative: the combination says (something ≥ 0) $\leq$ (a negative number), which is impossible. In Worked Example 5.5 we found such a combination by hand. Solvers return these certificates (often called "dual rays") whenever they report infeasibility.

13.4 The KKT Conditions for Linear Programs

Key Result KKT conditions

A vector $\mathbf{x}$ is optimal for (P) if and only if there is a vector $\mathbf{y}$ such that

$$\mathbf{A}\mathbf{x} \leq \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0} \qquad \text{(primal feasibility)} \tag{13.4}$$

$$\mathbf{A}^\top\mathbf{y} \geq \mathbf{c}, \quad \mathbf{y} \geq \mathbf{0} \qquad \text{(dual feasibility)} \tag{13.5}$$

$$\mathbf{y}^\top(\mathbf{b} - \mathbf{A}\mathbf{x}) = 0, \quad \mathbf{x}^\top(\mathbf{A}^\top\mathbf{y} - \mathbf{c}) = 0 \qquad \text{(complementary slackness)} \tag{13.6}$$

For linear programs these conditions are both necessary and sufficient, with no extra regularity assumptions.

Derivation The KKT conditions as stationarity of the Lagrangian

Step 1. Attach $\mathbf{y} \geq \mathbf{0}$ to $\mathbf{Ax} \leq \mathbf{b}$ and $\boldsymbol{\mu} \geq \mathbf{0}$ to $\mathbf{x} \geq \mathbf{0}$ (written $-\mathbf{x} \leq \mathbf{0}$). For maximisation the Lagrangian is $L = \mathbf{c}^\top \mathbf{x} - \mathbf{y}^\top(\mathbf{Ax} - \mathbf{b}) + \boldsymbol{\mu}^\top \mathbf{x}$.

Step 2. **Stationarity**: the gradient with respect to $\mathbf{x}$ vanishes: $\mathbf{c} - \mathbf{A}^\top \mathbf{y} + \boldsymbol{\mu} = \mathbf{0}$, so $\boldsymbol{\mu} = \mathbf{A}^\top \mathbf{y} - \mathbf{c}$.

Step 3. The condition $\boldsymbol{\mu} \geq \mathbf{0}$ is then dual feasibility $\mathbf{A}^\top \mathbf{y} \geq \mathbf{c}$.

Step 4. The complementarity conditions $y_i(b_i - \mathbf{a}_i^\top \mathbf{x}) = 0$ and $\mu_j x_j = 0$ are exactly (13.3).

These are the same KKT conditions used in every constrained optimisation problem in machine learning; for linear programs they are particularly clean.

Machine Learning Connection

Complementary slackness explains the sparsity of many learning methods. In the support vector machine (Module 21), a training point that lies strictly outside the margin has a slack constraint, so its dual multiplier is zero: only the support vectors (on or inside the margin) carry weight. In ℓ_1 regression and quantile regression (Modules 19 and 20), each data point has a dual variable in $[-1, 1]$ or $[\tau - 1, \tau]$ that equals the boundary value when the point lies above or below the fit, and is strictly inside only for points exactly on the fit. In optimal transport (Module 23), mass flows only along routes whose cost equals the difference of the dual potentials. Recognising these patterns lets you check solver output, design faster algorithms and explain models to stakeholders.

13.5 Worked Examples

Worked Example 13.1 Recovering Dual Prices by Complementary Slackness

For the cluster problem max $5x_1 + 4x_2 + 3x_3$ subject to $2x_1 + 3x_2 + x_3 \leq 5$, $4x_1 + x_2 + 2x_3 \leq 11$, $3x_1 + 4x_2 + 2x_3 \leq 8$, $\mathbf{x} \geq \mathbf{0}$, you are told that $\mathbf{x}^\star = (2, 0, 1)$. Find the dual solution using complementary slackness and confirm optimality.

Solution

Step 1. **Slacks at $\mathbf{x}^\star$.** Constraint 1: $4 + 0 + 1 = 5$, slack 0. Constraint 2: $8 + 0 + 2 = 10$, slack $1 > 0$. Constraint 3: $6 + 0 + 2 = 8$, slack 0.

Step 2. **Rule 1.** Constraint 2 has positive slack, so $y_2 = 0$.

Step 3. **Rule 2.** $x_1 = 2 > 0$ and $x_3 = 1 > 0$, so their dual constraints are equations:

$$2y_1 + 4y_2 + 3y_3 = 5, \qquad y_1 + 2y_2 + 2y_3 = 3.$$

Step 4. **Solve** with $y_2 = 0$: $2y_1 + 3y_3 = 5$ and $y_1 + 2y_3 = 3$. From the second, $y_1 = 3 - 2y_3$; substitute: $6 - 4y_3 + 3y_3 = 5$, so $y_3 = 1$, $y_1 = 1$.

Step 5. **Check the remaining dual constraint** (for x_2): $3y_1 + y_2 + 4y_3 = 3 + 0 + 4 = 7 \geq 4$. Holds. And $\mathbf{y} = (1, 0, 1) \geq \mathbf{0}$.

Step 6. **Compare values.** $\mathbf{b}^\top \mathbf{y} = 5 + 0 + 8 = 13 = 5(2) + 3(1) = \mathbf{c}^\top \mathbf{x}^\star$.

Answer. $\boxed{\mathbf{y}^\star = (1, 0, 1)}$, and $\mathbf{x}^\star = (2, 0, 1)$ is confirmed optimal with value 13.

Worked Example 13.2 Recovering the Primal from the Dual

The dual of the diet problem (Worked Example 12.4) has optimum $\mathbf{y}^\star = (0.6,\ 0.2)$. Use complementary slackness to find the optimal diet.

Solution

Step 1. **The pair.** Primal: $\min\ 2x_1 + 3x_2$ with $3x_1 + 4x_2 \geq 24$, $x_1 + 3x_2 \geq 12$, $\mathbf{x} \geq \mathbf{0}$. Dual: $\max\ 24y_1 + 12y_2$ with $3y_1 + y_2 \leq 2$, $4y_1 + 3y_2 \leq 3$, $\mathbf{y} \geq \mathbf{0}$.

Step 2. **Dual slacks at $\mathbf{y}^\star$.** $3(0.6) + 0.2 = 2$: tight. $4(0.6) + 3(0.2) = 3$: tight. No information forcing an x_j to zero.

Step 3. **Dual variables are positive** $(0.6 > 0, 0.2 > 0)$, so both primal constraints must be tight:

$$3x_1 + 4x_2 = 24, \qquad x_1 + 3x_2 = 12.$$

Step 4. **Solve.** From the second, $x_1 = 12 - 3x_2$; substitute: $36 - 9x_2 + 4x_2 = 24$, so $x_2 = 2.4$, $x_1 = 4.8$.

Step 5. **Check $\mathbf{x} \geq \mathbf{0}$** and cost $2(4.8) + 3(2.4) = 16.8 = 24(0.6) + 12(0.2)$.

Answer. $\boxed{\mathbf{x}^\star = (4.8,\ 2.4)}$, with equal primal and dual values 16.8.

Worked Example 13.3 Rejecting a Proposed Solution

A planner claims that making only chairs, $\mathbf{x} = (0, 10)$, is optimal for the furniture problem. Use complementary slackness to prove or refute the claim.

Solution

Step 1. **Feasibility.** Timber: $0 + 20 = 20 \leq 40$, slack 20. Labour: $0 + 30 = 30 \leq 30$, slack 0. Feasible, with profit 200.

Step 2. **Rule 1.** Timber has slack, so $y_1 = 0$.

Step 3. **Rule 2.** $x_2 = 10 > 0$, so its dual constraint is tight: $2y_1 + 3y_2 = 20$, giving $y_2 = \frac{20}{3}$.

Step 4. **Check the other dual constraint** (for x_1): $4y_1 + 2y_2 = 0 + \frac{40}{3} \approx 13.33$, but it must be at least 30. *Violated.*

Step 5. **Conclusion.** The only prices compatible with the plan fail dual feasibility, so the plan is not optimal. The violation says that at these prices a table earns £30 while consuming resources worth only £13.33: tables are underused.

Answer. The claim is $\boxed{\text{false}}$; the reduced cost of tables, $30 - 13.33 = 16.67 > 0$, shows they should be produced.

Worked Example 13.4 A Farkas Certificate of Infeasibility

Find a Farkas certificate proving that there is no $\mathbf{x} \geq \mathbf{0}$ with $x_1 + 2x_2 \leq 4$, $3x_1 + x_2 \geq 15$, $x_1 \leq 3$.

Solution

Step 1. Write as $\mathbf{Ax} \leq \mathbf{b}$. Flip the $\geq$ row: $-3x_1 - x_2 \leq -15$. So

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ -3 & -1 \\ 1 & 0 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 4 \\ -15 \\ 3 \end{pmatrix}.$$

Step 2. Requirements for $\mathbf{y} \geq \mathbf{0}$. $\mathbf{A}^\top \mathbf{y} \geq \mathbf{0}$ means

$$y_1 - 3y_2 + y_3 \geq 0, \qquad 2y_1 - y_2 \geq 0,$$

and we need $\mathbf{b}^\top \mathbf{y} = 4y_1 - 15y_2 + 3y_3 < 0$.

Step 3. Try $y_2 = 1$ (we must use the troublesome row). The second inequality needs $y_1 \geq 0.5$; take $y_1 = 0.5$. The first needs $0.5 - 3 + y_3 \geq 0$, so $y_3 \geq 2.5$; take $y_3 = 2.5$.

Step 4. Evaluate. $\mathbf{b}^\top \mathbf{y} = 4(0.5) - 15(1) + 3(2.5) = 2 - 15 + 7.5 = -5.5 < 0$.

Step 5. Read the certificate. Adding 0.5×(row 1) +1×(row 2) +2.5×(row 3):

$$(0.5 - 3 + 2.5)x_1 + (1 - 1 + 0)x_2 \leq -5.5, \qquad \text{that is} \qquad 0 \leq -5.5,$$

which is impossible.

Step 6. Connection. The value 5.5 is exactly the Phase I optimum of Worked Example 9.4: the smallest total violation.

Answer. $\boxed{\mathbf{y} = (0.5,\ 1,\ 2.5)}$ is a certificate: it combines the constraints into $0 \leq -5.5$.

Worked Example 13.5 Checking the KKT Conditions

For max $3x_1 + 2x_2$ subject to $x_1 + x_2 \leq 4$, $x_1 + 3x_2 \leq 6$, $x_1 \leq 3$, $\mathbf{x} \geq \mathbf{0}$, verify the KKT conditions at $\mathbf{x} = (3, 1)$ and find the multipliers.

Solution

Step 1. Primal feasibility. $3 + 1 = 4 \leq 4$ (tight); $3 + 3 = 6 \leq 6$ (tight); $3 \leq 3$ (tight). Three active constraints in the plane: a degenerate vertex.

Step 2. Complementary slackness on variables. $x_1, x_2 > 0$, so both dual constraints are equations:

$$y_1 + y_2 + y_3 = 3, \qquad y_1 + 3y_2 = 2.$$

Step 3. Solve. Two equations, three unknowns: the multipliers are not unique, as expected at a degenerate vertex. From the second, $y_1 = 2 - 3y_2$; then $y_3 = 3 - y_1 - y_2 = 1 + 2y_2$.

Step 4. Dual feasibility requires $y_1 = 2 - 3y_2 \geq 0$, $y_2 \geq 0$, $y_3 = 1 + 2y_2 \geq 0$, so any $y_2 \in [0, \frac{2}{3}]$ works.

Step 5. **Examples.** $y_2 = 0$: $\mathbf{y} = (2, 0, 1)$, value $8 + 0 + 3 = 11$. $y_2 = \frac{2}{3}$: $\mathbf{y} = (0, \frac{2}{3}, \frac{7}{3})$, value $0 + 4 + 7 = 11$. Both equal $\mathbf{c}^\top\mathbf{x} = 9 + 2 = 11$.

Step 6. **Conclusion.** The KKT conditions hold, so $\mathbf{x} = (3, 1)$ is optimal, with a whole segment of optimal dual solutions.

Answer. $\mathbf{x}^\star = (3, 1)$ is optimal with $z^\star = 11$; the multipliers are $\boxed{\mathbf{y} = (2 - 3\lambda,\ \lambda,\ 1 + 2\lambda),\ 0 \le \lambda \le \frac{2}{3}}$.

Worked Example 13.6 Complementary Slackness for the Median

The median of data $y_1, \dots, y_N$ minimises $\sum_i |y_i - m|$. For the data $(1,\ 2,\ 7)$: (a) write the LP and its dual; (b) use complementary slackness to show that $m^\star = 2$ and find the dual solution.

Solution

Step 1. **Primal** (Module 4): $\min\ \sum_i t_i$ subject to $t_i + m \ge y_i$ (multiplier $u_i \ge 0$) and $t_i - m \ge -y_i$ (multiplier $v_i \ge 0$), with m and t_i free.

Step 2. **Dual** (minimisation with $\ge$ rows gives a maximisation with nonnegative variables; free primal variables give equations). Columns: t_i gives $u_i + v_i = 1$; m gives $\sum_i (u_i - v_i) = 0$. Objective $\sum_i y_i(u_i - v_i)$. With $w_i = u_i - v_i$, the constraints $u_i, v_i \ge 0$, $u_i + v_i = 1$ mean exactly $w_i \in [-1, 1]$:

$$\max \sum_i y_i w_i \quad \text{subject to} \quad \sum_i w_i = 0, \quad -1 \le w_i \le 1.$$

Step 3. **Complementary slackness.** At the optimum $t_i = |y_i - m|$. If $y_i > m$, the constraint $t_i - m \ge -y_i$ is slack, so $v_i = 0$, $u_i = 1$, $w_i = 1$. If $y_i < m$, then $u_i = 0$ and $w_i = -1$. If $y_i = m$, w_i may lie anywhere in $[-1, 1]$.

Step 4. **Balance condition.** $\sum_i w_i = 0$ requires as many points above m as below, up to the flexible weights of points exactly at m. For $(1, 2, 7)$ with $m = 2$: $w_1 = -1$ (below), $w_3 = +1$ (above), and w_2 must be 0 to balance. Valid, since $0 \in [-1, 1]$.

Step 5. **Values.** Primal: $|1 - 2| + 0 + |7 - 2| = 6$. Dual: $1(-1) + 2(0) + 7(1) = 6$. Equal, so both optimal.

Step 6. **Why not $m = 3$?** Then points 1 and 2 are below ($w = -1$ each) and 7 above ($w = +1$): $\sum w_i = -1 \ne 0$, and no point sits at m to absorb the imbalance. Complementary slackness fails, so $m = 3$ is not optimal.

Answer. $\boxed{m^\star = 2,\ \mathbf{w}^\star = (-1, 0, 1)}$, total deviation 6. The dual weights are the "signs" of the residuals, the subgradient of the absolute loss.

Module Summary

- Duality gap: $\mathbf{b}^\top\mathbf{y} - \mathbf{c}^\top\mathbf{x} = \mathbf{s}^\top\mathbf{y} + \mathbf{t}^\top\mathbf{x}$, a sum of products of nonnegative numbers.
- Complementary slackness: $y_i s_i = 0$ and $x_j t_j = 0$ for all i, j, if and only if the feasible pair is optimal.

- From a primal optimum: zero prices on slack constraints, tight dual constraints for positive variables, solve, check.
- Farkas: either $\mathbf{Ax} \leq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$ is solvable, or some $\mathbf{y} \geq \mathbf{0}$ has $\mathbf{A}^\top\mathbf{y} \geq \mathbf{0}$, $\mathbf{b}^\top\mathbf{y} < 0$, never both.
- KKT for LP = primal feasibility + dual feasibility + complementary slackness; necessary and sufficient.
- In ML, complementary slackness produces support vectors, residual signs and sparse transport plans.

MODULE 14

The Dual Simplex Method and Reoptimisation

Learning Objectives

After studying this module you will be able to:

- explain the difference between primal feasibility and dual feasibility of a tableau;
- derive the leaving rule and the dual ratio test of the dual simplex method;
- solve minimisation problems with ≥ constraints directly, without artificial variables;
- reoptimise quickly after adding a new constraint or changing a right hand side;
- detect primal infeasibility with the dual simplex method;
- explain why warm started reoptimisation matters in machine learning pipelines.

14.1 Two Kinds of Feasibility in One Tableau

Every simplex tableau carries two vectors that can be checked separately.

- The **right hand side column $\mathbf{B}^{-1}\mathbf{b}$** gives the primal values. If it is nonnegative, the tableau is **primal feasible**.
- The **row 0 entries $\mathbf{y}^\top\mathbf{A} - \mathbf{c}^\top$** are the dual surpluses (Module 12). If they are all nonnegative (for maximisation), the multipliers $\mathbf{y}$ satisfy the dual constraints and the tableau is **dual feasible**. That is exactly the optimality test.

The primal simplex method of Module 8 keeps the tableau primal feasible and works towards dual feasibility. The **dual simplex method**, introduced by Lemke in 1954, does the opposite: it keeps the tableau dual feasible (row 0 nonnegative) and works towards primal feasibility. When both hold, the tableau is optimal.

Lucid Insight When is a dual feasible start available?

Two situations make the dual simplex method natural. First, a *minimisation* problem with nonnegative costs and ≥ constraints: after multiplying the ≥ rows by -1 and adding slacks, the slack basis is dual feasible (row 0 holds the costs, all nonnegative) but primal infeasible (negative right hand sides). Second, *reoptimisation*: after solving a problem, a constraint is added or a right hand side changes; the old optimal row 0 is still dual feasible, but some basic variable may become negative.

14.2 Deriving the Rules

Derivation The leaving variable and the dual ratio test

Consider a maximisation tableau whose row 0 entries $\delta_j \geq 0$ for every column j (dual feasible), but with some row r having $\bar{b}_r < 0$ (primal infeasible).

Step 1. Choose the leaving row. Pick a row with negative right hand side; the usual choice is the most negative $\bar{b}_r$. Its basic variable must leave, because it currently violates $x_{B(r)} \geq 0$.

Step 2. Which columns can enter? Row r reads $x_{B(r)} + \sum_{j \in N} \bar{a}_{rj} x_j = \bar{b}_r < 0$. To raise $x_{B(r)}$ to zero by increasing a nonbasic x_j, we need $\bar{a}_{rj} < 0$. If no entry in row r is negative, then the left side is at least $x_{B(r)} \geq 0$ for every nonnegative choice of variables, yet it must equal a negative number: the primal problem is **infeasible**.

Step 3. Preserve dual feasibility. Pivoting on $\bar{a}_{rq} < 0$ changes row 0 to $\delta'_j = \delta_j - \dfrac{\delta_q}{\bar{a}_{rq}} \bar{a}_{rj}$. For columns with $\bar{a}_{rj} \geq 0$ this only increases δ_j (since $-\delta_q / \bar{a}_{rq} \geq 0$). For columns with $\bar{a}_{rj} < 0$ we need $\delta'_j \geq 0$, which rearranges to

$$\frac{\delta_q}{|\bar{a}_{rq}|} \leq \frac{\delta_j}{|\bar{a}_{rj}|}.$$

Step 4. Dual ratio test. Hence the entering column must minimise

$$q = \underset{j \,:\, \bar{a}_{rj} < 0}{\arg\min} \frac{\delta_j}{|\bar{a}_{rj}|}. \tag{14.1}$$

Step 5. Effect on the objective. The new row 0 right hand side is $z' = z - \dfrac{\delta_q}{\bar{a}_{rq}} \bar{b}_r = z - \dfrac{\delta_q |\bar{b}_r|}{|\bar{a}_{rq}|} \leq z$. For a maximisation problem the objective value *decreases* towards the optimum from above, because a dual feasible tableau always shows an upper bound (weak duality).

where

δ_j is the row 0 entry under column j (nonnegative throughout);

$\bar{a}_{rj}$ is the entry of the leaving row r in column j;

$\bar{b}_r$ is the negative right hand side of the leaving row.

Key Result The dual simplex algorithm (maximisation tableau)

Start from a tableau with all row 0 entries ≥ 0.

1. If every right hand side is ≥ 0, stop: the tableau is optimal.
2. **Leaving row:** choose r with the most negative $\bar{b}_r$.
3. If row r has no negative entry, stop: the problem is infeasible.
4. **Entering column:** among columns with $\bar{a}_{rj} < 0$, choose the smallest ratio $\delta_j / |\bar{a}_{rj}|$.
5. Pivot on $\bar{a}_{rq}$ (a negative number) and return to step 1.

Lucid Insight The dual simplex method is the primal simplex method on the dual

Each dual simplex pivot performs exactly the pivot that the ordinary simplex method would perform on the dual problem: the leaving row chooses a dual variable to increase, and the dual ratio test is the ordinary ratio test of the dual. Nothing new is being invented; the same tableau is simply read the other way round.

14.3 Reoptimisation

Key Result Adding a constraint

To add a constraint $\mathbf{a}^\top \mathbf{x} \le \beta$ to a solved problem:

1. If the current optimum satisfies it, nothing changes.
2. Otherwise append the row $\mathbf{a}^\top \mathbf{x} + s_{\text{new}} = \beta$ with s_{new} basic.
3. Eliminate the current basic variables from the new row (subtract multiples of their rows) so that every basic column stays a unit column. The new right hand side becomes $\beta - \mathbf{a}^\top \mathbf{x}^\star < 0$.
4. Row 0 is unchanged and still dual feasible, so continue with the dual simplex method.

Key Result Changing a right hand side

If $\mathbf{b}$ changes to $\mathbf{b}'$, the optimal basis keeps its row 0 (dual feasibility does not depend on $\mathbf{b}$). Recompute $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}'$ and $z = \mathbf{y}^\top \mathbf{b}'$. If $\mathbf{x}_B \ge \mathbf{0}$ the basis remains optimal; otherwise continue with the dual simplex method.

Machine Learning Connection

Machine learning workflows rarely solve one linear program in isolation. Cutting plane methods for structured prediction and for LP relaxations of MAP inference add violated constraints one batch at a time. Hyperparameter sweeps solve the same model again with a different budget or regularisation level. Online and streaming settings update right hand sides as new data arrive. In every case the previous optimal basis remains dual feasible, and the dual simplex method typically restores optimality in a handful of pivots instead of starting from scratch. This is why commercial solvers default to the dual simplex method for repeated solves and for branch and bound in integer programming (Module 17).

14.4 Worked Examples

Worked Example 14.1 The Diet Problem without Artificial Variables

Solve min $2x_1 + 3x_2$ subject to $3x_1 + 4x_2 \ge 24$, $x_1 + 3x_2 \ge 12$, $\mathbf{x} \ge \mathbf{0}$ by the dual simplex method.

Solution

Step 1. **Set up.** Maximise $z = -2x_1 - 3x_2$. Multiply each $\geq$ row by -1 and add a slack: $-3x_1 - 4x_2 + s_1 = -24$, $-x_1 - 3x_2 + s_2 = -12$. Row 0 is $z + 2x_1 + 3x_2 = 0$, so its entries $2, 3, 0, 0$ are nonnegative: dual feasible. The right hand sides are negative: primal infeasible.

Step 2. **Iteration 1.** Most negative RHS: -24 in row s_1, which leaves. Dual ratios over negative entries: x_1: $2/3 \approx 0.667$; x_2: $3/4 = 0.75$. Minimum: x_1 enters; pivot -3.

Basic	x_1	x_2	s_1	s_2	RHS
z	2	3	0	0	0
s_1	$\boxed{-3}$	-4	1	0	-24
s_2	-1	-3	0	1	-12
Ratio	$\frac{2}{3}$	0.75			

Step 3. **Iteration 2.** Row s_2 has RHS -4 and leaves. Negative entries: x_2 ($-\frac{5}{3}$, ratio $\frac{1}{3} \div \frac{5}{3} = 0.2$) and s_1 ($-\frac{1}{3}$, ratio $\frac{2}{3} \div \frac{1}{3} = 2$). x_2 enters.

Basic	x_1	x_2	s_1	s_2	RHS
z	0	$\frac{1}{3}$	$\frac{2}{3}$	0	-16
x_1	1	$\frac{4}{3}$	$-\frac{1}{3}$	0	8
s_2	0	$\boxed{-\frac{5}{3}}$	$-\frac{1}{3}$	1	-4
Ratio		0.2	2		

Step 4. **Optimal.** All right hand sides are now nonnegative.

Basic	x_1	x_2	s_1	s_2	RHS
z	0	0	0.6	0.2	-16.8
x_1	1	0	-0.6	0.8	4.8
x_2	0	1	0.2	-0.6	2.4

Step 5. **Read.** $x_1 = 4.8$, $x_2 = 2.4$, $z = -16.8$, so the minimum cost is 16.8; the dual prices 0.6 and 0.2 appear under s_1 and s_2. Notice how the objective moved $0 \to -16 \to -16.8$: it fell towards the optimum, as the derivation predicted.

Answer. $\boxed{x^\star = (4.8,\ 2.4),\ \text{cost}^\star = 16.8}$, in two pivots and with no artificial variables.

Worked Example 14.2 A Three Variable Covering Problem

Solve min $3x_1 + 2x_2 + 4x_3$ subject to $2x_1 + x_2 + 3x_3 \geq 60$, $3x_1 + 3x_2 + 5x_3 \geq 120$, $\mathbf{x} \geq \mathbf{0}$.

Solution

Step 1. **Set up.** Maximise $-3x_1-2x_2-4x_3$; rows $-2x_1-x_2-3x_3+s_1=-60$ and $-3x_1-3x_2-5x_3+s_2=-120$.

Step 2. **Iteration 1.** Most negative RHS is -120 (row s_2). Ratios: x_1: $3/3=1$; x_2: $2/3\approx 0.667$; x_3: $4/5=0.8$. x_2 enters.

Basic	x_1	x_2	x_3	s_1	s_2	RHS
z	3	2	4	0	0	0
s_1	-2	-1	-3	1	0	-60
s_2	-3	$\boxed{-3}$	-5	0	1	-120
Ratio	1	$\frac{2}{3}$	0.8			

Step 3. **Iteration 2.** Row s_1 has RHS -20. Negative entries: x_1 (-1, ratio 1), x_3 ($-\frac{4}{3}$, ratio $\frac{2}{3}\div\frac{4}{3}=0.5$), s_2 ($-\frac{1}{3}$, ratio $\frac{2}{3}\div\frac{1}{3}=2$). x_3 enters.

Basic	x_1	x_2	x_3	s_1	s_2	RHS
z	1	0	$\frac{2}{3}$	0	$\frac{2}{3}$	-80
s_1	-1	0	$\boxed{-\frac{4}{3}}$	1	$-\frac{1}{3}$	-20
x_2	1	1	$\frac{5}{3}$	0	$-\frac{1}{3}$	40
Ratio	1		0.5		2	

Step 4. **Optimal.**

Basic	x_1	x_2	x_3	s_1	s_2	RHS
z	0.5	0	0	0.5	0.5	-90
x_3	0.75	0	1	-0.75	0.25	15
x_2	-0.25	1	0	1.25	-0.75	15

Step 5. **Check.** $x_2=x_3=15$: row 1: $15+45=60$; row 2: $45+75=120$. Cost $30+60=90$. The row 0 entry 0.5 under x_1 says forcing x_1 into the solution costs 0.5 per unit.

Answer. $\boxed{x^\star=(0,15,15),\ \text{cost}^\star=90}$; dual prices $(0.5,\ 0.5)$.

Worked Example 14.3 Adding a New Constraint after Solving

After solving the furniture problem (optimum $x_1=7.5$, $x_2=5$), the workshop learns that at most 6 tables can be finished per week. Reoptimise with the dual simplex method.

Solution

Step 1. Is the cut violated? $x_1 = 7.5 > 6$. Yes.

Step 2. Append the row $x_1 + s_3 = 6$ to the final tableau of Worked Example 8.1:

Basic	x_1	x_2	s_1	s_2	s_3	RHS
z	0	0	6.25	2.5	0	325
x_1	1	0	0.375	−0.25	0	7.5
x_2	0	1	−0.25	0.5	0	5
s_3	1	0	0	0	1	6

Step 3. Restore the unit column of x_1**.** Subtract the x_1 row from the new row: $(1-1,\ 0,\ 0-0.375,\ 0+0.25,\ 1 \mid 6-7.5) = (0,\ 0,\ -0.375,\ 0.25,\ 1 \mid -1.5)$.

Step 4. Dual simplex. Row s_3 leaves (-1.5). Only s_1 has a negative entry; ratio $6.25/0.375 = \frac{50}{3}$. s_1 enters.

Basic	x_1	x_2	s_1	s_2	s_3	RHS
z	0	0	6.25	2.5	0	325
x_1	1	0	0.375	−0.25	0	7.5
x_2	0	1	−0.25	0.5	0	5
s_3	0	0	$\boxed{-0.375}$	0.25	1	−1.5
Ratio			$\frac{50}{3}$			

Step 5. New optimum.

Basic	x_1	x_2	s_1	s_2	s_3	RHS
z	0	0	0	$\frac{20}{3}$	$\frac{50}{3}$	300
x_1	1	0	0	0	1	6
x_2	0	1	0	$\frac{1}{3}$	$-\frac{2}{3}$	6
s_1	0	0	1	$-\frac{2}{3}$	$-\frac{8}{3}$	4

Step 6. Read. $x_1 = 6$, $x_2 = 6$, $s_1 = 4$ (spare timber), profit 300. The cap costs $325 - 300 =$ £25 per week; the shadow price $\frac{50}{3} \approx 16.67$ under s_3 says that each extra table allowed would recover about £16.67.

Answer. $\boxed{x^\star = (6, 6),\ z^\star = 300}$, reached in one dual simplex pivot.

Worked Example 14.4 Reoptimising after a Capacity Change

In the annotation problem (Worked Example 8.6), team A's capacity rises from 180 to 260 hours. Use the optimal basis (x_1, x_2, s_3) with $\mathbf{B}^{-1}$ from Worked Example 11.6 to test whether it remains optimal, and reoptimise if necessary.

Solution

Step 1. New basic values. $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}'$ with $\mathbf{b}' = (260, 300, 240)$:

$$\mathbf{x}_B = \begin{pmatrix} 3/5 & -1/5 & 0 \\ -1/5 & 2/5 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 260 \\ 300 \\ 240 \end{pmatrix} = \begin{pmatrix} 156 - 60 \\ -52 + 120 \\ -260 + 240 \end{pmatrix} = \begin{pmatrix} 96 \\ 68 \\ -20 \end{pmatrix}.$$

Step 2. Verdict. $s_3 = -20 < 0$: the old basis is no longer primal feasible. Row 0 is unchanged, with RHS $\mathbf{y}^\top\mathbf{b}' = 2.6(260) + 0.8(300) = 916$, an upper bound that is not attainable.

Step 3. Dual simplex. Row s_3 leaves. Its only negative entry is under s_1 (-1, ratio 2.6). s_1 enters.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS
z	0	0	0.2	2.6	0.8	0	916
x_1	1	0	0.2	0.6	−0.2	0	96
x_2	0	1	0.6	−0.2	0.4	0	68
s_3	0	0	1	$\boxed{-1}$	0	1	−20
Ratio				2.6			

Step 4. New optimum.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	RHS
z	0	0	2.8	0	0.8	2.6	864
x_1	1	0	0.8	0	−0.2	0.6	84
x_2	0	1	0.4	0	0.4	−0.2	72
s_1	0	0	−1	1	0	−1	20

Step 5. Read. $x_1 = 84$, $x_2 = 72$, $s_1 = 20$, profit 864. Check: team A $168 + 72 = 240 \leq 260$; team B $84 + 216 = 300$; team C $168 + 72 = 240$.

Step 6. Lesson. The extra 80 hours raised profit by $864 - 708 = 156$, not $2.6 \times 80 = 208$. The shadow price 2.6 is valid only over a limited range of b_1; beyond it team C becomes the bottleneck and team A's price falls to zero. Module 15 computes that range.

Answer. $\boxed{x^\star = (84, 72, 0),\ z^\star = 864}$ after one pivot; 20 of the new hours go unused.

Worked Example 14.5 Detecting Infeasibility with the Dual Simplex Method

Apply the dual simplex method to max $-x_1 - x_2$ subject to $x_1 + x_2 \le 3$ and $x_1 + x_2 \ge 5$, $\mathbf{x} \ge \mathbf{0}$.

Solution

Step 1. Set up. Rows $x_1 + x_2 + s_1 = 3$ and $-x_1 - x_2 + s_2 = -5$. Row 0 entries $1, 1, 0, 0$: dual feasible.

Step 2. Iteration 1. Row s_2 leaves (-5). Ratios $1/1 = 1$ for both x_1 and x_2; take x_1.

Basic	x_1	x_2	s_1	s_2	RHS
z	1	1	0	0	0
s_1	1	1	1	0	3
s_2	$\boxed{-1}$	-1	0	1	-5
Ratio	1	1			

Step 3. Iteration 2. Row s_1 now has RHS -2, but its entries are $0,\ 0,\ 1,\ 1$: none negative.

Basic	x_1	x_2	s_1	s_2	RHS
z	0	0	0	1	-5
s_1	0	0	1	1	-2
x_1	1	1	0	-1	5
Ratio					

Step 4. Conclusion. Row s_1 says $s_1 + s_2 = -2$ with $s_1, s_2 \ge 0$, which is impossible. The problem is infeasible, as the derivation predicted, and the row itself is a Farkas certificate (Module 13): adding the two original constraints gives $0 \le -2$.

Answer. The problem is infeasible, proved by a row with negative RHS and no negative entries.

Worked Example 14.6 A Licence Cap on a Model Family

In the training cluster problem (Worked Example 8.3, optimum $(2, 0, 1)$, value 13) a software licence limits family 3 to at most 0.5 runs. Reoptimise.

Solution

Step 1. Violated? $x_3 = 1 > 0.5$. Yes.

Step 2. Append and clean the row. The new row $x_3 + s_4 = 0.5$, after subtracting the x_3 row of the final tableau, becomes $(0,\ 1,\ 0,\ 3,\ 0,\ -2,\ 1 \mid -0.5)$.

Step 3. Dual simplex. Row s_4 leaves; its only negative entry is under s_3 (-2), with ratio $1/2 = 0.5$. s_3 enters.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	s_4	RHS
z	0	3	0	1	0	1	0	13
x_1	1	2	0	2	0	−1	0	2
s_2	0	−5	0	−2	1	0	0	1
x_3	0	−1	1	−3	0	2	0	1
s_4	0	1	0	3	0	$\boxed{-2}$	1	−0.5
Ratio						0.5		

Step 4. New optimum.

Basic	x_1	x_2	x_3	s_1	s_2	s_3	s_4	RHS
z	0	3.5	0	2.5	0	0	0.5	12.75
x_1	1	1.5	0	0.5	0	0	−0.5	2.25
s_2	0	−5	0	−2	1	0	0	1
x_3	0	0	1	0	0	0	1	0.5
s_3	0	−0.5	0	−1.5	0	1	−0.5	0.25

Step 5. Read and check. $x_1 = 2.25$, $x_3 = 0.5$, $x_2 = 0$, value $11.25 + 1.5 = 12.75$. CPU: $4.5 + 0.5 = 5$ (tight); GPU: $9 + 1 = 10 \leq 11$; storage: $6.75 + 1 = 7.75 \leq 8$ (slack 0.25). The licence costs $13 - 12.75 = 0.25$ points, and its shadow price 0.5 says each extra half run allowed is worth 0.25.

Answer. $\boxed{x^\star = (2.25,\ 0,\ 0.5),\ z^\star = 12.75}$ after one pivot.

Module Summary

- Primal feasibility means RHS ≥ 0; dual feasibility means row $0 \geq 0$ (maximisation). Both together mean optimality.
- Dual simplex: leave the row with the most negative RHS; enter the column minimising $\delta_j / \left|\bar{a}_{rj}\right|$ over negative entries; pivot.
- A negative RHS row with no negative entries proves infeasibility.
- Minimisation problems with nonnegative costs and $\geq$ constraints can be solved without artificial variables.
- After adding a cut or changing **b**, the old basis is still dual feasible, so a few dual pivots restore optimality: the basis of fast warm started repeated solves.

MODULE 15

Sensitivity Analysis and Shadow Prices

Learning Objectives

After studying this module you will be able to:

- explain why sensitivity analysis is essential when data are estimated rather than known;
- derive the range over which a right hand side may change while the optimal basis stays optimal, and the linear change in the optimal value over that range;
- derive the ranges of optimality for objective coefficients of nonbasic and basic variables;
- decide, by pricing, whether a newly proposed activity should be introduced;
- describe the optimal value as a piecewise linear function of a resource, and interpret its slopes;
- apply these tools to resource decisions in machine learning projects.

15.1 Why Answers Must Come with Ranges

The certainty assumption of Module 1 is never exactly true. Profits are forecasts, capacities are estimates, and GPU prices change weekly. A manager who receives "make 48 units of product 1" will immediately ask: *how wrong could the data be before this advice changes, and what is an extra unit of each resource worth?* Sensitivity analysis answers those questions from the final tableau, without solving the problem again. Its key insight is that the optimal *basis* is often far more stable than the optimal *numbers*: the numbers move smoothly with the data, while the basis, and therefore the structure of the decision, stays the same over whole ranges.

Throughout, the problem is max $\mathbf{c}^\top \mathbf{x}$ subject to $\mathbf{A}\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$ (with slacks included), and $\mathbf{B}$ is an optimal basis with multipliers $\mathbf{y}^\top = \mathbf{c}_B^\top \mathbf{B}^{-1}$. Recall the two optimality conditions:

$$\text{primal feasibility: } \mathbf{B}^{-1}\mathbf{b} \geq \mathbf{0}, \qquad \text{dual feasibility: } c_j - \mathbf{y}^\top \mathbf{A}_j \leq 0 \ \text{ for all } j. \tag{15.1}$$

The first involves $\mathbf{b}$ but not $\mathbf{c}$; the second involves $\mathbf{c}$ but not $\mathbf{b}$. This separation is the whole secret of sensitivity analysis.

15.2 Changing a Right Hand Side

Derivation Range of feasibility for b_k

Change b_k to $b_k + \Delta$, that is $\mathbf{b}' = \mathbf{b} + \Delta \mathbf{e}_k$.

Step 1. Dual feasibility does not involve $\mathbf{b}$, so it still holds.

Step 2. The new basic values are

$$\mathbf{x}'_B = \mathbf{B}^{-1}(\mathbf{b} + \Delta \mathbf{e}_k) = \mathbf{B}^{-1}\mathbf{b} + \Delta\, \mathbf{B}^{-1}\mathbf{e}_k = \bar{\mathbf{b}} + \Delta \boldsymbol{\beta}_k,$$

where $\boldsymbol{\beta}_k$ is column k of $\mathbf{B}^{-1}$ (in the final tableau, the column under slack s_k).

Step 3. The basis stays optimal exactly when every component remains nonnegative: $\bar{b}_i + \Delta\beta_{ik} \geq 0$ for all i.

Step 4. Rows with $\beta_{ik} > 0$ give lower limits $\Delta \geq -\bar{b}_i/\beta_{ik}$; rows with $\beta_{ik} < 0$ give upper limits $\Delta \leq \bar{b}_i / |\beta_{ik}|$. Hence

$$\max_{i:\beta_{ik}>0} \left(-\frac{\bar{b}_i}{\beta_{ik}} \right) \leq \Delta \leq \min_{i:\beta_{ik}<0} \frac{\bar{b}_i}{|\beta_{ik}|}. \tag{15.2}$$

Step 5. Within this range the optimal value is $z' = \mathbf{y}^\top \mathbf{b}' = z^\star + y_k\Delta$.

where

Δ	is the change in resource k;
$\bar{b}_i$	is the current value of the ith basic variable;
β_{ik}	is entry i of column k of $\mathbf{B}^{-1}$;
y_k	is the shadow price of resource k.

Key Result Meaning of a shadow price

The shadow price y_k is the rate of change of the optimal value with respect to b_k, valid throughout the range (15.2). Outside the range the basis changes, and the rate changes with it.

Derivation The optimal value is concave and piecewise linear in b

Step 1. By strong duality, $z^\star(\mathbf{b}) = \min\{\mathbf{b}^\top \mathbf{y} : \mathbf{A}^\top \mathbf{y} \geq \mathbf{c},\ \mathbf{y} \geq \mathbf{0}\}$ (assuming the primal is feasible and bounded).

Step 2. The dual feasible region does not depend on $\mathbf{b}$, and its vertices $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)}$ are finitely many. So $z^\star(\mathbf{b}) = \min_k \mathbf{b}^\top \mathbf{y}^{(k)}$.

Step 3. Each $\mathbf{b}^\top \mathbf{y}^{(k)}$ is linear in $\mathbf{b}$, and the minimum of finitely many linear functions is concave and piecewise linear.

Step 4. Consequence: as one resource increases, its shadow price can only stay the same or fall. This is the law of *diminishing marginal returns*, derived rather than assumed.

15.3 Changing an Objective Coefficient

Derivation Range of optimality for a nonbasic variable

Change c_j to $c_j + \delta$ for a nonbasic x_j.

Step 1. $\mathbf{c}_B$ is unchanged, so $\mathbf{y}$ and all other reduced costs are unchanged.

Step 2. Only $\bar{c}_j = c_j + \delta - \mathbf{y}^\top \mathbf{A}_j$ changes. Optimality requires $\bar{c}_j \le 0$:

$$c_j + \delta \ \le \ \mathbf{y}^\top \mathbf{A}_j. \tag{15.3}$$

Step 3. In words, an unused activity stays unused until its profit reaches the value of the resources it would consume. The amount it may rise is exactly the row 0 entry under x_j.

Derivation Range of optimality for a basic variable

Change c_q to $c_q + \delta$ for the basic variable in row r.

Step 1. Now $\mathbf{c}_B$ changes by $\delta \mathbf{e}_r$, so $\mathbf{y}^\top$ changes by $\delta\, \mathbf{e}_r^\top \mathbf{B}^{-1}$ (row r of $\mathbf{B}^{-1}$).

Step 2. For each nonbasic j, the row 0 entry $\delta_j = \mathbf{y}^\top \mathbf{A}_j - c_j$ changes to $\delta_j + \delta\, \mathbf{e}_r^\top \mathbf{B}^{-1} \mathbf{A}_j = \delta_j + \delta\, \bar{a}_{rj}$, where $\bar{a}_{rj}$ is the entry of row r in column j of the final tableau.

Step 3. Optimality requires $\delta_j + \delta \bar{a}_{rj} \ge 0$ for every nonbasic j. Hence

$$\max_{j:\bar{a}_{rj}>0} \left(-\frac{\delta_j}{\bar{a}_{rj}} \right) \ \le \ \delta \ \le \ \min_{j:\bar{a}_{rj}<0} \frac{\delta_j}{|\bar{a}_{rj}|}. \tag{15.4}$$

Step 4. Within this range the optimal solution $\mathbf{x}^\star$ is unchanged and $z^\star$ changes by $\delta x_q^\star$.

15.4 Introducing a New Activity

Derivation Pricing out a new column

Suppose a new variable x_{new} with profit c_{new} and resource column $\mathbf{a}_{\text{new}}$ is proposed.

Step 1. Set $x_{\text{new}} = 0$: the current solution stays feasible.

Step 2. Its reduced cost is $\bar{c}_{\text{new}} = c_{\text{new}} - \mathbf{y}^\top \mathbf{a}_{\text{new}}$.

Step 3. If $\bar{c}_{\text{new}} \le 0$, the current solution remains optimal and the new activity is not worth starting.

Step 4. If $\bar{c}_{\text{new}} > 0$, append the column $\mathbf{B}^{-1}\mathbf{a}_{\text{new}}$ to the final tableau with row 0 entry $-\bar{c}_{\text{new}}$ and continue with the primal simplex method.

Common Pitfall

Shadow prices answer questions about *one* change at a time and only inside the computed range. For several simultaneous changes, the **100% rule** gives a safe test: if the sum over all changed coefficients of (actual change divided by the maximum allowed change in that direction) is at most 1, the basis remains optimal. If it exceeds 1, the basis may or may not remain optimal and the problem should be solved again. Also remember that at a degenerate optimum shadow prices are not unique (Worked Example 10.1), so a solver may report a price that holds in only one direction.

Machine Learning Connection

Sensitivity analysis is the language of resource planning in machine learning projects. The shadow price of a GPU hour constraint tells a team exactly how much extra compute is worth to the project's objective, and the range tells them how much they can buy before that value drops. Pricing a new column is the core step of *column generation*, used in LPBoost to decide whether a new weak learner improves the ensemble, and in large routing and scheduling problems that ML systems solve repeatedly. More broadly, the derivative of an optimal value with respect to its data, $\partial z^\star/\partial b_k = y_k$, is the idea behind *differentiable optimisation layers*, which embed LPs inside neural networks and back propagate through them using the dual solution.

15.5 Worked Examples

Worked Example 15.1 The Value and Range of Timber

For the furniture problem (optimal basis (x_1, x_2), $\mathbf{B}^{-1} = \begin{pmatrix} 3/8 & -1/4 \\ -1/4 & 1/2 \end{pmatrix}$, $\bar{\mathbf{b}} = (7.5, 5)$, $\mathbf{y} = (6.25, 2.5)$), find the range of timber availability b_1 for which the basis stays optimal, and the optimal profit as a function of b_1 in that range.

Solution

Step 1. Column of $\mathbf{B}^{-1}$ for $k = 1$: $\boldsymbol{\beta}_1 = (3/8,\ -1/4)$.

Step 2. Conditions. $x_1' = 7.5 + \frac{3}{8}\Delta \geq 0$ gives $\Delta \geq -20$. $x_2' = 5 - \frac{1}{4}\Delta \geq 0$ gives $\Delta \leq 20$.

Step 3. Range. $-20 \leq \Delta \leq 20$, that is $20 \leq b_1 \leq 60$.

Step 4. Optimal value. $z^\star(b_1) = 325 + 6.25(b_1 - 40)$ for $20 \leq b_1 \leq 60$, giving 200 at $b_1 = 20$ and 450 at $b_1 = 60$.

Step 5. Geometric check. At $b_1 = 60$: $x_1 = 7.5 + 7.5 = 15$, $x_2 = 0$, and labour $2(15) = 30$, still feasible: the corner has slid to the x_1 axis. Beyond that, extra timber has nothing to combine with and its value drops to zero. At $b_1 = 20$: $x_1 = 0$, $x_2 = 10$, the corner has slid to the x_2 axis.

Answer. The basis is optimal for $\boxed{20 \leq b_1 \leq 60}$, with $z^\star = 325 + 6.25(b_1 - 40)$.

Worked Example 15.2 Explaining the Capacity Puzzle of Module 14

In the annotation problem, the basis (x_1, x_2, s_3) has $\bar{\mathbf{b}} = (48, 84, 60)$, $\mathbf{y} = (2.6, 0.8, 0)$, and column 1 of $\mathbf{B}^{-1}$ equal to $(3/5,\ -1/5,\ -1)$. Find the range of team A's capacity b_1, and explain why raising it to 260 hours (Worked Example 14.4) gained only 156.

Solution

Step 1. **Conditions.** $48 + 0.6\Delta \ge 0 \Rightarrow \Delta \ge -80$. $84 - 0.2\Delta \ge 0 \Rightarrow \Delta \le 420$. $60 - \Delta \ge 0 \Rightarrow \Delta \le 60$.

Step 2. **Range.** $-80 \le \Delta \le 60$, so $100 \le b_1 \le 240$. The binding limit comes from s_3: team C's spare 60 hours are used up when b_1 reaches 240.

Step 3. **Value up to 240.** $z^\star(240) = 708 + 2.6(60) = 864$.

Step 4. **Beyond 240.** Team C becomes a bottleneck, the basis changes, and team A's price drops to 0 (Worked Example 14.4 found s_1 basic). So hours 241 to 260 add nothing.

Step 5. **Total gain.** $864 - 708 = 156 = 2.6 \times 60$, not 2.6×80.

Answer. Range $\boxed{100 \le b_1 \le 240}$; only the first 60 extra hours are valuable, worth 2.6 each.

Worked Example 15.3 How Profitable Must an Unused Product Become?

In the annotation problem, audio transcripts (x_3, profit 4) are not produced. How high must their profit rise before they enter the optimal plan?

Solution

Step 1. **Imputed value of resources** used by one batch of x_3, whose column is $(1, 2, 2)$: $\mathbf{y}^\top \mathbf{A}_3 = 2.6(1) + 0.8(2) + 0(2) = 4.2$.

Step 2. **Condition** (15.3). x_3 stays at zero while $c_3 \le 4.2$.

Step 3. **Cross check with the tableau.** The row 0 entry under x_3 in the final tableau is $0.2 = 4.2 - 4$, the allowable increase.

Step 4. **Above 4.2**, x_3 would have positive reduced cost and enter at the next simplex pivot.

Answer. Audio must earn more than $\boxed{4.2}$ (£420 per batch) before it is worth producing.

Worked Example 15.4 Range of Optimality for a Basic Variable

For the furniture problem, find the range of the table profit c_1 over which the plan (7.5, 5) stays optimal. Final tableau rows: x_1: $(1,\ 0,\ 0.375,\ -0.25 \mid 7.5)$; row 0: $(0,\ 0,\ 6.25,\ 2.5 \mid 325)$.

Solution

Step 1. **Row of x_1** ($r = 1$): entries under the nonbasic s_1 and s_2 are $\bar{a}_{1,s_1} = 0.375$ and $\bar{a}_{1,s_2} = -0.25$.

Step 2. **Conditions** (15.4). For s_1: $6.25 + 0.375\delta \ge 0 \Rightarrow \delta \ge -16.\overline{6}$. For s_2: $2.5 - 0.25\delta \ge 0 \Rightarrow \delta \le 10$.

Step 3. **Range.** $-\frac{50}{3} \le \delta \le 10$, so $\frac{40}{3} \approx 13.33 \le c_1 \le 40$.

Step 4. **Geometric check** (Module 5). The vertex $(7.5, 5)$ stays optimal while the slope of the objective, $c_1/c_2 = c_1/20$, lies between the slopes of the labour line $(2/3)$ and the timber line $(4/2 = 2)$. That gives $\frac{2}{3} \le c_1/20 \le 2$, so $13.33 \le c_1 \le 40$, in agreement.

Step 5. **Within the range** $z^\star = 325 + 7.5\delta$.

Answer. The plan is optimal for $\boxed{13.33 \le c_1 \le 40}$.

Worked Example 15.5 Should a New Product Be Launched?

The annotation company considers video labelling (x_4), earning 7 per batch and needing $(2, 2, 1)$ hours from teams A, B and C. Decide by pricing, and if worthwhile find the new optimum.

Solution

Step 1. **Price.** $\mathbf{y}^\top \mathbf{a}_4 = 2.6(2) + 0.8(2) + 0(1) = 6.8$. Reduced cost $\bar{c}_4 = 7 - 6.8 = 0.2 > 0$: launching is worthwhile.

Step 2. **New tableau column.** $\mathbf{B}^{-1}\mathbf{a}_4 = \begin{pmatrix} 3/5 & -1/5 & 0 \\ -1/5 & 2/5 & 0 \\ -1 & 0 & 1 \end{pmatrix}\begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1.2 - 0.4 \\ -0.4 + 0.8 \\ -2 + 1 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 0.4 \\ -1 \end{pmatrix}$, with row 0 entry -0.2.

Step 3. **Ratio test.** Row x_1: $48/0.8 = 60$. Row x_2: $84/0.4 = 210$. Row s_3: negative, none. x_1 leaves, $\theta^\star = 60$.

Step 4. **Update.** $x_4 = 60$, $x_2 = 84 - 0.4(60) = 60$, $s_3 = 60 + 60 = 120$, $x_1 = 0$. Profit $708 + 0.2(60) = 720$.

Step 5. **Optimality check.** New prices from the tight teams A and B with x_2, x_4 basic: $y_1 + 3y_2 = 5$ and $2y_1 + 2y_2 = 7$ give $y_2 = 0.75$, $y_1 = 2.75$. Reduced costs: x_1: $6 - (5.5 + 0.75) = -0.25$; x_3: $4 - (2.75 + 1.5) = -0.25$. Both negative: optimal.

Answer. Launch video: new plan $\boxed{x_2 = 60,\ x_4 = 60,\ z^\star = 720}$, replacing image boxes entirely.

Worked Example 15.6 The Value Curve of GPU Capacity

In the model routing problem of Worked Example 5.6, max $x_1 + 3x_2$ subject to $x_1 + 4x_2 \le G$ (GPU), $2x_1 + 3x_2 \le 45$ (memory), $x_1 + x_2 \ge 10$ (demand), $\mathbf{x} \ge \mathbf{0}$, the optimum at $G = 40$ is $(12, 7)$ with basis (x_1, x_2, e_3). (a) Find the range of G for this basis. (b) Describe the optimal value $z^\star(G)$ for all G and draw it.

Solution

Step 1. **(a) Column of $\mathbf{B}^{-1}$ for the GPU row.** Solve $\mathbf{B}\mathbf{d} = \mathbf{e}_1$ with $\mathbf{B}$ formed by the columns of x_1, x_2 and e_3: $d_1 + 4d_2 = 1$, $2d_1 + 3d_2 = 0$, $d_1 + d_2 - d_3 = 0$. From the second, $d_1 = -1.5d_2$; the first gives $2.5d_2 = 1$, so $d_2 = 0.4$, $d_1 = -0.6$, $d_3 = -0.2$.

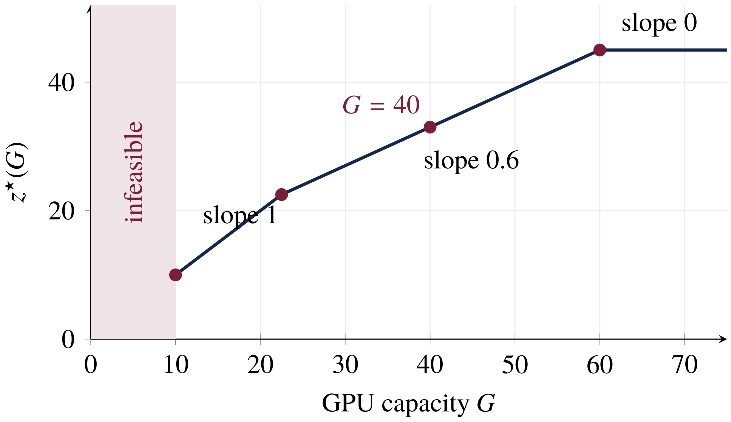
Step 2. **(a) Conditions.** $(x_1, x_2, e_3) = (12, 7, 9) + \Delta(-0.6,\ 0.4,\ -0.2) \ge \mathbf{0}$: $\Delta \le 20$, $\Delta \ge -17.5$, $\Delta \le 45$. So $-17.5 \le \Delta \le 20$, that is $22.5 \le G \le 60$, with slope $y_1 = 0.6$.

Step 3. **(b) Below 22.5.** Memory is no longer binding; the best plan uses only the small model, $x = (G, 0)$, value G, slope 1. This needs $G \ge 10$ to meet demand; below $G = 10$ the problem is infeasible.

Step 4. **(b) Above 60.** The plan (0, 15) uses all memory; extra GPU is worthless and the slope is 0.

Step 5. **(b) Value curve.**

$$z^\star(G) = \begin{cases} \text{infeasible}, & G < 10, \\ G, & 10 \le G \le 22.5, \\ 22.5 + 0.6(G - 22.5), & 22.5 \le G \le 60, \\ 45, & G \ge 60. \end{cases}$$



Step 6. **Interpretation.** The curve is concave: GPU capacity is worth £1 per unit while scarce, £0.60 in the middle range, and nothing once memory binds. A team bidding for GPU time should never pay more than the current slope.

Answer. Basis range $\boxed{22.5 \le G \le 60}$ with shadow price 0.6; the value curve has slopes 1, 0.6 and 0.

Module Summary

- Changing **b** affects only primal feasibility; changing **c** affects only dual feasibility.
- RHS range: $\bar{\mathbf{b}} + \Delta\boldsymbol{\beta}_k \ge \mathbf{0}$, where $\boldsymbol{\beta}_k$ is column k of $\mathbf{B}^{-1}$; inside it $z^\star$ changes by $y_k\Delta$.
- Nonbasic cost range: $c_j \le \mathbf{y}^\top \mathbf{A}_j$; the allowable increase is the row 0 entry under x_j.
- Basic cost range: $\delta_j + \delta\bar{a}_{rj} \ge 0$ for all nonbasic j.
- New activity: enter it if $c_{\text{new}} - \mathbf{y}^\top \mathbf{a}_{\text{new}} > 0$.
- $z^\star(\mathbf{b})$ is concave and piecewise linear: shadow prices can only fall as a resource grows.

Module 16

Interior Point Methods

Learning Objectives

After studying this module you will be able to:

- explain why the simplex method can, in the worst case, take exponentially many steps, and how interior point methods avoid the boundary;
- derive the logarithmic barrier problem and its optimality conditions;
- define the central path and prove that the duality gap on it equals $n\mu$;
- derive the Newton system of the primal dual interior point method and compute a step;
- compute the largest safe step length and update the barrier parameter;
- compare simplex and interior point methods for large machine learning problems.

16.1 Why Another Algorithm?

The simplex method walks along edges of the feasible polyhedron. In practice it is fast, but in 1972 Klee and Minty constructed problems, slightly squashed cubes, on which the classical rule visits all 2^n vertices. Whether some pivoting rule always needs only polynomially many steps is still an open question. In 1984 Narendra Karmarkar published a method that moves through the *interior* of the feasible region and provably needs only a polynomial number of iterations. Its modern descendants, primal dual interior point methods, are now the method of choice for very large linear programs, and their ideas run through all of modern convex optimisation, including the solvers behind many machine learning models.

Lucid Insight The picture to keep in mind

The simplex method is a careful walker who keeps one hand on the wall and follows the corridors. An interior point method is a hiker who stays in the middle of the valley, guided by a "force field" that repels from every wall, and gradually weakens that force so that the path bends towards the optimal corner.

Throughout this module we use the standard form pair

$$(P)\ \ \min\ \mathbf{c}^\top\mathbf{x} \text{ subject to } \mathbf{A}\mathbf{x}=\mathbf{b},\ \mathbf{x}\geq\mathbf{0}; \qquad (D)\ \ \max\ \mathbf{b}^\top\mathbf{y} \text{ subject to } \mathbf{A}^\top\mathbf{y}+\mathbf{s}=\mathbf{c},\ \mathbf{s}\geq\mathbf{0}, \tag{16.1}$$

where

$\mathbf{x}\in\mathbb{R}^n$ are the primal variables and $\mathbf{y}\in\mathbb{R}^m$ the dual variables;

$\mathbf{s} \in \mathbb{R}^n$ is the vector of dual slacks, $\mathbf{s} = \mathbf{c} - \mathbf{A}^\top \mathbf{y}$ (the reduced costs);

$\mathbf{A} \in \mathbb{R}^{m \times n}$ has full row rank.

By Module 13, $(\mathbf{x}, \mathbf{y}, \mathbf{s})$ is optimal exactly when it is primal feasible, dual feasible, and $x_j s_j = 0$ for every j.

16.2 The Logarithmic Barrier

Derivation The barrier problem

Step 1. The difficult part of (P) is the inequality $\mathbf{x} \geq \mathbf{0}$. Replace it by a penalty that becomes infinite at the boundary. The function $-\ln x_j$ tends to $+\infty$ as $x_j \to 0^+$ and is finite for $x_j > 0$.

Step 2. For a parameter $\mu > 0$, define the **barrier problem**

$$\min_{\mathbf{x}} \ \mathbf{c}^\top \mathbf{x} - \mu \sum_{j=1}^{n} \ln x_j \qquad \text{subject to} \qquad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad (\mathbf{x} > \mathbf{0}). \tag{16.2}$$

Step 3. The barrier term keeps every iterate strictly inside. As $\mu \to 0$ the penalty fades, and the solution approaches an optimal solution of (P).

Step 4. The objective is strictly convex (the second derivative of $-\mu \ln x_j$ is $\mu / x_j^2 > 0$), so the barrier problem has at most one minimiser.

where

μ is the **barrier parameter**, a positive number driven towards zero;

ln is the natural logarithm.

Derivation Optimality conditions of the barrier problem

Step 1. Attach multipliers $\mathbf{y}$ to $\mathbf{A}\mathbf{x} = \mathbf{b}$ and form the Lagrangian $\mathcal{L} = \mathbf{c}^\top \mathbf{x} - \mu \sum_j \ln x_j - \mathbf{y}^\top (\mathbf{A}\mathbf{x} - \mathbf{b})$.

Step 2. Set the derivative with respect to x_j to zero: $c_j - \mu / x_j - (\mathbf{A}^\top \mathbf{y})_j = 0$.

Step 3. Define $s_j = \mu / x_j$. Then the condition reads $\mathbf{A}^\top \mathbf{y} + \mathbf{s} = \mathbf{c}$, and the definition gives $x_j s_j = \mu$.

Step 4. Collecting everything:

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \qquad \mathbf{A}^\top \mathbf{y} + \mathbf{s} = \mathbf{c}, \qquad x_j s_j = \mu \ \ (j = 1, \ldots, n), \qquad \mathbf{x} > \mathbf{0}, \ \mathbf{s} > \mathbf{0}. \tag{16.3}$$

Compare with the LP optimality conditions: they are identical except that complementarity $x_j s_j = 0$ has been relaxed to $x_j s_j = \mu$.

16.3 The Central Path

Definition Central Path

For each $\mu > 0$, the unique solution $(\mathbf{x}(\mu), \mathbf{y}(\mu), \mathbf{s}(\mu))$ of (16.3) is a point of the **central path**. As μ decreases to 0, the central path converges to an optimal primal dual pair (in fact to the "analytic centre" of the optimal face).

Derivation The duality gap on the central path

Step 1. For any primal feasible $\mathbf{x}$ and dual feasible $(\mathbf{y}, \mathbf{s})$, the gap is $\mathbf{c}^\top\mathbf{x} - \mathbf{b}^\top\mathbf{y}$.

Step 2. Substitute $\mathbf{c} = \mathbf{A}^\top\mathbf{y} + \mathbf{s}$ and $\mathbf{b} = \mathbf{A}\mathbf{x}$: $\mathbf{c}^\top\mathbf{x} - \mathbf{b}^\top\mathbf{y} = \mathbf{y}^\top\mathbf{A}\mathbf{x} + \mathbf{s}^\top\mathbf{x} - \mathbf{y}^\top\mathbf{A}\mathbf{x} = \mathbf{s}^\top\mathbf{x}$.

Step 3. On the central path each $x_j s_j = \mu$, so

$$\mathbf{c}^\top\mathbf{x}(\mu) - \mathbf{b}^\top\mathbf{y}(\mu) = \sum_{j=1}^{n} x_j s_j = n\mu. \tag{16.4}$$

So to reach a gap of ε we need only drive μ down to ε/n. More generally, for any interior iterate the **duality measure** $\mu = \mathbf{x}^\top\mathbf{s}/n$ is the average complementarity product.

16.4 The Primal Dual Newton Step

Derivation The Newton system

We want to solve (16.3) for a target value $\tau = \sigma\mu$, where $\mu = \mathbf{x}^\top\mathbf{s}/n$ is the current measure and $\sigma \in (0, 1)$ is the **centring parameter**.

Step 1. Write the conditions as $F(\mathbf{x}, \mathbf{y}, \mathbf{s}) = \mathbf{0}$ with components $\mathbf{A}\mathbf{x} - \mathbf{b}$, $\mathbf{A}^\top\mathbf{y} + \mathbf{s} - \mathbf{c}$ and $x_j s_j - \tau$.

Step 2. Newton's method replaces F by its linearisation at the current point. For the products, $(x_j + \Delta x_j)(s_j + \Delta s_j) \approx x_j s_j + s_j \Delta x_j + x_j \Delta s_j$, dropping the small product $\Delta x_j \Delta s_j$.

Step 3. Setting the linearisation to zero gives

$$\begin{aligned} \mathbf{A}\,\Delta\mathbf{x} &= \mathbf{r}_p := \mathbf{b} - \mathbf{A}\mathbf{x}, \\ \mathbf{A}^\top\Delta\mathbf{y} + \Delta\mathbf{s} &= \mathbf{r}_d := \mathbf{c} - \mathbf{A}^\top\mathbf{y} - \mathbf{s}, \\ \mathbf{S}\,\Delta\mathbf{x} + \mathbf{X}\,\Delta\mathbf{s} &= \mathbf{r}_c := \tau\mathbf{1} - \mathbf{X}\mathbf{S}\mathbf{1}, \end{aligned} \tag{16.5}$$

where

$\mathbf{X} = \operatorname{diag}(\mathbf{x})$, $\mathbf{S} = \operatorname{diag}(\mathbf{s})$	are diagonal matrices holding the current values;
$\mathbf{r}_p, \mathbf{r}_d$	are the primal and dual residuals (zero if the current point is feasible);
$\mathbf{r}_c$	is the complementarity residual measured against the target τ;
$\mathbf{1}$	is the vector of ones.

Derivation Reducing the Newton system to normal equations

Step 1. From the second equation, $\Delta\mathbf{s} = \mathbf{r}_d - \mathbf{A}^\top\Delta\mathbf{y}$.

Step 2. Substitute into the third: $\mathbf{S}\Delta\mathbf{x} + \mathbf{X}\mathbf{r}_d - \mathbf{X}\mathbf{A}^\top\Delta\mathbf{y} = \mathbf{r}_c$, so $\Delta\mathbf{x} = \mathbf{S}^{-1}(\mathbf{r}_c - \mathbf{X}\mathbf{r}_d + \mathbf{X}\mathbf{A}^\top\Delta\mathbf{y})$.

Step 3. Substitute into the first and let $\mathbf{D} = \mathbf{X}\mathbf{S}^{-1}$ (diagonal, positive):

$$(\mathbf{A}\mathbf{D}\mathbf{A}^\top)\Delta\mathbf{y} = \mathbf{r}_p - \mathbf{A}\mathbf{S}^{-1}(\mathbf{r}_c - \mathbf{X}\mathbf{r}_d). \tag{16.6}$$

Step 4. Solve this $m \times m$ symmetric positive definite system (by Cholesky factorisation), then recover $\Delta\mathbf{x}$ and $\Delta\mathbf{s}$.

Almost all the work of an interior point method is in forming and factorising $\mathbf{A}\mathbf{D}\mathbf{A}^\top$, once per iteration.

Key Result Step length and update

The new point must keep $\mathbf{x} > \mathbf{0}$ and $\mathbf{s} > \mathbf{0}$. The largest steps that reach the boundary are

$$\alpha_P^{\max} = \min_{j:\Delta x_j<0} \frac{x_j}{-\Delta x_j}, \qquad \alpha_D^{\max} = \min_{j:\Delta s_j<0} \frac{s_j}{-\Delta s_j}, \tag{16.7}$$

and the method takes $\alpha_P = \min(1, \eta\,\alpha_P^{\max})$, $\alpha_D = \min(1, \eta\,\alpha_D^{\max})$ with a safety factor such as $\eta = 0.9$ or 0.99. Then $\mathbf{x} \leftarrow \mathbf{x} + \alpha_P\Delta\mathbf{x}$, $\mathbf{y} \leftarrow \mathbf{y} + \alpha_D\Delta\mathbf{y}$, $\mathbf{s} \leftarrow \mathbf{s} + \alpha_D\Delta\mathbf{s}$. Notice that (16.7) is the ratio test of the simplex method in a new role.

Key Result Complexity

A short step path following method with $\sigma = 1 - 0.4/\sqrt{n}$ keeps the iterates close to the central path and reduces μ by that factor at each iteration. Reducing the gap by a factor 10^{-k} then takes about $\frac{k \ln 10}{0.4}\sqrt{n}$ iterations: a number that grows only with $\sqrt{n}$. Practical long step methods with Mehrotra's predictor corrector heuristic usually need between 20 and 80 iterations regardless of problem size.

Common Pitfall

An interior point method returns a point *near* the optimal face, not a vertex. If an exact basic solution is required (for sensitivity analysis or for warm starts), solvers run a "crossover" procedure that moves to a nearby vertex with a few simplex pivots. Also, interior point methods are hard to warm start, so for sequences of closely related problems the dual simplex method of Module 14 is often faster.

Machine Learning Connection

Interior point methods solve the large LPs of machine learning in a predictable number of iterations: ℓ_1 regression and quantile regression with hundreds of thousands of observations, large optimal transport problems, and LP relaxations in structured prediction. The barrier idea also reaches far beyond LP. The log barrier is the same mathematics as the entropy term in entropic optimal transport (Module 23), where μ plays the role of the regularisation strength and the Sinkhorn algorithm follows an analogue of the central path; and the Newton system (16.5) is the template for second order solvers of support vector machines and logistic regression.

16.5 Worked Examples

Worked Example 16.1 A Barrier Trajectory in One Dimension

Consider max x subject to $0 \le x \le 1$, whose optimum is $x^\star = 1$. Write the barrier problem with barriers on both bounds, find its solution $x(\mu)$ in closed form, and evaluate it for $\mu = 1,\ 0.1,\ 0.01$.

Solution

Step 1. Barrier problem. Maximise $\phi(x) = x + \mu \ln x + \mu \ln(1 - x)$ for $0 < x < 1$ (for maximisation the barrier is added, since ln tends to $-\infty$ at the walls).

Step 2. Stationarity. $\phi'(x) = 1 + \dfrac{\mu}{x} - \dfrac{\mu}{1 - x} = 0.$

Step 3. Clear denominators. Multiply by $x(1 - x) > 0$: $x(1 - x) + \mu(1 - x) - \mu x = 0$, that is $-x^2 + (1 - 2\mu)x + \mu = 0$.

Step 4. Quadratic formula. $x^2 - (1 - 2\mu)x - \mu = 0$ gives

$$x(\mu) = \frac{(1 - 2\mu) + \sqrt{(1 - 2\mu)^2 + 4\mu}}{2},$$

taking the root in $(0, 1)$.

Step 5. Evaluate. $\mu = 1$: $x = \dfrac{-1 + \sqrt{5}}{2} \approx 0.6180$. $\mu = 0.1$: $x = \dfrac{0.8 + \sqrt{1.04}}{2} \approx 0.9099$. $\mu = 0.01$: $x = \dfrac{0.98 + \sqrt{0.9804}}{2} \approx 0.9901$.

Step 6. Interpretation. As μ shrinks the barrier solution approaches the optimum 1 from inside, staying strictly away from the boundary. For small μ, $1 - x(\mu) \approx \mu$.

Answer. $\boxed{x(1) \approx 0.618,\ x(0.1) \approx 0.910,\ x(0.01) \approx 0.990}$, converging to $x^\star = 1$.

Worked Example 16.2 Computing Points of the Central Path

For min $x_1 + 2x_2$ subject to $x_1 + x_2 = 1$, $\mathbf{x} \ge \mathbf{0}$ (optimum $\mathbf{x}^\star = (1, 0)$, $y^\star = 1$), compute the central path point for $\mu = 1,\ 0.1,\ 0.01$, and verify the gap formula (16.4).

Solution

Step 1. Conditions (16.3). Dual: $y + s_1 = 1$, $y + s_2 = 2$, so $s_1 = 1 - y$ and $s_2 = 2 - y$. Centrality: $x_1 = \mu/s_1$, $x_2 = \mu/s_2$. Primal: $x_1 + x_2 = 1$.

Step 2. One equation for y. $\dfrac{\mu}{1 - y} + \dfrac{\mu}{2 - y} = 1$ with $y < 1$.

Step 3. Solve. Multiply out: $\mu(2 - y) + \mu(1 - y) = (1 - y)(2 - y)$, that is $y^2 - (3 - 2\mu)y + (2 - 3\mu) = 0$. The root with $y < 1$ is $y = \dfrac{(3 - 2\mu) - \sqrt{1 + 4\mu^2}}{2}$.

Step 4. $\mu = 1$: $y = \dfrac{1 - \sqrt{5}}{2} \approx -0.618$; $s = (1.618,\ 2.618)$; $x = (0.618,\ 0.382)$.

Step 5. $\mu = 0.1$**:** $y = \dfrac{2.8 - \sqrt{1.04}}{2} \approx 0.8901$; $s \approx (0.1099,\ 1.1099)$; $x \approx (0.9099,\ 0.0901)$.

Step 6. $\mu = 0.01$**:** $y \approx 0.9899$; $s \approx (0.0101,\ 1.0101)$; $x \approx (0.9901,\ 0.0099)$.

Step 7. **Gap.** For $\mu = 0.1$: primal value $0.9099 + 2(0.0901) = 1.0901$, dual value 0.8901, gap $0.2000 = 2\mu$. Likewise the gaps for $\mu = 1$ and $\mu = 0.01$ are 2 and 0.02, exactly $n\mu$ with $n = 2$.

Answer. The path runs $\boxed{(0.618, 0.382) \to (0.910, 0.090) \to (0.990, 0.010) \to (1, 0)}$, with gap 2μ at each point.

Worked Example 16.3 Gap, Duality Measure and Target

An interior iterate of a problem with $n = 5$ has $\mathbf{x} = (2,\ 1,\ 0.5,\ 4,\ 1)$ and $\mathbf{s} = (0.1,\ 0.3,\ 0.2,\ 0.05,\ 0.4)$, and is primal and dual feasible. (a) Compute the duality gap and the duality measure. (b) With $\sigma = 0.2$, what is the target τ? (c) Which pairs are furthest from centrality?

Solution

Step 1. **Products** $x_j s_j$: 0.2, 0.3, 0.1, 0.2, 0.4.

Step 2. **(a) Gap** $= \mathbf{x}^\top \mathbf{s} = 0.2 + 0.3 + 0.1 + 0.2 + 0.4 = 1.2$. Duality measure $\mu = 1.2/5 = 0.24$.

Step 3. **(b) Target** $\tau = \sigma\mu = 0.2 \times 0.24 = 0.048$. The next Newton step aims to make every product about 0.048, cutting the gap to about $5 \times 0.048 = 0.24$.

Step 4. **(c) Centrality.** On the central path all products would equal $\mu = 0.24$. The largest relative deviations are $x_3 s_3 = 0.1$ (too small, the pair is near the boundary) and $x_5 s_5 = 0.4$ (too large). A common measure is $\|\mathbf{XS1} - \mu\mathbf{1}\|_2 / \mu = \sqrt{0.04^2 + 0.06^2 + 0.14^2 + 0.04^2 + 0.16^2}/0.24 = \sqrt{0.052}/0.24 \approx 0.950$.

Answer. (a) Gap $\boxed{1.2}$, $\mu = 0.24$; (b) $\tau = 0.048$; (c) pairs 3 and 5 deviate most, with centrality measure about 0.95.

Worked Example 16.4 One Primal Dual Newton Step by Hand

For the problem of Worked Example 16.2, start from the feasible interior point $\mathbf{x} = (0.5, 0.5)$, $y = 0$, $\mathbf{s} = (1, 2)$. Take one Newton step with $\sigma = 0.1$ and safety factor $\eta = 0.9$.

Solution

Step 1. **Measure and target.** $\mathbf{x}^\top \mathbf{s} = 0.5 + 1 = 1.5$, so $\mu = 0.75$ and $\tau = 0.075$.

Step 2. **Residuals.** $r_p = 1 - (0.5 + 0.5) = 0$; $\mathbf{r}_d = \mathbf{c} - \mathbf{A}^\top y - \mathbf{s} = (1, 2) - (0, 0) - (1, 2) = \mathbf{0}$; $\mathbf{r}_c = (0.075 - 0.5,\ 0.075 - 1) = (-0.425,\ -0.925)$.

Step 3. **Newton equations** (16.5).

$$\Delta x_1 + \Delta x_2 = 0, \qquad \Delta y + \Delta s_1 = 0, \qquad \Delta y + \Delta s_2 = 0,$$

$$1 \cdot \Delta x_1 + 0.5\,\Delta s_1 = -0.425, \qquad 2\,\Delta x_2 + 0.5\,\Delta s_2 = -0.925.$$

Step 4. **Eliminate.** $\Delta s_1 = \Delta s_2 = -\Delta y$ and $\Delta x_2 = -\Delta x_1$. The last two equations become $\Delta x_1 - 0.5\Delta y = -0.425$ and $-2\Delta x_1 - 0.5\Delta y = -0.925$.

Step 5. **Solve.** Subtract the second from the first: $3\Delta x_1 = 0.5$, so $\Delta x_1 = \frac{1}{6} \approx 0.1667$. Then $0.5\Delta y = \frac{1}{6} + 0.425$, so $\Delta y \approx 1.1833$. Hence $\Delta \mathbf{x} = (0.1667,\ -0.1667)$, $\Delta \mathbf{s} = (-1.1833,\ -1.1833)$.

Step 6. **Step lengths** (16.7)**.** Primal: only $\Delta x_2 < 0$, ratio $0.5/0.1667 = 3$, so $\alpha_P = \min(1,\ 0.9{\times}3) = 1$. Dual: ratios $1/1.1833 \approx 0.8451$ and $2/1.1833 \approx 1.690$; minimum 0.8451, so $\alpha_D = 0.9 \times 0.8451 \approx 0.7606$.

Step 7. **New point.** $\mathbf{x} = (0.6667,\ 0.3333)$; $y = 0.7606 \times 1.1833 \approx 0.9$; $\mathbf{s} = (1, 2) - 0.9 \cdot (1, 1) = (0.1,\ 1.1)$.

Step 8. **Progress.** New gap $\mathbf{x}^\top \mathbf{s} = 0.0667 + 0.3667 = 0.4333$, down from 1.5. Primal value 1.3333, dual value 0.9; their difference is the same 0.4333.

Answer. After one step $\boxed{\mathbf{x} = (0.667,\ 0.333),\ y = 0.9,\ \mathbf{s} = (0.1,\ 1.1)}$; the gap falls from 1.5 to 0.433.

Worked Example 16.5 Forming the Normal Equations

For the iterate of Worked Example 16.3 suppose the constraint matrix is $\mathbf{A} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}$. Compute $\mathbf{D} = \mathbf{X}\mathbf{S}^{-1}$ and the 2×2 normal matrix $\mathbf{A}\mathbf{D}\mathbf{A}^\top$, and explain why it is well defined.

Solution

Step 1. **Diagonal entries** $d_j = x_j/s_j$: $2/0.1 = 20$; $1/0.3 \approx 3.333$; $0.5/0.2 = 2.5$; $4/0.05 = 80$; $1/0.4 = 2.5$.

Step 2. **Entry formula.** $(\mathbf{A}\mathbf{D}\mathbf{A}^\top)_{ik} = \sum_j a_{ij} d_j a_{kj}$.

Step 3. **Entry (1,1)**: columns with $a_{1j} = 1$ are $j = 1, 2, 3$: $20 + 3.333 + 2.5 = 25.833$.

Step 4. **Entry (2,2)**: columns $j = 2, 4, 5$: $3.333 + 80 + 2.5 = 85.833$.

Step 5. **Entry (1,2) = (2,1)**: only $j = 2$ is in both rows: 3.333.

Step 6. **Result.**

$$\mathbf{A}\mathbf{D}\mathbf{A}^\top \approx \begin{pmatrix} 25.833 & 3.333 \\ 3.333 & 85.833 \end{pmatrix}, \qquad \det \approx 25.833(85.833) - 3.333^2 \approx 2206.2 > 0.$$

Step 7. **Why it is well defined.** Every $d_j > 0$ because the iterate is strictly interior, and $\mathbf{A}$ has full row rank, so $\mathbf{A}\mathbf{D}\mathbf{A}^\top$ is symmetric positive definite and Cholesky factorisation succeeds. Large d_j (here $d_4 = 80$) mark variables likely to be positive at the optimum; tiny d_j mark variables heading to zero. Near the optimum this spread becomes extreme, which is why careful numerical linear algebra matters.

Answer. $\boxed{\mathbf{A}\mathbf{D}\mathbf{A}^\top \approx \begin{pmatrix} 25.83 & 3.33 \\ 3.33 & 85.83 \end{pmatrix}}$, positive definite.

Worked Example 16.6 How Many Iterations?

A short step interior point method reduces μ by the factor $1-0.4/\sqrt{n}$ per iteration. Starting from a gap of 10^2, how many iterations are needed to reach a gap of 10^{-8} when (a) $n = 100$ and (b) $n = 10{,}000$? Compare with typical practice.

Solution

Step 1. Requirement. The gap must shrink by the factor 10^{-10}. After k iterations it is multiplied by $(1-0.4/\sqrt{n})^k$, so we need $(1-0.4/\sqrt{n})^k \le 10^{-10}$.

Step 2. Take logarithms (the logarithm of a number below one is negative, so the inequality flips when dividing):

$$k \ge \frac{10\ln 10}{-\ln(1-0.4/\sqrt{n})} = \frac{23.026}{-\ln(1-0.4/\sqrt{n})}.$$

Step 3. (a) $n = 100$**.** $0.4/\sqrt{100} = 0.04$ and $-\ln(0.96) \approx 0.04082$, so $k \ge 564.1$: about **565** iterations.

Step 4. (b) $n = 10{,}000$**.** $0.4/100 = 0.004$ and $-\ln(0.996) \approx 0.004008$, so $k \ge 5{,}745$: about **5,745** iterations.

Step 5. Scaling. Multiplying n by 100 multiplied the bound by only about 10, that is $\sqrt{100}$.

Step 6. Practice. Long step predictor corrector methods choose σ adaptively and typically finish both problems in 20 to 60 iterations. The theoretical bound guarantees polynomial behaviour; the practical method is far faster.

Answer. (a) $\boxed{\approx 565}$; (b) $\boxed{\approx 5{,}745}$ iterations in theory, growing like $\sqrt{n}$; tens of iterations in practice.

Module Summary

- The barrier problem $\min\ \mathbf{c}^\top\mathbf{x} - \mu\sum \ln x_j$ subject to $\mathbf{A}\mathbf{x} = \mathbf{b}$ keeps iterates inside; its conditions replace $x_j s_j = 0$ by $x_j s_j = \mu$.
- The central path is the curve of barrier solutions; on it the duality gap equals $n\mu$.
- Newton's method on the perturbed conditions gives the system (16.5), reduced to the normal equations $\mathbf{A}\mathbf{D}\mathbf{A}^\top\Delta\mathbf{y} = \cdots$ with $\mathbf{D} = \mathbf{X}\mathbf{S}^{-1}$.
- Step lengths come from a ratio test with a safety factor, keeping $\mathbf{x}, \mathbf{s} > \mathbf{0}$.
- Iteration bounds grow like $\sqrt{n}$; in practice tens of iterations suffice. Interior methods excel on large sparse problems; simplex excels at warm starts and exact vertices.

MODULE 17

Integer Programming, Branch and Bound, and Cutting Planes

Learning Objectives

After studying this module you will be able to:

- explain when divisibility fails and integer variables are needed;
- define the LP relaxation and prove that it bounds the integer optimum;
- model logical conditions such as yes or no decisions, fixed charges and "either or" constraints with binary variables;
- carry out branch and bound by hand, using bounds to prune the search tree;
- derive a Gomory fractional cut from a simplex tableau;
- recognise totally unimodular problems, such as assignment, whose LP relaxations are automatically integral.

17.1 When Fractions Are Not Allowed

The divisibility assumption of Module 1 lets variables take values like 7.5. Often that is acceptable: 7.5 tables per week is a production rate. Sometimes it is not: one cannot rent 0.4 of a server, hire 2.7 annotators, or select "half" of a feature. A linear program in which some or all variables must be integers is an **integer program** (IP), or a **mixed integer program** (MIP) when only some are. Integer programs are far harder to solve in general, but they are built on linear programming: every practical algorithm solves many LPs.

Definition Integer Program and LP Relaxation

An integer program has the form

$$z_{IP} = \max\ \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} \le \mathbf{b}, \quad \mathbf{x} \ge \mathbf{0}, \quad x_j \in \mathbb{Z} \ \text{ for } j \in J, \tag{17.1}$$

where

$\mathbb{Z}$ is the set of integers $\{\dots, -1, 0, 1, 2, \dots\}$;

J is the set of indices of integer variables (all of them for a pure IP);

binary variable means an integer variable with $0 \le x_j \le 1$, so $x_j \in \{0, 1\}$.

Its **LP relaxation** is the same problem with the integrality requirements removed; its optimal value is z_{LP}.

Derivation The relaxation gives a bound

Step 1. Every feasible point of the IP satisfies all the LP constraints, so the IP feasible set is a subset of the LP feasible set.

Step 2. Maximising over a larger set can only give a value at least as large. Hence

$$z_{IP} \le z_{LP} \quad \text{(maximisation)}, \qquad z_{IP} \ge z_{LP} \quad \text{(minimisation)}. \tag{17.2}$$

Step 3. If the LP optimum happens to be integral, it is feasible for the IP and attains the bound, so it is optimal for the IP.

The difference $z_{LP} - z_{IP}$ is the **integrality gap**.

Common Pitfall

Rounding the LP solution is not a reliable method. The rounded point may be infeasible, and even when it is feasible it may be far from optimal. Worked Example 17.1 shows both failures.

17.2 Modelling with Binary Variables

Key Result Standard binary modelling devices

Let $y \in \{0, 1\}$ be a binary decision and M a valid upper bound on the quantity involved.

1. **Yes or no selection**: $y_j = 1$ if item j is chosen. A budget is $\sum_j w_j y_j \le W$.
2. **Fixed charge**: a continuous amount $x \ge 0$ can be positive only if a set up cost f is paid: $x \le My$ and cost $fy + cx$. If $y = 0$ the constraint forces $x = 0$; if $y = 1$ it is inactive.
3. **Either or**: at least one of $\mathbf{a}_1^\top \mathbf{x} \le b_1$ and $\mathbf{a}_2^\top \mathbf{x} \le b_2$ must hold: $\mathbf{a}_1^\top \mathbf{x} \le b_1 + My$ and $\mathbf{a}_2^\top \mathbf{x} \le b_2 + M(1 - y)$.
4. **Implication**: “if $y_1 = 1$ then $y_2 = 1$” is $y_1 \le y_2$.
5. **At most k of n**: $\sum_j y_j \le k$; “exactly one” is $\sum_j y_j = 1$.

17.3 Branch and Bound

Derivation The logic of branch and bound

Step 1. **Bound.** Solve the LP relaxation of the current subproblem. Its value bounds every integer solution in that subproblem by (17.2).

Step 2. **Branch.** If the LP solution has a fractional integer variable $x_j = v \notin \mathbb{Z}$, split the subproblem into two: one with $x_j \le \lfloor v \rfloor$ and one with $x_j \ge \lceil v \rceil$. Every integer solution lies in exactly one of them, and the current fractional point lies in neither.

Step 3. **Prune.** Discard ("fathom") a subproblem if (a) its LP is infeasible, (b) its LP solution is integral (record it as the **incumbent** if it is the best so far), or (c) its LP bound is no better than the incumbent.

Step 4. **Stop** when no subproblems remain; the incumbent is optimal.

where

$\lfloor v \rfloor$ is the largest integer $\le v$ (the floor); $\lceil v \rceil$ is the smallest integer $\ge v$ (the ceiling);

incumbent is the best integer solution found so far.

If the objective coefficients are integers and all variables are integer, the objective of any integer solution is an integer, so a subproblem can also be pruned when $\lfloor z_{LP} \rfloor \le z_{\text{incumbent}}$.

17.4 Cutting Planes

Derivation The Gomory fractional cut

Take a row of an optimal simplex tableau for a pure integer program (all variables, including slacks, integer, which holds when **A** and **b** are integral):

$$x_{B(r)} + \sum_{j\in N} \bar{a}_{rj} x_j = \bar{b}_r, \qquad \bar{b}_r \notin \mathbb{Z}.$$

Step 1. Write every coefficient as an integer part plus a fractional part: $\bar{a}_{rj} = \lfloor \bar{a}_{rj} \rfloor + f_j$ and $\bar{b}_r = \lfloor \bar{b}_r \rfloor + f_0$, with $0 \le f_j < 1$ and $0 < f_0 < 1$.

Step 2. Move all integer parts to the left: $x_{B(r)} + \sum_j \lfloor \bar{a}_{rj} \rfloor x_j - \lfloor \bar{b}_r \rfloor = f_0 - \sum_j f_j x_j$.

Step 3. For any integer solution the left side is an integer, so the right side is an integer too.

Step 4. The right side is at most $f_0 < 1$ (since $f_j x_j \ge 0$). An integer less than 1 is at most 0. Hence

$$\sum_{j\in N} f_j x_j \;\ge\; f_0. \tag{17.3}$$

Step 5. At the current LP optimum all nonbasic $x_j = 0$, so the left side is $0 < f_0$: the cut is violated by the LP solution but satisfied by every integer solution.

Adding (17.3) and reoptimising with the dual simplex method (Module 14) tightens the relaxation. Modern solvers combine many kinds of cuts with branch and bound ("branch and cut").

17.5 Problems That Are Integral for Free

Definition Total Unimodularity

A matrix is **totally unimodular** (TU) if every square submatrix has determinant 0, 1 or −1.

Key Result Integral vertices

If $\mathbf{A}$ is totally unimodular and $\mathbf{b}$ is integral, every vertex of $\{\mathbf{x} : \mathbf{A}\mathbf{x} \le \mathbf{b},\ \mathbf{x} \ge \mathbf{0}\}$ is integral. Reason: a vertex solves $\mathbf{B}\mathbf{x}_B = \mathbf{b}$ for a square submatrix $\mathbf{B}$, and by Cramer's rule $x_i = \det \mathbf{B}_i / \det \mathbf{B}$ where $\mathbf{B}_i$ has integer entries; since $\det \mathbf{B} = \pm 1$, each x_i is an integer. The node arc incidence matrices of networks and the constraint matrices of assignment and transportation problems are totally unimodular, so their LPs automatically have integer optimal vertices (Module 18).

Machine Learning Connection

Integer programs appear throughout machine learning: best subset feature selection with a cardinality limit, verification of neural networks with ReLU activations (each ReLU is an "either or" constraint), optimal decision trees, MAP inference in graphical models, and scheduling of training jobs on clusters. LP relaxations are equally central: they give fast approximate inference with guarantees, and when the constraint matrix is totally unimodular, as in bipartite matching for assigning annotators to tasks or in the Hungarian algorithm used to match predictions to ground truth in object detection (for example in DETR), the relaxation is exact.

17.6 Worked Examples

Worked Example 17.1 Why Rounding Fails

Consider max $5x_1 + 8x_2$ subject to $x_1 + x_2 \le 6$, $5x_1 + 9x_2 \le 45$, $x_1, x_2 \ge 0$ and integer. Solve the LP relaxation, then examine every rounding of its solution.

Solution

Step 1. **LP relaxation.** Both constraints are tight at the optimum: $x_1 + x_2 = 6$ and $5x_1 + 9x_2 = 45$. Substituting $x_1 = 6 - x_2$: $30 + 4x_2 = 45$, so $x_2 = 3.75$, $x_1 = 2.25$, and $z_{LP} = 11.25 + 30 = 41.25$.

Step 2. **Four roundings.**

(x_1, x_2)	$x_1 + x_2 \le 6$	$5x_1 + 9x_2 \le 45$	z
$(2, 3)$	5 ✓	37 ✓	34
$(2, 4)$	6 ✓	46 ×	
$(3, 3)$	6 ✓	42 ✓	39
$(3, 4)$	7 ×	51 ×	

Step 3. **Best rounding** is $(3, 3)$ with $z = 39$. But the point $(0, 5)$ is feasible ($5 \le 6$, $45 \le 45$) with $z = 40$, better than any rounding, and it is far from the LP solution.

Answer. $z_{LP} = 41.25$; rounding gives at best 39, while the true integer optimum (Worked Example 17.2) is $\boxed{40 \text{ at } (0,5)}$.

Worked Example 17.2 Branch and Bound by Hand

Solve the integer program of Worked Example 17.1 by branch and bound, exploring the “down” branch first.

Solution

Step 1. **Node P0** (root): LP solution $(2.25,\ 3.75)$, $z = 41.25$. Since the objective coefficients are integers, no integer solution exceeds $\lfloor 41.25 \rfloor = 41$. Branch on x_1: $x_1 \le 2$ or $x_1 \ge 3$.

Step 2. **Node P1** ($x_1 \le 2$): with $x_1 = 2$, the second constraint gives $x_2 \le 35/9 \approx 3.889$; LP solution $(2,\ 3.889)$, $z \approx 41.11$. Branch on x_2: $x_2 \le 3$ or $x_2 \ge 4$.

Step 3. **Node P2** ($x_1 \le 2$, $x_2 \le 3$): LP solution $(2,3)$, $z = 34$, integral. *Incumbent* 34.

Step 4. **Node P3** ($x_1 \le 2$, $x_2 \ge 4$): with $x_2 = 4$, $5x_1 \le 9$, so $x_1 \le 1.8$; LP solution $(1.8,\ 4)$, $z = 41$. Branch on x_1: $x_1 \le 1$ or $x_1 \ge 2$.

Step 5. **Node P4** ($x_1 \le 1$, $x_2 \ge 4$): $x_1 = 1$ gives $x_2 \le 40/9 \approx 4.444$; LP solution $(1,\ 4.444)$, $z \approx 40.56$. Branch on x_2: $x_2 \le 4$ or $x_2 \ge 5$.

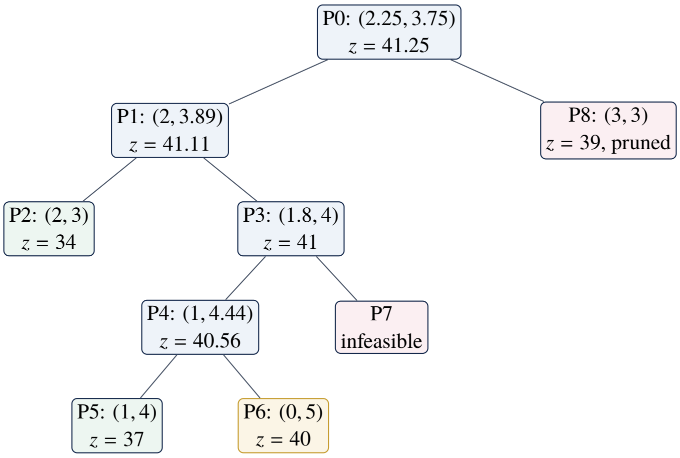
Step 6. **Node P5** ($x_1 \le 1$, $x_2 = 4$): LP solution $(1,4)$, $z = 37$, integral, better than 34. *Incumbent* 37.

Step 7. **Node P6** ($x_1 \le 1$, $x_2 \ge 5$): $5x_1 + 45 \le 45$ forces $x_1 = 0$; LP solution $(0,5)$, $z = 40$, integral. *Incumbent* 40.

Step 8. **Node P7** ($x_1 = 2$, $x_2 \ge 4$): $10 + 36 = 46 > 45$. *Infeasible*, pruned.

Step 9. **Node P8** ($x_1 \ge 3$): with $x_1 = 3$, $x_2 \le 3$; LP solution $(3,3)$, $z = 39 \le 40$. Pruned by bound (it is integral but worse).

Step 10. **Tree summary.**



Answer. $\boxed{x^\star = (0,5),\ z_{IP} = 40}$; the integrality gap is $41.25 - 40 = 1.25$.

Worked Example 17.3 A Gomory Cut from the Optimal Tableau

The optimal LP tableau for Worked Example 17.1 has the rows

$$x_1 + \tfrac{9}{4}s_1 - \tfrac{1}{4}s_2 = \tfrac{9}{4}, \qquad x_2 - \tfrac{5}{4}s_1 + \tfrac{1}{4}s_2 = \tfrac{15}{4}.$$

Derive the Gomory cut from the x_1 row, express it in terms of x_1, x_2, and check that it removes the LP optimum but keeps all integer points.

Solution

Step 1. Fractional parts. $\frac{9}{4} = 2 + \frac{1}{4}$, so $f_{s_1} = \frac{1}{4}$. $-\frac{1}{4} = -1 + \frac{3}{4}$, so $f_{s_2} = \frac{3}{4}$. $\frac{9}{4} = 2 + \frac{1}{4}$, so $f_0 = \frac{1}{4}$.

Step 2. Cut (17.3)**.** $\frac{1}{4}s_1 + \frac{3}{4}s_2 \geq \frac{1}{4}$, that is $s_1 + 3s_2 \geq 1$.

Step 3. Substitute the slacks. $s_1 = 6 - x_1 - x_2$ and $s_2 = 45 - 5x_1 - 9x_2$:

$$6 - x_1 - x_2 + 135 - 15x_1 - 27x_2 \geq 1 \implies 16x_1 + 28x_2 \leq 140 \implies 4x_1 + 7x_2 \leq 35.$$

Step 4. LP optimum removed. $4(2.25) + 7(3.75) = 9 + 26.25 = 35.25 > 35$.

Step 5. Integer points kept. $(0,5)$: $35 \leq 35$. $(3,3)$: $33 \leq 35$. $(1,4)$: $32 \leq 35$. $(2,3)$: $29 \leq 35$. In general, s_1 and s_2 are integers at integer points, and $s_1 + 3s_2 \geq 1$ fails only when $s_1 = s_2 = 0$, i.e. at the fractional point $(2.25, 3.75)$.

Answer. Gomory cut: $\boxed{4x_1 + 7x_2 \leq 35}$, which passes through the integer optimum $(0,5)$.

Worked Example 17.4 Feature Selection under a Budget

Five candidate features have estimated accuracy gains $(12, 10, 8, 7, 4)$ (tenths of a percentage point) and acquisition costs $(5, 4, 4, 3, 2)$ (thousands of pounds). The budget is 10. Choose features to maximise total gain by branch and bound with binary variables.

Solution

Step 1. Model. max $12y_1 + 10y_2 + 8y_3 + 7y_4 + 4y_5$ subject to $5y_1 + 4y_2 + 4y_3 + 3y_4 + 2y_5 \leq 10$, $y_j \in \{0,1\}$.

Step 2. LP relaxations are easy. For a single constraint with bounds $0 \leq y_j \leq 1$, the LP optimum fills items in decreasing order of gain per cost (Module 1). Ratios: 2.4, 2.5, 2.0, 2.333, 2.0, so the order is $y_2, y_1, y_4, y_3/y_5$.

Step 3. Root. $y_2 = 1$ (cost 4), $y_1 = 1$ (cost 9), then $\frac{1}{3}$ of y_4: bound $10 + 12 + \frac{7}{3} \approx 24.33$. Since gains are integers, no selection exceeds 24. Branch on y_4.

Step 4. Search summary (exploring $y_j = 1$ first):

Fixed variables	LP bound and solution	Action
$y_4 = 1$	24.2; $y_1 = 0.6$	branch on y_1
$y_4 = 1, y_1 = 1$	24.0; $y_2 = 0.5$	branch on y_2
$y_4 = y_1 = y_2 = 1$	cost $12 > 10$	infeasible
$y_4 = y_1 = 1, y_2 = 0$	23.0; $y_3 = 0.5$	branch on y_3
$y_3 = 1$	cost $12 > 10$	infeasible
$y_3 = 0$	23; $\{1, 4, 5\}$ integral	incumbent 23
$y_4 = 1, y_1 = 0$	23.0	pruned (≤ 23)
$y_4 = 0$	24.0; $y_5 = 0.5$	branch on y_5
$y_4 = 0, y_5 = 1$	23.6	pruned ($\lfloor 23.6 \rfloor \leq 23$)
$y_4 = 0, y_5 = 0$	24.0; $y_3 = 0.25$	branch on y_3
$y_3 = 1$	22.8	pruned
$y_3 = 0$	22; $\{1, 2\}$	pruned

Step 5. **Result.** Incumbent $\{1, 4, 5\}$: cost $5 + 3 + 2 = 10$, gain $12 + 7 + 4 = 23$. The greedy choice $\{1, 2\}$ (the two best ratios) gains only 22, because it leaves one unit of budget unused.

Answer. Select features $\boxed{1,\ 4,\ 5}$ for a gain of 23 (2.3 percentage points).

Worked Example 17.5 A Fixed Charge Compute Rental

A team needs 40 GPU hours. Provider A charges a set up fee of £90 plus £2 per hour (capacity 60 hours). Provider B charges £20 plus £3 per hour (capacity 30). Provider C has no fee and charges £5 per hour (capacity 25). (a) Formulate a mixed integer program. (b) Solve it by examining the provider combinations. (c) Compare with the LP relaxation.

Solution

Step 1. **(a) Variables.** $x_A, x_B, x_C \geq 0$ hours; $y_A, y_B \in \{0, 1\}$ for paying the fees (C needs none).

$$\begin{aligned} \min\ & 90y_A + 20y_B + 2x_A + 3x_B + 5x_C \\ \text{subject to}\ & x_A + x_B + x_C \geq 40, \quad x_A \leq 60y_A, \quad x_B \leq 30y_B, \quad x_C \leq 25. \end{aligned}$$

Step 2. **(b) Enumerate the fee decisions** and, for each, buy hours cheapest first:

Open	Hours	Cost
C only	capacity $25 < 40$	infeasible
B and C	B 30, C 10	$20 + 90 + 50 = 160$
A and C	A 40	$90 + 80 = 170$
A, B and C	A 40	$110 + 80 = 190$

Step 3. **Optimum**: open B and use C for the remainder, cost £160.

Step 4. **(c) LP relaxation.** With y_A continuous, the cheapest choice sets $y_A = x_A/60$, so A effectively costs $2 + 90/60 = 3.5$ per hour; similarly B costs $3 + 20/30 \approx 3.67$ per hour. The relaxation buys all 40 hours from A: cost $3.5 \times 40 = 140$, with $y_A = \frac{2}{3}$.

Step 5. **Gap.** $160 - 140 = 20$. The relaxation spreads the fixed fee as if it could be paid in fractions, which is why fixed charge models have weak relaxations and need branch and bound.

Answer. Open provider B (30 hours) and use C for 10 hours: $\boxed{\text{cost £160}}$; the LP relaxation bound is £140.

Worked Example 17.6 Assigning Annotators: An LP That Is Automatically Integral

Three annotators must each take exactly one of three labelling tasks. The time (hours) for annotator i on task j is given by

$$\mathbf{C} = \begin{pmatrix} 9 & 4 & 7 \\ 6 & 8 & 3 \\ 5 & 7 & 9 \end{pmatrix}.$$

Formulate the assignment LP, explain why it has an integral optimum, and solve it.

Solution

Step 1. **Model.** $x_{ij} \geq 0$ is the fraction of task j given to annotator i:

$$\min \sum_{i,j} c_{ij} x_{ij}$$

$$\text{subject to} \sum_j x_{ij} = 1 \quad (i = 1,2,3), \qquad \sum_i x_{ij} = 1 \quad (j = 1,2,3), \qquad x_{ij} \geq 0.$$

Step 2. **Integrality.** Each variable appears in exactly one “annotator” row and one “task” row with coefficient 1. Such matrices (bipartite incidence matrices) are totally unimodular, so every vertex of the feasible region is a 0 or 1 matrix: a permutation. The simplex method therefore returns a genuine assignment without any integer constraints.

Step 3. **Solve by comparing permutations** (there are $3! = 6$):

Assignment (task of annotator 1, 2, 3)	Total
(1,2,3)	$9 + 8 + 9 = 26$
(1,3,2)	$9 + 3 + 7 = 19$
(2,1,3)	$4 + 6 + 9 = 19$
(2,3,1)	$4 + 3 + 5 = 12$
(3,1,2)	$7 + 6 + 7 = 20$
(3,2,1)	$7 + 8 + 5 = 20$

Step 4. **Optimal.** Annotator 1 takes task 2, annotator 2 takes task 3, annotator 3 takes task 1: total 12 hours. An LP solver returns exactly this 0 or 1 solution.

Step 5. **Dual check.** With row potentials $u = (4,3,5)$ and column potentials $v = (0,0,0)$, every $c_{ij} - u_i - v_j \geq 0$ ($9-4$, $7-4$, $6-3$, $8-3$, $7-5$, $9-5$ are all positive) and equals 0 on the chosen cells, and $\sum u_i + \sum v_j = 12$: a dual certificate by Module 13.

Answer. $\boxed{1 \to 2,\ 2 \to 3,\ 3 \to 1,\ \text{total 12 hours}}$, found by plain LP thanks to total unimodularity.

Module Summary

- Integer requirements break divisibility; the LP relaxation bounds the integer optimum, $z_{IP} \le z_{LP}$ for maximisation.
- Binary variables model selection, fixed charges ($x \le My$), either or conditions and logical implications.
- Branch and bound: solve relaxations, branch on a fractional variable with $\le \lfloor v \rfloor$ and $\ge \lceil v \rceil$, prune by infeasibility, integrality or bound.
- Gomory cut from a tableau row: $\sum_j f_j x_j \ge f_0$, built from fractional parts.
- Totally unimodular matrices with integral **b** give integral vertices: assignment, transportation and network flow LPs need no integer constraints.

Part IV Linear Programming in Machine Learning

MODULE 18

Network Flow Models and Their Role in Machine Learning

Learning Objectives

After studying this module you will be able to:

- represent a network by its node arc incidence matrix and write flow conservation as $\mathbf{Ax} = \mathbf{b}$;
- formulate and solve transportation, shortest path, maximum flow and minimum cost flow problems as linear programs;
- interpret the dual variables of network problems as node potentials and verify optimality with reduced arc costs;
- derive the max flow min cut theorem from LP duality;
- model binary image segmentation as a minimum cut, a classical computer vision application.

18.1 Why Networks Deserve Their Own Module

A remarkable share of practical linear programs are *network* problems: shipping goods, routing data packets, assigning workers, matching predictions to labels, scheduling jobs, and partitioning images. Their constraint matrices have a special structure that makes them totally unimodular (Module 17), so their optimal vertices are integral, and specialised versions of the simplex method solve them hundreds of times faster than general LP codes. Their dual variables have a clear physical meaning, potentials at the nodes. Many machine learning tasks, including bipartite matching in object detection, graph cut segmentation and optimal transport (Module 23), are network flows in disguise.

18.2 Networks and Flow Conservation

Definition Directed Network and Incidence Matrix

A **directed network** consists of a set of nodes V with $|V| = p$ and a set of arcs E with $|E| = q$, each arc (u, v) pointing from node u to node v. The **node arc incidence matrix** $\mathbf{A} \in \mathbb{R}^{p\times q}$ has, in the column of arc (u, v), the entry $+1$ in row u, the entry -1 in row v, and 0 elsewhere.

Derivation Flow conservation as Ax = b

Let $x_{uv} \geq 0$ be the flow on arc (u, v) and let b_v be the **net supply** of node v (positive at sources, negative at sinks, zero at transit nodes).

Step 1. Conservation at node v says: flow out minus flow in equals net supply,

$$\sum_{w:(v,w)\in E} x_{vw} - \sum_{u:(u,v)\in E} x_{uv} = b_v.$$

Step 2. Row v of $\mathbf{A}$ has $+1$ under arcs leaving v and -1 under arcs entering v, so $(\mathbf{Ax})_v$ is exactly the left side.

Step 3. Hence all conservation equations together read

$$\mathbf{Ax} = \mathbf{b}. \tag{18.1}$$

Step 4. Summing all rows of $\mathbf{A}$ gives zero (each column has one $+1$ and one -1), so a solution requires $\sum_v b_v = 0$: total supply equals total demand.

Definition Minimum Cost Flow Problem

$$\min \sum_{(u,v)\in E} c_{uv} x_{uv} \quad \text{subject to} \quad \mathbf{Ax} = \mathbf{b}, \qquad 0 \leq x_{uv} \leq k_{uv}, \tag{18.2}$$

where

c_{uv} is the cost per unit of flow on arc (u, v);

k_{uv} is the capacity of arc (u, v) (possibly infinite).

Transportation, assignment, shortest path and maximum flow problems are all special cases.

Derivation Node potentials and reduced arc costs

Step 1. The dual of (18.2) attaches a free variable π_v to each conservation equation (a **node potential**) and a variable to each capacity.

Step 2. The reduced cost of arc (u, v) is its cost minus the dual price of its column, $c_{uv} - (\pi_u - \pi_v)$, because the column has $+1$ in row u and -1 in row v:

$$\bar{c}_{uv} = c_{uv} - \pi_u + \pi_v. \tag{18.3}$$

Step 3. Complementary slackness (Module 13) for bounded variables then says that a flow is optimal if and only if there are potentials with

$$\bar{c}_{uv} > 0 \Rightarrow x_{uv} = 0, \qquad \bar{c}_{uv} < 0 \Rightarrow x_{uv} = k_{uv}, \qquad 0 < x_{uv} < k_{uv} \Rightarrow \bar{c}_{uv} = 0. \tag{18.4}$$

where

$\pi_u - \pi_v$ is the “price difference” between the two ends of an arc: the value of moving one unit from u to v.

Potentials are determined only up to a common constant, because adding the same number to every π_v leaves (18.3) unchanged.

18.3 Maximum Flow and Minimum Cut

Definition Maximum Flow and Cut

Given a source s, a sink t and capacities k_{uv}, the **maximum flow problem** maximises the net flow ϕ leaving s subject to conservation at every other node except t and $0 \le x_{uv} \le k_{uv}$. An $s\,t$ **cut** is a partition of the nodes into a set S containing s and its complement $\bar{S}$ containing t; its **capacity** is the total capacity of arcs from S to $\bar{S}$.

Derivation Weak duality for flows and cuts

Step 1. Sum the conservation equations over all nodes in S. Arcs with both ends in S cancel (each contributes $+x$ once and $-x$ once). The source contributes the net flow ϕ; other nodes in S contribute zero. Hence

$$\phi = \sum_{(u,v):u\in S, v\in\bar{S}} x_{uv} - \sum_{(u,v):u\in\bar{S}, v\in S} x_{uv}.$$

Step 2. The first sum is at most the cut capacity (each $x_{uv} \le k_{uv}$); the second sum is at least zero. Therefore

$$\phi \le \operatorname{cap}(S, \bar{S}) \quad \text{for every flow and every cut.} \tag{18.5}$$

Key Result Max flow min cut theorem

The maximum flow value equals the minimum cut capacity. This is strong duality for the max flow LP: its dual is a relaxation of the minimum cut problem whose vertices are integral (by total unimodularity), so the LP dual optimum is attained by an actual cut. At the optimum every arc leaving S is saturated and every arc entering S is empty.

Lucid Insight Bottlenecks

A minimum cut is the network's bottleneck: the cheapest set of arcs whose removal disconnects s from t. Strong duality says you can always push exactly as much flow as the narrowest bottleneck allows, no more and no less.

Machine Learning Connection

Graph cuts in vision. Binary image segmentation chooses for each pixel the label "object" or "background" to minimise an energy made of unary costs (how well a pixel's colour fits each label) and pairwise costs (a penalty λ when neighbouring pixels disagree). Boykov, Veksler and Zabih showed that such energies are minimised exactly by a minimum $s\,t$ cut, solved via maximum flow. **Matching.** The Hungarian algorithm used to pair predicted and true objects in detection transformers solves an assignment problem, a minimum cost flow. **Transport.** The Wasserstein distance of Module 23 is a transportation problem. **Clustering and fairness.** Balanced assignment of points to clusters with capacity limits is a minimum cost flow.

18.4 Worked Examples

Worked Example 18.1 A Transportation Problem for Data Centres

Two data centres can process 50 and 70 terabytes of jobs per day. Three regions submit 30, 40 and 50 terabytes. The cost (pounds per terabyte) of serving region j from centre i is $\begin{pmatrix} 4 & 6 & 9 \\ 5 & 3 & 8 \end{pmatrix}$. Find the cheapest plan and certify it with potentials.

Solution

Step 1. **Model.** $x_{ij} \geq 0$ terabytes from centre i to region j:

$$\begin{aligned} \min \quad & 4x_{11} + 6x_{12} + 9x_{13} + 5x_{21} + 3x_{22} + 8x_{23} \\ \text{subject to} \quad & x_{11} + x_{12} + x_{13} = 50, \quad x_{21} + x_{22} + x_{23} = 70, \\ & x_{11} + x_{21} = 30, \quad x_{12} + x_{22} = 40, \quad x_{13} + x_{23} = 50. \end{aligned}$$

Supply 120 equals demand 120.

Step 2. **Intuition.** Region 1 is cheapest from centre 1 ($4 < 5$), region 2 from centre 2 ($3 < 6$), and region 3 slightly cheaper from centre 2 ($8 < 9$) but centre 2 has limited capacity.

Step 3. **Candidate plan.** $x_{11} = 30$, $x_{22} = 40$; centre 2 has $70 - 40 = 30$ left for region 3, so $x_{23} = 30$, and centre 1 sends the remaining $x_{13} = 20$. Cost $120 + 120 + 240 + 180 = 660$.

Step 4. **Potentials.** For a transportation problem the dual has u_i for centres and v_j for regions, and used routes must satisfy $u_i + v_j = c_{ij}$. Set $u_2 = 0$. Route $(2,2)$: $v_2 = 3$. Route $(2,3)$: $v_3 = 8$. Route $(1,3)$: $u_1 = 9 - 8 = 1$. Route $(1,1)$: $v_1 = 4 - 1 = 3$.

Step 5. **Reduced costs of unused routes.** $(1,2)$: $6 - (1+3) = 2 \geq 0$. $(2,1)$: $5 - (0+3) = 2 \geq 0$. Both nonnegative, so no route can lower the cost: optimal.

Step 6. **Dual value.** $50u_1 + 70u_2 + 30v_1 + 40v_2 + 50v_3 = 50 + 0 + 90 + 120 + 400 = 660$, equal to the primal cost.

Answer. $\boxed{x_{11} = 30,\ x_{13} = 20,\ x_{22} = 40,\ x_{23} = 30,\ \text{cost £}660}$ per day.

Worked Example 18.2 Shortest Path as a Linear Program

Arcs and lengths: $s \to a$ 2, $s \to b$ 5, $a \to b$ 1, $a \to c$ 6, $b \to c$ 2, $b \to t$ 7, $c \to t$ 1. Formulate the shortest s to t path as a minimum cost flow, solve it, and interpret the potentials.

Solution

Step 1. **Model.** Send one unit from s to t: $b_s = 1$, $b_t = -1$, other $b_v = 0$; minimise $\sum c_{uv}x_{uv}$ with $x \geq 0$ and no capacities. By total unimodularity an optimal vertex has $x_{uv} \in \{0,1\}$, describing a path.

Step 2. **Distances to** t by working backwards (Bellman's principle): $d(t) = 0$; $d(c) = 1$; $d(b) = \min(7,\ 2+1) = 3$; $d(a) = \min(1+3,\ 6+1) = 4$; $d(s) = \min(2+4,\ 5+3) = 6$.

Step 3. **Path.** Follow the minimising choices: $s \to a \to b \to c \to t$, length $2 + 1 + 2 + 1 = 6$.

Step 4. **Potentials.** Take $\pi_v = d(v)$. Reduced costs (18.3): $s \to a$: $2-6+4 = 0$; $a \to b$: $1-4+3 = 0$; $b \to c$: $2 - 3 + 1 = 0$; $c \to t$: $1 - 1 + 0 = 0$ (all used arcs have zero reduced cost); $s \to b$: $5 - 6 + 3 = 2$; $a \to c$: $6 - 4 + 1 = 3$; $b \to t$: $7 - 3 + 0 = 4$ (unused arcs nonnegative).

Step 5. **Interpretation.** The LP dual of shortest path is max $\pi_s - \pi_t$ subject to $\pi_u - \pi_v \leq c_{uv}$: stretch the nodes apart as far as possible without stretching any arc beyond its length. The optimum stretch equals the shortest distance.

Answer. Shortest path $\boxed{s \to a \to b \to c \to t, \text{ length } 6}$, with potentials $(6, 4, 3, 1, 0)$.

Worked Example 18.3 Maximum Flow and Minimum Cut

Capacities: $s \to a$ 10, $s \to b$ 8, $a \to b$ 5, $a \to c$ 7, $b \to c$ 6, $b \to t$ 4, $c \to t$ 12. Find a maximum flow and a minimum cut.

Solution

Step 1. **Augment along** $s \to a \to c \to t$**:** bottleneck $\min(10, 7, 12) = 7$. Flow value 7.

Step 2. **Augment along** $s \to b \to t$**:** bottleneck $\min(8, 4) = 4$. Value 11.

Step 3. **Augment along** $s \to b \to c \to t$**:** remaining capacities $8 - 4 = 4$, 6, $12 - 7 = 5$; bottleneck 4. Value 15.

Step 4. **Augment along** $s \to a \to b \to c \to t$**:** remaining $10 - 7 = 3$, 5, $6 - 4 = 2$, $5 - 4 = 1$; bottleneck 1. Value 16.

Step 5. **No more augmenting paths.** Arc $c \to t$ is full (12) and $b \to t$ is full (4), and these are the only arcs into t.

Step 6. **Minimum cut.** Take $S = \{s, a, b, c\}$, $\bar{S} = \{t\}$. Arcs from S to $\bar{S}$: $b \to t$ (4) and $c \to t$ (12), capacity 16. By (18.5) no flow exceeds 16, and we found 16, so both are optimal.

Step 7. **Other cuts for comparison.** $\{s\}$: $10 + 8 = 18$. $\{s, a, b\}$: $7 + 6 + 4 = 17$. Both larger.

Answer. Maximum flow $\boxed{16}$, equal to the minimum cut $\{b \to t,\ c \to t\}$.

Worked Example 18.4 Minimum Cost Flow with Potentials

On the network of Worked Example 18.3, with unit costs $s \to a$ 2, $s \to b$ 4, $a \to b$ 1, $a \to c$ 3, $b \to c$ 1, $b \to t$ 5, $c \to t$ 2, send 12 units from s to t at minimum cost. Verify the flow $x_{sa} = 10$, $x_{sb} = 2$, $x_{ab} = 4$, $x_{ac} = 6$, $x_{bc} = 6$, $x_{bt} = 0$, $x_{ct} = 12$ with the potentials $\pi = (\pi_s, \pi_a, \pi_b, \pi_c, \pi_t) = (4, 1, 0, -2, -5)$.

Solution

Step 1. **Conservation.** Node s: out $10+2 = 12$. Node a: in 10, out $4+6 = 10$. Node b: in $2+4 = 6$, out $6 + 0 = 6$. Node c: in $6 + 6 = 12$, out 12. Node t: in 12. All balanced; capacities respected.

Step 2. **Cost.** $10(2) + 2(4) + 4(1) + 6(3) + 6(1) + 0(5) + 12(2) = 20 + 8 + 4 + 18 + 6 + 0 + 24 = 80$.

Step 3. **Reduced costs** $\bar{c}_{uv} = c_{uv} - \pi_u + \pi_v$ and the conditions (18.4):

Arc	c_{uv}	$\bar{c}_{uv}$	x_{uv} / k_{uv}	**Condition**
$s \to a$	2	$2 - 4 + 1 = -1$	10/10	negative, at capacity ✓
$s \to b$	4	$4 - 4 + 0 = 0$	2/8	zero, interior ✓
$a \to b$	1	$1 - 1 + 0 = 0$	4/5	zero, interior ✓
$a \to c$	3	$3 - 1 - 2 = 0$	6/7	zero, interior ✓
$b \to c$	1	$1 - 0 - 2 = -1$	6/6	negative, at capacity ✓
$b \to t$	5	$5 - 0 - 5 = 0$	0/4	zero, at zero ✓
$c \to t$	2	$2 + 2 - 5 = -1$	12/12	negative, at capacity ✓

Step 4. **Conclusion.** Every arc satisfies (18.4), so the flow is optimal. The potential difference $\pi_s - \pi_t = 9$ is the marginal cost of a thirteenth unit: it would travel along $s \to b \to t$ at cost $4 + 5 = 9$.

Answer. The flow is optimal with $\boxed{\text{cost } 80}$; the next unit would cost 9.

Worked Example 18.5 Image Segmentation by Minimum Cut

A one dimensional "image" has four pixels in a row. The cost of labelling pixel i as object is $F = (4, 3, 1, 1)$ and as background is $G = (1, 2, 5, 4)$. Each pair of neighbours with different labels costs $\lambda = 2$. Find the minimum energy labelling and explain the cut construction.

Solution

Step 1. **Energy.** For labels $\ell_i \in \{0 = \text{background},\ 1 = \text{object}\}$:

$$E(\boldsymbol{\ell}) = \sum_{i=1}^{4} (\ell_i F_i + (1 - \ell_i) G_i) + \lambda \sum_{i=1}^{3} |\ell_i - \ell_{i+1}|\,.$$

Step 2. **Graph construction.** Add a source s (object side) and a sink t (background side). For each pixel, an arc $s \to i$ with capacity G_i and an arc $i \to t$ with capacity F_i; between neighbours, arcs in both directions with capacity λ. Put pixels labelled object in S and background in $\bar{S}$.

Step 3. **Cut equals energy.** An object pixel ($i \in S$) cuts its arc $i \to t$, paying F_i; a background pixel ($i \in \bar{S}$) cuts $s \to i$, paying G_i; each disagreeing neighbour pair cuts one arc of capacity λ. So $\text{cap}(S, \bar{S}) = E(\boldsymbol{\ell})$, and the minimum cut gives the minimum energy.

Step 4. **Candidate labellings.** $(0, 0, 1, 1)$: $1 + 2 + 1 + 1 + 2(1) = 7$. $(0, 1, 1, 1)$: $1 + 3 + 1 + 1 + 2 = 8$. $(1, 1, 1, 1)$: $4 + 3 + 1 + 1 = 9$. $(0, 0, 0, 0)$: $1 + 2 + 5 + 4 = 12$. $(0, 0, 0, 1)$: $1 + 2 + 5 + 1 + 2 = 11$.

Step 5. **Optimal.** Checking all $2^4 = 16$ labellings (or computing a max flow of value 7) confirms the minimum 7 at $(0, 0, 1, 1)$: the first two pixels are background and the last two object, with one boundary paying $\lambda = 2$.

Step 6. **Effect of λ.** If λ were 5, the boundary would cost too much and the all object labelling (9) would win over $(0, 0, 1, 1)$ (10): larger λ gives smoother segmentations.

Answer. Minimum energy $\boxed{7}$ with labelling (background, background, object, object).

Worked Example 18.6 Total Unimodularity of an Incidence Matrix

For the network with arcs $1 \to 2, 1 \to 3, 2 \to 3$, write the incidence matrix, compute the determinants of all its 2×2 submatrices formed from rows 1 and 2, and explain why integer supplies give integer flows.

Solution

Step 1. Incidence matrix (rows nodes 1, 2, 3; columns arcs 12, 13, 23):

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & -1 \end{pmatrix}.$$

Step 2. 2×2 minors from rows 1 and 2. Columns $\{12, 13\}$: $1 \cdot 0 - 1 \cdot (-1) = 1$. Columns $\{12, 23\}$: $1 \cdot 1 - 0 \cdot (-1) = 1$. Columns $\{13, 23\}$: $1 \cdot 1 - 0 \cdot 0 = 1$.

Step 3. Full determinant. Expanding along the first row: $1(0\cdot(-1)-1\cdot(-1))-1((-1)(-1)-1\cdot 0)+0 = 1 - 1 = 0$. This reflects the dependency that the rows sum to zero.

Step 4. General argument. Every column has exactly one $+1$ and one -1. A classical theorem (Poincaré) shows that such matrices are totally unimodular: every square submatrix has determinant 0 or ± 1. All the minors computed above agree.

Step 5. Consequence. With integer supplies $\mathbf{b}$ and integer capacities, every basic solution solves $\mathbf{B}\mathbf{x}_B = \mathbf{b}$ with $\det \mathbf{B} = \pm 1$, so by Cramer's rule $\mathbf{x}_B$ is integral. Network LPs therefore deliver whole units of flow, matched pairs and paths without integer programming.

Answer. All minors are in $\boxed{\{0, \pm 1\}}$; network LPs have integral optimal vertices.

Module Summary

- Flow conservation is $\mathbf{A}\mathbf{x} = \mathbf{b}$ with the node arc incidence matrix; supplies must balance.
- Minimum cost flow covers transportation, assignment, shortest path and maximum flow.
- Dual variables are node potentials; arc reduced costs $c_{uv} - \pi_u + \pi_v$ certify optimality through complementary slackness.
- Max flow equals min cut by LP duality; saturated arcs leaving S form the bottleneck.
- Incidence matrices are totally unimodular, so network LPs have integral solutions.
- ML uses: graph cut segmentation, Hungarian matching, transport distances, balanced clustering.

MODULE 19

Least Absolute Deviation Regression

Learning Objectives

After studying this module you will be able to:

- formulate least absolute deviation (LAD, or ℓ_1) regression as a linear program in two equivalent ways;
- derive its dual and interpret the dual variables as bounded residual signs;
- prove that some optimal LAD fit passes exactly through at least p data points;
- explain why LAD regression is robust to outliers in the response, and contrast it with least squares;
- compute LAD fits by hand for small data sets and certify them with dual solutions;
- add linear prior knowledge, such as sign or bound constraints, to a LAD model.

19.1 Motivation: When Squared Errors Mislead

Ordinary least squares (OLS) chooses weights that minimise the *sum of squared* residuals. Squaring makes large errors enormously expensive: a single residual of 10 costs as much as one hundred residuals of 1. When a data set contains a few gross errors, such as a mistyped sensor reading, a fraudulent transaction or a mislabelled example, OLS bends the whole fit towards them. Least absolute deviation regression replaces squares by absolute values. Each residual then costs in proportion to its size, outliers lose their leverage over the response, and, as Module 4 showed, the problem becomes a linear program. LAD regression dates back to Boscovich in 1757 and Laplace, earlier than least squares itself, and it is the simplest robust regression method used in machine learning practice today.

19.2 The Model

Definition LAD Regression

Given data $(\mathbf{x}_i, y_i)$ for $i = 1, \dots, N$, with feature vectors $\mathbf{x}_i \in \mathbb{R}^p$ (including a constant 1 for the intercept) and responses $y_i \in \mathbb{R}$, LAD regression solves

$$\min_{\mathbf{w}\in\mathbb{R}^p} \sum_{i=1}^{N} \left|y_i - \mathbf{w}^\top \mathbf{x}_i\right| = \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_1 , \tag{19.1}$$

where

$\mathbf{w} \in \mathbb{R}^p$ is the weight vector (its first entry is the intercept when $\mathbf{x}_i$ begins with 1);

$\mathbf{X} \in \mathbb{R}^{N\times p}$ is the data matrix whose ith row is $\mathbf{x}_i^\top$;

$\mathbf{y} \in \mathbb{R}^N$ is the vector of responses;

$r_i = y_i - \mathbf{w}^\top \mathbf{x}_i$ is the ith residual;

$\|\cdot\|_1$ is the ℓ_1 norm, the sum of absolute values.

Derivation Two linear programming formulations

Step 1. Bounding form (Module 4). Introduce $t_i \geq |r_i|$ via $t_i \geq y_i - \mathbf{w}^\top \mathbf{x}_i$ and $t_i \geq -(y_i - \mathbf{w}^\top \mathbf{x}_i)$:

$$\min_{\mathbf{w},\mathbf{t}} \ \mathbf{1}^\top \mathbf{t} \quad \text{subject to} \quad -\mathbf{t} \leq \mathbf{y} - \mathbf{X}\mathbf{w} \leq \mathbf{t}. \tag{19.2}$$

Step 2. Splitting form. Write each residual as a difference of nonnegative parts, $y_i - \mathbf{w}^\top \mathbf{x}_i = u_i - v_i$ with $u_i, v_i \geq 0$:

$$\min_{\mathbf{w},\mathbf{u},\mathbf{v}} \ \mathbf{1}^\top \mathbf{u} + \mathbf{1}^\top \mathbf{v} \quad \text{subject to} \quad \mathbf{X}\mathbf{w} + \mathbf{u} - \mathbf{v} = \mathbf{y}, \quad \mathbf{u}, \mathbf{v} \geq \mathbf{0}, \quad \mathbf{w} \text{ free}. \tag{19.3}$$

Step 3. Why the forms agree. In (19.3), if both $u_i > 0$ and $v_i > 0$, reducing each by $\min(u_i, v_i)$ keeps the equation and lowers the cost. So at an optimum $u_i v_i = 0$ and $u_i + v_i = |r_i|$. In the simplex method this happens automatically, because the columns of u_i and v_i are negatives of each other and cannot both be basic.

where

u_i is the positive part of residual i (point above the fit);

v_i is the negative part (point below the fit).

19.3 The Dual of LAD Regression

Derivation Deriving the dual

Start from (19.3), a minimisation with N equality constraints.

Step 1. Attach a free dual variable d_i to the ith equation. The dual objective is $\mathbf{y}^\top \mathbf{d}$, to be maximised.

Step 2. Columns of the free variables $\mathbf{w}$: their dual constraints are equations, $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$.

Step 3. Column of u_i (the unit vector $\mathbf{e}_i$, cost 1, $u_i \geq 0$): $d_i \leq 1$.

Step 4. Column of v_i ($-\mathbf{e}_i$, cost 1, $v_i \geq 0$): $-d_i \leq 1$, that is $d_i \geq -1$.

Step 5. The dual is therefore

$$\max_{\mathbf{d}} \ \mathbf{y}^\top \mathbf{d} \quad \text{subject to} \quad \mathbf{X}^\top \mathbf{d} = \mathbf{0}, \quad -1 \leq d_i \leq 1 \ \ (i = 1, \ldots, N). \tag{19.4}$$

Key Result Complementary slackness for LAD

At optimal $(\mathbf{w}^\star, \mathbf{d}^\star)$:

$$r_i > 0 \Rightarrow d_i^\star = +1, \qquad r_i < 0 \Rightarrow d_i^\star = -1, \qquad r_i = 0 \Rightarrow d_i^\star \in [-1, 1]. \tag{19.5}$$

So $d_i^\star$ is the sign of the residual, or a "fractional sign" for points exactly on the fit. The condition $\mathbf{X}^\top \mathbf{d}^\star = \mathbf{0}$ is the LAD analogue of the OLS normal equations $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$: *the signs of the residuals, not the residuals themselves, must balance against every feature*. With an intercept, the first component of $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$ says that roughly as many points lie above the fit as below it.

19.4 Interpolation and Robustness

Derivation Some optimal LAD fit interpolates p points

Step 1. Formulation (19.3) has N equality constraints, so a basic feasible solution has at most N basic variables (Module 7).

Step 2. If $\mathbf{X}$ has rank p, the free weights can be taken basic, using p of the N basic positions.

Step 3. That leaves at most $N - p$ basic positions for the $2N$ variables u_i, v_i. Hence at least p indices i have both $u_i = v_i = 0$, that is $r_i = 0$.

Step 4. By the Fundamental Theorem (Module 7), an optimal basic solution exists. Therefore there is an optimal LAD fit passing exactly through at least p data points.

For a straight line ($p = 2$) this means: *some best LAD line passes through two of the data points*, which gives a simple hand method: check the lines through every pair of points.

Key Result Robustness to response outliers

If a data point lies strictly above the optimal LAD fit and its response y_i is increased by any amount, the same $\mathbf{w}^\star$ remains optimal. Proof: the primal stays feasible with the same weights and a larger u_i, and the dual solution $\mathbf{d}^\star$ stays feasible (the constraints (19.4) do not involve $\mathbf{y}$) and still satisfies complementary slackness because the sign of r_i is unchanged. By contrast, the OLS weights move linearly with y_i and can be dragged arbitrarily far by a single point.

Common Pitfall

LAD is robust to outliers in the *response*, not to outliers in the *features*. A point with an extreme $\mathbf{x}_i$ (a high leverage point) can still pull the LAD fit, because the fit must balance residual signs weighted by feature values in $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$. For protection against both kinds, statisticians use methods such as least trimmed squares, which are much harder to compute.

Machine Learning Connection

LAD regression is the maximum likelihood estimator when noise follows a Laplace distribution, whose heavy tails model occasional large errors. In machine learning practice the absolute loss is called the **L1 loss** or mean absolute error (MAE) loss, used for depth estimation, bounding box regression and forecasting, where it is far less sensitive to annotation errors than the squared loss. The Huber loss and the "smooth L1" loss of object detectors such as Fast R CNN blend the two: quadratic near zero, linear (LAD like) in the tails. When solved exactly as an LP, LAD fits also provide dual certificates of optimality, something gradient methods do not.

19.5 Worked Examples

Worked Example 19.1 A LAD Line through Data with an Outlier

Fit a line $\hat{y} = w_0 + w_1 x$ by LAD to the data $x = (1, 2, 3, 4, 5)$, $y = (2,\ 4,\ 6,\ 9,\ 20)$, where the last response is suspicious. Compare with OLS.

Solution

Step 1. **Method.** By the interpolation result, it suffices to check the $\binom{5}{2} = 10$ lines through pairs of points and keep the one with the smallest total absolute residual.

Step 2. **Line through points 2 and 4**, $(2, 4)$ and $(4, 9)$: slope $w_1 = (9 - 4)/(4 - 2) = 2.5$, intercept $w_0 = 4 - 2.5(2) = -1$. Predictions $1.5,\ 4,\ 6.5,\ 9,\ 11.5$; residuals $0.5,\ 0,\ -0.5,\ 0,\ 8.5$; total 9.5.

Step 3. **Line through points 1 and 4**: slope $7/3$, intercept $-\frac{1}{3}$; residuals $0,\ -0.333,\ -0.667,\ 0,\ 8.667$; total 9.667.

Step 4. **Line through points 1 and 2**: slope 2, intercept 0; residuals $0, 0, 0, 1, 10$; total 11.

Step 5. **The remaining seven pairs** all give totals larger than 9.5 (for instance points 3 and 5 give a slope of 7 and total 19). The best is the line through points 2 and 4.

Step 6. **OLS for comparison.** With $\bar{x} = 3$, $\bar{y} = 8.2$: $\sum(x_i - \bar{x})(y_i - \bar{y}) = (-2)(-6.2) + (-1)(-4.2) + 0 + (1)(0.8) + (2)(11.8) = 41$ and $\sum(x_i - \bar{x})^2 = 10$, so $w_1 = 4.1$ and $w_0 = 8.2 - 12.3 = -4.1$. Residuals $2,\ -0.1,\ -2.2,\ -3.3,\ 3.6$.

Step 7. **Comparison.** OLS has slope 4.1 because the outlier pulls it up; it misfits every one of the first four points by up to 3.3. The LAD slope 2.5 follows the bulk of the data.

Answer. LAD: $\boxed{\hat{y} = -1 + 2.5x}$ with total absolute error 9.5; OLS: $\hat{y} = -4.1 + 4.1x$ with total absolute error 11.2.

Worked Example 19.2 A Dual Certificate for the LAD Line

Prove that $\hat{y} = -1 + 2.5x$ is optimal for the data of Worked Example 19.1 by constructing a dual solution of (19.4).

Solution

Step 1. **Signs from (19.5).** Residuals $0.5 > 0$, 0, $-0.5 < 0$, 0, $8.5 > 0$ give $d_1 = +1$, $d_3 = -1$, $d_5 = +1$; d_2 and d_4 must lie in $[-1, 1]$.

Step 2. **Balance conditions $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$**, with rows of $\mathbf{X}$ equal to $(1, x_i)$:

$$\text{intercept: } d_1 + d_2 + d_3 + d_4 + d_5 = 0, \qquad \text{slope: } 1d_1 + 2d_2 + 3d_3 + 4d_4 + 5d_5 = 0.$$

Step 3. **Substitute the known signs.** Intercept: $1 + d_2 - 1 + d_4 + 1 = 0$, so $d_2 + d_4 = -1$. Slope: $1 + 2d_2 - 3 + 4d_4 + 5 = 0$, so $2d_2 + 4d_4 = -3$.

Step 4. **Solve.** From the first, $d_2 = -1 - d_4$; substitute: $-2 - 2d_4 + 4d_4 = -3$, so $d_4 = -0.5$ and $d_2 = -0.5$.

Step 5. **Feasibility.** Both lie in $[-1, 1]$. So $\mathbf{d} = (1, -0.5, -1, -0.5, 1)$ is dual feasible.

Step 6. **Equal objectives.** Dual: $\mathbf{y}^\top \mathbf{d} = 2 - 2 - 6 - 4.5 + 20 = 9.5$, equal to the primal total. By weak duality both are optimal.

Answer. Certificate $\boxed{\mathbf{d} = (1, -0.5, -1, -0.5, 1)}$ with $\mathbf{y}^\top \mathbf{d} = 9.5$.

Worked Example 19.3 Robustness When the Outlier Grows

Change the last response in Worked Example 19.1 from 20 to 100. What happens to the LAD and OLS fits? What if it changes to 10 instead?

Solution

Step 1. **LAD with $y_5 = 100$.** Point 5 stays above the line $-1 + 2.5x$ (prediction 11.5), so its residual sign is unchanged. The dual certificate of Worked Example 19.2 is still feasible (its constraints do not involve $\mathbf{y}$) and still matches signs. The new primal total is $0.5 + 0.5 + 88.5 = 89.5$ and the dual value is $2 - 2 - 6 - 4.5 + 100 = 89.5$. Equal, so the same line remains optimal.

Step 2. **OLS with $y_5 = 100$.** Now $\bar{y} = 24.2$ and $\sum (x_i - \bar{x})(y_i - \bar{y})$ increases by $2 \times 80 = 160$ to 201, so $w_1 = 20.1$ and $w_0 = 24.2 - 60.3 = -36.1$. The fit is ruined: it predicts -16 at $x = 1$.

Step 3. **LAD with $y_5 = 10$.** Now point 5 lies *below* the old line ($10 < 11.5$), so the residual sign flips and the old certificate fails. Rechecking pairs, the line through points 1 and 2 ($\hat{y} = 2x$) has residuals $0, 0, 0, 1, 0$, total 1; it passes through four of the five points. The data now contain no outlier, and LAD and OLS agree closely (OLS gives $-0.1 + 2.1x$).

Answer. With $y_5 = 100$ the LAD fit is $\boxed{\text{unchanged}}$ while OLS jumps to slope 20.1; with $y_5 = 10$ LAD becomes $\hat{y} = 2x$.

Worked Example 19.4 The Median as LAD with an Intercept Only

Find all LAD fits $\hat{y} = w_0$ (intercept only) for the data $y = (3, 8, 1, 10)$, and explain the result using the dual.

Solution

Step 1. Objective. $f(w_0) = |3 - w_0| + |8 - w_0| + |1 - w_0| + |10 - w_0|$.

Step 2. Sorted data $1,\ 3,\ 8,\ 10$. For w_0 between 3 and 8, the two smaller points are below and the two larger above, so $f = (w_0 - 1) + (w_0 - 3) + (8 - w_0) + (10 - w_0) = 14$, constant.

Step 3. Outside that interval. For $w_0 < 3$, at least three points lie above and only one below, and moving w_0 up lowers f (slope -2). For $w_0 > 8$ the slope is $+2$.

Step 4. Set of optima. Every $w_0 \in [3, 8]$ is optimal with $f = 14$. The LP has multiple optima (Module 10); its basic optimal solutions are the two endpoints 3 and 8, each passing through $p = 1$ data point.

Step 5. Dual. With $\mathbf{X} = \mathbf{1}$, the dual is max $\mathbf{y}^\top\mathbf{d}$ subject to $\sum_i d_i = 0$, $-1 \le d_i \le 1$. Take $d = -1$ for $y = 1, 3$ and $d = +1$ for $y = 8, 10$: $\sum d_i = 0$ and $\mathbf{y}^\top\mathbf{d} = -1 - 3 + 8 + 10 = 14$. Optimal.

Answer. Every $\boxed{w_0 \in [3, 8]}$ is optimal with total deviation 14: the median of an even sample is an interval.

Worked Example 19.5 LAD with Two Features

Fit $\hat{y} = w_0 + w_1x_1 + w_2x_2$ by LAD to the six observations

i	1	2	3	4	5	6
x_1	0	1	2	1	3	2
x_2	1	0	1	2	2	3
y	3	3	6	5	8	9

and certify the answer $\mathbf{w} = (1.5,\ 1.5,\ 1.5)$ with the dual.

Solution

Step 1. Predictions $1.5(1 + x_1 + x_2)$: $3,\ 3,\ 6,\ 6,\ 9,\ 9$.

Step 2. Residuals $y - \hat{y}$: $0,\ 0,\ 0,\ -1,\ -1,\ 0$. Total absolute error 2. Four points lie on the fitted plane, at least $p = 3$ as the theory guarantees.

Step 3. Dual signs. Points 4 and 5 lie below: $d_4 = d_5 = -1$. The others are free in $[-1, 1]$.

Step 4. Balance $\mathbf{X}^\top\mathbf{d} = \mathbf{0}$:

$$\begin{aligned} d_1 + d_2 + d_3 + d_4 + d_5 + d_6 &= 0, \\ 0d_1 + 1d_2 + 2d_3 + 1d_4 + 3d_5 + 2d_6 &= 0, \\ 1d_1 + 0d_2 + 1d_3 + 2d_4 + 2d_5 + 3d_6 &= 0. \end{aligned}$$

Step 5. Try $d_1 = d_2 = 0$, $d_3 = d_6 = 1$: first equation $0+0+1-1-1+1 = 0$; second $0+2-1-3+2 = 0$; third $0 + 1 - 2 - 2 + 3 = 0$. All hold, and every $d_i \in [-1, 1]$.

Step 6. Dual value. $\mathbf{y}^\top\mathbf{d} = 6 - 5 - 8 + 9 = 2$, equal to the primal total. Optimal.

Answer. $\boxed{\hat{y} = 1.5 + 1.5x_1 + 1.5x_2}$ with total error 2, certified by $\mathbf{d} = (0, 0, 1, -1, -1, 1)$.

Worked Example 19.6 LAD with Prior Knowledge

For the data of Worked Example 19.1, domain experts insist that the slope cannot exceed 2. Fit the constrained LAD line.

Solution

Step 1. **Model.** Add $w_1 \le 2$ to (19.3). The problem remains a linear program, an advantage of the LP view: any linear prior knowledge can be imposed exactly.

Step 2. **Is the constraint active?** The unconstrained slope 2.5 violates it, so at the optimum $w_1 = 2$.

Step 3. **Reduce to a median.** With $w_1 = 2$ fixed, the residuals are $y_i - 2x_i - w_0 = z_i - w_0$, where $z = (0,\ 0,\ 0,\ 1,\ 10)$. Minimising $\sum |z_i - w_0|$ over w_0 gives the median of z, which is 0.

Step 4. **Fit.** $\hat{y} = 2x$, residuals $0, 0, 0, 1, 10$, total 11.

Step 5. **Price of the prior.** The constrained optimum 11 exceeds the unconstrained 9.5 by 1.5; the dual variable of $w_1 \le 2$ measures the marginal cost of tightening the bound.

Answer. $\boxed{\hat{y} = 2x}$ with total absolute error 11.

Module Summary

- LAD regression minimises $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_1$ and is an LP: $\min\ \mathbf{1}^\top(\mathbf{u} + \mathbf{v})$ subject to $\mathbf{X}\mathbf{w} + \mathbf{u} - \mathbf{v} = \mathbf{y}$, $\mathbf{u}, \mathbf{v} \ge \mathbf{0}$.
- Its dual is $\max\ \mathbf{y}^\top\mathbf{d}$ subject to $\mathbf{X}^\top\mathbf{d} = \mathbf{0}$, $-1 \le d_i \le 1$; d_i is the sign of residual i.
- Some optimal fit passes through at least p points (a basic solution); for lines, check pairs of points.
- Moving a response outlier further away on the same side does not change the LAD fit; OLS can be dragged arbitrarily.
- With an intercept only, LAD gives the median, which may be an interval.
- Linear prior knowledge (bounds, signs, monotonicity) can be added without leaving the LP framework.

MODULE 20

Chebyshev Regression and Quantile Regression

Learning Objectives

After studying this module you will be able to:

- formulate Chebyshev (minimax, or ℓ_∞) regression as a linear program and explain when the worst case error is the right criterion;
- describe the equioscillation property of Chebyshev fits and use it to compute small fits by hand;
- define the pinball (check) loss and derive why minimising it produces a quantile;
- formulate quantile regression as a linear program, write its dual and interpret the dual bounds $[\tau - 1, \tau]$;
- build prediction intervals from a pair of quantile regressions and assess their coverage.

20.1 Beyond the Average Error

Least squares controls the average squared error and LAD the average absolute error. Two other questions are just as important in practice. First, *what is the worst error?* A model that controls a robot arm, sets a safety margin or approximates a function inside a chip must keep every error small, not just the average. That leads to **Chebyshev regression**. Second, *what is the range of likely outcomes?* A demand forecast that predicts only the mean cannot say how much stock to hold to avoid shortages nine days out of ten. That leads to **quantile regression**. Both are linear programs, built with the epigraph and splitting tricks of Module 4.

20.2 Chebyshev Regression

Definition Chebyshev Regression

Chebyshev regression chooses the weights that minimise the largest absolute residual:

$$\min_{\mathbf{w}\in\mathbb{R}^p} \max_{i=1,\dots,N} \left|y_i - \mathbf{w}^\top \mathbf{x}_i\right| = \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_\infty , \tag{20.1}$$

where

$\|\mathbf{r}\|_\infty = \max_i |r_i|$ is the maximum (Chebyshev) norm;

$\mathbf{X}, \mathbf{y}, \mathbf{w}$ are as in Module 19.

Derivation The Chebyshev LP and its dual

Step 1. Epigraph variable. Let t bound every absolute residual. By Module 4, $t \geq |r_i|$ is the pair $t \geq y_i - \mathbf{w}^\top \mathbf{x}_i$ and $t \geq -(y_i - \mathbf{w}^\top \mathbf{x}_i)$.

Step 2. Primal LP.

$$\min_{\mathbf{w},\,t} \ t \quad \text{subject to} \quad -t \leq y_i - \mathbf{w}^\top \mathbf{x}_i \leq t, \qquad i = 1, \ldots, N. \tag{20.2}$$

Step 3. Dual. Attach $\alpha_i \geq 0$ to $\mathbf{w}^\top \mathbf{x}_i + t \geq y_i$ and $\beta_i \geq 0$ to $-\mathbf{w}^\top \mathbf{x}_i + t \geq -y_i$. The free variables $\mathbf{w}$ and t give equations: $\sum_i (\alpha_i - \beta_i)\mathbf{x}_i = \mathbf{0}$ and $\sum_i (\alpha_i + \beta_i) = 1$. With $d_i = \alpha_i - \beta_i$ the dual becomes

$$\max_{\mathbf{d}} \ \mathbf{y}^\top \mathbf{d} \quad \text{subject to} \quad \mathbf{X}^\top \mathbf{d} = \mathbf{0}, \quad \sum_{i=1}^{N} |d_i| \leq 1. \tag{20.3}$$

Compare with the LAD dual (19.4): the box $|d_i| \leq 1$ has become the ℓ_1 ball $\sum |d_i| \leq 1$. This is a general pattern: the dual of minimising an ℓ_1 norm involves an ℓ_∞ constraint, and vice versa.

Key Result Equioscillation

For fitting a polynomial of degree $p - 1$ (including a straight line, $p = 2$) to points with distinct x values, an optimal Chebyshev fit has at least $p + 1$ points at which the residual equals $\pm t^\star$, with *alternating* signs in order of x. In LP terms: an optimal basic solution has at least $p + 1$ active constraints, because $\mathbf{w}$ and t together have $p + 1$ components; complementary slackness makes the corresponding d_i nonzero with the sign of the residual.

Lucid Insight Why alternation means optimality

Suppose the residuals at the extreme points alternate $+, -, +$. Any change in a straight line that lowers the first and third (which are positive, above the line) must raise the line at both ends, which raises the middle prediction too and makes the middle residual (negative, below the line) larger in size. No line can shrink all three at once. That is the whole proof idea, and it is exactly what the dual solution encodes.

20.3 Quantile Regression

Definition Pinball Loss

For a level $\tau \in (0, 1)$, the **pinball** (or check) loss of a residual r is

$$\rho_\tau(r) = \begin{cases} \tau\, r, & r \geq 0, \\ (\tau - 1)\, r, & r < 0, \end{cases} \qquad \text{equivalently} \qquad \rho_\tau(r) = \max\left(\tau r,\ (\tau - 1) r\right). \tag{20.4}$$

where

τ is the target quantile level (for example 0.9 for the 90th percentile);

$r = y - \hat{y}$ is the residual: positive when the prediction is too low.

Under predicting by one unit costs τ; over predicting by one unit costs $1 - \tau$. For $\tau = 0.5$, $\rho_\tau(r) = \frac{1}{2}|r|$ and quantile regression reduces to LAD.

Derivation Minimising the pinball loss gives a sample quantile

Consider $f(q) = \sum_{i=1}^{N} \rho_\tau(y_i - q)$ for a single number q.

Step 1. Let $N_>(q)$, $N_<(q)$ and $N_=(q)$ count the data above, below and equal to q.

Step 2. Moving q up by a small amount $\varepsilon > 0$ decreases each residual of a point above q, saving $\tau\varepsilon$ each, and increases the overshoot of each point below or equal to q, costing $(1 - \tau)\varepsilon$ each. The right slope is $-\tau N_> + (1 - \tau)(N_< + N_=)$.

Step 3. Similarly the left slope (moving down) is $-\tau(N_> + N_=) + (1 - \tau)N_<$.

Step 4. q is optimal when the right slope is ≥ 0 and the left slope is ≤ 0. Using $N_> + N_< + N_= = N$, these become

$$N_<(q) \;\leq\; \tau N \;\leq\; N_<(q) + N_=(q). \tag{20.5}$$

Step 5. In words: at most a fraction τ of the data lie strictly below q, and at least a fraction τ lie at or below it. That is the definition of a sample τ quantile.

Derivation Quantile regression as a linear program

Step 1. Replace the constant q by a linear predictor $\mathbf{w}^\top \mathbf{x}_i$ and split residuals as in LAD: $y_i - \mathbf{w}^\top \mathbf{x}_i = u_i - v_i$, $u_i, v_i \geq 0$.

Step 2. The pinball loss of residual i is $\tau u_i + (1 - \tau)v_i$ when $u_i v_i = 0$, which holds at an optimum.

Step 3. The LP is

$$\min_{\mathbf{w},\mathbf{u},\mathbf{v}} \; \tau\,\mathbf{1}^\top \mathbf{u} + (1 - \tau)\,\mathbf{1}^\top \mathbf{v} \quad \text{subject to} \quad \mathbf{X}\mathbf{w} + \mathbf{u} - \mathbf{v} = \mathbf{y}, \quad \mathbf{u}, \mathbf{v} \geq \mathbf{0}. \tag{20.6}$$

Step 4. **Dual** (exactly as in Module 19, with costs τ and $1 - \tau$):

$$\max_{\mathbf{d}} \; \mathbf{y}^\top \mathbf{d} \quad \text{subject to} \quad \mathbf{X}^\top \mathbf{d} = \mathbf{0}, \quad \tau - 1 \leq d_i \leq \tau. \tag{20.7}$$

Step 5. **Complementary slackness:** $d_i = \tau$ for points above the fit, $d_i = \tau - 1$ for points below, and $d_i \in [\tau - 1, \tau]$ for points on the fit.

With an intercept, the first equation of $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$ gives $\tau N_> - (1 - \tau)N_< +$(terms for points on the fit) $= 0$, the regression analogue of (20.5): roughly a fraction τ of the points lie below the fitted hyperplane.

Common Pitfall

Quantile lines fitted separately for different τ can occasionally cross, especially far from the data, which would claim that a 90th percentile lies below a 10th percentile. Remedies include adding the non crossing constraints $\mathbf{w}_{0.9}^\top \mathbf{x} \geq \mathbf{w}_{0.1}^\top \mathbf{x}$ at chosen points, which keeps the joint problem a linear program.

Machine Learning Connection

The pinball loss is the standard training loss for probabilistic forecasting: deep forecasting models such as MQ RNN and the Temporal Fusion Transformer output several quantiles at once and are trained on the sum of their pinball losses. Quantile regression underlies *conformalised quantile regression*, a popular way to produce prediction intervals with guaranteed coverage, and gradient boosting libraries (LightGBM, XGBoost) offer a "quantile" objective built on the same loss. Chebyshev fitting appears in minimax approximation of activation functions for hardware, in worst case calibration, and in robust control; its dual (20.3) reappears in the analysis of adversarial robustness, where an ℓ_∞ bounded perturbation is paired with an ℓ_1 dual norm.

20.4 Worked Examples

Worked Example 20.1 A Chebyshev Line through Three Points

Find the line $\hat{y} = w_0 + w_1 x$ that minimises the maximum absolute error for the points $(0,1)$, $(1,3)$, $(2,4)$.

Solution

Step 1. **Equioscillation guess.** With $p = 2$ we expect all three points to have error $\pm t$ with alternating signs: $r_1 = -t$, $r_2 = +t$, $r_3 = -t$ (the middle point lies above the line, because the data bend downwards).

Step 2. **Equations.**

$$1 - w_0 = -t, \qquad 3 - w_0 - w_1 = t, \qquad 4 - w_0 - 2w_1 = -t.$$

Step 3. **Solve.** Subtract the first from the third: $3 - 2w_1 = 0$, so $w_1 = 1.5$. Add the first and second: $4 - 2w_0 - w_1 = 0$, so $2w_0 = 4 - 1.5 = 2.5$ and $w_0 = 1.25$. Then $t = w_0 - 1 = 0.25$.

Step 4. **Check.** Predictions $1.25,\ 2.75,\ 4.25$; residuals $-0.25,\ +0.25,\ -0.25$. Alternating, all of size 0.25.

Step 5. **Dual certificate.** Choose $\mathbf{d} = (-\frac{1}{4}, \frac{1}{2}, -\frac{1}{4})$: $\sum d_i = 0$, $\sum x_i d_i = 0 + \frac{1}{2} - \frac{1}{2} = 0$, $\sum |d_i| = 1$, and $\mathbf{y}^\top \mathbf{d} = -\frac{1}{4} + \frac{3}{2} - 1 = \frac{1}{4} = t$. Strong duality holds, so the line is optimal.

Answer. $\boxed{\hat{y} = 1.25 + 1.5x,\ t^\star = 0.25}$.

Worked Example 20.2 A Chebyshev Fit to Four Points

Fit a Chebyshev line to $(0,1)$, $(1,2)$, $(2,2)$, $(3,5)$.

Solution

Step 1. Choose three points for equioscillation. The fourth point $(3, 5)$ is high, the third $(2, 2)$ low. Try the points at $x = 0, 2, 3$ with signs $+, -, +$ (residual $= y - \hat{y}$).

Step 2. Equations.

$$1 - w_0 = t, \qquad 2 - w_0 - 2w_1 = -t, \qquad 5 - w_0 - 3w_1 = t.$$

Step 3. Solve. Subtract the first from the third: $4 - 3w_1 = 0$, so $w_1 = \frac{4}{3}$. Add the first and second: $3 - 2w_0 - 2w_1 = 0$, so $w_0 = \frac{3-8/3}{2} = \frac{1}{6}$. Then $t = 1 - \frac{1}{6} = \frac{5}{6}$.

Step 4. Check the remaining point $(1, 2)$: prediction $\frac{1}{6} + \frac{4}{3} = 1.5$, residual $0.5 \le \frac{5}{6}$. Feasible.

Step 5. Dual certificate. Put weights on points 1, 3, 4 with signs $+, -, +$: $\mathbf{d} = (a,\ 0,\ -b,\ c)$ with $a - b + c = 0$, $-2b + 3c = 0$, $a + b + c = 1$. From the second, $b = 1.5c$; then $a = 0.5c$; then $3c = 1$, so $c = \frac{1}{3}$, $b = \frac{1}{2}$, $a = \frac{1}{6}$. All have the required signs. $\mathbf{y}^\top\mathbf{d} = \frac{1}{6} - 1 + \frac{5}{3} = \frac{5}{6} = t$.

Answer. $\boxed{\hat{y} = \frac{1}{6} + \frac{4}{3}x,\ t^\star = \frac{5}{6} \approx 0.833}$, with extreme errors at $x = 0, 2, 3$.

Worked Example 20.3 Sample Quantiles from the Pinball Loss

For the ten values $y = (4, 7, 1, 9, 3, 12, 6, 5, 10, 8)$, find all minimisers of $\sum_i \rho_\tau(y_i - q)$ for $\tau = 0.25$, 0.5 and 0.9.

Solution

Step 1. Sort. $1, 3, 4, 5, 6, 7, 8, 9, 10, 12$; $N = 10$.

Step 2. $\tau = 0.25$**:** $\tau N = 2.5$. Condition (20.5) needs $N_<(q) \le 2.5 \le N_<(q) + N_=(q)$. At $q = 4$: $N_< = 2$, $N_= = 1$, so $2 \le 2.5 \le 3$. Holds. At any other value, it fails (for $3 < q < 4$, $N_< = 2$ and $N_= = 0$). Unique answer $q = 4$.

Step 3. $\tau = 0.5$**:** $\tau N = 5$. Any $q \in [6, 7]$ works: at $q = 6$, $N_< = 4$, $N_= = 1$ $(4 \le 5 \le 5)$; for $6 < q < 7$, $N_< = 5$, $N_= = 0$ $(5 \le 5 \le 5)$; at $q = 7$, $N_< = 5$. The median is the interval $[6, 7]$.

Step 4. $\tau = 0.9$**:** $\tau N = 9$. Any $q \in [10, 12]$ works, by the same counting.

Step 5. Objective at the 0.9 quantile $q = 10$: only 12 lies above, costing $0.9 \times 2 = 1.8$; the other nine lie at or below, costing $0.1 \times ((10-1) + (10-3) + (10-4) + (10-5) + (10-6) + (10-7) + (10-8) + (10-9) + 0) = 0.1 \times 37 = 3.7$. Total 5.5.

Answer. $\boxed{q_{0.25} = 4,\ q_{0.5} \in [6, 7],\ q_{0.9} \in [10, 12]}$ (minimum pinball loss 5.5 at $\tau = 0.9$).

Worked Example 20.4 A Lower Quartile Regression Line

For the eight points $x = (1, \dots, 8)$, $y = (2, 5, 3, 7, 6, 10, 8, 13)$, verify that $\hat{y} = 1 + x$ is a $\tau = 0.25$ quantile regression line, and compute its pinball loss.

Solution

Step 1. Predictions $2, 3, 4, 5, 6, 7, 8, 9$; residuals $y - \hat{y} = 0,\ 2,\ -1,\ 2,\ 0,\ 3,\ 0,\ 4$.

Step 2. Counts. Below: one point ($x = 3$). On the line: three points ($x = 1, 5, 7$). Above: four points. Check (20.5) for the line: $N_< = 1 \le \tau N = 2 \le N_< + N_= = 4$. Consistent with a 25% quantile.

Step 3. Pinball loss. Points above cost τr: $0.25(2 + 2 + 3 + 4) = 2.75$. The point below costs $(1 - \tau)\,|r| = 0.75 \times 1 = 0.75$. Total 3.5.

Step 4. Basic solution. The line passes through three points, more than the $p = 2$ required by the vertex property of Module 19, so the solution is degenerate. Optimality is confirmed by the dual in the next example.

Answer. $\boxed{\hat{y}_{0.25} = 1 + x}$ with pinball loss 3.5.

Worked Example 20.5 A Dual Certificate for Quantile Regression

Certify the line of Worked Example 20.4 by exhibiting $\mathbf{d}$ satisfying (20.7) with $\mathbf{y}^\top\mathbf{d} = 3.5$.

Solution

Step 1. Fixed values from complementary slackness. Points above ($x = 2, 4, 6, 8$): $d = \tau = \frac{1}{4}$. Point below ($x = 3$): $d = \tau - 1 = -\frac{3}{4}$. Points on the line ($x = 1, 5, 7$): $d_1, d_5, d_7 \in [-\frac{3}{4}, \frac{1}{4}]$.

Step 2. Balance equations $\sum d_i = 0$ and $\sum x_i d_i = 0$. Known contributions: sum $= 4(\frac{1}{4}) - \frac{3}{4} = \frac{1}{4}$; weighted sum $= \frac{1}{4}(2 + 4 + 6 + 8) - \frac{3}{4}(3) = 5 - 2.25 = 2.75$. So

$$d_1 + d_5 + d_7 = -\tfrac{1}{4}, \qquad d_1 + 5d_5 + 7d_7 = -2.75.$$

Step 3. Try $d_5 = 0$. Then $d_1 + d_7 = -\frac{1}{4}$ and $d_1 + 7d_7 = -2.75$. Subtracting, $6d_7 = -2.5$, so $d_7 = -\frac{5}{12} \approx -0.417$, and $d_1 = -\frac{1}{4} + \frac{5}{12} = \frac{1}{6} \approx 0.167$. Both lie in $[-0.75,\ 0.25]$.

Step 4. Dual value. $\mathbf{y}^\top\mathbf{d} = 2(\frac{1}{6}) + 5(\frac{1}{4}) + 3(-\frac{3}{4}) + 7(\frac{1}{4}) + 6(0) + 10(\frac{1}{4}) + 8(-\frac{5}{12}) + 13(\frac{1}{4})$. The quarter terms give $\frac{1}{4}(5 + 7 + 10 + 13) = 8.75$; the rest give $\frac{1}{3} - 2.25 - \frac{10}{3} = -5.25$. Total 3.5.

Step 5. Conclusion. Dual feasible with value equal to the primal loss 3.5: the line is optimal.

Answer. $\boxed{\mathbf{d} = (\frac{1}{6},\ \frac{1}{4},\ -\frac{3}{4},\ \frac{1}{4},\ 0,\ \frac{1}{4},\ -\frac{5}{12},\ \frac{1}{4})}$ certifies optimality.

Worked Example 20.6 A Prediction Band from Two Quantile Lines

For the data of Worked Example 20.4, the $\tau = 0.75$ quantile regression line is $\hat{y}_{0.75} = 1 + 1.5x$. Combine it with $\hat{y}_{0.25} = 1 + x$ to form a band, report its coverage on the eight training points, predict the band at $x = 10$, and comment.

Solution

Step 1. **Check the upper line.** Residuals $y-(1+1.5x)$: $-0.5,\ 1,\ -2.5,\ 0,\ -2.5,\ 0,\ -3.5,\ 0$. One point above, three on, four below: $N_< = 4 \le \tau N = 6 \le N_< + N_= = 7$. Consistent with a 75% quantile.

Step 2. **Band membership** (lower $\le y \le$ upper):

x	y	$1+x$	$1+1.5x$	inside?
1	2	2	2.5	yes
2	5	3	4	no (above)
3	3	4	5.5	no (below)
4	7	5	7	yes
5	6	6	8.5	yes
6	10	7	10	yes
7	8	8	11.5	yes
8	13	9	13	yes

Step 3. **Coverage.** 6 of 8 points, or 75%, against a nominal $0.75 - 0.25 = 50\%$. The excess comes from the six points lying exactly *on* a boundary, a consequence of the vertex property. On new data from the same source, coverage would typically be nearer the nominal level; with very small samples, in sample coverage is unreliable, which is why calibration methods such as conformal prediction use a separate calibration set.

Step 4. **Prediction at $x = 10$.** Lower 11, upper 16: the band $[11, 16]$, which widens with x because the data spread out as x grows.

Answer. Band $\boxed{[1+x,\ 1+1.5x]}$; training coverage $6/8$; at $x = 10$ the band is $[11, 16]$.

Module Summary

- Chebyshev regression: $\min t$ subject to $-t \le y_i - \mathbf{w}^\top \mathbf{x}_i \le t$; dual $\max\ \mathbf{y}^\top \mathbf{d}$ subject to $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$, $\|\mathbf{d}\|_1 \le 1$.
- Optimal Chebyshev fits equioscillate: at least $p+1$ points attain the maximum error with alternating signs.
- Pinball loss $\rho_\tau(r) = \max(\tau r, (\tau-1)r)$; its minimiser over constants is a sample τ quantile: $N_< \le \tau N \le N_< + N_=$.
- Quantile regression LP: $\min\ \tau \mathbf{1}^\top \mathbf{u} + (1-\tau)\mathbf{1}^\top \mathbf{v}$ subject to $\mathbf{X}\mathbf{w} + \mathbf{u} - \mathbf{v} = \mathbf{y}$; dual bounds $\tau - 1 \le d_i \le \tau$.
- Pairs of quantile fits give prediction bands; in sample coverage is inflated by points on the fitted lines.

Module 21

Linear Classification and the ℓ_1 Support Vector Machine

Learning Objectives

After studying this module you will be able to:

- decide whether two classes of points are linearly separable by solving a feasibility LP;
- prove infeasibility of separation with a Farkas certificate, as in the XOR problem;
- formulate the hard and soft margin ℓ_1 norm support vector machines as linear programs;
- derive the dual of the ℓ_1 SVM and identify support vectors through complementary slackness;
- explain the geometric meaning of minimising $||\mathbf{w}||_1$ (an ℓ_∞ margin) and why it produces sparse classifiers;
- compute hinge losses and study the effect of the regularisation constant C.

21.1 Classification as Optimisation

A binary classifier assigns to each feature vector $\mathbf{x} \in \mathbb{R}^p$ a label $y \in \{-1, +1\}$. A **linear classifier** predicts

$$\hat{y}(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b), \tag{21.1}$$

where

$\mathbf{w} \in \mathbb{R}^p$ is the weight vector, normal to the decision boundary (Module 2);

$b \in \mathbb{R}$ is the bias (offset);

$\mathbf{w}^\top \mathbf{x} + b = 0$ is the decision boundary, a hyperplane;

$\text{sign}(u)$ equals $+1$ if $u > 0$ and -1 if $u < 0$.

A training point $(\mathbf{x}_i, y_i)$ is classified correctly exactly when $y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0$. The quantity $m_i = y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ is called the **functional margin** of point i: positive means correct, and larger means more confident.

The most famous classifier of this kind, the support vector machine (SVM) of Cortes and Vapnik, uses the Euclidean norm and leads to a quadratic program. Replacing the Euclidean norm by the ℓ_1 norm keeps almost all of its good properties, makes the training problem a linear program, and adds automatic feature selection. That is the subject of this module.

21.2 Separability as a Feasibility Problem

Derivation From strict to non strict inequalities

Step 1. The data are **linearly separable** if some $(\mathbf{w}, b)$ has $y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0$ for every i. Strict inequalities are not allowed in an LP (Module 1).

Step 2. With finitely many points, let $\delta = \min_i y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0$. Divide $\mathbf{w}$ and b by δ; the classifier (21.1) is unchanged because sign ignores positive scaling, and now every margin is at least 1.

Step 3. Hence the data are separable if and only if the system

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \qquad i = 1, \ldots, N \tag{21.2}$$

has a solution, which an LP with any objective (for example zero) can decide.

Key Result Certificate of non separability

By Farkas' lemma (Module 13), system (21.2) has no solution if and only if there are weights $\lambda_i \geq 0$, not all zero, with

$$\sum_i \lambda_i y_i \mathbf{x}_i = \mathbf{0} \qquad \text{and} \qquad \sum_i \lambda_i y_i = 0. \tag{21.3}$$

Dividing by $\sum_i \lambda_i$, this says that a convex combination of the positive points equals a convex combination of the negative points: *the convex hulls of the two classes intersect*. Two finite point sets are linearly separable exactly when their convex hulls are disjoint.

21.3 The ℓ_1 Norm Support Vector Machine

Derivation Margins measured in the ℓ_∞ norm

Step 1. For any norm, the distance from $\mathbf{x}_0$ to the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ is $\left|\mathbf{w}^\top \mathbf{x}_0 + b\right| / \|\mathbf{w}\|_*$, where $\|\cdot\|_*$ is the *dual norm*. For Euclidean distance the dual norm is again Euclidean (Module 2). For distance measured in the ℓ_∞ norm (the largest coordinate change), the dual norm is ℓ_1.

Step 2. Under the scaling (21.2), every point has $\left|\mathbf{w}^\top \mathbf{x}_i + b\right| \geq 1$, so the smallest ℓ_∞ distance to the boundary is at least $1/\|\mathbf{w}\|_1$.

Step 3. Maximising this guaranteed margin is the same as minimising $\|\mathbf{w}\|_1$.

The ℓ_∞ margin is natural when we want the classifier to tolerate a change of up to ε in *every* feature at once, exactly the adversarial perturbation model of Module 24.

Definition Hard and Soft Margin ℓ_1 SVM

Hard margin (separable data):

$$\min_{\mathbf{w}, b} \|\mathbf{w}\|_1 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1,\ i = 1, \ldots, N. \tag{21.4}$$

Soft margin (general data), with slack variables $\xi_i \ge 0$:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \|\mathbf{w}\|_1 + C\sum_{i=1}^{N}\xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^\top\mathbf{x}_i + b) \ge 1 - \xi_i, \quad \xi_i \ge 0, \tag{21.5}$$

where

ξ_i is the amount by which point i violates the margin; at the optimum $\xi_i = \max\left(0,\ 1 - y_i(\mathbf{w}^\top\mathbf{x}_i + b)\right)$, the **hinge loss**;

$C > 0$ is the regularisation constant, trading margin size against training violations.

Both become LPs by splitting $\mathbf{w} = \mathbf{w}^+ - \mathbf{w}^-$ with $\mathbf{w}^\pm \ge \mathbf{0}$, so that $\|\mathbf{w}\|_1 = \mathbf{1}^\top(\mathbf{w}^+ + \mathbf{w}^-)$ at the optimum.

Derivation The dual of the soft margin ℓ_1 SVM

Attach $\alpha_i \ge 0$ to each margin constraint $y_i\mathbf{x}_i^\top\mathbf{w}^+ - y_i\mathbf{x}_i^\top\mathbf{w}^- + y_i b + \xi_i \ge 1$.

Step 1. **Dual objective:** max $\sum_i \alpha_i$ (the right hand sides are all 1).

Step 2. **Column of** w_j^+ (cost 1): $\sum_i \alpha_i y_i x_{ij} \le 1$. **Column of** w_j^-: $-\sum_i \alpha_i y_i x_{ij} \le 1$. Together: $\left|\sum_i \alpha_i y_i x_{ij}\right| \le 1$ for each feature j.

Step 3. **Column of** b (free, cost 0): $\sum_i \alpha_i y_i = 0$.

Step 4. **Column of** ξ_i (cost C): $\alpha_i \le C$.

Step 5. The dual is

$$\max_{\boldsymbol{\alpha}} \ \sum_{i=1}^{N}\alpha_i \quad \text{subject to} \quad \left\|\sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i\right\|_\infty \le 1, \quad \sum_{i=1}^{N}\alpha_i y_i = 0, \quad 0 \le \alpha_i \le C. \tag{21.6}$$

For the hard margin problem, drop the upper bound C.

Key Result Support vectors and sparse weights

Complementary slackness (Module 13) gives two kinds of sparsity.

1. **Few support vectors.** If $y_i(\mathbf{w}^\top\mathbf{x}_i + b) > 1$ (point safely outside the margin), then $\alpha_i = 0$. Only points on or inside the margin, the **support vectors**, have $\alpha_i > 0$. If $\xi_i > 0$ (margin violated), then $\alpha_i = C$.
2. **Few features.** If $\left|\sum_i \alpha_i y_i x_{ij}\right| < 1$, then $w_j = 0$. A basic optimal solution has at most as many nonzero variables as constraints, so with N points at most N of the weights and slacks are nonzero, and in practice many w_j vanish. The ℓ_1 SVM therefore selects features automatically.

Lucid Insight Two norms, two geometries

The Euclidean SVM finds the widest "street" between the classes, measured with a ruler. The ℓ_1 SVM finds the widest street measured in the ℓ_∞ sense, where moving one unit in every coordinate at once counts as distance one. Corners of the ℓ_1 ball lie on the axes, so the optimal weight vector tends to sit on an axis: that is the geometric origin of sparsity, the same reason the lasso (Module 22) produces sparse models.

Machine Learning Connection

The ℓ_1 SVM, introduced by Bradley and Mangasarian (1998) and studied by Zhu, Rosset, Hastie and Tibshirani (2004), is used in genomics and text classification where there are many more features than samples and an interpretable sparse classifier is needed. The hinge loss (21.5) is also the loss of the standard SVM, of structured SVMs and of the multiclass hinge loss used to train some neural networks. The LP view gives exact solutions with certificates and makes it easy to add linear side constraints, such as fairness requirements on group average scores, without changing the solver.

21.4 Worked Examples

Worked Example 21.1 Is the Training Set Separable?

Negative points: $(0, 0)$ and $(1, 2)$. Positive points: $(3, 1)$ and $(4, 3)$. Write the separability system (21.2) and find a solution.

Solution

Step 1. System. With $\mathbf{w} = (w_1, w_2)$:

$$\begin{aligned} -(0w_1 + 0w_2 + b) &\geq 1, & -(1w_1 + 2w_2 + b) &\geq 1, \\ 3w_1 + 1w_2 + b &\geq 1, & 4w_1 + 3w_2 + b &\geq 1. \end{aligned}$$

Step 2. Try a vertical boundary ($w_2 = 0$): the conditions become $b \leq -1$, $w_1 + b \leq -1$, $3w_1 + b \geq 1$, $4w_1 + b \geq 1$.

Step 3. Choose $w_1 = 1$. Then $b \leq -2$ (from the second) and $b \geq -2$ (from the third). So $b = -2$ works, and the first and fourth conditions give $-2 \leq -1$ and $2 \geq 1$.

Step 4. Classifier. $\hat{y} = \operatorname{sign}(x_1 - 2)$: the boundary is the vertical line $x_1 = 2$.

Step 5. Margins $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$: 2, 1, 1, 2. All at least 1.

Answer. Separable: $\boxed{\mathbf{w} = (1, 0),\ b = -2}$ satisfies every constraint.

Worked Example 21.2 Solving the Hard Margin ℓ_1 SVM

For the data of Worked Example 21.1, solve (21.4) and report the ℓ_∞ margin.

Solution

Step 1. A lower bound from the two closest points. The constraints for $(1, 2)$ (negative) and $(3, 1)$ (positive) are $-w_1 - 2w_2 - b \geq 1$ and $3w_1 + w_2 + b \geq 1$. Adding them eliminates b: $2w_1 - w_2 \geq 2$.

Step 2. Bound on the norm. For any numbers, $2w_1 - w_2 \leq 2|w_1| + |w_2| \leq 2(|w_1| + |w_2|)$. Hence

$2 \le 2\,\|\mathbf{w}\|_1$, so $\|\mathbf{w}\|_1 \ge 1$ for every feasible classifier.

Step 3. **Attain the bound.** The classifier $\mathbf{w} = (1, 0)$, $b = -2$ from Worked Example 21.1 is feasible with $\|\mathbf{w}\|_1 = 1$. It is therefore optimal.

Step 4. **Margin.** The guaranteed ℓ_∞ distance is $1/\|\mathbf{w}\|_1 = 1$. The two closest points, $(1, 2)$ and $(3, 1)$, lie exactly at ℓ_∞ distance 1 from the line $x_1 = 2$, so any perturbation that changes every coordinate by less than one unit leaves every training point correctly classified.

Step 5. **Sparsity.** The optimal weight vector uses only feature x_1; feature x_2 is discarded.

Answer. $\boxed{\mathbf{w}^\star = (1, 0),\ b^\star = -2,\ \left\|\mathbf{w}^\star\right\|_1 = 1}$; margin 1; only feature 1 is used.

Worked Example 21.3 The Dual Solution and the Support Vectors

Find the optimal dual solution of the hard margin problem in Worked Example 21.2 and identify the support vectors.

Solution

Step 1. **Complementary slackness.** Points $(0, 0)$ and $(4, 3)$ have margin $2 > 1$, so $\alpha_1 = \alpha_4 = 0$. Only $(1, 2)$ and $(3, 1)$ can be support vectors.

Step 2. **Bias condition** $\sum \alpha_i y_i = 0$: $-\alpha_2 + \alpha_3 = 0$, so $\alpha_2 = \alpha_3 = a$.

Step 3. **Weight condition.** $\sum \alpha_i y_i \mathbf{x}_i = a((3, 1) - (1, 2)) = a(2,\ -1)$. Its ℓ_∞ norm is $2a$, which must be ≤ 1, so $a \le 0.5$.

Step 4. **Maximise** $\sum \alpha_i = 2a$: take $a = 0.5$, giving dual value 1.

Step 5. **Strong duality.** The dual value 1 equals the primal value $\|\mathbf{w}\|_1 = 1$.

Step 6. **Feature sparsity explained.** The vector $\sum \alpha_i y_i \mathbf{x}_i = (1,\ -0.5)$ reaches the bound 1 only in coordinate 1. In coordinate 2 the bound is slack $(0.5 < 1)$, so complementary slackness forces $w_2 = 0$.

Answer. $\boxed{\alpha^\star = (0,\ 0.5,\ 0.5,\ 0)}$; support vectors $(1, 2)$ and $(3, 1)$; dual value 1.

Worked Example 21.4 The XOR Problem Is Not Linearly Separable

Positive points $(0, 0)$ and $(1, 1)$; negative points $(1, 0)$ and $(0, 1)$. Prove that no linear classifier separates them by exhibiting a Farkas certificate (21.3).

Solution

Step 1. **Constraints** (21.2).

$$b \ge 1, \qquad w_1 + w_2 + b \ge 1, \qquad -w_1 - b \ge 1, \qquad -w_2 - b \ge 1.$$

Step 2. **Take all multipliers equal to one** and add the four inequalities. The left side is $b + (w_1 +$

$w_2 + b) + (-w_1 - b) + (-w_2 - b) = 0$; the right side is 4. The sum says $0 \geq 4$, which is false. So no $(\mathbf{w}, b)$ satisfies all four constraints.

Step 3. Check (21.3)**.** With $\lambda = (1,1,1,1)$: $\sum \lambda_i y_i \mathbf{x}_i = (0,0) + (1,1) - (1,0) - (0,1) = (0,0)$ and $\sum \lambda_i y_i = 1 + 1 - 1 - 1 = 0$.

Step 4. Geometry. With weights $\frac{1}{2}$, the midpoint of the positive points, $(0.5, 0.5)$, equals the midpoint of the negative points. The convex hulls (the two diagonals of the square) cross at the centre.

Step 5. Remedy. Adding the feature $x_1 x_2$ makes the data separable, for example by $\operatorname{sign}(2x_1x_2 - x_1 - x_2 + 0.5)$: values $0.5, 0.5, -0.5, -0.5$. This is the motivation for feature maps, kernels and hidden layers in neural networks.

Answer. Certificate $\boxed{\lambda = (1,1,1,1)}$ proves that XOR is not linearly separable.

Worked Example 21.5 A Soft Margin ℓ_1 SVM in One Dimension

Five points on a line: $x = (1,2,3,4,5)$ with labels $y = (-1,-1,+1,-1,+1)$. Verify that $w = \frac{2}{3}$, $b = -\frac{7}{3}$ solves the soft margin problem (21.5) with $C = 1$, using the dual.

Solution

Step 1. Margins $m_i = y_i(wx_i + b)$ with $wx_i + b = (2x_i - 7)/3$: values $(-\frac{5}{3},\ -1,\ -\frac{1}{3},\ \frac{1}{3},\ 1)$ for $x = 1, \ldots, 5$, so $m = (\frac{5}{3},\ 1,\ -\frac{1}{3},\ -\frac{1}{3},\ 1)$.

Step 2. Hinge losses $\xi_i = \max(0, 1 - m_i)$: $(0,\ 0,\ \frac{4}{3},\ \frac{4}{3},\ 0)$. Points 3 and 4 are misclassified (the label pattern cannot be separated by one threshold).

Step 3. Primal objective. $|w| + C\sum \xi_i = \frac{2}{3} + \frac{8}{3} = \frac{10}{3} \approx 3.333$.

Step 4. Dual candidate from complementary slackness. $m_1 > 1$ gives $\alpha_1 = 0$. $\xi_3, \xi_4 > 0$ give $\alpha_3 = \alpha_4 = C = 1$. Points 2 and 5 sit exactly on the margin, so $\alpha_2, \alpha_5 \in [0, 1]$.

Step 5. Bias condition. $-\alpha_2 + 1 - 1 + \alpha_5 = 0$, so $\alpha_2 = \alpha_5 = a$.

Step 6. Weight condition. Since $w \neq 0$, the bound must be tight: $\sum \alpha_i y_i x_i = -2a + 3 - 4 + 5a = 3a - 1 = 1$ (the sign matches $w > 0$), giving $a = \frac{2}{3}$.

Step 7. Dual value. $\sum \alpha_i = 0 + \frac{2}{3} + 1 + 1 + \frac{2}{3} = \frac{10}{3}$, equal to the primal value. Both are optimal.

Answer. $\boxed{w = \frac{2}{3},\ b = -\frac{7}{3},\ \text{objective } \frac{10}{3}}$; support vectors at $x = 2, 3, 4, 5$ with $\boldsymbol{\alpha} = (0, \frac{2}{3}, 1, 1, \frac{2}{3})$.

Worked Example 21.6 The Effect of the Regularisation Constant

For the data of Worked Example 21.5, show that with $C = 0.2$ the optimal classifier is $w = 0$, and explain the result.

Solution

Step 1. **With** $w = 0$, the classifier predicts $\text{sign}(b)$ for every point. Choose $b = -1$: margins $y_i b$ are $1, 1, -1, 1, -1$, hinge losses $0, 0, 2, 0, 2$.

Step 2. **Objective.** $|0| + 0.2 \times 4 = 0.8$.

Step 3. **Dual certificate.** Take $\alpha_i = C = 0.2$ for the two violated points (3 and 5), and $\alpha = 0.2$ for two of the negative points to satisfy $\sum \alpha_i y_i = 0$, say points 1 and 4: $\boldsymbol{\alpha} = (0.2,\ 0,\ 0.2,\ 0.2,\ 0.2)$. Check $\sum \alpha_i y_i = -0.2 + 0.2 - 0.2 + 0.2 = 0$ and $|\sum \alpha_i y_i x_i| = |-0.2 + 0.6 - 0.8 + 1.0| = 0.6 \le 1$. Dual value 0.8.

Step 4. **Optimal.** Primal and dual values agree at 0.8.

Step 5. **Interpretation.** With a small C, violations are cheap compared with any nonzero weight, so the model prefers to predict the majority class everywhere. As C grows the model fits the data more closely ($C = 1$ gave a genuine threshold). Choosing C by cross validation balances these extremes.

Answer. With $C = 0.2$: $\boxed{w^\star = 0,\ b^\star = -1,\ \text{objective } 0.8}$; the classifier is constant.

Module Summary

- A linear classifier is $\text{sign}(\mathbf{w}^\top \mathbf{x} + b)$; correct classification with margin means $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1$.
- Separability is an LP feasibility question; non separability has a Farkas certificate: the convex hulls of the classes intersect.
- The ℓ_1 SVM minimises $\|\mathbf{w}\|_1 + C \sum \xi_i$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1 - \xi_i$; it maximises an ℓ_∞ margin.
- Dual: $\max \sum \alpha_i$ subject to $\|\sum \alpha_i y_i \mathbf{x}_i\|_\infty \le 1$, $\sum \alpha_i y_i = 0$, $0 \le \alpha_i \le C$.
- Complementary slackness yields support vectors ($\alpha_i > 0$ only on or inside the margin) and sparse weights ($w_j = 0$ when the jth dual bound is slack).
- Small C favours simple (even constant) classifiers; large C favours fitting the training data.

MODULE 22

Sparse Recovery, Basis Pursuit and ℓ_1 Regularisation

Learning Objectives

After studying this module you will be able to:

- explain why sparse solutions are valuable and why minimising the number of nonzeros directly is intractable;
- formulate basis pursuit, $\min \|\mathbf{x}\|_1$ subject to $\mathbf{A}\mathbf{x} = \mathbf{b}$, as a linear program and write its dual;
- certify that a sparse vector is the unique basis pursuit solution using a dual certificate;
- state and prove the null space property, which characterises exact recovery of all sparse vectors;
- formulate the Dantzig selector as an LP and derive soft thresholding for orthonormal designs;
- compute a regularisation path for ℓ_1 penalised LAD regression.

22.1 The Value of Sparsity

A vector is **sparse** if most of its entries are zero. Sparse models are easier to interpret (a diagnosis that depends on five genes rather than twenty thousand), cheaper to deploy (a pruned network), and often more accurate when data are limited, because they cannot fit noise through many small coefficients. Sparsity also makes *compressed sensing* possible: a sparse signal can be reconstructed from far fewer linear measurements than its length, which is how fast MRI scanners and single pixel cameras work.

Definition The ℓ_0 Count and Sparse Recovery

For $\mathbf{x} \in \mathbb{R}^n$, $\|\mathbf{x}\|_0$ denotes the number of nonzero entries (not a true norm). A vector with $\|\mathbf{x}\|_0 \leq k$ is k **sparse**. Given a measurement matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m < n$ and measurements $\mathbf{b} = \mathbf{A}\mathbf{x}^0$ of an unknown sparse $\mathbf{x}^0$, the ideal recovery problem is

$$\min_{\mathbf{x}} \ \|\mathbf{x}\|_0 \quad \text{subject to} \quad \mathbf{A}\mathbf{x} = \mathbf{b}. \tag{22.1}$$

Problem (22.1) is combinatorial: one would have to test supports of size $1, 2, 3, \ldots$, and it is NP hard in general. The key idea of this module is to replace $\|\mathbf{x}\|_0$ by $\|\mathbf{x}\|_1$, the closest convex function, which turns the problem into a linear program.

Lucid Insight Why the ℓ_1 ball finds sparse points

Picture the solution set $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b}\}$, an affine subspace, and inflate an ℓ_1 ball around the origin until it first touches that subspace. The ℓ_1 ball is a diamond whose corners lie on the coordinate axes; the first contact typically happens at a corner, a sparse point. A Euclidean ball is round and touches at a generic, dense point. That picture is the whole intuition of compressed sensing.

22.2 Basis Pursuit

Derivation Basis pursuit as a linear program

Step 1. **Problem.** Basis pursuit (Chen, Donoho and Saunders, 1998) solves

$$\min_{\mathbf{x}} \ \|\mathbf{x}\|_1 \quad \text{subject to} \quad \mathbf{A}\mathbf{x} = \mathbf{b}. \tag{22.2}$$

Step 2. **Split.** Write $\mathbf{x} = \mathbf{x}^+ - \mathbf{x}^-$ with $\mathbf{x}^\pm \geq \mathbf{0}$. Then $\|\mathbf{x}\|_1 \leq \mathbf{1}^\top(\mathbf{x}^+ + \mathbf{x}^-)$, with equality when $x_j^+ x_j^- = 0$, which holds at every basic solution.

Step 3. **LP.**

$$\min \ \mathbf{1}^\top\mathbf{x}^+ + \mathbf{1}^\top\mathbf{x}^- \quad \text{subject to} \quad \mathbf{A}\mathbf{x}^+ - \mathbf{A}\mathbf{x}^- = \mathbf{b}, \quad \mathbf{x}^+, \mathbf{x}^- \geq \mathbf{0}. \tag{22.3}$$

Step 4. **Sparsity of vertex solutions.** The LP has m equations, so a basic optimal solution has at most m nonzero entries (Module 7). Basis pursuit always returns an m sparse vector at worst.

Step 5. **Dual.** Attach a free $\mathbf{y} \in \mathbb{R}^m$ to the equations. The column of x_j^+ gives $\mathbf{A}_j^\top\mathbf{y} \leq 1$ and that of x_j^- gives $-\mathbf{A}_j^\top\mathbf{y} \leq 1$:

$$\max_{\mathbf{y}} \ \mathbf{b}^\top\mathbf{y} \quad \text{subject to} \quad \left\|\mathbf{A}^\top\mathbf{y}\right\|_\infty \leq 1. \tag{22.4}$$

Key Result Dual certificate for exact recovery

Let $\mathbf{x}^0$ be feasible with support $S = \{j : x_j^0 \neq 0\}$, and suppose the columns $\{\mathbf{A}_j : j \in S\}$ are linearly independent. If there is $\mathbf{y}$ with

$$\mathbf{A}_j^\top\mathbf{y} = \operatorname{sign}(x_j^0) \ \text{ for } j \in S, \qquad \left|\mathbf{A}_j^\top\mathbf{y}\right| < 1 \ \text{ for } j \notin S, \tag{22.5}$$

then $\mathbf{x}^0$ is the *unique* solution of basis pursuit.

Derivation Proof of the certificate

Step 1. **Optimality.** $\mathbf{y}$ is dual feasible. Its value is $\mathbf{b}^\top\mathbf{y} = (\mathbf{A}\mathbf{x}^0)^\top\mathbf{y} = \sum_{j\in S} x_j^0\mathbf{A}_j^\top\mathbf{y} = \sum_{j\in S} x_j^0 \operatorname{sign}(x_j^0) = \left\|\mathbf{x}^0\right\|_1$. Equal primal and dual values prove optimality.

Step 2. **Uniqueness.** Let $\mathbf{x}$ be any other feasible point, $\mathbf{h} = \mathbf{x} - \mathbf{x}^0 \neq \mathbf{0}$, so $\mathbf{A}\mathbf{h} = \mathbf{0}$. Then $\|\mathbf{x}\|_1 \geq \mathbf{y}^\top\mathbf{A}\mathbf{x} + \sum_{j\notin S}(1 - \left|\mathbf{A}_j^\top\mathbf{y}\right|)\left|x_j\right|$, because for $j \in S$ we use $\left|x_j\right| \geq \operatorname{sign}(x_j^0)x_j = (\mathbf{A}_j^\top\mathbf{y})x_j$ and for $j \notin S$ we use $\left|x_j\right| \geq (\mathbf{A}_j^\top\mathbf{y})x_j + (1 - \left|\mathbf{A}_j^\top\mathbf{y}\right|)\left|x_j\right|$.

Step 3. Since $\mathbf{y}^\top\mathbf{A}\mathbf{x} = \mathbf{y}^\top\mathbf{b} = \left\|\mathbf{x}^0\right\|_1$, we get $\|\mathbf{x}\|_1 \geq \left\|\mathbf{x}^0\right\|_1 + \sum_{j\notin S}(1 - \left|\mathbf{A}_j^\top\mathbf{y}\right|)\left|x_j\right|$.

Step 4. If $\mathbf{x}$ were also optimal, every x_j with $j \notin S$ would vanish (the coefficients are strictly positive). Then $\mathbf{h}$ is supported on S with $\mathbf{Ah} = \mathbf{0}$, and independence of the columns forces $\mathbf{h} = \mathbf{0}$, a contradiction.

22.3 When Does ℓ_1 Recover Every Sparse Vector?

Definition Null Space Property

$\mathbf{A}$ satisfies the **null space property of order** k if for every nonzero $\mathbf{h}$ with $\mathbf{Ah} = \mathbf{0}$ and every index set S with $|S| \le k$,

$$\|\mathbf{h}_S\|_1 < \left\|\mathbf{h}_{\bar{S}}\right\|_1, \tag{22.6}$$

where

$\mathbf{h}_S$ keeps the entries of $\mathbf{h}$ indexed by S and sets the others to zero;

$\bar{S}$ is the complement of S.

In words: no null space vector concentrates half or more of its ℓ_1 mass on k coordinates.

Key Result Exact recovery theorem

Every k sparse $\mathbf{x}^0$ is the unique solution of basis pursuit with $\mathbf{b} = \mathbf{Ax}^0$ if and only if $\mathbf{A}$ satisfies the null space property of order k.

Derivation Proof that the null space property suffices

Step 1. Let $\mathbf{x}^0$ be supported on S, $|S| \le k$, and let $\mathbf{x} = \mathbf{x}^0 + \mathbf{h}$ be another feasible point, so $\mathbf{Ah} = \mathbf{0}$, $\mathbf{h} \neq \mathbf{0}$.

Step 2. Split the norm over S and $\bar{S}$: $\|\mathbf{x}\|_1 = \left\|\mathbf{x}^0_S + \mathbf{h}_S\right\|_1 + \left\|\mathbf{h}_{\bar{S}}\right\|_1$, since $\mathbf{x}^0$ vanishes off S.

Step 3. Reverse triangle inequality: $\left\|\mathbf{x}^0_S + \mathbf{h}_S\right\|_1 \ge \left\|\mathbf{x}^0\right\|_1 - \|\mathbf{h}_S\|_1$.

Step 4. Hence $\|\mathbf{x}\|_1 \ge \left\|\mathbf{x}^0\right\|_1 + \left(\left\|\mathbf{h}_{\bar{S}}\right\|_1 - \|\mathbf{h}_S\|_1\right) > \left\|\mathbf{x}^0\right\|_1$ by (22.6).

Conversely, if (22.6) fails for some $\mathbf{h}$ and S, the vector $\mathbf{x}^0 = \mathbf{h}_S$ (which is k sparse) and $\mathbf{x} = -\mathbf{h}_{\bar{S}}$ have the same measurements and $\|\mathbf{x}\|_1 \le \left\|\mathbf{x}^0\right\|_1$, so $\mathbf{x}^0$ is not the unique solution.

Common Pitfall

The null space property is hard to verify for large matrices. Compressed sensing theory therefore proves it with high probability for *random* matrices: an $m \times n$ Gaussian matrix satisfies it of order k when m is about $C\,k\log(n/k)$ for a modest constant C. Matrices with highly correlated columns, common in real feature sets, can fail it badly, and then ℓ_1 methods may select the wrong features.

22.4 Noisy Measurements: The Dantzig Selector

Derivation The Dantzig selector LP

With noise, $\mathbf{b} = \mathbf{A}\mathbf{x}^0 + \mathbf{z}$, insisting on $\mathbf{A}\mathbf{x} = \mathbf{b}$ would fit the noise. Candès and Tao (2007) proposed

$$\min_{\mathbf{x}} \|\mathbf{x}\|_1 \quad \text{subject to} \quad \left\|\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\mathbf{x})\right\|_\infty \le \delta, \tag{22.7}$$

which requires the residual to be nearly uncorrelated with every column.

Step 1. Split $\mathbf{x} = \mathbf{x}^+ - \mathbf{x}^-$ as before.

Step 2. The constraint $\|\mathbf{u}\|_\infty \le \delta$ is the $2n$ linear inequalities $-\delta \le u_j \le \delta$.

Step 3. So (22.7) is an LP with $2n$ variables and $2n$ inequality constraints.

where

$\delta \ge 0$ is a tuning parameter, typically proportional to the noise level times $\sqrt{2 \log n}$.

Derivation Soft thresholding for an orthonormal design

Suppose $\mathbf{A}^\top\mathbf{A} = \mathbf{I}$ (orthonormal columns). Let $\mathbf{c} = \mathbf{A}^\top\mathbf{b}$.

Step 1. Then $\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\mathbf{x}) = \mathbf{c} - \mathbf{x}$, and (22.7) separates into n scalar problems: minimise $|x_j|$ subject to $|c_j - x_j| \le \delta$.

Step 2. If $|c_j| \le \delta$, the value $x_j = 0$ is allowed, and it has the smallest possible absolute value.

Step 3. If $c_j > \delta$, the allowed interval $[c_j - \delta,\ c_j + \delta]$ lies to the right of 0; the point closest to 0 is $c_j - \delta$. Symmetrically for $c_j < -\delta$.

Step 4. Hence

$$x_j^\star = \operatorname{sign}(c_j) \max\left(|c_j| - \delta,\ 0\right), \tag{22.8}$$

the **soft thresholding** operator: small coefficients are set to zero, large ones are shrunk towards zero by δ.

Machine Learning Connection

ℓ_1 regularisation is one of the most used tools in machine learning. The lasso, $\min \frac{1}{2}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1$, is a quadratic program whose solution for orthonormal designs is exactly the soft thresholding (22.8); when the squared loss is replaced by the absolute loss or the hinge loss, the whole problem is an LP (the LAD lasso and the ℓ_1 SVM of Module 21). ℓ_1 penalties prune neural network weights, select features in genomics, and drive sparse coding and dictionary learning. Compressed sensing powers accelerated MRI and inspired deep "unrolled" networks (such as LISTA) that imitate iterative soft thresholding.

22.5 Worked Examples

Worked Example 22.1 Basis Pursuit Finds the Sparse Solution, Least Norm Does Not

Let $\mathbf{A} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$ and $\mathbf{b} = (1,1)^\top$, generated by the sparse vector $\mathbf{x}^0 = (0,0,1)$. Find all solutions of $\mathbf{A}\mathbf{x} = \mathbf{b}$, the basis pursuit solution and the minimum Euclidean norm solution.

Solution

Step 1. All solutions. The equations are $x_1 + x_3 = 1$ and $x_2 + x_3 = 1$. With $x_3 = t$: $\mathbf{x}(t) = (1-t,\ 1-t,\ t)$ for any real t.

Step 2. ℓ_1 norm along the line. $f(t) = 2\,|1-t| + |t|$. For $0 \le t \le 1$: $f = 2 - 2t + t = 2 - t$, decreasing. For $t \ge 1$: $f = 2t - 2 + t = 3t - 2$, increasing. For $t \le 0$: $f = 2 - 3t$, decreasing as t rises. Minimum at $t = 1$, $f = 1$.

Step 3. Basis pursuit solution: $\mathbf{x} = (0,0,1) = \mathbf{x}^0$, exact recovery.

Step 4. Euclidean minimum. $g(t) = 2(1-t)^2 + t^2$. Setting $g'(t) = -4(1-t) + 2t = 0$ gives $t = \frac{2}{3}$, so $\mathbf{x} = (\frac{1}{3},\ \frac{1}{3},\ \frac{2}{3})$: every entry nonzero.

Step 5. Comparison. The ℓ_1 solution has one nonzero; the ℓ_2 solution spreads the signal over all coordinates.

Answer. Basis pursuit returns $\boxed{(0,0,1)}$ (exact); minimum ℓ_2 norm returns the dense $(\frac{1}{3}, \frac{1}{3}, \frac{2}{3})$.

Worked Example 22.2 A Dual Certificate of Exact Recovery

For Worked Example 22.1, find $\mathbf{y}$ satisfying (22.5) and conclude that $\mathbf{x}^0 = (0,0,1)$ is the unique basis pursuit solution.

Solution

Step 1. Compute $\mathbf{A}^\top\mathbf{y}$ $= (y_1,\ y_2,\ y_1 + y_2)$.

Step 2. Support condition. $S = \{3\}$ with $x_3^0 = 1 > 0$, so we need $y_1 + y_2 = 1$.

Step 3. Off support conditions. $|y_1| < 1$ and $|y_2| < 1$.

Step 4. Choose $\mathbf{y}$ $= (\frac{1}{2},\ \frac{1}{2})$: $y_1 + y_2 = 1$, $|y_1| = |y_2| = \frac{1}{2} < 1$. The single support column is nonzero, hence independent.

Step 5. Conclusion. The certificate exists, so $\mathbf{x}^0$ is the unique solution. Dual value $\mathbf{b}^\top\mathbf{y} = 1 = \|\mathbf{x}^0\|_1$.

Answer. $\boxed{\mathbf{y} = (\frac{1}{2}, \frac{1}{2})}$ certifies unique recovery of $(0,0,1)$.

Worked Example 22.3 When Basis Pursuit Fails

Let $\mathbf{A} = \begin{pmatrix} 1 & 0 & 0.4 \\ 0 & 1 & 0.4 \end{pmatrix}$ and $\mathbf{x}^0 = (0, 0, 1)$, so $\mathbf{b} = (0.4,\ 0.4)$. Solve basis pursuit and explain the failure.

Solution

Step 1. All solutions. $x_1 + 0.4x_3 = 0.4$ and $x_2 + 0.4x_3 = 0.4$: $\mathbf{x}(t) = (0.4 - 0.4t,\ 0.4 - 0.4t,\ t)$.

Step 2. ℓ_1 norm. For $0 \le t \le 1$: $f(t) = 2(0.4 - 0.4t) + t = 0.8 + 0.2t$, increasing. For $t < 0$ it increases as t decreases. For $t > 1$: $f = 2(0.4t - 0.4) + t = 1.8t - 0.8$, increasing. Minimum at $t = 0$: $f = 0.8$.

Step 3. Basis pursuit returns $(0.4,\ 0.4,\ 0)$, a 2 sparse vector with the wrong support, while the true $\mathbf{x}^0$ has ℓ_1 norm $1 > 0.8$.

Step 4. Why. Column 3 is “short”: representing the measurement with columns 1 and 2 costs less ℓ_1 mass (0.8) than with the genuine column 3 (1). No dual certificate can exist: we would need $0.4(y_1 + y_2) = 1$ with $|y_1|, |y_2| < 1$, but then $0.4(y_1 + y_2) < 0.8$.

Step 5. Remedy. Normalising the columns to equal length (a standard preprocessing step) would restore recovery here.

Answer. Basis pursuit returns $\boxed{(0.4, 0.4, 0)}$, not $\mathbf{x}^0$: recovery fails.

Worked Example 22.4 Checking the Null Space Property

Determine whether the matrices of Worked Examples 22.1 and 22.3 satisfy the null space property of order 1, and reconcile the answers with those examples.

Solution

Step 1. Matrix of 22.1. Null space: solve $h_1 + h_3 = 0$, $h_2 + h_3 = 0$. So $\mathbf{h} = s(1, 1, -1)$ for $s \neq 0$.

Step 2. Test (22.6) **for every single index.** $\|\mathbf{h}\|_1 = 3\,|s|$. Any single coordinate carries $|s|$, the rest $2\,|s|$, and $|s| < 2\,|s|$. The property holds: *every* 1 sparse vector is recovered, including $(0, 0, 1)$.

Step 3. Matrix of 22.3. Null space: $h_1 + 0.4h_3 = 0$, $h_2 + 0.4h_3 = 0$, so $\mathbf{h} = s(-0.4,\ -0.4,\ 1)$.

Step 4. Test. For $S = \{3\}$: $\|\mathbf{h}_S\|_1 = |s|$ and $\left\|\mathbf{h}_{\bar{S}}\right\|_1 = 0.8\,|s|$. Since $|s| > 0.8\,|s|$, the property fails for $S = \{3\}$.

Step 5. Reconcile. The failing set is exactly the support of the vector that was not recovered. For $S = \{1\}$ the test passes $(0.4 < 1.4)$, so 1 sparse vectors supported on coordinate 1 or 2 are still recovered by the second matrix.

Answer. First matrix: $\boxed{\text{property holds}}$; second matrix: $\boxed{\text{fails on } S = \{3\}}$, exactly as observed.

Worked Example 22.5 The Dantzig Selector with an Orthonormal Design

With $\mathbf{A} = \mathbf{I}_3$ and noisy measurements $\mathbf{b} = (3,\ -0.5,\ 1.2)$, solve the Dantzig selector (22.7) with $\delta = 1$.

Solution

Step 1. **Orthonormal case.** $\mathbf{c} = \mathbf{A}^\top \mathbf{b} = \mathbf{b}$, so (22.8) applies coordinate by coordinate.

Step 2. **Coordinate 1.** $c_1 = 3 > 1$: $x_1 = 3 - 1 = 2$.

Step 3. **Coordinate 2.** $|c_2| = 0.5 \le 1$: $x_2 = 0$.

Step 4. **Coordinate 3.** $c_3 = 1.2 > 1$: $x_3 = 1.2 - 1 = 0.2$.

Step 5. **Check the constraint.** $\mathbf{c} - \mathbf{x} = (1,\ -0.5,\ 1)$, whose largest absolute entry is $1 \le \delta$. The ℓ_1 norm is 2.2, and no feasible point has smaller norm because each coordinate was individually minimised.

Step 6. **Interpretation.** The weak, probably noisy coordinate is removed; strong coordinates are kept but shrunk by δ, which reduces variance at the price of some bias.

Answer. $\boxed{\mathbf{x}^\star = (2,\ 0,\ 0.2)}$.

Worked Example 22.6 A Regularisation Path for ℓ_1 Penalised LAD

Fit $\hat{y} = wx$ (no intercept) to $x = (1, 2, 4)$, $y = (2, 3, 9)$ by minimising $\sum_i |y_i - wx_i| + \lambda |w|$ for $\lambda = 0$, 2 and 8.

Solution

Step 1. **Rewrite as a weighted median.** $|y_i - wx_i| = x_i |y_i/x_i - w|$ for $x_i > 0$. So the loss is $\sum_i x_i |r_i - w|$ with ratios $r = (2,\ 1.5,\ 2.25)$ and weights $(1, 2, 4)$. The penalty $\lambda |w| = \lambda |0 - w|$ acts like an extra point at ratio 0 with weight λ.

Step 2. **Weighted median rule.** By the subgradient argument of Module 20 (with $\tau = 0.5$), the minimiser is a value where at most half the total weight lies strictly below and at most half strictly above.

Step 3. **$\lambda = 0$.** Sorted ratios $1.5\,(2)$, $2\,(1)$, $2.25\,(4)$; total weight 7, half 3.5. Cumulative weights 2, 3, 7: the half is crossed at 2.25. So $w = 2.25$; loss $|2 - 2.25| + |3 - 4.5| + |9 - 9| = 0.25 + 1.5 = 1.75$.

Step 4. **$\lambda = 2$.** Add ratio 0 with weight 2: sorted $0\,(2)$, $1.5\,(2)$, $2\,(1)$, $2.25\,(4)$; total 9, half 4.5; cumulative $2, 4, 5, 9$: crossed at 2. So $w = 2$; objective $0 + 1 + 1 + 2(2) = 6$.

Step 5. **$\lambda = 8$.** Total 15, half 7.5; the extra point alone has weight $8 > 7.5$, so $w = 0$: the model is switched off.

Step 6. **Path.** w falls from 2.25 to 2 to 0 as λ grows: the characteristic shrink and select behaviour of ℓ_1 penalties. Each value is the solution of an LP, and the whole path is piecewise constant in this one dimensional case.

Answer. $\boxed{w(0) = 2.25,\ w(2) = 2,\ w(8) = 0}$.

Module Summary

- Minimising $\|\mathbf{x}\|_0$ is intractable; $\|\mathbf{x}\|_1$ is its convex surrogate and gives LPs.
- Basis pursuit: $\min \|\mathbf{x}\|_1$ subject to $\mathbf{A}\mathbf{x} = \mathbf{b}$; dual $\max\ \mathbf{b}^\top \mathbf{y}$ subject to $\left\|\mathbf{A}^\top \mathbf{y}\right\|_\infty \leq 1$.
- A dual vector equal to the signs on the support and strictly less than one in size elsewhere certifies unique recovery.
- Null space property: every null vector has more ℓ_1 mass off any k set than on it; it is equivalent to exact recovery of all k sparse vectors.
- The Dantzig selector is an LP; for orthonormal designs it reduces to soft thresholding $\operatorname{sign}(c)\max(|c| - \delta, 0)$.
- ℓ_1 penalties shrink coefficients and set some exactly to zero as the penalty grows.

MODULE 23

Optimal Transport and the Wasserstein Distance

Learning Objectives

After studying this module you will be able to:

- state the Kantorovich optimal transport problem between two discrete distributions as a linear program;
- derive its dual in terms of potentials and interpret the Kantorovich Rubinstein form of the Wasserstein W_1 distance;
- compute W_1 on the line through cumulative distribution functions;
- derive the entropic regularisation of optimal transport and the Sinkhorn scaling algorithm;
- use transport distances to compare label distributions, generated samples and data sets in machine learning.

23.1 Moving Mass at Minimum Cost

In 1781 Gaspard Monge asked how to move a pile of earth into a hole with the least total effort. In 1942 Leonid Kantorovich reformulated the question as a linear program, one of the first ever written, and later received a Nobel prize for related work. Today optimal transport is a central tool of machine learning: it compares probability distributions in a way that respects the geometry of the space they live on. Two histograms that are shifted slightly apart are “close” in transport distance, whereas many classical divergences (such as the Kullback Leibler divergence) see them as completely different. This property made the Wasserstein distance the basis of Wasserstein GANs, domain adaptation methods, and evaluation metrics for generative models.

23.2 The Kantorovich Linear Program

Definition Discrete Optimal Transport

Let $\mathbf{a} \in \mathbb{R}^n$ and $\mathbf{b} \in \mathbb{R}^m$ be probability vectors ($\mathbf{a}, \mathbf{b} \geq \mathbf{0}$, $\mathbf{1}^\top \mathbf{a} = \mathbf{1}^\top \mathbf{b} = 1$) placed on points $x_1, \dots, x_n$ and $z_1, \dots, z_m$. Let $\mathbf{C} \in \mathbb{R}^{n \times m}$ hold the costs $C_{ij} = c(x_i, z_j)$ of moving one unit of mass from x_i to z_j.

The optimal transport problem is

$$\mathrm{OT}(\mathbf{a},\mathbf{b}) = \min_{\mathbf{P}\in\mathbb{R}^{n\times m}} \sum_{i=1}^{n}\sum_{j=1}^{m} C_{ij}P_{ij} \quad \text{subject to} \quad \mathbf{P1} = \mathbf{a}, \quad \mathbf{P}^\top\mathbf{1} = \mathbf{b}, \quad \mathbf{P} \geq \mathbf{0}, \tag{23.1}$$

where

P_{ij} is the amount of mass moved from x_i to z_j; **P** is called a **transport plan** or **coupling**;

$\mathbf{P1} = \mathbf{a}$ says that each source ships out exactly its mass (row sums);

$\mathbf{P}^\top\mathbf{1} = \mathbf{b}$ says that each destination receives exactly its mass (column sums).

When $c(x, z) = |x - z|^p$ (or $\|x - z\|^p$), the value $W_p = \mathrm{OT}^{1/p}$ is the **Wasserstein distance** of order p. For $p = 1$ it is also called the **earth mover's distance**.

Lucid Insight It is a transportation problem

Problem (23.1) is exactly the transportation problem of Module 18 with supplies **a** and demands **b**. Everything from that module applies: the constraint matrix is totally unimodular, an optimal plan exists at a vertex, and a vertex plan has at most $n + m - 1$ nonzero entries (the number of independent equations). Optimal plans are therefore sparse.

Derivation The dual: Kantorovich potentials

Step 1. Attach a free dual variable f_i to each row constraint $\sum_j P_{ij} = a_i$ and g_j to each column constraint $\sum_i P_{ij} = b_j$.

Step 2. The column of P_{ij} has a 1 in row constraint i and a 1 in column constraint j, and $P_{ij} \geq 0$ with cost C_{ij}. The dual constraint is therefore $f_i + g_j \leq C_{ij}$.

Step 3. The dual problem is

$$\mathrm{OT}(\mathbf{a},\mathbf{b}) = \max_{\mathbf{f},\mathbf{g}} \ \mathbf{a}^\top\mathbf{f} + \mathbf{b}^\top\mathbf{g} \quad \text{subject to} \quad f_i + g_j \leq C_{ij} \ \text{ for all } i, j. \tag{23.2}$$

Step 4. **Complementary slackness.** Mass flows only along routes where the constraint is tight: $P_{ij} > 0 \Rightarrow f_i + g_j = C_{ij}$.

where

f_i is the price a shipping company charges for collecting a unit at x_i;

g_j is the price for delivering a unit to z_j (often negative);

$f_i + g_j \leq C_{ij}$ says the company can never charge more than the cost of doing the job yourself.

Key Result Kantorovich Rubinstein duality for W_1

When $C_{ij} = d(x_i, z_j)$ is a distance (and both measures live on the same set of points), the optimal potentials satisfy $g = -f$, and the dual becomes

$$W_1(\mathbf{a},\mathbf{b}) = \max_{\varphi} \Big(\sum_i a_i\varphi(x_i) - \sum_j b_j\varphi(z_j)\Big) \quad \text{subject to} \quad \big|\varphi(x) - \varphi(x')\big| \le d(x,x'), \tag{23.3}$$

a maximum over **1 Lipschitz functions** φ. The Wasserstein GAN uses exactly this form: its "critic" network is trained to be (approximately) 1 Lipschitz and to maximise the difference of its average values on real and generated data.

Derivation W_1 on the real line through distribution functions

Let both distributions live on equally spaced points $0, 1, \dots, K$ with $C_{ij} = |i - j|$, and let $F_k = \sum_{i\le k} a_i$ and $G_k = \sum_{j\le k} b_j$ be their cumulative distribution functions.

Step 1. Consider the "gap" between positions k and $k+1$. Any transport plan must carry net mass $F_k - G_k$ across this gap: if $F_k > G_k$, more mass sits at or below k in $\mathbf{a}$ than in $\mathbf{b}$, and the excess must move up across the gap (and vice versa).

Step 2. Each unit crossing a gap of length 1 costs at least 1, so the total cost is at least $\sum_{k=0}^{K-1} |F_k - G_k|$.

Step 3. The plan that moves mass monotonically (never crossing a gap in both directions) achieves exactly this cost.

Step 4. Hence

$$W_1(\mathbf{a},\mathbf{b}) = \sum_{k=0}^{K-1} |F_k - G_k| \times (\text{spacing}). \tag{23.4}$$

For general points on the line, the same argument gives $W_1 = \int |F(t) - G(t)|\, dt$, the area between the two distribution functions.

23.3 Entropic Regularisation and the Sinkhorn Algorithm

Exact OT costs about n^3 operations for n points, too slow for large data. Cuturi (2013) showed that adding an entropy term gives a problem that can be solved by simple matrix scaling, which runs on GPUs and differentiates easily.

Derivation The structure of the entropic optimal plan

For a regularisation strength $\varepsilon > 0$, consider

$$\min_{\mathbf{P}\ge\mathbf{0}} \sum_{i,j} C_{ij}P_{ij} + \varepsilon \sum_{i,j} P_{ij}(\ln P_{ij} - 1) \quad \text{subject to} \quad \mathbf{P}\mathbf{1} = \mathbf{a},\ \mathbf{P}^\top\mathbf{1} = \mathbf{b}. \tag{23.5}$$

Step 1. Form the Lagrangian with multipliers f_i and g_j: $\mathcal{L} = \sum_{ij}(C_{ij}P_{ij} + \varepsilon P_{ij}(\ln P_{ij} - 1)) - \sum_i f_i(\sum_j P_{ij} - a_i) - \sum_j g_j(\sum_i P_{ij} - b_j)$.

Step 2. Differentiate with respect to P_{ij}: $C_{ij} + \varepsilon \ln P_{ij} - f_i - g_j = 0$ (the derivative of $P(\ln P - 1)$ is $\ln P$).

Step 3. Solve: $P_{ij} = \exp\left((f_i + g_j - C_{ij})/\varepsilon\right) = e^{f_i/\varepsilon}\, e^{-C_{ij}/\varepsilon}\, e^{g_j/\varepsilon}$.

Step 4. Define $u_i = e^{f_i/\varepsilon}$, $v_j = e^{g_j/\varepsilon}$ and the **Gibbs kernel** $K_{ij} = e^{-C_{ij}/\varepsilon}$. Then

$$\mathbf{P} = \operatorname{diag}(\mathbf{u})\, \mathbf{K}\, \operatorname{diag}(\mathbf{v}). \tag{23.6}$$

The optimal plan is the kernel with its rows and columns rescaled. The logarithm also keeps every $P_{ij} > 0$: the entropy acts as a barrier, exactly like the logarithmic barrier of Module 16.

Key Result Sinkhorn iterations

Start with $\mathbf{v} = \mathbf{1}$ and repeat until the marginals match:

$$\mathbf{u} \leftarrow \mathbf{a} \oslash (\mathbf{K}\mathbf{v}), \qquad \mathbf{v} \leftarrow \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}), \tag{23.7}$$

where $\oslash$ denotes division entry by entry. The first update makes the row sums of (23.6) equal $\mathbf{a}$; the second makes the column sums equal $\mathbf{b}$. As $\varepsilon \to 0$ the entropic plan tends to an optimal plan of (23.1); larger ε gives faster convergence but a blurrier plan.

Common Pitfall

The entropic cost $\sum C_{ij} P_{ij}$ of the Sinkhorn plan is always at least the exact OT cost, and with large ε it can be far larger, because the plan spreads mass over expensive routes. For small ε the kernel entries $e^{-C_{ij}/\varepsilon}$ underflow to zero in floating point; practical codes work with $\log \mathbf{u}$ and $\log \mathbf{v}$ ("log domain stabilisation").

Machine Learning Connection

Optimal transport appears across modern machine learning: the **Wasserstein GAN** critic (Kantorovich Rubinstein dual); the **Fréchet Inception Distance** used to evaluate image generators, which is the closed form W_2 between Gaussian approximations; **domain adaptation**, which transports source features onto target features; **flow matching** and rectified flow generative models, which learn transport maps; word mover's distance between documents; and **label shift** analysis, which measures how far class proportions move between training and deployment. In each case the core computation is the LP (23.1) or its entropic version.

23.4 Worked Examples

Worked Example 23.1 An Exact Transport Plan

Distribution $\mathbf{a} = (0.5,\ 0.3,\ 0.2)$ and distribution $\mathbf{b} = (0.1,\ 0.3,\ 0.6)$ both live on the points $0, 1, 2$ with cost $C_{ij} = |i - j|$. Find an optimal plan and W_1.

Solution

Step 1. **Cost matrix.** $\mathbf{C} = \begin{pmatrix} 0 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 0 \end{pmatrix}$.

Step 2. **Keep what can stay.** Mass that does not need to move costs nothing: $P_{00} = \min(0.5, 0.1) = 0.1$, $P_{11} = \min(0.3, 0.3) = 0.3$, $P_{22} = \min(0.2, 0.6) = 0.2$.

Step 3. **Remaining.** Source 0 still holds 0.4; destination 2 still needs 0.4. Move it: $P_{02} = 0.4$ at cost 2 per unit.

Step 4. **Plan.**

$$\mathbf{P} = \begin{pmatrix} 0.1 & 0 & 0.4 \\ 0 & 0.3 & 0 \\ 0 & 0 & 0.2 \end{pmatrix}, \qquad \text{row sums } (0.5, 0.3, 0.2), \text{ column sums } (0.1, 0.3, 0.6).$$

Step 5. **Cost.** $0.4 \times 2 = 0.8$.

Step 6. **Alternative optimum.** Moving 0.3 from 0 to 1, 0.3 from 1 to 2 and 0.1 from 0 to 2 also costs $0.3 + 0.3 + 0.2 = 0.8$: with the cost $|i - j|$ on a line, a long jump costs the same as a chain of short ones, so optimal plans are not unique.

Answer. $\boxed{W_1(\mathbf{a}, \mathbf{b}) = 0.8}$, attained for example by the plan above.

Worked Example 23.2 W_1 from Distribution Functions

Verify Worked Example 23.1 with the formula (23.4).

Solution

Step 1. **Cumulative sums.** $F = (0.5,\ 0.8,\ 1.0)$ and $G = (0.1,\ 0.4,\ 1.0)$.

Step 2. **Gaps.** Between 0 and 1: $|F_0 - G_0| = 0.4$. Between 1 and 2: $|F_1 - G_1| = 0.4$.

Step 3. **Sum** with spacing 1: $0.4 + 0.4 = 0.8$.

Step 4. **Interpretation.** 0.4 units of mass must cross each of the two gaps from left to right, whatever the plan. That is a lower bound for every plan, and the plan of Worked Example 23.1 meets it.

Answer. $\boxed{W_1 = 0.4 + 0.4 = 0.8}$, matching the LP.

Worked Example 23.3 Certifying the Plan with Kantorovich Potentials

For Worked Example 23.1, show that $\mathbf{f} = (2, 1, 0)$ and $\mathbf{g} = (-2, -1, 0)$ solve the dual (23.2), and identify the 1 Lipschitz function of (23.3).

Solution

Step 1. **Feasibility** $f_i + g_j \le |i-j|$: the sums $f_i + g_j = (2-i) - (2-j) = j - i \le |i-j|$. Every constraint holds.

Step 2. **Tight routes.** $f_i + g_j = |i-j|$ exactly when $j \ge i$. The plan uses $(0,0)$, $(1,1)$, $(2,2)$ and $(0,2)$, all with $j \ge i$, so complementary slackness holds.

Step 3. **Dual value.** $\mathbf{a}^\top \mathbf{f} + \mathbf{b}^\top \mathbf{g} = (0.5 \cdot 2 + 0.3 \cdot 1 + 0) + (0.1 \cdot (-2) + 0.3 \cdot (-1) + 0) = 1.3 - 0.5 = 0.8$. Equal to the primal cost.

Step 4. **Lipschitz form.** $\varphi(x) = 2 - x$ has slope -1, so $|\varphi(x) - \varphi(x')| = |x - x'|$: it is 1 Lipschitz. Then $\sum a_i \varphi(i) = 1.3$ and $\sum b_j \varphi(j) = 0.5$, difference 0.8.

Step 5. **Meaning.** The best “critic” simply decreases at the maximum allowed rate, rewarding mass that sits to the left, where **a** has more of it than **b**.

Answer. $\boxed{\mathbf{f} = (2,1,0),\ \mathbf{g} = (-2,-1,0)}$ are optimal; the critic is $\varphi(x) = 2 - x$.

Worked Example 23.4 Measuring Label Shift between Training and Deployment

A classifier was trained on class proportions (cat, dog, car) = (0.5, 0.3, 0.2) and is deployed on (0.2, 0.4, 0.4). Confusing two animal classes costs 1, an animal with a car costs 4. Compute the transport distance and interpret it.

Solution

Step 1. **Cost matrix.** $\mathbf{C} = \begin{pmatrix} 0 & 1 & 4 \\ 1 & 0 & 4 \\ 4 & 4 & 0 \end{pmatrix}$ (rows and columns: cat, dog, car).

Step 2. **Surpluses and deficits.** Cat: $0.5 - 0.2 = 0.3$ surplus. Dog: $0.3 - 0.4 = -0.1$ deficit. Car: $0.2 - 0.4 = -0.2$ deficit.

Step 3. **Option A.** Cat to dog 0.1 (cost 0.1) and cat to car 0.2 (cost 0.8): total 0.9.

Step 4. **Option B.** Cat to dog 0.3 and dog to car 0.2: cost $0.3 + 0.8 = 1.1$.

Step 5. **Optimal.** Any plan must move 0.2 of animal mass to cars at cost at least 4 each, giving at least 0.8, plus at least 0.1 from cat to dog at cost at least 1: at least 0.9. Option A attains it.

Step 6. **Plan.** $P_{\text{cat,cat}} = 0.2$, $P_{\text{cat,dog}} = 0.1$, $P_{\text{cat,car}} = 0.2$, $P_{\text{dog,dog}} = 0.3$, $P_{\text{car,car}} = 0.2$.

Step 7. **Interpretation.** Most of the shift distance (0.8 of 0.9) comes from the semantically large move towards cars. A divergence that ignores geometry would treat cat to dog and cat to car shifts as equally severe.

Answer. Label shift distance $\boxed{0.9}$, dominated by the move of 0.2 mass from cats to cars.

Worked Example 23.5 One Sinkhorn Iteration

Let $\mathbf{a} = \mathbf{b} = (0.5, 0.5)$ on two points at distance 1, so $\mathbf{C} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. With $\varepsilon = 0.5$, run Sinkhorn from $\mathbf{v} = \mathbf{1}$ and compare the entropic plan with the exact plan.

Solution

Step 1. Kernel. $K_{ij} = e^{-C_{ij}/0.5}$: diagonal $e^0 = 1$, off diagonal $e^{-2} \approx 0.13534$.

Step 2. Update u. $\mathbf{Kv} = (1 + 0.13534,\ 0.13534 + 1) = (1.13534,\ 1.13534)$, so $u_i = 0.5/1.13534 \approx 0.44040$.

Step 3. Update v. $\mathbf{K}^\top\mathbf{u} = 0.44040 \times 1.13534 = 0.5$ in each entry, so $v_j = 0.5/0.5 = 1$. The iteration has already converged, by symmetry.

Step 4. Plan (23.6): $P_{ii} = 0.44040 \times 1 \times 1 = 0.44040$ and $P_{ij} = 0.44040 \times 0.13534 \approx 0.05960$ for $i \neq j$. Row and column sums are 0.5.

Step 5. Transport cost. $\sum C_{ij}P_{ij} = 2 \times 0.05960 = 0.11920$.

Step 6. Exact OT. The distributions are identical, so the exact plan is diagonal with cost 0. Entropy has spread 11.9% of the mass onto the costly routes. With $\varepsilon = 0.1$ the off diagonal share would be $e^{-10}/(1 + e^{-10}) \approx 0.0045\%$ of each row, almost exact.

Answer. $\boxed{\mathbf{P}_\varepsilon \approx \begin{pmatrix} 0.4404 & 0.0596 \\ 0.0596 & 0.4404 \end{pmatrix}}$ with cost 0.119, versus exact cost 0.

Worked Example 23.6 Matching Generated Samples to Real Samples

Three real data values are $\{0,\ 2,\ 5\}$ and three generated values are $\{1,\ 3,\ 4\}$, each with mass $\frac{1}{3}$. Find the optimal transport between the two empirical distributions for the costs $|x - z|$ and $(x - z)^2$, and compute W_1 and W_2.

Solution

Step 1. Structure. With equal masses $\frac{1}{3}$ on three points each, the transport polytope is the set of doubly stochastic matrices scaled by $\frac{1}{3}$. By the Birkhoff theorem (a consequence of total unimodularity, Module 17) its vertices are permutation matrices: an optimal plan is a one to one matching.

Step 2. Sorted matching. On the line, for any convex cost the optimal matching pairs the sorted lists in order: $0 \leftrightarrow 1$, $2 \leftrightarrow 3$, $5 \leftrightarrow 4$.

Step 3. Absolute cost. Each pair moves distance 1: $W_1 = \frac{1}{3}(1 + 1 + 1) = 1$.

Step 4. Squared cost. Each pair costs 1: $W_2^2 = \frac{1}{3}(1 + 1 + 1) = 1$, so $W_2 = 1$.

Step 5. A crossing matching for comparison. $0 \leftrightarrow 3$, $2 \leftrightarrow 1$, $5 \leftrightarrow 4$: absolute cost $\frac{1}{3}(3+1+1) = \frac{5}{3}$; squared cost $\frac{1}{3}(9 + 1 + 1) = \frac{11}{3}$. Both worse.

Step 6. Use in evaluation. Averaging such distances over many samples measures how well a generator reproduces the data distribution; the FID score applies the same idea to Gaussian summaries of deep features.

Answer. Optimal matching $\boxed{0 \to 1,\ 2 \to 3,\ 5 \to 4}$; $W_1 = 1$ and $W_2 = 1$.

Module Summary

- Kantorovich OT: $\min \sum C_{ij} P_{ij}$ subject to $\mathbf{P1} = \mathbf{a}$, $\mathbf{P}^\top \mathbf{1} = \mathbf{b}$, $\mathbf{P} \geq \mathbf{0}$, a transportation LP with sparse vertex plans.
- Dual: $\max\ \mathbf{a}^\top \mathbf{f} + \mathbf{b}^\top \mathbf{g}$ subject to $f_i + g_j \leq C_{ij}$; mass moves only on tight routes.
- For a distance cost, W_1 is the maximum of $\mathbb{E}_a \varphi - \mathbb{E}_b \varphi$ over 1 Lipschitz φ (the WGAN critic).
- On the line, W_1 is the area between the cumulative distribution functions.
- Entropic OT has solution $\mathrm{diag}(\mathbf{u}) \mathbf{K} \, \mathrm{diag}(\mathbf{v})$ with $K_{ij} = e^{-C_{ij}/\varepsilon}$, computed by Sinkhorn scaling; small ε approaches the exact plan.

MODULE 24

Zero Sum Games, Minimax and Adversarial Robustness

Learning Objectives

After studying this module you will be able to:

- model a two player zero sum game by a payoff matrix and explain mixed strategies;
- derive the linear programs of both players and prove the minimax theorem from LP duality;
- solve small games by formula, graphically and by LP;
- derive the worst case effect of an ℓ_∞ bounded perturbation on a linear classifier and train a robust classifier by LP;
- formulate group distributionally robust model selection as a game and solve it.

24.1 Learning against an Adversary

Many modern learning problems are contests. A spam filter competes with spammers; an image classifier must resist small, deliberately chosen pixel changes; a fair model must perform well on its *worst* demographic group; a generative adversarial network trains a generator against a discriminator. The mathematics of such contests is game theory, and for two players with opposed interests it reduces, remarkably, to a pair of dual linear programs. John von Neumann proved the minimax theorem in 1928; in 1947 he recognised, in a famous conversation with Dantzig, that it is the same mathematics as LP duality.

24.2 Matrix Games and Mixed Strategies

Definition Two Player Zero Sum Matrix Game

A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ defines a game: the row player chooses a row i, the column player simultaneously chooses a column j, and the column player pays the row player a_{ij} (negative entries mean the row player pays). A **mixed strategy** is a probability vector: $\mathbf{p} \in \Delta_m$ for the row player and $\mathbf{q} \in \Delta_n$ for the column player, where

$$\Delta_k = \{\mathbf{z} \in \mathbb{R}^k : \mathbf{z} \geq \mathbf{0},\ \mathbf{1}^\top \mathbf{z} = 1\}. \tag{24.1}$$

The expected payoff to the row player is

$$\mathbb{E}[\text{payoff}] = \sum_{i=1}^{m} \sum_{j=1}^{n} p_i a_{ij} q_j = \mathbf{p}^\top \mathbf{A} \mathbf{q}. \tag{24.2}$$

where

p_i is the probability that the row player chooses row i;

q_j is the probability that the column player chooses column j;

Δ_k is the probability simplex in $\mathbb{R}^k$.

Lucid Insight Why randomise?

In rock, paper, scissors any fixed choice is exploited at once. Randomising makes you unpredictable: whatever your opponent does, your expected result is the same. Mixed strategies are the mathematical form of not being predictable, and they are exactly what is needed to make the worst case as good as possible.

24.3 Each Player's Linear Program

Derivation The row player's problem

Step 1. The row player fears the worst. For a fixed $\mathbf{p}$, the column player picks the column that hurts most. Because $\mathbf{p}^\top \mathbf{A} \mathbf{q}$ is linear in $\mathbf{q}$, its minimum over the simplex is attained at a vertex, a single column (Module 3):

$$\min_{\mathbf{q} \in \Delta_n} \mathbf{p}^\top \mathbf{A} \mathbf{q} = \min_j (\mathbf{A}^\top \mathbf{p})_j = \min_j \sum_i p_i a_{ij}.$$

Step 2. The row player maximises this guaranteed amount: $\max_{\mathbf{p} \in \Delta_m} \min_j (\mathbf{A}^\top \mathbf{p})_j$, a maximin problem.

Step 3. **Epigraph trick** (Module 4): introduce v with $v \le (\mathbf{A}^\top \mathbf{p})_j$ for every j:

$$\max_{\mathbf{p}, v} \; v \quad \text{subject to} \quad \mathbf{A}^\top \mathbf{p} \ge v \mathbf{1}, \quad \mathbf{1}^\top \mathbf{p} = 1, \quad \mathbf{p} \ge \mathbf{0}. \tag{24.3}$$

Derivation The column player's problem is the dual

Step 1. Symmetrically, the column player solves $\min_{\mathbf{q} \in \Delta_n} \max_i (\mathbf{A}\mathbf{q})_i$, that is

$$\min_{\mathbf{q}, w} \; w \quad \text{subject to} \quad \mathbf{A}\mathbf{q} \le w \mathbf{1}, \quad \mathbf{1}^\top \mathbf{q} = 1, \quad \mathbf{q} \ge \mathbf{0}. \tag{24.4}$$

Step 2. Dualise (24.3) with the rules of Module 12. Write its constraints as $v\mathbf{1} - \mathbf{A}^\top \mathbf{p} \le \mathbf{0}$ (multipliers $q_j \ge 0$) and $\mathbf{1}^\top \mathbf{p} = 1$ (free multiplier w).

Step 3. The dual objective is $0 \cdot \mathbf{q} + 1 \cdot w = w$, minimised.

Step 4. The column of the free variable v has coefficient 1 in each of the first constraints and cost 1, so $\sum_j q_j = 1$.

Step 5. The column of $p_i \geq 0$ has entries $-a_{ij}$ and 1 and cost 0, giving $-\sum_j a_{ij} q_j + w \geq 0$, that is $(\mathbf{Aq})_i \leq w$.

Step 6. This is exactly (24.4). The two players' problems are a primal dual pair.

Key Result Von Neumann's minimax theorem

For every matrix $\mathbf{A}$,

$$\max_{\mathbf{p}\in\Delta_m} \min_{\mathbf{q}\in\Delta_n} \mathbf{p}^\top \mathbf{A}\mathbf{q} = \min_{\mathbf{q}\in\Delta_n} \max_{\mathbf{p}\in\Delta_m} \mathbf{p}^\top \mathbf{A}\mathbf{q} =: v^\star, \tag{24.5}$$

the **value** of the game. Proof: both LPs are feasible (any strategy with a sufficiently low v or high w works), so by strong duality (Module 12) their optimal values are equal. Optimal strategies $\mathbf{p}^\star, \mathbf{q}^\star$ form a **saddle point**: neither player can gain by deviating alone.

Derivation Closed form for 2×2 games without a saddle point in pure strategies

Let $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ and suppose neither player has a dominant pure strategy, so both mix.

Step 1. At the optimum the row player's mixture $(p, 1-p)$ makes the column player indifferent: $ap + c(1-p) = bp + d(1-p)$ (otherwise the column player would play only the better column).

Step 2. Rearranging, $p(a-b-c+d) = d-c$, so

$$p = \frac{d-c}{a-b-c+d}, \qquad q = \frac{d-b}{a-b-c+d}, \qquad v^\star = \frac{ad-bc}{a-b-c+d}, \tag{24.6}$$

where q follows from the symmetric indifference condition $aq + b(1-q) = cq + d(1-q)$, and $v^\star = ap + c(1-p)$ simplifies to the stated ratio.

24.4 Adversarial Robustness of Linear Classifiers

Derivation The worst case ℓ_∞ perturbation

An adversary may move a test point $\mathbf{x}$ to $\mathbf{x} + \boldsymbol{\delta}$ with $\|\boldsymbol{\delta}\|_\infty \leq \varepsilon$ (each feature changes by at most ε).

Step 1. The functional margin becomes $y(\mathbf{w}^\top(\mathbf{x}+\boldsymbol{\delta}) + b) = y(\mathbf{w}^\top \mathbf{x} + b) + y\mathbf{w}^\top\boldsymbol{\delta}$.

Step 2. The adversary minimises $y\mathbf{w}^\top\boldsymbol{\delta} = \sum_j y w_j \delta_j$ term by term: choose $\delta_j = -\varepsilon \operatorname{sign}(yw_j)$, giving $-\varepsilon \left|w_j\right|$ for each j.

Step 3. Therefore

$$\min_{\|\boldsymbol{\delta}\|_\infty \leq \varepsilon} y(\mathbf{w}^\top(\mathbf{x}+\boldsymbol{\delta}) + b) = y(\mathbf{w}^\top \mathbf{x} + b) - \varepsilon \|\mathbf{w}\|_1 . \tag{24.7}$$

where

δ is the adversarial perturbation; for neural networks this choice $\delta_j = -\varepsilon \operatorname{sign}(\cdot)$ is the *fast gradient sign method*;

ε is the perturbation budget.

Robust training requires $y_i(\mathbf{w}^\top \mathbf{x}_i + b) - \varepsilon \|\mathbf{w}\|_1 \geq 1$ for each training point. With $\|\mathbf{w}\|_1 \leq \mathbf{1}^\top \mathbf{t}$, $-\mathbf{t} \leq \mathbf{w} \leq \mathbf{t}$, these are linear constraints: robust training of a linear classifier is an LP.

24.5 Distributionally Robust Model Selection

Definition Group Distributionally Robust Mixture

Suppose K candidate models have error rates E_{kg} on G population groups. A randomised predictor that uses model k with probability q_k has expected error $\sum_k q_k E_{kg}$ on group g. The **group robust** mixture minimises the worst group error:

$$\min_{\mathbf{q} \in \Delta_K,\, t} \; t \quad \text{subject to} \quad \sum_{k=1}^{K} q_k E_{kg} \leq t, \quad g = 1, \ldots, G. \tag{24.8}$$

This is the column player's problem (24.4) for the game with payoff matrix $\mathbf{E}^\top$: an adversary ("nature") chooses the group, we choose the model. The dual variables are the adversary's worst case weights over groups.

Machine Learning Connection

Minimax and duality run through modern machine learning. **Adversarial training** solves $\min_\theta \mathbb{E} \max_{\|\delta\| \leq \varepsilon}$ loss; for linear models the inner maximum has the closed form (24.7). **Generative adversarial networks** are (nonconvex) two player games. **Group DRO** (Sagawa and colleagues, 2020) trains networks to minimise the worst group loss, using exactly the dual weights of (24.8) to reweight groups. **Boosting** can be viewed as a game between a booster choosing a distribution over examples and a weak learner choosing a hypothesis; LPBoost solves its LP directly. Multiplicative weights algorithms, which solve these games approximately, are among the most widely used ideas in learning theory.

24.6 Worked Examples

Worked Example 24.1 Rock, Paper, Scissors

The row player's payoff matrix (rows and columns: rock, paper, scissors) is

$$\mathbf{A} = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}.$$

Find the value and optimal strategies.

Solution

Step 1. **Row LP** (24.3)**.** Maximise v subject to $\mathbf{A}^\top \mathbf{p} \geq v\mathbf{1}$:

$$p_2 - p_3 \geq v, \qquad -p_1 + p_3 \geq v, \qquad p_1 - p_2 \geq v, \qquad p_1 + p_2 + p_3 = 1.$$

Step 2. **Upper bound on v.** Adding the three inequalities: $0 \geq 3v$, so $v \leq 0$ for every strategy.

Step 3. **Attain it.** $\mathbf{p} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ makes each left side 0, so $v = 0$ is achieved.

Step 4. **Column player.** By symmetry ($\mathbf{A}^\top = -\mathbf{A}$, a symmetric game) $\mathbf{q} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ gives $\mathbf{Aq} = \mathbf{0} \leq 0 \cdot \mathbf{1}$.

Step 5. **Uniqueness.** If $v = 0$, the three inequalities $p_2 \geq p_3$, $p_3 \geq p_1$, $p_1 \geq p_2$ force $p_1 = p_2 = p_3$.

Answer. $\boxed{v^\star = 0,\ \mathbf{p}^\star = \mathbf{q}^\star = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})}$: the game is fair and only uniform play is safe.

Worked Example 24.2 A 2 × 2 Game by Formula and by LP

Solve the game $\mathbf{A} = \begin{pmatrix} 3 & -1 \\ -2 & 4 \end{pmatrix}$.

Solution

Step 1. **No pure saddle point.** Row minima are -1 and -2 (best guaranteed pure payoff -1); column maxima are 3 and 4 (best guaranteed pure loss 3). Since $-1 \neq 3$, both players must mix.

Step 2. **Formula** (24.6) with $a = 3$, $b = -1$, $c = -2$, $d = 4$: denominator $3 + 1 + 2 + 4 = 10$.

$$p = \frac{4 - (-2)}{10} = 0.6, \qquad q = \frac{4 - (-1)}{10} = 0.5, \qquad v^\star = \frac{12 - 2}{10} = 1.$$

Step 3. **Check indifference.** Against column 1 the row strategy $(0.6, 0.4)$ earns $1.8 - 0.8 = 1$; against column 2, $-0.6 + 1.6 = 1$. Against row 1 the column strategy $(0.5, 0.5)$ pays $1.5 - 0.5 = 1$; against row 2, $-1 + 2 = 1$.

Step 4. **LP certificate.** $\mathbf{p}^\star$ is feasible for (24.3) with $v = 1$, and $\mathbf{q}^\star$ is feasible for (24.4) with $w = 1$. Equal objectives prove optimality of both (weak duality).

Answer. $\boxed{\mathbf{p}^\star = (0.6, 0.4),\ \mathbf{q}^\star = (0.5, 0.5),\ v^\star = 1}$.

Worked Example 24.3 A 2 × 3 Game Solved Graphically

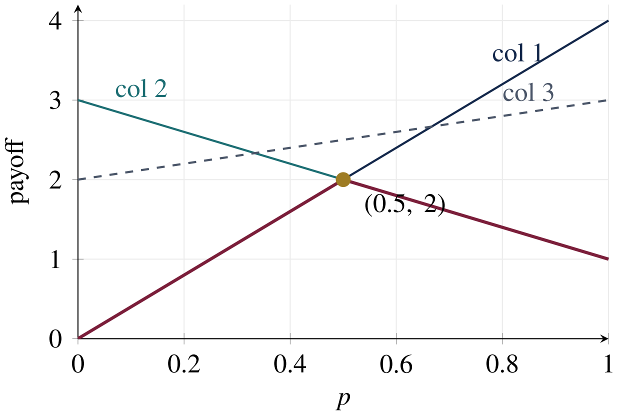
Solve $\mathbf{A} = \begin{pmatrix} 4 & 1 & 3 \\ 0 & 3 & 2 \end{pmatrix}$.

Solution

Step 1. **Row strategy** $(p, 1 - p)$. Expected payoff against each column:

$$\text{col 1: } 4p, \qquad \text{col 2: } p + 3(1 - p) = 3 - 2p, \qquad \text{col 3: } 3p + 2(1 - p) = 2 + p.$$

Step 2. Lower envelope. The row player's guarantee is $g(p) = \min(4p,\ 3-2p,\ 2+p)$, drawn below.



Step 3. Maximise the envelope. For $p \le 0.5$ the minimum is $4p$; for $p \ge 0.5$ it is $3-2p$; column 3 lies above both near the peak. The peak is where $4p = 3-2p$: $p = 0.5$, $v^\star = 2$.

Step 4. Column strategy. Only the active columns 1 and 2 are used ($q_3 = 0$, because at $p = 0.5$ column 3 gives $2.5 > 2$: complementary slackness). Make the row player indifferent: $4q_1 + 1q_2 = 0q_1 + 3q_2$ with $q_1 + q_2 = 1$, so $4q_1 = 2q_2$, giving $q_1 = \frac{1}{3}$, $q_2 = \frac{2}{3}$. Value check: $4(\frac{1}{3}) + \frac{2}{3} = 2$.

Answer. $\boxed{\mathbf{p}^\star = (0.5, 0.5),\ \mathbf{q}^\star = (\tfrac{1}{3}, \tfrac{2}{3}, 0),\ v^\star = 2}$.

Worked Example 24.4 Group Robust Model Selection

Three models have the following error rates on three population groups:

	Group 1	Group 2	Group 3
Model A	0.10	0.30	0.20
Model B	0.25	0.10	0.30
Model C	0.30	0.25	0.05

Find the randomised mixture of models that minimises the worst group error, and compare with the best single model.

Solution

Step 1. Single models. Worst group errors: A 0.30, B 0.30, C 0.30. Every single model is poor on some group.

Step 2. LP (24.8). Minimise t subject to

$$\begin{aligned} 0.10q_A + 0.25q_B + 0.30q_C &\le t, \\ 0.30q_A + 0.10q_B + 0.25q_C &\le t, \qquad q_A + q_B + q_C = 1,\ \mathbf{q} \ge \mathbf{0}. \\ 0.20q_A + 0.30q_B + 0.05q_C &\le t, \end{aligned}$$

Step 3. **Equalise the groups.** Each model is best on a different group, so at the optimum all three constraints should be tight. Subtract the first from the second: $0.20q_A - 0.15q_B - 0.05q_C = 0$, i.e. $4q_A - 3q_B - q_C = 0$. Subtract the first from the third: $0.10q_A + 0.05q_B - 0.25q_C = 0$, i.e. $2q_A + q_B - 5q_C = 0$.

Step 4. **Solve.** From the second relation $q_B = 5q_C - 2q_A$. Substitute into the first: $4q_A - 15q_C + 6q_A - q_C = 0$, so $q_A = 1.6q_C$, and then $q_B = 5q_C - 3.2q_C = 1.8q_C$. Normalise: $q_C(1.6 + 1.8 + 1) = 1$, so $q_C = \frac{1}{4.4} = \frac{5}{22}$, $q_A = \frac{8}{22} = \frac{4}{11}$, $q_B = \frac{9}{22}$.

Step 5. **Worst group error.** Group 1: $0.10 \cdot \frac{8}{22} + 0.25 \cdot \frac{9}{22} + 0.30 \cdot \frac{5}{22} = \frac{0.8+2.25+1.5}{22} = \frac{4.55}{22} \approx 0.2068$. Groups 2 and 3 give the same value by construction.

Step 6. **Dual check.** The adversary's group weights $\boldsymbol{\lambda} = (\frac{7}{22}, \frac{17}{44}, \frac{13}{44})$ make every model equally bad: for model A, $0.10 \cdot \frac{14}{44} + 0.30 \cdot \frac{17}{44} + 0.20 \cdot \frac{13}{44} = \frac{1.4+5.1+2.6}{44} = \frac{9.1}{44} \approx 0.2068$, and similarly for B and C. Equal primal and dual values certify optimality.

Answer. Use A, B, C with probabilities $\boxed{(\frac{4}{11}, \frac{9}{22}, \frac{5}{22})}$: worst group error ≈ 0.207, down from 0.30.

Worked Example 24.5 Training a Robust Linear Classifier

For the data of Worked Example 21.1 (negatives $(0,0)$, $(1,2)$; positives $(3,1)$, $(4,3)$), find the minimum ℓ_1 norm classifier whose training margins survive every perturbation with $\|\delta\|_\infty \le \varepsilon$, for $\varepsilon = 0.25$ and $\varepsilon = 0.5$. What happens as $\varepsilon \to 1$?

Solution

Step 1. **Robust constraints** from (24.7): $y_i(\mathbf{w}^\top \mathbf{x}_i + b) - \varepsilon \|\mathbf{w}\|_1 \ge 1$.

Step 2. **Try the same direction** $\mathbf{w} = (s, 0)$ with $s > 0$, $b = -2s$ (boundary $x_1 = 2$). Margins $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ become $2s$, s, s, $2s$; the binding points are $(1,2)$ and $(3,1)$, which need $s - \varepsilon s \ge 1$, so $s \ge \dfrac{1}{1-\varepsilon}$.

Step 3. **Optimality.** Adding the robust constraints of $(1,2)$ and $(3,1)$ gives $2w_1 - w_2 - 2\varepsilon \|\mathbf{w}\|_1 \ge 2$. Since $2w_1 - w_2 \le 2\|\mathbf{w}\|_1$, this implies $2(1-\varepsilon)\|\mathbf{w}\|_1 \ge 2$, so every robust classifier has $\|\mathbf{w}\|_1 \ge \dfrac{1}{1-\varepsilon}$, which the choice above attains.

Step 4. $\varepsilon = 0.25$**:** $s = \frac{4}{3}$, $\mathbf{w} = (\frac{4}{3}, 0)$, $b = -\frac{8}{3}$, $\|\mathbf{w}\|_1 = \frac{4}{3}$.

Step 5. $\varepsilon = 0.5$**:** $s = 2$, $\mathbf{w} = (2, 0)$, $b = -4$, $\|\mathbf{w}\|_1 = 2$.

Step 6. $\varepsilon \to 1$**.** The required norm $1/(1-\varepsilon)$ blows up, and for $\varepsilon \ge 1$ the LP is infeasible: the two closest points are at ℓ_∞ distance exactly 1 from the boundary (Worked Example 21.2), so a perturbation of size 1 in every coordinate can push them onto it. Robustness beyond the data's intrinsic ℓ_∞ margin is impossible.

Answer. $\boxed{\varepsilon = 0.25:\ \mathbf{w} = (\tfrac{4}{3}, 0);\quad \varepsilon = 0.5:\ \mathbf{w} = (2,0)}$; infeasible for $\varepsilon \ge 1$.

Worked Example 24.6 Saddle Point Verification

For the game of Worked Example 24.2, verify the saddle point inequalities $\mathbf{p}^\top\mathbf{A}\mathbf{q}^\star \le \mathbf{p}^{\star\top}\mathbf{A}\mathbf{q}^\star \le \mathbf{p}^{\star\top}\mathbf{A}\mathbf{q}$ for all mixed strategies, and compute what the row player loses by playing the pure strategy row 1 against an informed opponent.

Solution

Step 1. **Column player fixed at $\mathbf{q}^\star = (0.5, 0.5)$.** $\mathbf{A}\mathbf{q}^\star = (1.5 - 0.5,\ -1 + 2) = (1, 1)$. For any $\mathbf{p}$, $\mathbf{p}^\top\mathbf{A}\mathbf{q}^\star = p_1 + p_2 = 1$. So the left inequality holds with equality: the row player cannot do better than 1.

Step 2. **Row player fixed at $\mathbf{p}^\star = (0.6, 0.4)$.** $\mathbf{A}^\top\mathbf{p}^\star = (1.8 - 0.8,\ -0.6 + 1.6) = (1, 1)$. For any $\mathbf{q}$, $\mathbf{p}^{\star\top}\mathbf{A}\mathbf{q} = 1$. The right inequality holds with equality: the column player cannot pay less than 1.

Step 3. **Saddle value.** $\mathbf{p}^{\star\top}\mathbf{A}\mathbf{q}^\star = 1$.

Step 4. **Pure row 1 against an informed opponent.** The column player answers with column 2, paying -1: the row player loses 1 per round instead of winning 1. The cost of predictability is 2 per round.

Answer. Both inequalities hold with value $\boxed{1}$; predictable play loses 2 per round relative to the optimal mixture.

Module Summary

- A zero sum game with payoff matrix $\mathbf{A}$ has expected payoff $\mathbf{p}^\top\mathbf{A}\mathbf{q}$ for mixed strategies $\mathbf{p}, \mathbf{q}$.
- Row LP: $\max v$ subject to $\mathbf{A}^\top\mathbf{p} \ge v\mathbf{1}$, $\mathbf{p} \in \Delta_m$. Column LP: $\min w$ subject to $\mathbf{A}\mathbf{q} \le w\mathbf{1}$, $\mathbf{q} \in \Delta_n$. They are duals.
- Minimax theorem: $\max_{\mathbf{p}} \min_{\mathbf{q}} \mathbf{p}^\top\mathbf{A}\mathbf{q} = \min_{\mathbf{q}} \max_{\mathbf{p}} \mathbf{p}^\top\mathbf{A}\mathbf{q}$, by strong duality.
- For 2×2 games without pure saddle points: $p = (d-c)/(a-b-c+d)$, $v = (ad-bc)/(a-b-c+d)$.
- Worst ℓ_∞ attack on a linear classifier lowers the margin by $\varepsilon\,\|\mathbf{w}\|_1$; robust training is an LP.
- Group robust mixtures equalise the worst groups; their dual is an adversarial weighting of groups.

MODULE 25

Markov Decision Processes and Reinforcement Learning by Linear Programming

Learning Objectives

After studying this module you will be able to:

- define a finite Markov decision process (MDP), a policy and its discounted value;
- evaluate a fixed policy by solving a linear system, and derive the Bellman optimality equation;
- derive the primal linear program whose solution is the optimal value function;
- derive the dual linear program over discounted state action occupancy measures and extract an optimal policy from it;
- solve constrained MDPs, where safety budgets lead to randomised optimal policies;
- relate the LP view to value iteration and to modern reinforcement learning methods.

25.1 Sequential Decisions

Every module so far has made one decision at one time. Reinforcement learning (RL) makes a sequence of decisions, each changing the situation in which the next is made: a robot moves, a recommender shows items, a trading agent buys and sells. Surprisingly, the problem of finding the best long run strategy in a finite Markov decision process is itself a linear program, first observed by Manne (1960) and d'Epenoux (1963). This final module brings together almost every idea in the book: linear systems (Module 2), formulation (Module 4), duality (Module 12), complementary slackness (Module 13) and the interpretation of dual variables as flows (Modules 18 and 23).

25.2 Markov Decision Processes

Definition Finite Discounted MDP

An MDP consists of

- a finite set of **states** $\mathcal{S}$ and, for each state s, a finite set of **actions** $\mathcal{A}(s)$;
- **rewards** $r(s, a)$ received when taking action a in state s;
- **transition probabilities** $P(s' \mid s, a)$ of moving to state s', with $\sum_{s'} P(s' \mid s, a) = 1$;
- a **discount factor** $\gamma \in [0, 1)$ that makes a reward received t steps later worth γ^t times as much;
- an **initial distribution** $\mu(s) > 0$ over starting states.

A (stationary) **policy** π chooses an action in each state, possibly at random: $\pi(a \mid s)$ is the probability of action a in state s.

Derivation The value of a policy solves a linear system

Step 1. Define the value $V^\pi(s)$ as the expected discounted sum of rewards starting from s and following π: $V^\pi(s) = \mathbb{E}\big[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \,\big|\, s_0 = s\big]$.

Step 2. Split off the first step. The first reward is $r(s, a)$; afterwards the process restarts from s', discounted by γ. For a deterministic policy $a = \pi(s)$:

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) V^\pi(s').$$

Step 3. In vector form with $\mathbf{r}_\pi(s) = r(s, \pi(s))$ and $(\mathbf{P}_\pi)_{ss'} = P(s' \mid s, \pi(s))$:

$$\mathbf{v}^\pi = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}^\pi \qquad \Longrightarrow \qquad (\mathbf{I} - \gamma \mathbf{P}_\pi)\mathbf{v}^\pi = \mathbf{r}_\pi. \tag{25.1}$$

Step 4. The matrix $\mathbf{I} - \gamma \mathbf{P}_\pi$ is invertible because $\gamma < 1$ and $\mathbf{P}_\pi$ has rows summing to one (its eigenvalues have size at most one), so the value is unique: $\mathbf{v}^\pi = (\mathbf{I} - \gamma \mathbf{P}_\pi)^{-1} \mathbf{r}_\pi$.

Key Result Bellman optimality equation

The optimal value $V^\star(s) = \max_\pi V^\pi(s)$ satisfies

$$V^\star(s) = \max_{a \in \mathcal{A}(s)} \Big(r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\star(s') \Big) \quad \text{for every } s, \tag{25.2}$$

and any policy that chooses a maximising action in each state (a **greedy** policy) is optimal.

25.3 The Primal Linear Program

Derivation From the Bellman equation to an LP

Step 1. Replace the maximum in (25.2) by a family of inequalities: $V(s) \geq r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)V(s')$ for every state and every action. A function satisfying all of them is called **superharmonic** (an upper bound on what any action can achieve).

Step 2. **Every superharmonic V dominates $V^\star$.** For any policy π, the inequalities for the actions of π give $\mathbf{v} \geq \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}$. Substituting this inequality into itself repeatedly (allowed because $\mathbf{P}_\pi$ has nonnegative entries) gives $\mathbf{v} \geq \sum_{t=0}^{k} \gamma^t \mathbf{P}_\pi^t \mathbf{r}_\pi + \gamma^{k+1}\mathbf{P}_\pi^{k+1}\mathbf{v}$, and letting $k \to \infty$ gives $\mathbf{v} \geq \mathbf{v}^\pi$. Hence $\mathbf{v} \geq \mathbf{v}^\star$.

Step 3. $V^\star$ itself is superharmonic, by (25.2).

Step 4. So $V^\star$ is the smallest superharmonic function, and for any weights $\mu(s) > 0$ it solves

$$\min_{\mathbf{v}} \sum_s \mu(s)V(s) \quad \text{subject to} \quad V(s) - \gamma \sum_{s'} P(s' \mid s,a)V(s') \ \geq\ r(s,a) \quad \forall s,\ \forall a \in \mathcal{A}(s). \tag{25.3}$$

where

$V(s)$ are free variables, one per state;

one constraint per state action pair, so the LP has $|\mathcal{S}|$ variables and $\sum_s |\mathcal{A}(s)|$ constraints.

25.4 The Dual: Occupancy Measures

Derivation The dual LP

Step 1. Attach a multiplier $x(s,a) \geq 0$ to each constraint of (25.3).

Step 2. The dual objective is max $\sum_{s,a} r(s,a)x(s,a)$.

Step 3. The column of the free variable $V(s')$ has entry 1 in constraints whose state is s' and entry $-\gamma P(s' \mid s,a)$ in every constraint (s,a); its cost is $\mu(s')$. Hence an equation:

$$\sum_a x(s',a) - \gamma \sum_{s,a} P(s' \mid s,a)\, x(s,a) = \mu(s') \quad \text{for every } s'. \tag{25.4}$$

Step 4. The dual is

$$\max_{\mathbf{x} \geq \mathbf{0}} \sum_{s,a} r(s,a)x(s,a) \quad \text{subject to} \quad (25.4). \tag{25.5}$$

Key Result Interpretation and policy extraction

- $x(s,a) = \mathbb{E}[\sum_t \gamma^t \mathbf{1}\{s_t = s, a_t = a\}]$ is the **discounted occupancy measure**: how much (discounted) time the process spends taking action a in state s when started from μ.
- Constraint (25.4) is flow conservation (Module 18): discounted time spent in s' equals the initial mass there plus the discounted inflow from all states.

- Summing (25.4) over s' gives $\sum_{s,a} x(s,a) = \dfrac{1}{1-\gamma}$.
- An optimal policy is recovered by $\pi(a \mid s) = x(s,a) / \sum_{a'} x(s,a')$.
- Complementary slackness: $x(s,a) > 0$ only for actions that attain the maximum in the Bellman equation.

Lucid Insight Why the dual is so useful

The primal works with values; the dual works with *behaviour*. Constraints on behaviour, such as "spend at most this much discounted time in risky states", are linear in $\mathbf{x}$, so they can be added to (25.5) directly. The result is a **constrained MDP**, and its optimal policy may need to randomise, exactly as in the games of Module 24.

Common Pitfall

The LP has one variable per state, so it is practical only for small or moderately sized state spaces. Large problems (games, robotics) use approximate methods: *approximate linear programming* restricts V to a linear combination of features, $V \approx \boldsymbol{\Phi}\boldsymbol{\theta}$, which keeps an LP with few variables but many constraints; deep RL replaces the LP by stochastic gradient methods on related objectives.

Machine Learning Connection

The LP view underlies several modern RL ideas. **Occupancy measures** are the variables of imitation learning methods such as GAIL, which match the expert's occupancy measure. Offline RL methods of the "DICE" family (DualDICE, GenDICE, AlgaeDICE) estimate ratios of occupancy measures by solving regularised versions of (25.5). **Constrained policy optimisation** for safe RL is built on the Lagrangian of the constrained dual. And value iteration, the basis of Q learning and deep Q networks, is a fixed point method for the same Bellman equation that (25.3) solves exactly.

25.5 Worked Examples

The examples use one MDP. State s_1 is "preparing" and s_2 is "producing"; $\gamma = 0.9$; $\mu = (0.5, 0.5)$.

State	**Action**	$r(s,a)$	**Transitions**
s_1	stay	1	stay in s_1
s_1	move	0	to s_2 with prob. 0.8, stay in s_1 with prob. 0.2
s_2	safe	2	stay in s_2
s_2	risky	4	back to s_1 with prob. 0.5, stay in s_2 with prob. 0.5

Worked Example 25.1 Evaluating Fixed Policies

Compute the value of each of the four deterministic policies by solving (25.1).

Solution

Step 1. **(stay, safe).** Each state keeps itself: $V_1 = 1 + 0.9V_1$ so $V_1 = 10$; $V_2 = 2 + 0.9V_2$ so $V_2 = 20$.

Step 2. **(move, safe).** $V_2 = 20$ as before. $V_1 = 0 + 0.9(0.2V_1 + 0.8 \cdot 20) = 0.18V_1 + 14.4$, so $0.82V_1 = 14.4$ and $V_1 = \frac{14.4}{0.82} \approx 17.561$.

Step 3. **(stay, risky).** $V_1 = 10$. $V_2 = 4 + 0.9(0.5 \cdot 10 + 0.5V_2) = 8.5 + 0.45V_2$, so $V_2 = \frac{8.5}{0.55} \approx 15.455$.

Step 4. **(move, risky).** Two coupled equations:

$$V_1 = 0.18V_1 + 0.72V_2, \qquad V_2 = 4 + 0.45V_1 + 0.45V_2.$$

From the first, $0.82V_1 = 0.72V_2$, so $V_1 = \frac{36}{41}V_2$. Substitute into the second: $0.55V_2 = 4 + 0.45 \cdot \frac{36}{41}V_2$, so $V_2(0.55 - \frac{16.2}{41}) = 4$, that is $V_2 \cdot \frac{6.35}{41} = 4$, giving $V_2 = \frac{164}{6.35} = \frac{3280}{127} \approx 25.827$ and $V_1 = \frac{2880}{127} \approx 22.677$.

Step 5. **Summary.**

Policy	$V(s_1)$	$V(s_2)$
(stay, safe)	10	20
(move, safe)	17.561	20
(stay, risky)	10	15.455
(move, risky)	22.677	25.827

Answer. (move, risky) has the largest value in both states: $\boxed{V \approx (22.677,\ 25.827)}$.

Worked Example 25.2 The Optimal Value by the Primal LP

Write the primal LP (25.3) for the MDP and show that $\mathbf{v}^\star = (\frac{2880}{127}, \frac{3280}{127})$ is optimal.

Solution

Step 1. **Constraints**, one per state action pair:

$$\begin{aligned}
(s_1, \text{stay}) &: \quad V_1 - 0.9V_1 \geq 1 && \Leftrightarrow \ 0.1V_1 \geq 1,\\
(s_1, \text{move}) &: \quad V_1 - 0.9(0.2V_1 + 0.8V_2) \geq 0 && \Leftrightarrow \ 0.82V_1 - 0.72V_2 \geq 0,\\
(s_2, \text{safe}) &: \quad V_2 - 0.9V_2 \geq 2 && \Leftrightarrow \ 0.1V_2 \geq 2,\\
(s_2, \text{risky}) &: \quad V_2 - 0.9(0.5V_1 + 0.5V_2) \geq 4 && \Leftrightarrow \ -0.45V_1 + 0.55V_2 \geq 4.
\end{aligned}$$

Objective: minimise $0.5V_1 + 0.5V_2$.

Step 2. **Feasibility of $\mathbf{v}^\star \approx (22.677,\ 25.827)$.** Stay: $2.268 \geq 1$. Move: $18.595 - 18.595 = 0 \geq 0$ (tight). Safe: $2.583 \geq 2$. Risky: $-10.205 + 14.205 = 4 \geq 4$ (tight).

Step 3. **Bellman check.** In s_1: stay gives $1 + 0.9(22.677) = 21.409$; move gives $0.9(0.2 \cdot 22.677 + 0.8 \cdot 25.827) = 22.677$. Maximum $22.677 = V_1$. In s_2: safe gives $2 + 0.9(25.827) = 25.244$; risky gives $4 + 0.45(22.677 + 25.827) = 25.827$. Maximum $25.827 = V_2$. The Bellman equation holds, so $\mathbf{v}^\star$ is optimal.

Step 4. **Objective.** $0.5(22.677 + 25.827) = 24.252$.

Answer. $\boxed{V^\star(s_1) \approx 22.677,\ V^\star(s_2) \approx 25.827}$, LP value ≈ 24.252; the tight constraints are move and risky.

Worked Example 25.3 The Dual Occupancy Measure and the Optimal Policy

Solve the dual LP (25.5) using complementary slackness, verify that its value matches the primal, and extract the optimal policy.

Solution

Step 1. **Complementary slackness.** Only tight primal constraints can have positive x: $x(s_1, \text{stay}) = x(s_2, \text{safe}) = 0$. Write $x_m = x(s_1, \text{move})$ and $x_r = x(s_2, \text{risky})$.

Step 2. **Flow equations** (25.4)**.** For s_1: inflow comes from move staying behind (prob. 0.2) and risky returning (prob. 0.5):

$$x_m - 0.9(0.2x_m + 0.5x_r) = 0.5 \quad\Longleftrightarrow\quad 0.82x_m - 0.45x_r = 0.5.$$

For s_2: inflow from move succeeding (prob. 0.8) and risky staying (prob. 0.5):

$$x_r - 0.9(0.8x_m + 0.5x_r) = 0.5 \quad\Longleftrightarrow\quad -0.72x_m + 0.55x_r = 0.5.$$

Step 3. **Solve.** Add the equations: $0.10x_m + 0.10x_r = 1$, so $x_m + x_r = 10 = \frac{1}{1-\gamma}$, as predicted. Substitute $x_r = 10 - x_m$ into the first: $0.82x_m - 4.5 + 0.45x_m = 0.5$, so $1.27x_m = 5$, $x_m = \frac{500}{127} \approx 3.937$ and $x_r = \frac{770}{127} \approx 6.063$.

Step 4. **Dual value.** $0 \cdot x_m + 4x_r = \frac{3080}{127} \approx 24.252$, equal to the primal value. Both are optimal.

Step 5. **Policy.** In s_1 all occupancy is on move; in s_2 all on risky: $\pi^\star = (\text{move}, \text{risky})$, deterministic.

Step 6. **Interpretation.** Starting half the time in each state, the discounted process spends about 3.9 units of time preparing and 6.1 producing with the risky action.

Answer. $\boxed{x(s_1, \text{move}) = \frac{500}{127},\ x(s_2, \text{risky}) = \frac{770}{127}}$; optimal policy (move, risky).

Worked Example 25.4 Value Iteration Approaches the LP Solution

Run four steps of value iteration $V_{k+1}(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V_k(s')\right)$ from $V_0 = (0,0)$ and compare with $V^\star$.

Solution

Step 1. **Step 1.** s_1: $\max(1 + 0,\ 0) = 1$. s_2: $\max(2,\ 4) = 4$. $V_1 = (1,\ 4)$.

Step 2. **Step 2.** s_1: stay $1 + 0.9 = 1.9$; move $0.9(0.2 + 3.2) = 3.06$. s_2: safe $2 + 3.6 = 5.6$; risky $4 + 0.45(1 + 4) = 6.25$. $V_2 = (3.06,\ 6.25)$.

Step 3. **Step 3.** s_1: stay 3.754; move $0.9(0.612 + 5) = 5.0508$. s_2: safe 7.625; risky $4 + 0.45(9.31) = 8.1895$. $V_3 = (5.0508,\ 8.1895)$.

Step 4. **Step 4.** s_1: move $0.9(1.01016 + 6.5516) = 6.8056$. s_2: risky $4 + 0.45(13.2403) = 9.9581$. $V_4 \approx (6.806,\ 9.958)$.

Step 5. **Comparison.** The greedy actions are already (move, risky) from step 2 on, but the values are still far from $V^\star \approx (22.68,\ 25.83)$. The error shrinks by about the factor $\gamma = 0.9$ per step, so roughly $\ln(10^{-3})/\ln 0.9 \approx 66$ more steps are needed for three digit accuracy. The LP gives the exact answer at once, at the price of solving a linear program.

Answer. $\boxed{V_4 \approx (6.806,\ 9.958)}$, converging slowly towards $V^\star$; the greedy policy is correct early.

Worked Example 25.5 A Safety Constrained MDP

The risky action wears out equipment. Management requires the discounted number of risky actions to be at most 3: $x(s_2, \text{risky}) \le 3$. Solve the constrained dual LP and describe the optimal policy.

Solution

Step 1. **Model.** Maximise $x(s_1, \text{stay}) + 2x(s_2, \text{safe}) + 4x(s_2, \text{risky})$ subject to the two flow equations of Worked Example 25.3 (now with all four variables) and $x(s_2, \text{risky}) \le 3$.

Step 2. **The cap binds.** Unconstrained, $x_r = 6.06 > 3$, so set $x_r = 3$. Preparing is best done by moving, so keep $x(s_1, \text{stay}) = 0$, and let x_m and $x_f = x(s_2, \text{safe})$ be unknown.

Step 3. **Flow at s_1.** $x_m - 0.9(0.2x_m + 0.5 \cdot 3) = 0.5$, so $0.82x_m = 0.5 + 1.35 = 1.85$ and $x_m = \frac{185}{82} \approx 2.256$.

Step 4. **Flow at s_2.** $x_f + 3 - 0.9(0.8x_m + x_f + 1.5) = 0.5$, so $0.1x_f = 0.5 - 3 + 1.35 + 0.72x_m = -1.15 + 1.6244 = 0.4744$, and $x_f \approx 4.744$.

Step 5. **Check the total.** $2.256 + 4.744 + 3 = 10 = \frac{1}{1-\gamma}$.

Step 6. **Value.** $2(4.744) + 4(3) = 9.488 + 12 = 21.488$, down from 24.252.

Step 7. **Policy.** In s_2 the occupancy is split: $\pi(\text{risky} \mid s_2) = \frac{3}{3+4.744} \approx 0.387$ and $\pi(\text{safe} \mid s_2) \approx 0.613$. In s_1 always move. The optimal constrained policy is *randomised*, something no deterministic policy can match: (move, safe) gives only $0.5(17.561 + 20) = 18.78$.

Answer. Optimal constrained policy: move in s_1; in s_2 play risky with probability $\boxed{\approx 0.387}$; value ≈ 21.49.

Worked Example 25.6 A Deterministic MDP Is a Shortest Path Problem

A robot on a line of three cells $1, 2, 3$ must reach cell 3 (absorbing, reward 0 forever). In cells 1 and 2 it may step right (reward -1) or jump two cells right (reward -3, available only from cell 1). With $\gamma = 1$ for this finite horizon task, write the primal LP, solve it, and relate it to Module 18.

Solution

Step 1. **Primal LP** (minimise $V_1 + V_2$ with $V_3 = 0$):

$$(1, \text{step}) : V_1 \geq -1 + V_2, \qquad (1, \text{jump}) : \ V_1 \geq -3 + V_3 = -3,$$
$$(2, \text{step}) : V_2 \geq -1 + V_3 = -1.$$

Step 2. **Smallest feasible values.** $V_2 = -1$; then $V_1 \geq \max(-1 + V_2,\ -3) = \max(-2, -3) = -2$, so $V_1 = -2$.

Step 3. **Optimal policy.** In cell 1, step (gives -2) beats jump (gives -3); in cell 2, step. Path $1 \to 2 \to 3$ with total reward -2.

Step 4. **Link to shortest paths.** With costs 1 for a step and 3 for a jump, $-V(s)$ is the shortest distance from s to the goal: the primal LP is the dual of the shortest path LP of Worked Example 18.2, and the values are node potentials.

Answer. $\boxed{V^\star = (-2,\ -1,\ 0)}$; policy: always step; the MDP LP reproduces shortest path potentials.

Module Summary

- A fixed policy's value solves $(\mathbf{I} - \gamma \mathbf{P}_\pi)\mathbf{v} = \mathbf{r}_\pi$.
- The optimal value is the smallest superharmonic function: $\min \sum_s \mu(s)V(s)$ subject to $V(s) \geq r(s, a) + \gamma \sum_{s'} P(s' \mid s, a)V(s')$.
- The dual variables are discounted occupancy measures, satisfying flow conservation and summing to $1/(1 - \gamma)$; the policy is $\pi(a \mid s) \propto x(s, a)$.
- Complementary slackness puts occupancy only on Bellman optimal actions.
- Linear behaviour constraints give constrained MDPs, whose optimal policies may randomise.
- Value iteration converges geometrically at rate γ; the LP gives the exact fixed point.

Appendix A Formula Reference

This appendix collects the central results of the manual on a few pages, for revision and quick reference. The module in which each result is derived is shown in the last column.

Topic	**Result**	**Module**
Canonical LP	$\max\ \mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$	1
Standard LP	$\min\ \mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} = \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$, $\mathbf{b} \ge \mathbf{0}$	6
2×2 inverse	$\begin{pmatrix} p & q \\ r & s \end{pmatrix}^{-1} = \dfrac{1}{ps - qr}\begin{pmatrix} s & -q \\ -r & p \end{pmatrix}$	2
Distance to hyperplane	$\left\lvert\mathbf{a}^\top\mathbf{x}_0 - b\right\rvert / \lVert\mathbf{a}\rVert_2$	2
Vertex test	n linearly independent active constraints	3
Recession direction	$\mathbf{A}\mathbf{d} \le \mathbf{0}$; unbounded if $\mathbf{c}^\top\mathbf{d} > 0$	3
Absolute value	$t \ge \lvert u\rvert \Leftrightarrow t \ge u,\ t \ge -u$	4
Minimax	$\min\max_k(\mathbf{d}_k^\top\mathbf{x} + e_k) = \min\{t : t \ge \mathbf{d}_k^\top\mathbf{x} + e_k\}$	4
Fractional LP	$\mathbf{y} = \tau\mathbf{x}$, $\tau = 1/(\mathbf{q}^\top\mathbf{x} + \beta)$	4
Basic solution	$\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$, $\mathbf{x}_N = \mathbf{0}$	7
Reduced cost	$\bar{c}_j = c_j - \mathbf{c}_B^\top\mathbf{B}^{-1}\mathbf{A}_j = c_j - \mathbf{y}^\top\mathbf{A}_j$	8, 11
Ratio test	$\theta^\star = \min\{\bar{b}_i/\bar{a}_{iq} : \bar{a}_{iq} > 0\}$	8
Optimality (max)	all row 0 entries ≥ 0	8
Eta update	$\mathbf{B}'^{-1} = \mathbf{E}\mathbf{B}^{-1}$, $\eta_r = 1/d_r$, $\eta_i = -d_i/d_r$	11

Topic	Result	Module
Dual of canonical LP	min $\mathbf{b}^\top\mathbf{y}$ subject to $\mathbf{A}^\top\mathbf{y} \ge \mathbf{c}, \mathbf{y} \ge \mathbf{0}$	12
Weak duality	$\mathbf{c}^\top\mathbf{x} \le \mathbf{b}^\top\mathbf{y}$	12
Duality gap	$\mathbf{b}^\top\mathbf{y} - \mathbf{c}^\top\mathbf{x} = \mathbf{s}^\top\mathbf{y} + \mathbf{t}^\top\mathbf{x}$	13
Complementary slackness	$y_i s_i = 0, x_j t_j = 0$	13
Farkas	$\mathbf{Ax} \le \mathbf{b}, \mathbf{x} \ge \mathbf{0}$ infeasible $\Leftrightarrow \exists\mathbf{y} \ge \mathbf{0}$: $\mathbf{A}^\top\mathbf{y} \ge \mathbf{0}, \mathbf{b}^\top\mathbf{y} < 0$	13
Dual simplex ratio	$\min\{\delta_j / \lvert\bar{a}_{rj}\rvert : \bar{a}_{rj} < 0\}$	14
RHS ranging	$\bar{\mathbf{b}} + \Delta\,\mathbf{B}^{-1}\mathbf{e}_k \ge \mathbf{0}$; $\Delta z^\star = y_k\Delta$	15
Central path	$\mathbf{Ax} = \mathbf{b}, \mathbf{A}^\top\mathbf{y} + \mathbf{s} = \mathbf{c}, x_j s_j = \mu$; gap $= n\mu$	16
Newton system	$\mathbf{A}\Delta\mathbf{x} = \mathbf{r}_p, \mathbf{A}^\top\Delta\mathbf{y} + \Delta\mathbf{s} = \mathbf{r}_d, \mathbf{S}\Delta\mathbf{x} + \mathbf{X}\Delta\mathbf{s} = \mathbf{r}_c$	16
Gomory cut	$\sum_j f_j x_j \ge f_0$	17
Reduced arc cost	$c_{uv} - \pi_u + \pi_v$	18
LAD regression	min $\lVert\mathbf{y} - \mathbf{Xw}\rVert_1$; dual $\mathbf{X}^\top\mathbf{d} = \mathbf{0}, \lvert d_i\rvert \le 1$	19
Chebyshev regression	min $\lVert\mathbf{y} - \mathbf{Xw}\rVert_\infty$; dual $\mathbf{X}^\top\mathbf{d} = \mathbf{0}, \lVert\mathbf{d}\rVert_1 \le 1$	20
Pinball loss	$\rho_\tau(r) = \max(\tau r, (\tau - 1)r)$; dual $\tau - 1 \le d_i \le \tau$	20
ℓ_1 SVM	min $\lVert\mathbf{w}\rVert_1 + C\sum\xi_i$ subject to $y_i(\mathbf{w}^\top\mathbf{x}_i + b) \ge 1 - \xi_i$	21
Basis pursuit	min $\lVert\mathbf{x}\rVert_1$ subject to $\mathbf{Ax} = \mathbf{b}$; dual $\lVert\mathbf{A}^\top\mathbf{y}\rVert_\infty \le 1$	22
Soft thresholding	$\operatorname{sign}(c)\max(\lvert c\rvert - \delta, 0)$	22
Optimal transport	min $\sum C_{ij}P_{ij}$, $\mathbf{P1} = \mathbf{a}$, $\mathbf{P}^\top\mathbf{1} = \mathbf{b}$; dual $f_i + g_j \le C_{ij}$	23
Sinkhorn	$\mathbf{u} \leftarrow \mathbf{a} \oslash \mathbf{Kv}, \mathbf{v} \leftarrow \mathbf{b} \oslash \mathbf{K}^\top\mathbf{u}, K_{ij} = e^{-C_{ij}/\varepsilon}$	23
Matrix game	$\max\{v : \mathbf{A}^\top\mathbf{p} \ge v\mathbf{1},\ \mathbf{p} \in \Delta_m\}$ = value	24
Robust margin	$y(\mathbf{w}^\top\mathbf{x} + b) - \varepsilon\lVert\mathbf{w}\rVert_1$	24
MDP primal	min $\sum\mu(s)V(s)$, $V(s) \ge r(s,a) + \gamma\sum_{s'}P(s' \mid s,a)V(s')$	25
MDP dual	occupancy $x(s,a) \ge 0$ with flow conservation; $\sum x = 1/(1-\gamma)$	25

Appendix B Solving Linear Programs in Python

Every worked example in this manual was checked with the open source HiGHS solver through the SciPy library. The listings below show how to reproduce the main computations. To keep the code consistent with the typographical conventions of this manual, arguments are passed by position: the call linprog(c, A, b, Aeq, beq, bounds) means “minimise $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$, $\mathbf{A}_{\mathrm{eq}}\mathbf{x} = \mathbf{b}_{\mathrm{eq}}$ and the given bounds”. Remember that linprog always *minimises*; to maximise, negate the objective and negate the reported optimal value.

B.1 The furniture workshop (Worked Examples 1.1, 5.1 and 8.1)

```
import numpy as np
from scipy.optimize import linprog

c = np.array([30, 20])          # profits per table and per chair
A = np.array([[4, 2],           # timber used per unit
        [2, 3]])          # labour hours per unit
b = np.array([40, 30])          # timber and labour available

res = linprog(−c, A, b, method=”highs”)  # negate to maximise
print(res.x)                    # optimal plan [7.5, 5.0]
print(−res.fun)                 # optimal profit 325.0
print(−res.ineqlin.marginals)   # shadow prices [6.25, 2.5]
```

B.2 Least absolute deviation regression (Module 19)

```
import numpy as np
from scipy.optimize import linprog

x = np.array([1, 2, 3, 4, 5.0])
y = np.array([2, 4, 6, 9, 20.0])
X = np.vstack([np.ones(5), x]).T     # intercept and slope columns
N, p = X.shape

# variables: w (p, free), u (N, >= 0), v (N, >= 0)
cost = np.concatenate([np.zeros(p), np.ones(2 ∗ N)])
Aeq = np.hstack([X, np.eye(N), −np.eye(N)])     # X w + u − v = y
bounds = [(None, None)] ∗ p + [(0, None)] ∗ (2 ∗ N)
```

```
res = linprog(cost, None, None, Aeq, y, bounds, method="highs")
print(res.x[:p])          # [−1.0, 2.5]
print(res.fun)            # total absolute error 9.5
print(res.eqlin.marginals) # dual vector d, the residual signs
```

B.3 Quantile regression (Module 20)

```
def quantile(X, y, tau):
    N, p = X.shape
    cost = np.concatenate([np.zeros(p),
                           tau * np.ones(N),        # under prediction
                           (1 − tau) * np.ones(N)]) # over prediction
    Aeq = np.hstack([X, np.eye(N), −np.eye(N)])
    bounds = [(None, None)] * p + [(0, None)] * (2 * N)
    res = linprog(cost, None, None, Aeq, y, bounds, method="highs")
    return res.x[:p]
```

B.4 Optimal transport (Module 23)

```
def transport(a, b, C):
    n, m = C.shape
    rows = np.kron(np.eye(n), np.ones(m))  # each row of P sums to a
    cols = np.kron(np.ones(n), np.eye(m))  # each column of P sums to b
    Aeq = np.vstack([rows, cols])
    beq = np.concatenate([a, b])
    res = linprog(C.ravel(), None, None, Aeq, beq, method="highs")
    return res.x.reshape(n, m), res.fun

a = np.array([0.5, 0.3, 0.2])
b = np.array([0.1, 0.3, 0.6])
pts = np.array([0, 1, 2.0])
C = np.abs(pts[:, None] − pts[None, :])
plan, cost = transport(a, b, C)           # cost 0.8
```

B.5 Sinkhorn iterations (Module 23)

```
def sinkhorn(a, b, C, eps, iters=500):
    K = np.exp(−C / eps)
    v = np.ones(len(b))
    for k in range(iters):
        u = a / (K @ v)
        v = b / (K.T @ u)
    return np.diag(u) @ K @ np.diag(v)
```

B.6 Matrix games (Module 24)

```
def solvegame(A):
    m, n = A.shape
    # variables: p (m entries, >= 0) and v (free); maximise v
    cost = np.concatenate([np.zeros(m), [−1.0]])
    Aub = np.hstack([−A.T, np.ones((n, 1))])    # v − (A^T p)j <= 0
    Aeq = np.concatenate([np.ones(m), [0.0]]).reshape(1, −1)
    bounds = [(0, None)] ∗ m + [(None, None)]
    res = linprog(cost, Aub, np.zeros(n), Aeq, [1.0], bounds,
                  method=”highs”)
    return res.x[:m], res.x[−1]

p, value = solvegame(np.array([[3, −1], [−2, 4.0]]))   # (0.6, 0.4), 1
```

B.7 Markov decision process (Module 25)

```
gamma = 0.9
# rows: (state, action) pairs; each row gives V(s) − gamma P V >= r
rows = [([1 − gamma, 0], 1),                    # s1 stay
        ([1 − 0.9 ∗ 0.2, −0.9 ∗ 0.8], 0),       # s1 move
        ([0, 1 − gamma], 2),                    # s2 safe
        ([−0.9 ∗ 0.5, 1 − 0.9 ∗ 0.5], 4)]       # s2 risky
Aub = −np.array([r[0] for r in rows])
bub = −np.array([r[1] for r in rows])
res = linprog([0.5, 0.5], Aub, bub, bounds=[(None, None)] ∗ 2,
              method=”highs”)
print(res.x)                  # optimal values [22.677, 25.827]
print(−res.ineqlin.marginals)   # occupancy measures x(s, a)
```

Appendix C Glossary

Active constraint
A constraint satisfied with equality at a given point; also called tight or binding.

Artificial variable
A temporary nonnegative variable added to ≥ or = rows to obtain a starting basis; it must be driven to zero.

Basis
A set of m linearly independent columns of $\mathbf{A}$; its matrix $\mathbf{B}$ is invertible.

Basic feasible solution
The solution $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b} \geq \mathbf{0}$, $\mathbf{x}_N = \mathbf{0}$ of a basis; geometrically a vertex.

Bland's rule
Choose the entering and leaving variables with the smallest index; prevents cycling.

Branch and bound
Enumeration of integer solutions that uses LP relaxations to prune subproblems.

Central path
The curve of solutions of the barrier problem, parameterised by $\mu > 0$.

Complementary slackness
At optimality each primal slack times its dual variable is zero, and each primal variable times its dual surplus is zero.

Convex set
A set containing the segment between any two of its points.

Cycling
An infinite loop of degenerate simplex pivots returning to a previous basis.

Degenerate solution
A basic solution in which some basic variable equals zero.

Dual problem
The LP whose optimal value gives the best bound obtainable by combining primal constraints.

Duality gap
The difference between dual and primal objective values for a feasible pair.

Epigraph variable
An auxiliary variable bounding a maximum or absolute value from above.

Farkas' lemma
Exactly one of a linear system and its alternative (a certificate of infeasibility) has a solution.

Feasible region
The set of all points satisfying every constraint.

Hinge loss
$\max(0, 1 - y(\mathbf{w}^\top \mathbf{x} + b))$, the loss of support vector machines.

Integrality gap
The difference between the optimal values of an integer program and its LP relaxation.

Interior point method
An algorithm that follows the central path through the interior of the feasible region.

Kantorovich potentials
The dual variables of an optimal transport problem.

Null space property
A condition on $\mathbf{A}$ equivalent to exact ℓ_1 recovery of all sparse vectors.

Occupancy measure
The discounted frequency of state action pairs under a policy; the dual variables of the MDP LP.

Pinball loss
The asymmetric absolute loss whose minimiser is a quantile.

Pivot
One Gaussian elimination step that exchanges a basic and a nonbasic variable.

Polyhedron
An intersection of finitely many half spaces; a polytope if bounded.

Recession direction
A direction along which one can move forever without leaving the feasible region.

Reduced cost
The rate of change of the objective when a nonbasic variable increases.

Shadow price
The rate of change of the optimal value with respect to a right hand side; the optimal dual variable.

Simplex multipliers
$\mathbf{y}^\top = \mathbf{c}_B^\top \mathbf{B}^{-1}$, the dual vector associated with a basis.

Slack and surplus
Nonnegative variables converting $\leq$ and $\geq$ constraints into equations.

Support vector
A training point with a positive dual multiplier in an SVM.

Totally unimodular
A matrix whose square submatrices all have determinant 0 or ±1; gives integral vertices.

Wasserstein distance
The optimal transport cost between two distributions with a distance based cost.

Appendix D Further Reading

The following books and papers develop the topics of this manual further. They are grouped by theme.

Linear programming and optimisation

- D. Bertsimas and J. N. Tsitsiklis, *Introduction to Linear Optimization*, Athena Scientific, 1997.
- V. Chvátal, *Linear Programming*, W. H. Freeman, 1983.
- G. B. Dantzig and M. N. Thapa, *Linear Programming 1: Introduction*, Springer, 1997.
- R. J. Vanderbei, *Linear Programming: Foundations and Extensions*, 5th edition, Springer, 2020.
- S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- S. J. Wright, *Primal Dual Interior Point Methods*, SIAM, 1997.
- L. A. Wolsey, *Integer Programming*, 2nd edition, Wiley, 2020.

Regression, classification and sparsity

- R. Koenker, *Quantile Regression*, Cambridge University Press, 2005.
- P. S. Bradley and O. L. Mangasarian, Feature selection via concave minimization and support vector machines, *Proceedings of ICML*, 1998.
- J. Zhu, S. Rosset, T. Hastie and R. Tibshirani, 1 norm support vector machines, *Advances in Neural Information Processing Systems*, 2004.
- S. S. Chen, D. L. Donoho and M. A. Saunders, Atomic decomposition by basis pursuit, *SIAM Journal on Scientific Computing*, 1998.
- E. Candès and T. Tao, The Dantzig selector, *Annals of Statistics*, 2007.
- T. Hastie, R. Tibshirani and M. Wainwright, *Statistical Learning with Sparsity*, CRC Press, 2015.

Transport, games and reinforcement learning

- G. Peyré and M. Cuturi, *Computational Optimal Transport*, Foundations and Trends in Machine Learning, 2019.
- M. Cuturi, Sinkhorn distances: lightspeed computation of optimal transport, *Advances in Neural Information Processing Systems*, 2013.
- M. Arjovsky, S. Chintala and L. Bottou, Wasserstein generative adversarial networks, *Proceedings of ICML*, 2017.

- S. Sagawa, P. W. Koh, T. B. Hashimoto and P. Liang, Distributionally robust neural networks, *Proceedings of ICLR*, 2020.
- M. L. Puterman, *Markov Decision Processes*, Wiley, 1994.
- E. Altman, *Constrained Markov Decision Processes*, Chapman and Hall, 1999.

Appendix E Practice Exercises with Answers

These exercises let you test each module independently. Try each one before reading its answer, which appears directly below the exercises of every module. Every numerical answer was checked with a solver.

Module 1 The Anatomy of a Linear Program

1. A bakery earns £5 per batch of bread and £4 per batch of cake. Bread uses 6 kg of flour and 1 oven hour per batch; cake uses 4 kg and 2 oven hours. There are 24 kg of flour and 6 oven hours. Formulate the LP and find the best of the plans $(4, 0)$, $(0, 3)$ and $(3, 1.5)$.

2. Rewrite "at least 30% of total output must be product 1" as a linear constraint in x_1, x_2.

3. Give $\mathbf{c}$, $\mathbf{A}$, $\mathbf{b}$ for max $x_1 + 2x_2 - x_3$ subject to $x_1 + x_3 \le 4$, $x_2 - x_1 \ge 1$, $\mathbf{x} \ge \mathbf{0}$, using only $\le$ rows.

1. max $5x_1 + 4x_2$ subject to $6x_1 + 4x_2 \le 24$, $x_1 + 2x_2 \le 6$, $\mathbf{x} \ge \mathbf{0}$. All three plans are feasible, with profits 20, 12 and 21; the best is $(3, 1.5)$ with $z = 21$ (it is in fact optimal). 2. $x_1 \ge 0.3(x_1 + x_2)$, that is $0.7x_1 - 0.3x_2 \ge 0$. 3. $\mathbf{c} = (1, 2, -1)$, $\mathbf{A} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{pmatrix}$, $\mathbf{b} = (4, -1)$.

Module 2 Linear Algebra Essentials

1. Compute $\mathbf{Ax}$ for $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 0 & 3 \\ 1 & 1 \end{pmatrix}$ and $\mathbf{x} = (1, 2)$.

2. Invert $\begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$.

3. Find the distance from $(1, 1)$ to the line $3x_1 + 4x_2 = 12$.

1. $(0,\ 6,\ 3)$. 2. Determinant 2; inverse $\begin{pmatrix} 1 & -0.5 \\ -2 & 1.5 \end{pmatrix}$. 3. $|3 + 4 - 12| / 5 = 1$.

Module 3 Convex Sets, Polyhedra and Extreme Points

1. List the vertices of $\{x_1 + x_2 \le 5,\ x_1 + 3x_2 \le 9,\ \mathbf{x} \ge \mathbf{0}\}$ and maximise $2x_1 + 3x_2$ over it.

2. Is $\{\mathbf{x} \in \mathbb{R}^2 : x_1^2 + x_2^2 \ge 1\}$ convex?

3. Find all recession directions of $\{\mathbf{x} \geq \mathbf{0} : x_1 - x_2 \leq 1\}$.

1. $(0,0)$, $(5,0)$, $(3,2)$, $(0,3)$; maximum 12 at $(3,2)$. 2. No: $(1,0)$ and $(-1,0)$ belong to it but their midpoint $(0,0)$ does not. 3. All nonzero $\mathbf{d} \geq \mathbf{0}$ with $d_1 \leq d_2$.

Module 4 Modelling Techniques and Linearisation

1. Minimise $|x_1 - 3| + |x_2 + 1|$ over free x_1, x_2 with $x_1 + x_2 \geq 4$.

2. Minimise $\max(x,\ 2 - x,\ 0.5x + 1)$ over $x \in \mathbb{R}$.

3. Apply the Charnes and Cooper substitution to max $(2x_1 + x_2)/(x_1 + x_2 + 1)$ subject to $\mathbf{Ax} \leq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$.

1. Value 2 (for example at $(5,-1)$ or $(3,1)$), since $|x_1 - 3| + |x_2 + 1| \geq (x_1 + x_2) - 2 \geq 2$. 2. Value $\frac{4}{3}$ at $x = \frac{2}{3}$, where $2 - x = 0.5x + 1$. 3. max $2y_1 + y_2$ subject to $\mathbf{Ay} - \mathbf{b}\tau \leq \mathbf{0}$, $y_1 + y_2 + \tau = 1$, $\mathbf{y}, \tau \geq 0$; then $\mathbf{x} = \mathbf{y}/\tau$.

Module 5 The Graphical Method

1. Solve max $3x_1 + 2x_2$ subject to $x_1 + x_2 \leq 6$, $2x_1 + x_2 \leq 10$, $x_1 \leq 4$, $\mathbf{x} \geq \mathbf{0}$.

2. Solve min $4x_1 + 3x_2$ subject to $2x_1 + x_2 \geq 8$, $x_1 + x_2 \geq 6$, $\mathbf{x} \geq \mathbf{0}$.

3. In Exercise 1, for which values of c_1 (with $c_2 = 2$) does $(4,2)$ remain optimal?

1. $(4,2)$, $z = 16$ (a degenerate vertex with three active constraints). 2. $(2,4)$, cost 20. 3. $c_1 \geq 2$: then $(c_1, 2) = (c_1 - 2)(1,0) + 2(1,1)$ lies in the cone of the normals of $x_1 \leq 4$ and $x_1 + x_2 \leq 6$.

Module 6 Standard Form and Conversions

1. Convert max $x_1 - x_2$ subject to $x_1 + 2x_2 \geq 3$, $x_1 \leq 5$, $x_1 \geq 0$, x_2 free, to standard form.

2. How many equations and variables does the standard form have for 4 constraints of type $\leq$, 1 of type $\geq$, 2 equations, 5 nonnegative and 1 free variable?

3. For the furniture problem, compute the slacks of the plan $(6,4)$.

1. min $-x_1 + x_2^+ - x_2^-$ subject to $x_1 + 2x_2^+ - 2x_2^- - e_1 = 3$, $x_1 + s_2 = 5$, all variables ≥ 0. 2. 7 equations and $5 + 2 + 4 + 1 = 12$ variables. 3. $s_1 = 40 - 24 - 8 = 8$, $s_2 = 30 - 12 - 12 = 6$.

Module 7 Basic Feasible Solutions

1. List all basic solutions of $x_1 + x_2 + s_1 = 5$, $x_1 + 3x_2 + s_2 = 9$ and mark the feasible ones.

2. What is the maximum possible number of bases for $m = 3$, $n = 8$?

3. Is the vertex $(2,0)$ of $\{x_1 + x_2 \leq 2,\ x_1 \leq 2,\ \mathbf{x} \geq \mathbf{0}\}$ degenerate?

1. $\{x_1, x_2\}$: $(3,2)$ feasible; $\{x_1, s_1\}$: $x_1 = 9$, $s_1 = -4$ infeasible; $\{x_1, s_2\}$: $(5,\ s_2 = 4)$ feasible; $\{x_2, s_1\}$: $(3,\ s_1 = 2)$ feasible; $\{x_2, s_2\}$: $x_2 = 5$, $s_2 = -6$ infeasible; $\{s_1, s_2\}$: $(5,9)$ feasible. 2. $\binom{8}{3} = 56$. 3. Yes: three constraints $(x_1 + x_2 = 2,\ x_1 = 2,\ x_2 = 0)$ are active, so one basic variable is zero.

Module 8 The Simplex Method

1. Solve the bakery problem of Exercise 1.1 by the simplex method and read the shadow prices.

2. Solve Exercise 5.1 by the simplex method.

3. A final row 0 reads (0, 0, 1.5, 0, 2 | 40) under $(x_1, x_2, x_3, s_1, s_2)$. What is the optimal value, which resource is not binding, and what does the 1.5 mean?

1. x_1 enters (s_1 leaves), then x_2 enters (s_2 leaves): $(3, 1.5)$, $z = 21$, shadow prices 0.75 (flour) and 0.5 (oven). 2. $(4, 2)$, $z = 16$, with a tie in the ratio test. 3. $z^\star = 40$; resource 1 (shadow price 0); forcing one unit of x_3 would lower z by 1.5.

Module 9 Artificial Variables

1. Solve min $3x_1 + 5x_2$ subject to $x_1 + 2x_2 \geq 8$, $3x_1 + x_2 \geq 9$, $\mathbf{x} \geq \mathbf{0}$.

2. What does Phase I report for $x_1 + x_2 \geq 6$, $x_1 + x_2 \leq 4$?

3. Solve min $2x_1 + x_2$ subject to $x_1 + x_2 \geq 4$, $x_1 - x_2 \leq 2$, $\mathbf{x} \geq \mathbf{0}$.

1. $(2, 3)$, cost 21. 2. Minimum total artificial value $2 > 0$: infeasible. 3. $(0, 4)$, cost 4.

Module 10 Special Cases

1. Is max $x_1 + 2x_2$ subject to $-x_1 + x_2 \leq 2$, $x_2 \leq 5$, $\mathbf{x} \geq \mathbf{0}$ bounded?

2. Describe all optimal solutions of max $x_1 + x_2$ subject to $x_1 + x_2 \leq 4$, $x_1 \leq 3$, $\mathbf{x} \geq \mathbf{0}$.

3. State Bland's rule.

1. Unbounded: $\mathbf{d} = (1, 0)$ satisfies $-1 \leq 0$, $0 \leq 0$ and $\mathbf{c}^\top\mathbf{d} = 1 > 0$. 2. The segment from $(3, 1)$ to $(0, 4)$, value 4. 3. Enter the improving variable with the smallest index; among tied ratio rows, remove the basic variable with the smallest index.

Module 11 The Revised Simplex Method

1. For the bakery basis $\{x_1, x_2\}$, compute $\mathbf{B}^{-1}$, $\mathbf{x}_B$ and $\mathbf{y}$.

2. Price a new product with profit 3 and resource column $(2, 3)$ at those prices.

3. Find the eta vector for $\mathbf{d} = (2, -1, 4)$ and pivot row $r = 1$.

1. $\mathbf{B}^{-1} = \frac{1}{8}\begin{pmatrix} 2 & -4 \\ -1 & 6 \end{pmatrix}$, $\mathbf{x}_B = (3,\ 1.5)$, $\mathbf{y} = (0.75,\ 0.5)$. 2. $\bar{c} = 3 - (1.5 + 1.5) = 0$: indifferent. 3. $\boldsymbol{\eta} = (0.5,\ 0.5,\ -2)$.

Module 12 Duality Theory

1. Write and solve the dual of max $6x_1 + 5x_2$ subject to $x_1 + x_2 \leq 5$, $3x_1 + 2x_2 \leq 12$, $\mathbf{x} \geq \mathbf{0}$.

2. Write the dual of min $\mathbf{c}^\top\mathbf{x}$ subject to $\mathbf{A}\mathbf{x} \geq \mathbf{b}$, $\mathbf{x}$ free.

3. If a feasible primal plan earns 25 and a feasible dual vector costs 28 in Exercise 1, what can you conclude?

1. min $5y_1 + 12y_2$ subject to $y_1 + 3y_2 \geq 6$, $y_1 + 2y_2 \geq 5$, $\mathbf{y} \geq \mathbf{0}$; $\mathbf{y}^\star = (3, 1)$, value 27, equal to the primal optimum at $(2, 3)$. 2. max $\mathbf{b}^\top \mathbf{y}$ subject to $\mathbf{A}^\top \mathbf{y} = \mathbf{c}$, $\mathbf{y} \geq \mathbf{0}$. 3. $25 \leq z^\star \leq 28$.

Module 13 Complementary Slackness and Farkas' Lemma

1. Recover the dual solution of Exercise 12.1 from $\mathbf{x}^\star = (2, 3)$.

2. Give a Farkas certificate that $x_1 + x_2 \geq 6$, $x_1 + x_2 \leq 4$ has no solution.

3. Use complementary slackness to show that $(4, 0)$ is not optimal in Exercise 12.1.

1. Both $x_j > 0$, so $y_1 + 3y_2 = 6$ and $y_1 + 2y_2 = 5$, giving $\mathbf{y} = (3, 1)$. 2. Weights $(1, 1)$: $(-x_1 - x_2) + (x_1 + x_2) \leq -6 + 4$, that is $0 \leq -2$. 3. Slack in row 1 gives $y_1 = 0$; $x_1 > 0$ gives $3y_2 = 6$, so $y_2 = 2$; then $y_1 + 2y_2 = 4 < 5$ violates dual feasibility.

Module 14 The Dual Simplex Method

1. Solve min $5x_1 + 4x_2$ subject to $3x_1 + x_2 \geq 9$, $x_1 + 2x_2 \geq 8$, $\mathbf{x} \geq \mathbf{0}$ by the dual simplex method.

2. Add $x_2 \leq 2$ to Exercise 12.1 and reoptimise.

3. What tableau pattern proves infeasibility in the dual simplex method?

1. $(2, 3)$, cost 22. 2. $x_2 = 2$, $x_1 = \frac{8}{3}$, $z = 26$. 3. A row with negative right hand side and no negative entries.

Module 15 Sensitivity Analysis

1. In Exercise 12.1, find the range of b_1 for which the basis $\{x_1, x_2\}$ stays optimal.

2. Find the range of c_2 (with $c_1 = 6$) keeping $(2, 3)$ optimal.

3. Should a product with profit 8 and column $(2, 1)$ be introduced?

1. $\mathbf{B}^{-1} = \begin{pmatrix} -2 & 1 \\ 3 & -1 \end{pmatrix}$; $(2, 3) + \Delta(-2, 3) \geq \mathbf{0}$ gives $-1 \leq \Delta \leq 1$, so $4 \leq b_1 \leq 6$, with $z = 27 + 3\Delta$. 2. $4 \leq c_2 \leq 6$. 3. Yes: $\mathbf{y}^\top \mathbf{a} = 3(2) + 1(1) = 7 < 8$.

Module 16 Interior Point Methods

1. Find the barrier solution of max x subject to $0 \leq x \leq 2$ for $\mu = 1$.

2. For $\mathbf{x} = (1, 2, 1)$ and $\mathbf{s} = (0.2, 0.1, 0.6)$, compute the gap and duality measure.

3. How many short step iterations reduce the gap by 10^{-10} when $n = 400$?

1. $1 + 1/x - 1/(2 - x) = 0$ gives $x^2 = 2$, $x = \sqrt{2} \approx 1.414$. 2. Gap $0.2 + 0.2 + 0.6 = 1.0$, $\mu = \frac{1}{3}$. 3. $23.03/(-\ln 0.98) \approx 1{,}140$.

Module 17   Integer Programming

1. Solve max $3x_1 + 2x_2$ subject to $2x_1 + 2x_2 \leq 9$, $3x_1 + x_2 \leq 8$, $\mathbf{x} \geq \mathbf{0}$ integer, and compare with the LP relaxation.

2. Model "choose at most two of four features" with binaries.

3. Derive the Gomory cut from the row $x_1 + 0.5s_1 - 1.5s_2 = 2.25$.

1. LP: $(1.75,\ 2.75)$, $z = 10.75$; integer optimum $(2, 2)$, $z = 10$.   2. $y_1 + y_2 + y_3 + y_4 \leq 2$, $y_j \in \{0, 1\}$.   3. $0.5s_1 + 0.5s_2 \geq 0.25$.

Module 18   Network Flows

1. Solve the transportation problem with supplies $(30, 20)$, demands $(25, 25)$ and costs $\begin{pmatrix} 2 & 4 \\ 3 & 1 \end{pmatrix}$.

2. Find the maximum flow with capacities $s \to a$ 3, $s \to b$ 2, $a \to t$ 2, $b \to t$ 3, $a \to b$ 1.

3. Why do network LPs have integral solutions?

1. $x_{11} = 25$, $x_{12} = 5$, $x_{22} = 20$, cost 90; potentials $u = (0, -3)$, $v = (2, 4)$ give reduced cost $4 \geq 0$ on the unused route.   2. 5 (paths *sat*: 2, *sbt*: 2, *sabt*: 1; cut $\{s\}$ has capacity 5).   3. Their incidence matrices are totally unimodular.

Module 19   LAD Regression

1. Find the LAD line for $(0, 0)$, $(1, 1)$, $(2, 4)$.

2. Find the LAD constant for $(5, 1, 9, 3, 7)$.

3. What are the bounds on the LAD dual variables?

1. $\hat{y} = 2x$, total error 1 (the other pair lines give 2).   2. The median, 5.   3. $-1 \leq d_i \leq 1$, with $\mathbf{X}^\top \mathbf{d} = \mathbf{0}$.

Module 20   Chebyshev and Quantile Regression

1. Find the Chebyshev line for $(0, 0)$, $(1, 2)$, $(2, 2)$.

2. Find all 0.8 quantiles of $1, 2, \ldots, 10$.

3. Compute $\rho_{0.9}(-2)$.

1. $\hat{y} = 0.5 + x$, maximum error 0.5 with residual signs $-, +, -$.   2. Any $q \in [8, 9]$.   3. $(0.9 - 1)(-2) = 0.2$.

Module 21   Linear Classification

1. For $w = 1$, $b = -2$, compute the hinge losses of $(x, y) = (1, -1)$, $(3, +1)$, $(2.5, -1)$.

2. Are negatives $\{1, 4\}$ and positives $\{2, 3\}$ on a line separable?

3. Compute the ℓ_∞ distance from $(1, 1)$ to $2x_1 - x_2 = 0$.

1. Margins $1,\ 1,\ -0.5$; losses $0,\ 0,\ 1.5$. 2. No: the midpoint 2.5 of the positives equals the midpoint of the negatives. 3. $|2-1| \,/\, \|(2,-1)\|_1 = \frac{1}{3}$.

Module 22 Sparse Recovery

1. Solve basis pursuit for $\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$, $\mathbf{b} = (1, 1)$.

2. Soft threshold $\mathbf{c} = (-2.5,\ 0.3,\ 1.0)$ at $\delta = 0.5$.

3. Does a matrix whose null space is spanned by $(1, -2, 1)$ satisfy the null space property of order 1?

1. Solutions $(1-t,\ t,\ 1-t)$; the ℓ_1 norm $2\,|1-t| + |t|$ is minimised at $t = 1$: $\mathbf{x} = (0, 1, 0)$. 2. $(-2,\ 0,\ 0.5)$. 3. No: for $S = \{2\}$, $\|\mathbf{h}_S\|_1 = 2$ is not strictly less than $\left\|\mathbf{h}_{\bar{S}}\right\|_1 = 2$.

Module 23 Optimal Transport

1. Find W_1 between $(0.6, 0.4)$ and $(0.2, 0.8)$ on the points $\{0, 1\}$.

2. Find W_1 between $(0.2, 0.3, 0.5)$ and $(0.5, 0.3, 0.2)$ on $\{0, 1, 2\}$.

3. Compute the Gibbs kernel for $\mathbf{C} = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$ and $\varepsilon = 1$.

1. 0.4. 2. CDFs $(0.2, 0.5, 1)$ and $(0.5, 0.8, 1)$: $0.3 + 0.3 = 0.6$. 3. $\begin{pmatrix} 1 & e^{-2} \\ e^{-2} & 1 \end{pmatrix} \approx \begin{pmatrix} 1 & 0.135 \\ 0.135 & 1 \end{pmatrix}$.

Module 24 Games and Adversarial Robustness

1. Solve the game $\begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}$.

2. Does $\begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$ have a pure saddle point?

3. Compute the worst case margin of $\mathbf{w} = (1, -2)$, $b = 0.5$ at $\mathbf{x} = (1, 0)$, $y = 1$, $\varepsilon = 0.1$.

1. $\mathbf{p} = (0.4, 0.6)$, $\mathbf{q} = (0.4, 0.6)$, $v^\star = 0.2$. 2. Yes: row 2, column 2, value 2 (the largest row minimum equals the smallest column maximum). 3. $1.5 - 0.1 \times 3 = 1.2$.

Module 25 Markov Decision Processes

1. A single state MDP gives reward 3 each step with $\gamma = 0.8$. Find its value.

2. States $s_1 \to s_2$ (reward 1) and $s_2 \to s_2$ (reward 0), $\gamma = 0.5$. Find V.

3. What is the total occupancy $\sum_{s,a} x(s, a)$ when $\gamma = 0.8$?

1. $3/(1-0.8) = 15$. 2. $V(s_2) = 0$, $V(s_1) = 1$. 3. $1/(1-0.8) = 5$.

End of the manual